

Tanta University

Faculty of Computers and Informatics



رومانى صبحى رزق جرجس مطر

Computer Security

2025/2026 2025/2026 2025/2026

Third Year – 1st Semester 2026 / 2027

Prepared By

Dr. Moustafa El-Ashry

Course Specification

(2025)

1. Basic Information

Course Title (according to the bylaw)	**Computer Security**			
Course Code (according to the bylaw)	**CS 311**			
Department/s participating in delivery of the course	**CS**			
Number of credit hours/points of the course (according to the bylaw)	**Theoretical**	**Practical**	**Other (specify)**	**Total**
	2	**2**		
Course Type	اجباري			
Academic level at which the course is taught	الفرقة/المستوي الثالث			
Academic Program	**First Term 2025 / 2026**			
Faculty/Institute	**Faculty of Computers and Informatics**			
University/Academy	**Tanta University**			
Name of Course Coordinator	**Dr. Moustafa El-Ashry**			
Course Specification Approval Date	**9/19/2026**			
Course Specification Approval (Attach the decision/minutes of the department /committee/council ....)				

2. Course Overview (Brief summary of scientific content)

. The course presents a comprehensive study of the principles of computer system security. The course covers various topics including: Access control, Key managements, Cryptography, Program and programming security that includes various types of Malwares, operating system security, database security, network security.

3. Course Learning Outcomes CLOs

Matrix of course learning outcomes CLOs with program outcomes POs (NARS/ARS)

Program Outcomes (NARS/ARS) (according to the matrix in the program specs)		Course Learning Outcomes Upon completion of the course, the student will be able to:	
Code	**Text**	**Code**	**Text**
A3	Demonstrate basic knowledge and understanding of a core of analysis, algebra, applied mathematics and statistics.	A1	Recognize the fundamental notions of threat, vulnerability, attack, countermeasure and security goals of a computing system.
A5	Know and understand the principles and techniques of a number of application areas informed by the research directions of the subject, such as artificial intelligence, natural language processing, data mining, databases and computer graphics.	A2	Describe the major types of cryptographic algorithms and typical applications.
A7	Understand the fundamental topics in Computer Science, including hardware and software architectures, software engineering principles and methodologies, operating systems, compilers, parallel and distributed computing, systems and software tools.	A3	Explain the mechanisms available in an OS to control access to resources, concepts of authentication, and authorization

Program Outcomes (NARS/ARS) (according to the matrix in the program specs)		Course Learning Outcomes Upon completion of the course, the student will be able to:	
Code	**Text**	**Code**	**Text**
B1	Define traditional and nontraditional problems, set goals towards solving them, and. observes results.	**B1**	Create simple extensions of cryptographic protocols, using known protocols and cryptographic primitives.
B5	Summarize the proposed solutions and their results.	**B2**	Explain the mechanisms available in an OS to control access to resources.
C1	Use appropriate programming languages, web-based systems and tools, design methodologies, and knowledge and database systems.	**C1**	Compare and contrast two security models.
C3	Perform independent information acquisition and management, using the scientific literature and Web sources.	**C2** 2025/2026	Rewrite a simple program to remove a simple vulnerability. 2025/2026
C3	Evaluate systems in terms of general quality attributes and possible tradeoffs presented within the given problem.	**C3**	Summarize the features and limitations of an operating system used to provide protection and security.
D3	Demonstrate skills in group working, team management, time management and organizational skills.	**D1**	Show ability to work in teams in accordance with ethics and proper code of conduct.

7. Teaching and Learning Methods

1. Lecture
2. Discussion (Brain Storming)
3. Self-learning
4. Exercise
5. Web searching

Course Schedule

Number of the Week	Scientific content of the course (Course Topics)	Total Weekly Hours	Expected number of the Learning Hours			
			Theoretical teaching (lectures/discussion groups/ .....)	Training (Practical/Clinical/ .....)	Self-learning (Tasks/ Assignments/ Projects/ ...)	Other (to be determined)
1	The meaning of computer security	**3**	**2**	**2**		
2	Security models	**3**	**2**	**2**		
3	Threats to the security of a computing system	**3**	**2**	**2**		
4	Software and hardware controls	**3**	**2**	**2**		
5	Authentication	**3**	**2**	**2**		
6	Access control	**3**	**2**	**2**		
7	cryptography	**2**				
8	Midterm Exam	**3**	**2**	**2**		
9	stream and Block ciphers	**3**	**2**	**2**		
10	Program security	**3**	**2**	**2**		
11	Operating system security	**3**	**2**	**2**		
12	Database security	**3**	**2**	**2**		
13	Network security	**3**	**2**	**2**		
14	Legal Issues and Ethics	**3**	**2**	**2**		
15	**Overview**	**3**	**2**	**2**		

رومانی صبحی رزق جرجس مطر

5. Methods of students' assessment

No.	Assessment Methods *	Assessment Timing (Week Number)	Marks/ Scores	Percentage of total course Marks
1	Exam 1written (Semester work)	7	25	25%
2	Final Written Exam	16	50	50%
3	Final Practical/Clinical/… Exam	14	10	10%
4	Final Oral Exam	13	5	5%
5	Assignments / Project /Portfolio/ Logbook	10	10	10%

*** The methods mentioned are examples, the organization may add and/or delete**

6. Learning Resources and Supportive Facilities *

Learning resources (books, scientific references, etc.) *	**The main (essential) reference for the course** (must be written in full according to the scientific documentation method)	Security in Computing, 5th Edition - Charles P. Pfleeger
	Other References	Course notes
	Electronic Sources (Links must be added)	
	Learning Platforms (Links must be added)	
	Other (to be mentioned)	
Supportive facilities & equipment or teaching and learning *	**Devices/Instruments**	
	Supplies	
	Electronic Programs	
	Skill Labs/ Simulators	
	Virtual Labs	
	Other (to be mentioned)	

*** The list mentioned is an example, the institution may add and/or delete depending on the nature of the course**

Name and Signature
Course Coordinator
Dr. Moustafa El-Ashry

Name and Signature
Program Coordinator

Table of Contents

***Chapter 1: Introduction*……...……..8**

1.1 What Is Computer Security?……..…….8

1.2 Threats ……..……..…12

1.3 Harm ……..……....31

1.4 Vulnerabilities ……...38

1.5 Controls ……..……....39

***Chapter 2: Authentication, Access Control, and Cryptography*…….....44**

2.1 Authentication ……..……..…45

2.2 Access Control ……..…….....72

2.3 Cryptography.……..……..110

***Chapter 3: Programs and Programming* ……...……....119**

3.1 Unintentional (Nonmalicious) Programming Oversights ……121

3.2 Malicious Code—Malware ……..……..155

***Chapter 4: Operating Systems*…….…....172**

4.1 Security in Operating Systems ……..…..…172

4.2 Security in the Design of Operating Systems ……..….201

***Chapter 5: Databases*……..……217**

5.1 Introduction to Databases ……...….218

5.2 Security Requirements of Databases ……..……..223

5.3 Reliability and Integrity……230

5.4 Database Disclosure……..……237

***Chapter 6: Legal Issues and Ethics*……..……...259**

6.1 Protecting Programs and Data ……..……261

6.2 Computer Crime ……..……...274

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

Chapter 1

Introduction

This book is about the security of computers, their data, and the devices and objects to which they relate. In this book you will learn some of the ways computers can fail—or be made to fail—and how to protect against those failures. We begin that study in the way any good report does: by answering the basic questions of what, who, why, and how.

1.1 What Is Computer Security?

Computer security is the protection of the items you value, called the ***assets*** of a computer or computer system. There are many types of assets, involving hardware, software, data, people, processes, or combinations of these. To determine what to protect, we must first identify what has value and to whom.

A computer device (including hardware, added components, and accessories) is certainly an asset. Because most computer hardware is pretty useless without programs, the software is also an asset. Software includes the operating system, utilities and device handlers; applications such as word processing, media players or email handlers; and even programs that you may have written yourself. Much hardware and software is off-the-shelf, meaning that it is commercially available (not custom-made for your purpose) and that you can easily get a replacement. The thing that makes your computer unique and important to you is its content: photos, tunes, papers, email messages, projects, calendar information, ebooks (with your annotations), contact information, code you created, and the like. Thus, data items on a computer are assets, too. Unlike most hardware and software, data can be hard—if not impossible—to recreate or replace. These assets are all shown in Figure 1-2

These three things—hardware, software, and data—contain or express things like the design for your next new product, the photos from your recent vacation, the chapters of your new book, or the genome sequence resulting from your recent research. All of these things represent intellectual endeavor or property, and they have value that differs from one person or organization to another. It is that value that makes them assets worthy of protection, and they are the elements we want to protect. Other assets—such as access to data, quality of service,

2025/2026 2025/2026 2025/2026

processes, human users, and network connectivity—deserve protection, too; they are affected or enabled by the hardware, software, and data. So in most cases, protecting hardware, software, and data covers these other assets as well



Hardware:	Software:	Data:
• Computer • Devices (disk drives, memory, printer) • Network gear	• Operating system • Utilities (antivirus) • Commercial applications (word processing, photo editing) • Individual applications	• Documents • Photos • Music, videos • Email • Class projects

FIGURE 1-2 Computer Objects of Value

Computer systems—hardware, software, and data—have value and deserve security protection.

In this book, unless we specifically distinguish between hardware, software, and data, we refer to all these assets as the computer system, or sometimes as the computer. And because processors are embedded in so many devices, we also need to think about such variations as mobile phones, implanted pacemakers, heating controllers, and automobiles. Even if the primary purpose of the device is not computing, the device's embedded computer can be involved in security incidents and represents an asset worthy of protection.

Values of Assets

After identifying the assets to protect, we next determine their value. We make value based decisions frequently, even when we are not aware of them. For example, when you go for a swim you can leave a bottle of water and a towel on the beach, but not your wallet or cell phone. The difference relates to the value of the assets.

The value of an asset depends on the asset owner's or user's perspective, and it may be independent of monetary cost, as shown in Figure 1-3. Your photo of your sister, worth only a few cents in terms of paper and ink, may have high value to you and no value to your roommate. Other items' value depends on replacement cost; some computer data are difficult or impossible to replace. For example, that photo of you and your friends at a party may have cost you nothing, but it is invaluable because there is no other copy. On the other hand, the DVD of your favorite film may have cost a significant portion of your take-home pay, but you can buy another one if the DVD is stolen or corrupted. Similarly, timing has bearing on asset value. For example, the value of the plans for a company's new product line is very high, especially to competitors. But once the new product is released, the plans' value drops dramatically.

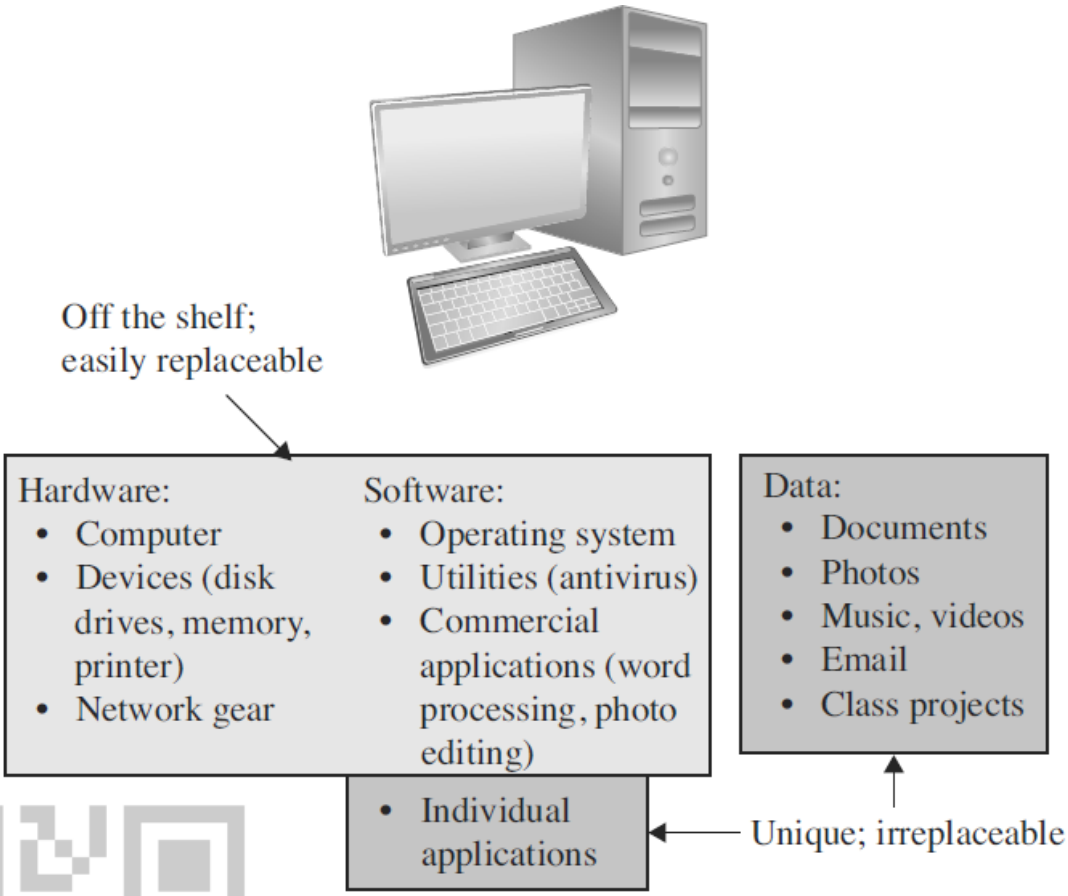


FIGURE 1-3 Values of Assets

Assets' values are personal, time dependent, and often imprecise.

رومانی صبحی رزق جرجس مطر

The Vulnerability–Threat–Control Paradigm

The goal of computer security is protecting valuable assets. To study different ways of protection, we use a framework that describes how assets may be harmed and how to counter or mitigate that harm.

A Vulnerability is a weakness in the system, for example, in procedures, design, or implementation that might be exploited to cause loss or harm. For instance, a particular system may be vulnerable to unauthorized data manipulation because the system does not verify a user's identity before allowing data access.

A vulnerability is a weakness that could be exploited to cause harm.

A threat to a computing system is a set of circumstances that has the potential to cause loss or harm. To see the difference between a threat and a vulnerability, consider the illustration in Figure 1-4. Here, a wall is holding water back. The water to the left of the wall is a threat to the man on the right of the wall: The water could rise, overflowing onto the man, or it could stay beneath the height of the wall, causing the wall to collapse. So the threat of harm is the potential for the man to get wet, get hurt, or be drowned. For now, the wall is intact, so the threat to the man is unrealized.

2025/2026 2025/2026 2025/2026

A threat is a set of circumstances that could cause harm.

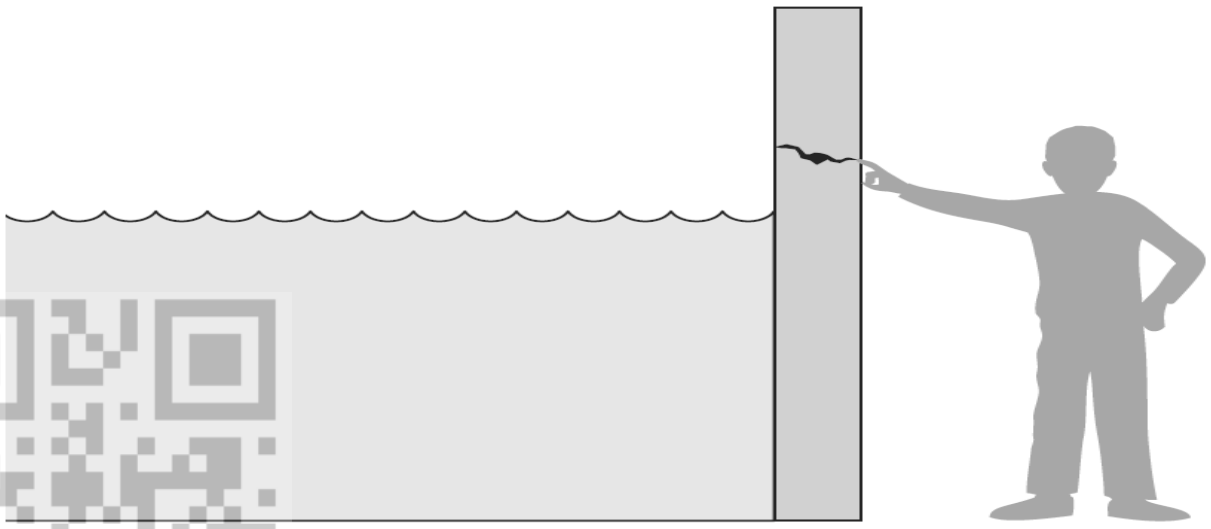


FIGURE 1-4 Threat and Vulnerability

However, we can see a small crack in the wall—a vulnerability that threatens the man's security. If the water rises to or beyond the level of the crack, it will exploit the vulnerability and harm the man.

There are many threats to a computer system, including human-initiated and computer initiated ones. We have all experienced the results of inadvertent human errors, hardware design flaws, and software failures. But natural disasters are threats, too; they can bring a system down when the computer room is flooded or the data center collapses from an earthquake, for example.

A human who exploits a vulnerability perpetrates an attack on the system. An attack can also be launched by another system, as when one system sends an overwhelming flood of messages to another, virtually shutting down the second system's ability to function. Unfortunately, we have seen this type of attack frequently, as denial-of-service attacks deluge servers with more messages than they can handle. (We take a closer look at denial of service later.)

How do we address these problems? We use a control or countermeasure as protection. That is, a control is an action, device, procedure, or technique that removes or reduces a vulnerability. In Figure 1-4, the man is placing his finger in the hole, controlling the threat of water leaks until he finds a more permanent solution to the problem. In general, we can describe the relationship between threats, controls, and vulnerabilities in this way:

Controls prevent threats from exercising vulnerabilities.

A threat is blocked by control of a vulnerability.

Before we can protect assets, we need to know the kinds of harm we have to protect them against, so now we explore threats to valuable assets.

1.2 Threats

We can consider potential harm to assets in two ways: First, we can look at what bad things can happen to assets, and second, we can look at who or what can cause or allow those bad things to happen. These two perspectives enable us to determine how to protect assets.

Think for a moment about what makes your computer valuable to you. First, you use it as a tool for sending and receiving email, searching the web, writing papers, and performing many other tasks, and you expect it to be available for use when you want it. Without your computer these

tasks would be harder, if not impossible. Second, you rely heavily on your computer's integrity. When you write a paper and save it, you trust that the paper will reload exactly as you saved it. Similarly, you expect that the photo a friend passes you on a flash drive will appear the same when you load it into your computer as when you saw it on your friend's computer. Finally, you expect the "personal" aspect of a personal computer to stay personal, meaning you want it to protect your confidentiality. For example, you want your email messages to be just between you and your listed recipients; you don't want them broadcast to other people. And when you write an essay, you expect that no one can copy it without your permission.

These three aspects, confidentiality, integrity, and availability, make your computer valuable to you. But viewed from another perspective, they are three possible ways to make it less valuable, that is, to cause you harm. If someone steals your computer, scrambles data on your disk, or looks at your private data files, the value of your computer has been diminished or your computer use has been harmed. These characteristics are both basic security properties and the objects of security threats.

We can define these three properties as follows.

2025/2026 2025/2026 2025/2026

- ***Availability***: the ability of a system to ensure that an asset can be used by any authorized parties
- ***Integrity***: the ability of a system to ensure that an asset is modified only by authorized parties
- ***Confidentiality***: the ability of a system to ensure that an asset is viewed only by authorized parties

These three properties, hallmarks of solid security, appear in the literature as early as James P. Anderson's essay on computer security [AND73] and reappear frequently in more recent computer security papers and discussions. Taken together (and rearranged), the properties are called the C-I-A triad or the security triad. ISO 7498-2 [ISO89] adds to them two more properties that are desirable, particularly in communication networks:

:

- ***Authentication***: the ability of a system to confirm the identity of a sender
- ***Nonrepudiation or accountability***: the ability of a system to confirm that a sender cannot convincingly deny having sent something

رومانی صبحی رزق جرجس مطر

The U.S. Department of Defense [DOD85] adds auditability: the ability of a system to trace all actions related to a given asset. The C-I-A triad forms a foundation for thinking about security. Authenticity and nonrepudiation extend security notions to network communications, and auditability is important in establishing individual accountability for computer activity. In this book we generally use the C-I-A triad as our security taxonomy so that we can frame threats, vulnerabilities, and controls in terms of the C-I-A properties affected. We highlight one of these other properties when it is relevant to a particular threat we are describing. For now, we focus on just the three elements of the triad.

C-I-A triad: confidentiality, integrity, availability

What can happen to harm the confidentiality, integrity, or availability of computer assets? If a thief steals your computer, you no longer have access, so you have lost availability; furthermore, if the thief looks at the pictures or documents you have stored, your confidentiality is compromised. And if the thief changes the content of your music files but then gives them back with your computer, the integrity of your data has been harmed. You can envision many scenarios based around these three properties.

2025/2026 2025/2026 2025/2026

The C-I-A triad can be viewed from a different perspective: the nature of the harm caused to assets. Harm can also be characterized by four acts: ***interception, interruption, modification, and fabrication***. These four acts are depicted in Figure 1-5. From this point of view, confidentiality can suffer if someone intercepts data, availability is lost if someone or something interrupts a flow of data or access to a computer, and integrity can fail if someone or something modifies data or fabricates false data. Thinking of these four kinds of acts can help you determine what threats might exist against the computers you are trying to protect.

To analyze harm, we next refine the C-I-A triad, looking more closely at each of its elements.

Confidentiality

Some things obviously need confidentiality protection. For example, students' grades, financial transactions, medical records, and tax returns are sensitive. A proud student may run out of a classroom screaming "I got an A!" but the student should be the one to choose whether to reveal that grade to others. Other things, such as diplomatic and military secrets, companies' marketing

رومانی صبحی رزق جرجس مطر

and product development plans, and educators' tests, also must be carefully controlled. Sometimes, however, it is not so obvious that something is sensitive. For example, a military food order may seem like innocuous information, but a sudden increase in the order could be a sign of incipient engagement in conflict. Purchases of food, hourly changes in location, and access to books are not things you would ordinarily consider confidential, but they can reveal something that someone wants to be kept confidential.

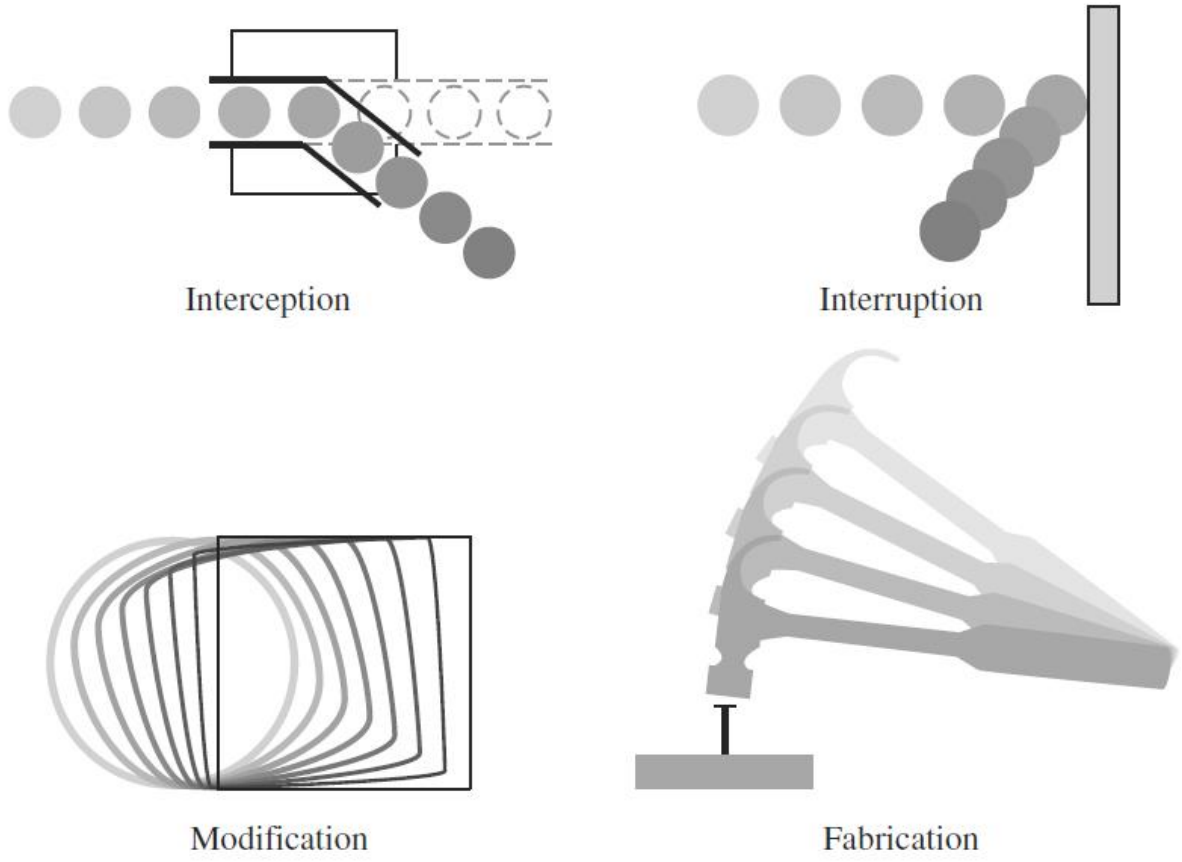


FIGURE 1-5 Four Acts to Cause Security Harm

The definition of confidentiality is straightforward: Only authorized people or systems can access protected data. However, as we see in later chapters, ensuring confidentiality can be difficult. For example, who determines which people or systems are authorized to access the current system? By "accessing" data, do we mean that an authorized party can access a single bit? the whole collection? pieces of data out of context? Can someone who is authorized disclose data to other parties? Sometimes there is even a question of who owns the data: If you visit a

رومانی صبحی رزق جرجس مطر

web page, do you own the fact that you clicked on a link, or does the web page owner, the Internet provider, someone else, or all of you?

In spite of these complicating examples, confidentiality is the security property we understand best because its meaning is narrower than that of the other two. We also understand confidentiality well because we can relate computing examples to those of preserving confidentiality in the real world.

Confidentiality relates most obviously to data, although we can think of the confidentiality of a piece of hardware (a novel invention) or a person (the whereabouts of a wanted criminal). Here are some properties that could mean a failure of data confidentiality:

- An unauthorized person accesses a data item.
- An unauthorized process or program accesses a data item.
- A person authorized to access certain data accesses other data not authorized (which is a specialized version of "an unauthorized person accesses a data item").
- An unauthorized person accesses an approximate data value (for example, not knowing someone's exact salary but knowing that the salary falls in a particular range or exceeds a particular amount).

2025/2026 2025/2026 2025/2026

- An unauthorized person learns the existence of a piece of data (for example, knowing that a company is developing a certain new product or that talks are underway about the merger of two companies).

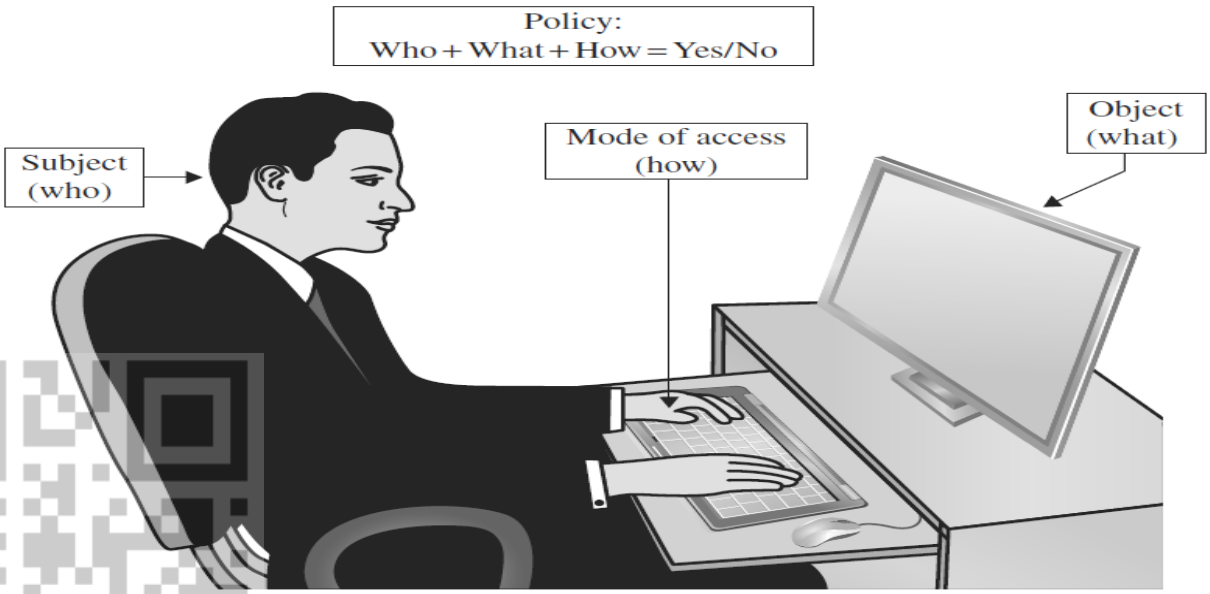


FIGURE 1-6 Access Control

رومانی صبحی رزق جرجس مطر

Notice the general pattern of these statements: A person, process, or program is (or is not) authorized to access a data item in a particular way. We call the person, process, or program a ***subject***, the data item an ***object***, the kind of access (such as read, write, or execute) an ***access mode***, and the authorization a ***policy,*** as shown in Figure 1-6. These four terms reappear throughout this book because they are fundamental aspects of computer security.

One word that captures most aspects of confidentiality is view, although you should not take that term literally. A failure of confidentiality does not necessarily mean that someone sees an object and, in fact, it is virtually impossible to look at bits in any meaningful way (although you may look at their representation as characters or pictures). The word view does connote another aspect of confidentiality in computer security, through the association with viewing a movie or a painting in a museum: look but do not touch. In computer security, confidentiality usually means obtaining but not modifying.

Modification is the subject of integrity, which we consider in the next section.

Integrity

2025/2026 2025/2026 2025/2026

Examples of integrity failures are easy to find. A number of years ago a malicious macro in a Word document inserted the word “not” after some random instances of the word “is;” you can imagine the havoc that ensued. Because the document was generally syntactically correct, people did not immediately detect the change. In another case, a model of the Pentium computer chip produced an incorrect result in certain circumstances of floating-point arithmetic. Although the circumstances of failure were rare, Intel decided to manufacture and replace the chips. Many of us receive mail that is misaddressed because someone typed something wrong when transcribing from a written list. A worse situation occurs when that inaccuracy is propagated to other mailing lists such that we can never seem to correct the root of the problem. Other times we find that a spreadsheet seems to be wrong, only to find that someone typed “space 123” in a cell, changing it from a numeric value to text, so the spreadsheet program misused that cell in computation.

Suppose someone converted numeric data to roman numerals: One could argue that IV is the same as 4, but IV would not be useful in most applications, nor would it be obviously meaningful to someone expecting 4 as an answer. These cases show some of the breadth of examples of integrity failures.

رومانی صبحی رزق جرجس مطر

Integrity is harder to pin down than confidentiality. As Stephen Welke and Terry Mayfield [WEL90, MAY91, NCS91a] point out, integrity means different things in different contexts. When we survey the way some people use the term, we find several different meanings. For example, if we say that we have preserved the integrity of an item, we may mean that the item is

- precise
- accurate
- unmodified
- modified only in acceptable ways
- modified only by authorized people
- modified only by authorized processes
- consistent
- internally consistent
- meaningful and usable

Integrity can also mean two or more of these properties. Welke and Mayfield recognize three particular aspects of integrity—authorized actions, separation and protection of resources, and error detection and correction. Integrity can be enforced in much the same way as can confidentiality: by rigorous control of who or what can access which resources in what ways.

2025/2026 2025/2026 2025/2026

Availability

A computer user's worst nightmare: You turn on the switch and the computer does nothing. Your data and programs are presumably still there, but you cannot get at them. Fortunately, few of us experience that failure. Many of us do experience overload, however: access gets slower and slower; the computer responds but not in a way we consider normal or acceptable.

Availability applies both to data and to services (that is, to information and to information processing), and it is similarly complex. As with the notion of confidentiality, different people expect availability to mean different things. For example, an object or service is thought to be available if the following are true:

- It is present in a usable form.
- It has enough capacity to meet the service's needs.
- It is making clear progress, and, if in wait mode, it has a bounded waiting time.
- The service is completed in an acceptable period of time.

رومانی صبحی رزق جرجس مطر

We can construct an overall description of availability by combining these goals. Following are some criteria to define availability.

- There is a timely response to our request.
- Resources are allocated fairly so that some requesters are not favored over others.
- Concurrency is controlled; that is, simultaneous access, deadlock management, and exclusive access are supported as required.
- The service or system involved follows a philosophy of fault tolerance, whereby hardware or software faults lead to graceful cessation of service or to work-arounds rather than to crashes and abrupt loss of information. (Cessation does mean end; whether it is graceful or not, ultimately the system is unavailable. However, with fair warning of the system's stopping, the user may be able to move to another system and continue work.)
- The service or system can be used easily and in the way it was intended to be used. (This is a characteristic of usability, but an unusable system may also cause an availability failure.)

As you can see, expectations of availability are far-reaching. In Figure 1-7 we depict some of the properties with which availability overlaps. Indeed, the security community is just beginning to understand what availability implies and how to ensure it.

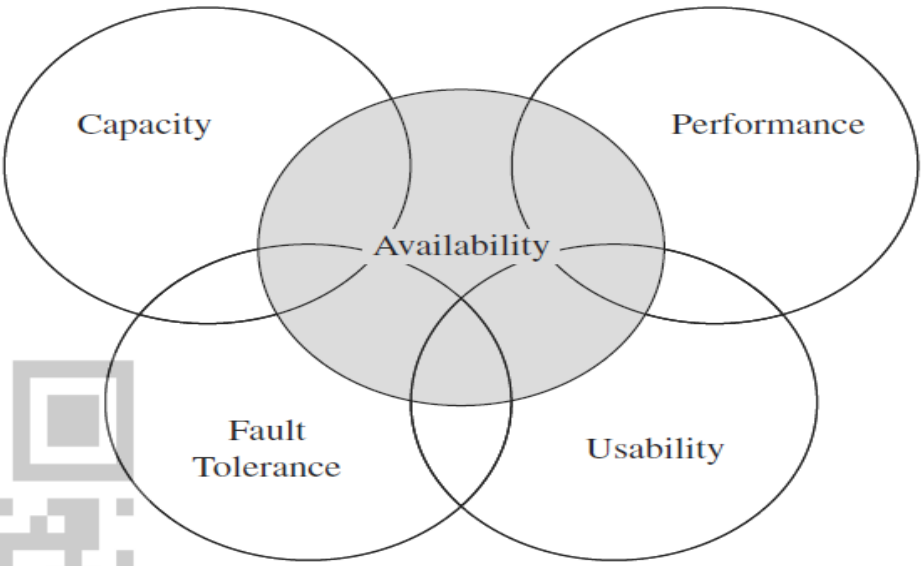


FIGURE 1-7 Availability and Related Aspects

رومانی صبحی رزق جرجس مطر

A person or system can do three basic things with a data item: view it, modify it, or use it. Thus, viewing (confidentiality), modifying (integrity), and using (availability) are the basic modes of access that computer security seeks to preserve.

Computer security seeks to prevent unauthorized viewing (confidentiality) or modification (integrity) of data while preserving access (availability).

A paradigm of computer security is access control: To implement a policy, computer security controls all accesses by all subjects to all protected objects in all modes of access. A small, centralized control of access is fundamental to preserving confidentiality and integrity, but it is not clear that a single access control point can enforce availability. Indeed, experts on dependability will note that single points of control can become single points of failure, making it easy for an attacker to destroy availability by disabling the single control point. Much of computer security's past success has focused on confidentiality and integrity; there are models of confidentiality and integrity, for example, see David Bell and Leonard La Padula [BEL73, BEL76] and Kenneth Biba [BIB77]. Availability is security's next great challenge.

2025/2026 2025/2026 2025/2026

We have just described the C-I-A triad and the three fundamental security properties it represents. Our description of these properties was in the context of things that need protection. To motivate your understanding we gave some examples of harm and threats to cause harm. Our next step is to think about the nature of threats themselves.

Types of Threats

For some ideas of harm, look at Figure 1-8, taken from Willis Ware's report [WAR70]. Although it was written when computers were so big, so expensive, and so difficult to operate that only large organizations like universities, major corporations, or government departments would have one, Ware's discussion is still instructive today. Ware was concerned primarily with the protection of classified data that is, preserving confidentiality. In the figure, he depicts humans such as programmers and maintenance staff gaining access to data, as well as radiation by which data can escape as signals. From the figure you can see some of the many kinds of threats to a computer system.

رومانی صبحی رزق جرجس مطر

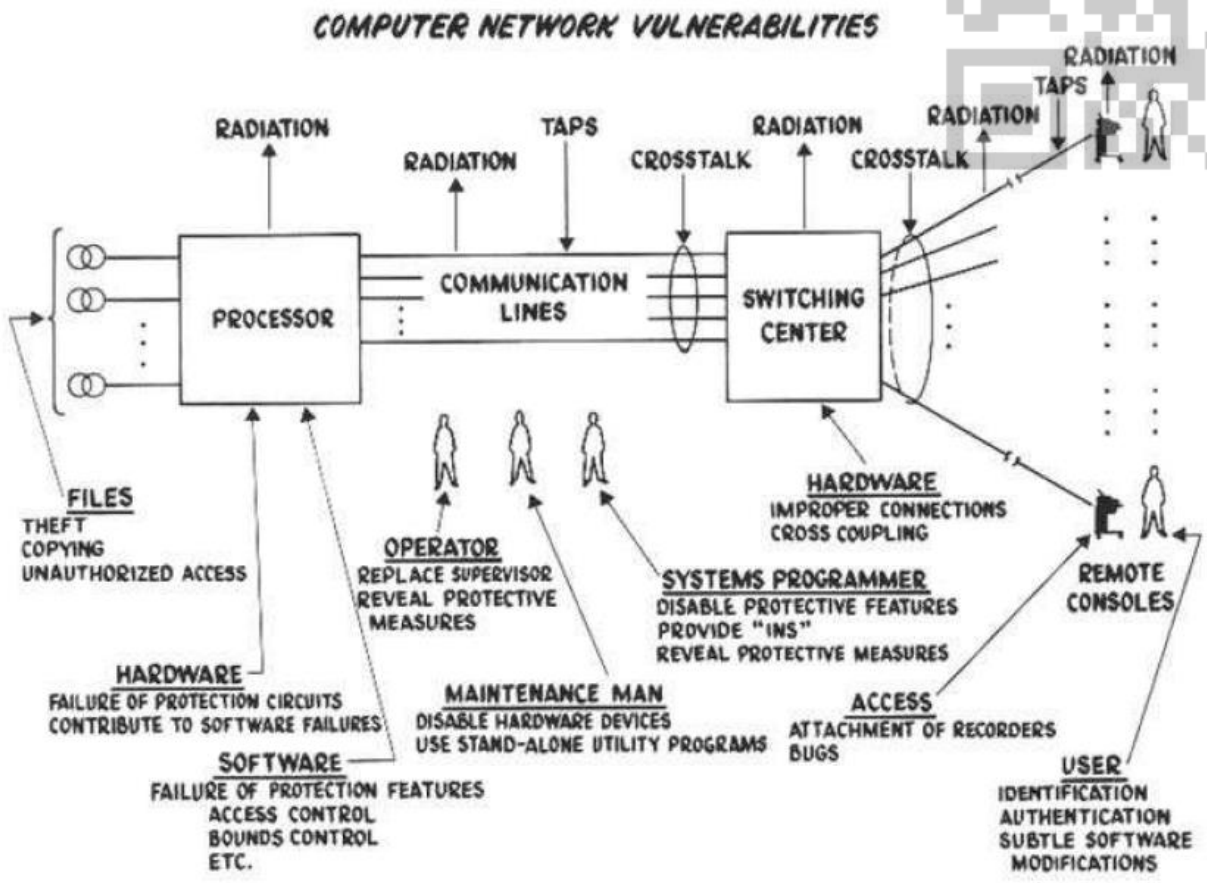


FIGURE 1-8 Computer [Network] Vulnerabilities (from [WAR70])

One way to analyze harm is to consider the cause or source. We call a potential cause of harm a threat. Harm can be caused by either nonhuman events or humans. Examples of nonhuman threats include natural disasters like fires or floods; loss of electrical power; failure of a component such as a communications cable, processor chip, or disk drive; or attack by a wild boar.

Threats are caused both by human and other sources.

Human threats can be either benign (nonmalicious) or malicious. Nonmalicious kinds of harm include someone's accidentally spilling a soft drink on a laptop, unintentionally deleting text, inadvertently sending an email message to the wrong person, and carelessly typing "12" instead of "21" when entering a phone number or clicking "yes" instead of "no" to overwrite a file. These inadvertent, human errors happen to most people; we just hope that the seriousness of harm is not too great, or if it is, that we will not repeat the mistake.

Threats can be malicious or not.

Most computer security activity relates to malicious, human-caused harm: A malicious person actually wants to cause harm, and so we often use the term attack for a malicious computer security event. Malicious attacks can be random or directed. In a random attack the attacker wants to harm any computer or user; such an attack is analogous to accosting the next pedestrian who walks down the street. An example of a random attack is malicious code posted on a website that could be visited by anybody.

In a directed attack, the attacker intends harm to specific computers, perhaps at one organization (think of attacks against a political organization) or belonging to a specific individual (think of trying to drain a specific person's bank account, for example, by impersonation). Another class of directed attack is against a particular product, such as any computer running a particular browser. (We do not want to split hairs about whether such an attack is directed—at that one software product—or random, against any user of that product; the point is not semantic perfection but protecting against the attacks.) The range of possible directed attacks is practically unlimited. Different kinds of threats are shown in Figure 1-9.

2025/2026 2025/2026 2025/2026

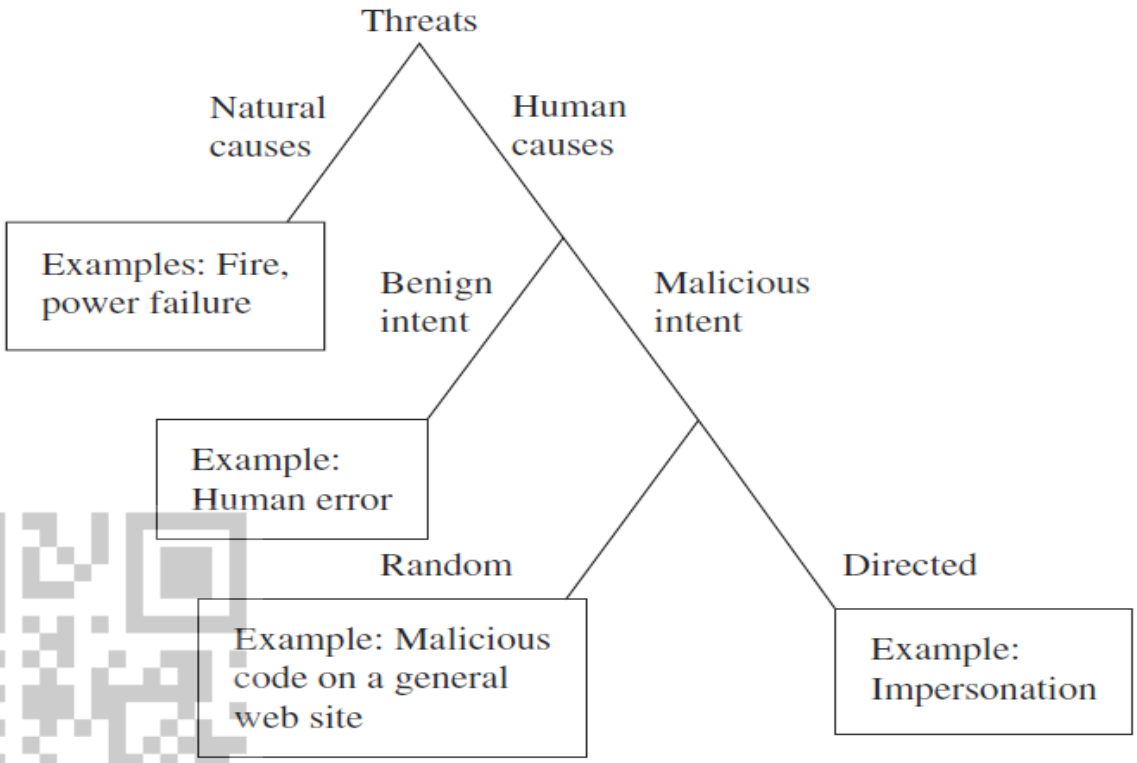


FIGURE 1-9 Kinds of Threats

رومانی صبحی رزق جرجس مطر

Threats can be targeted or random

Although the distinctions shown in Figure 1-9 seem clear-cut, sometimes the nature of an attack is not obvious until the attack is well underway, or perhaps even ended. A normal hardware failure can seem like a directed, malicious attack to deny access, and hackers often try to conceal their activity to look like ordinary, authorized users. As computer security experts we need to anticipate what bad things might happen, instead of waiting for the attack to happen or debating whether the attack is intentional or accidental.

Neither this book nor any checklist or method can show you all the kinds of harm that can happen to computer assets. There are too many ways to interfere with your use of these assets. Two retrospective lists of known vulnerabilities are of interest, however. The Common Vulnerabilities and Exposures (CVE) list (see http://cve.mitre.org/) is a dictionary of publicly known security vulnerabilities and exposures. CVE's common identifiers enable data exchange between security products and provide a baseline index point for evaluating coverage of security tools and services. To measure the extent of harm, the Common Vulnerability Scoring System (CVSS) (see http://nvd.nist.gov/cvss.cfm) provides a standard measurement system that allows accurate and consistent scoring of vulnerability impact.

2025/2026 2025/2026 2025/2026

Advanced Persistent Threat

Security experts are becoming increasingly concerned about a type of threat called advanced persistent threat. A lone attacker might create a random attack that snares a few, or a few million, individuals, but the resulting impact is limited to what that single attacker can organize and manage. A collection of attackers—think, for example, of the cyber equivalent of a street gang or an organized crime squad—might work together to purloin credit card numbers or similar financial assets to fund other illegal activity. Such attackers tend to be opportunistic, picking unlucky victims' pockets and moving on to other activities.

Advanced persistent threat attacks come from organized, well financed, patient assailants. Often affiliated with governments or quasi-governmental groups, these attackers engage in long term campaigns. They carefully select their targets, crafting attacks that appeal to specifically those targets; email messages called spear phishing (described in Chapter 4) are intended to seduce

their recipients. Typically the attacks are silent, avoiding any obvious impact that would alert a victim, thereby allowing the attacker to exploit the victim's access rights over a long time.

The motive of such attacks is sometimes unclear. One popular objective is economic espionage. A series of attacks, apparently organized and supported by the Chinese government, was used in 2012 and 2013 to obtain product designs from aerospace companies in the United States. There is evidence the stub of the attack code was loaded into victim machines long in advance of the attack; then, the attackers installed the more complex code and extracted the desired data. In May 2014 the Justice Department indicted five Chinese hackers in absentia for these attacks.

In the summer of 2014 a series of attacks against J.P. Morgan Chase bank and up to a dozen similar financial institutions allowed the assailants access to 76 million names, phone numbers, and email addresses. The attackers—and even their country of origin— remain unknown, as does the motive. Perhaps the attackers wanted more sensitive financial data, such as account numbers or passwords, but were only able to get the less valuable contact information. It is also not known if this attack was related to an attack a year earlier that disrupted service to that bank and several others.

To imagine the full landscape of possible attacks, you may find it useful to consider the kinds of people who attack computer systems. Although potentially anyone is an attacker, certain classes of people stand out because of their backgrounds or objectives. Thus, in the following sections we look at profiles of some classes of attackers.

Types of Attackers

Who are attackers? As we have seen, their motivations range from chance to a specific target. Putting aside attacks from natural and benign causes, we can explore who the attackers are and what motivates them.

Most studies of attackers actually analyze computer criminals, that is, people who have actually been convicted of a crime, primarily because that group is easy to identify and study. The ones who got away or who carried off an attack without being detected may have characteristics different from those of the criminals who have been caught. Worse, by studying only the criminals we have caught, we may not learn how to catch attackers who know how to abuse the system without being apprehended.

رومانی صبحی رزق جرجس مطر

What does a cyber criminal look like? In television and films the villains wore shabby clothes, looked mean and sinister, and lived in gangs somewhere out of town. By contrast, the sheriff dressed well, stood proud and tall, was known and respected by everyone in town, and struck fear in the hearts of most criminals.

To be sure, some computer criminals are mean and sinister types. But many more wear business suits, have university degrees, and appear to be pillars of their communities. Some are high school or university students. Others are middle-aged business executives. Some are mentally deranged, overtly hostile, or extremely committed to a cause, and they attack computers as a symbol. Others are ordinary people tempted by personal profit, revenge, challenge, advancement, or job security—like perpetrators of any crime, using a computer or not. Researchers have tried to find the psychological traits that distinguish attackers, as described in Sidebar 1-1. These studies are far from conclusive, however, and the traits they identify may show correlation but not necessarily causality. To appreciate this point, suppose a study found that a disproportionate number of people convicted of computer crime were left-handed. Does that result imply that all left-handed people are computer criminals or that only left-handed people are? Certainly not. No single profile captures the characteristics of a "typical" computer attacker, and the characteristics of some notorious attackers also match many people who are not attackers. As shown in Figure 1-10, attackers look just like anybody in a crowd.

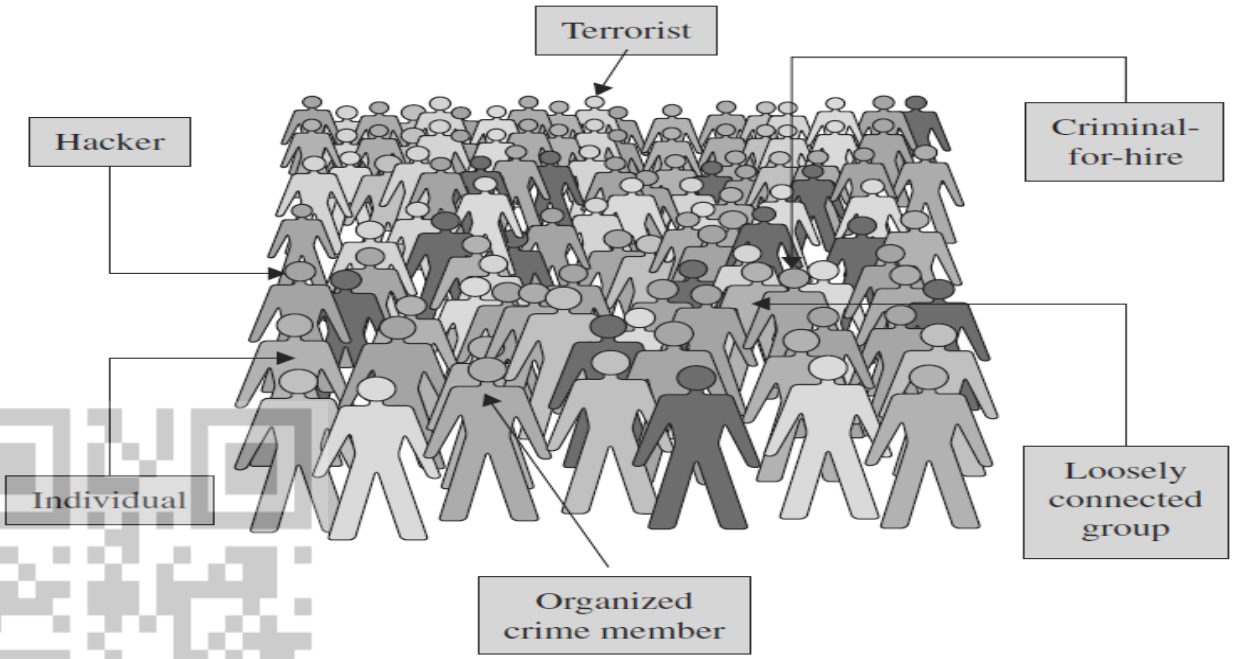


FIGURE 1-10 Attackers

رومانی صبحی رزق جرجس مطر

No one pattern matches all attackers.

Sidebar 1-1 An Attacker's Psychological Profile?

Temple Grandin, a professor of animal science at Colorado State University and a sufferer from a mental disorder called Asperger syndrome (AS), thinks that Kevin Mitnick and several other widely described hackers show classic symptoms of Asperger syndrome. Although quick to point out that no research has established a link between AS and hacking, Grandin notes similar behavior traits among Mitnick, herself, and other AS sufferers. An article in USA Today (29 March 2001) lists the following AS traits:

- poor social skills, often associated with being loners during childhood; the classic "computer nerd"
- fidgeting, restlessness, inability to make eye contact, lack of response to cues in social interaction, such as facial expressions or body language
- exceptional ability to remember long strings of number ,
- ability to focus on a technical problem intensely and for a long time, although easily distracted on other problems and unable to manage several tasks at once

2025/2026 2025/2026 2025/2026

- deep honesty and respect for laws

Donn Parker [PAR98] has studied hacking and computer crime for many years. He states "hackers are characterized by an immature, excessively idealistic attitude … They delight in presenting themselves to the media as idealistic do-gooders, champions of the underdog."

Consider the following excerpt from an interview [SHA00] with "Mixter," the German programmer who admitted he was the author of a widespread piece of attack software called Tribal Flood Network (TFN) and its sequel TFN2K:

Q: Why did you write the software?

A: I first heard about Trin00 [another piece of attack software] in July '99 and I considered it as interesting from a technical perspective, but also potentially powerful in a negative way. I knew some facts of how Trin00 worked, and since I didn't manage to get Trin00 sources or binaries at that time, I wrote my own server-client network that was capable of performing denial of service.

Q: Were you involved … in any of the recent high-profile attacks?

A: No. The fact that I authored these tools does in no way mean that I condone their active use. I must admit I was quite shocked to hear about the latest attacks. It seems that the attackers are

pretty clueless people who misuse powerful resources and tools for generally harmful and senseless activities just “because they can.”

Notice that from some information about denial-of-service attacks, he wrote his own server-client network and then a sophisticated attack. But he was “quite shocked” to hear they were used for harm.

More research is needed before we can define the profile of a hacker. And even more work will be needed to extend that profile to the profile of a (malicious) attacker. Not all hackers become attackers; some hackers become extremely dedicated and conscientious system administrators, developers, or security experts. But some psychologists see in AS the rudiments of a hacker’s profile.

Individuals

Originally, computer attackers were individuals, acting with motives of fun, challenge, or revenge. Early attackers acted alone. Two of the most well known among them are Robert Morris Jr., the Cornell University graduate student who brought down the Internet in 1988 [SPA89], and Kevin Mitnick, the man who broke into and stole data from dozens of computers, including the San Diego Supercomputer Center [MAR95].

Organized, Worldwide Groups

More recent attacks have involved groups of people. An attack against the government of the country of Estonia (described in more detail in Chapter 13) is believed to have been an uncoordinated outburst from a loose federation of attackers from around the world. Kevin Poulsen [POU05] quotes Tim Rosenberg, a research professor at George Washington University, warning of “multinational groups of hackers backed by organized crime” and showing the sophistication of prohibition-era mobsters. He also reports that Christopher Painter, deputy director of the U.S. Department of Justice’s computer crime section, argues that cyber criminals and serious fraud artists are increasingly working in concert or are one and the same. According to Painter, loosely connected groups of criminals all over the world work together to break into systems and steal and sell information, such as credit card numbers. For instance, in October 2004, U.S. and Canadian authorities arrested 28 people from 6 countries involved in an international, organized cybercrime ring to buy and sell credit card information and identities.

Whereas early motives for computer attackers such as Morris and Mitnick were personal, such as prestige or accomplishment, recent attacks have been heavily influenced by financial gain. Security firm McAfee reports "Criminals have realized the huge financial gains to be made from the Internet with little risk. They bring the skills, knowledge, and connections needed for large scale, high-value criminal enterprise that, when combined with computer skills, expand the scope and risk of cybercrime." [MCA05]

رومانی صبحی رزق جرجس مطر

Organized Crime

Attackers' goals include fraud, extortion, money laundering, and drug trafficking, areas in which organized crime has a well-established presence. Evidence is growing that organized crime groups are engaging in computer crime. In fact, traditional criminals are recruiting hackers to join the lucrative world of cybercrime. For example, Albert Gonzales was sentenced in March 2010 to 20 years in prison for working with a crime ring to steal 40 million credit card numbers from retailer TJMaxx and others, costing over $200 million (Reuters, 26 March 2010).

Organized crime may use computer crime (such as stealing credit card numbers or bank account details) to finance other aspects of crime. Recent attacks suggest that professional criminals have discovered just how lucrative computer crime can be. Mike Danseglio, a security project manager with Microsoft, said, "In 2006, the attackers want to pay the rent. They don't want to write a worm that destroys your hardware. They want to assimilate your computers and use them to make money." [NAR06a] Mikko Hyppönen, Chief Research Officer with Finnish security company f-Secure, agrees that today's attacks often come from Russia, Asia, and Brazil; the motive is now profit, not fame [BRA06]. Ken Dunham, Director of the Rapid Response Team for VeriSign says he is "convinced that groups of well-organized mobsters have taken control of a global billion-dollar crime network powered by skillful hackers." [NAR06b]

Organized crime groups are discovering that computer crime can be lucrative.

McAfee also describes the case of a hacker-for-hire: a businessman who hired a 16- year-old New Jersey hacker to attack the websites of his competitors. The hacker barraged the site for a five-month period and damaged not only the target companies but also their Internet service providers (ISPs) and other unrelated companies that used the same ISPs. By FBI estimates, the

attacks cost all the companies over $2 million; the FBI arrested both hacker and businessman in March 2005 [MCA05].

Brian Snow [SNO05] observes that hackers want a score or some kind of evidence to give them bragging rights. Organized crime wants a resource; such criminals want to stay under the radar to be able to extract profit from the system over time. These different objectives lead to different approaches to computer crime: The novice hacker can use a crude attack, whereas the professional attacker wants a neat, robust, and undetectable method that can deliver rewards for a long time.

Terrorists

The link between computer security and terrorism is quite evident. We see terrorists using computers in four ways:

- Computer as target of attack: Denial-of-service attacks and website defacements are popular activities for any political organization because they attract attention to the cause and bring undesired negative attention to the object of the attack. An example is the massive denial-of-service attack launched against the country of Estonia, detailed later.
- Computer as method of attack: Launching offensive attacks requires the use of computers. Stuxnet, an example of malicious computer code called a worm, is known to attack automated control systems, specifically a model of control system manufactured by Siemens. Experts say the code is designed to disable machinery used in the control of nuclear reactors in Iran [MAR10]. The persons behind the attack are unknown, but the infection is believed to have spread through USB flash drives brought in by engineers maintaining the computer controllers. (We examine the Stuxnet worm in more detail in Chapters 6 and 13.)
- Computer as enabler of attack: Websites, web logs, and email lists are effective, fast, and inexpensive ways to allow many people to coordinate. According to the Council on Foreign Relations, the terrorists responsible for the November 2008 attack that killed over 200 people in Mumbai used GPS systems to guide their boats, Blackberries for their communication, and Google Earth to plot their routes.
- Computer as enhancer of attack: The Internet has proved to be an invaluable means for terrorists to spread propaganda and recruit agents. In October 2009 the FBI arrested

2025/2026 2025/2026 2025/2026
رومانی صبحی رزق جرجس مطر

Colleen LaRose, also known as JihadJane, after she had spent months using email, YouTube, MySpace, and electronic message boards to recruit radicals in Europe and South Asia to "wage violent jihad," according to a federal indictment.

We cannot accurately measure the degree to which terrorists use computers, because terrorists keep secret the nature of their activities and because our definitions and measurement tools are rather weak. Still, incidents like the one described in Sidebar 1-2 provide evidence that all four of these activities are increasing

Sidebar 1-2 The Terrorists, Inc., IT Department

In 2001, a reporter for the Wall Street Journal bought a used computer in Afghanistan. Much to his surprise, he found that the hard drive contained what appeared to be files from a senior al Qaeda operative. The reporter, Alan Cullison [CUL04], reports that he turned the computer over to the FBI. In his story published in 2004 in The Atlantic, he carefully avoids revealing anything he thinks might be sensitive.

The disk contained over 1,000 documents, many of them encrypted with relatively weak encryption. Cullison found draft mission plans and white papers setting forth ideological and philosophical arguments for the attacks of 11 September 2001. Also found were copies of news stories on terrorist activities. Some of the found documents indicated that al Qaeda was not originally interested in chemical, biological, or nuclear weapons, but became interested after reading public news articles accusing al Qaeda of having those capabilities.

Perhaps most unexpected were email messages of the kind one would find in a typical office: recommendations for promotions, justifications for petty cash expenditures, and arguments concerning budgets.

The computer appears to have been used by al Qaeda from 1999 to 2001. Cullison notes that Afghanistan in late 2001 was a scene of chaos, and it is likely the laptop's owner fled quickly, leaving the computer behind, where it fell into the hands of a secondhand goods merchant who did not know its contents.

But this computer's contents illustrate an important aspect of computer security and confidentiality: We can never predict the time at which a security disaster will strike, and thus we must always be prepared to act immediately if it suddenly happens.

2025/2026 2025/2026 2025/2026

If someone on television sneezes, you do not worry about the possibility of catching a cold. But if someone standing next to you sneezes, you may become concerned. In the next section we examine the harm that can come from the presence of a computer security threat on your own computer systems.

1.3 Harm

The negative consequence of an actualized threat is harm; we protect ourselves against threats in order to reduce or eliminate harm. We have already described many examples of computer harm: a stolen computer, modified or lost file, revealed private letter, or denied access to data. These events cause harm that we want to avoid.

In our earlier discussion of assets, we noted that value depends on owner or outsider perception and need. Some aspects of value are immeasurable, such as the value of the paper you need to submit to your professor tomorrow; if you lose the paper (that is, if its availability is lost), no amount of money will compensate you for it. Items on which you place little or no value might be more valuable to someone else; for example, the group photograph taken at last night's party can reveal that your friend was not where he told his wife he would be. Even though it may be difficult to assign a specific number as the value of an asset, you can usually assign a value on a generic scale, such as moderate or minuscule or incredibly high, depending on the degree of harm that loss or damage to the object would cause. Or you can assign a value relative to other assets, based on comparable loss: This version of the file is more valuable to you than that version.

In their 2010 global Internet threat report, security firm Symantec surveyed the kinds of goods and services offered for sale on underground web pages. The item most frequently offered in both 2009 and 2008 was credit card numbers, at prices ranging from $0.85 to $30.00 each. (Compare those prices to an individual's effort to deal with the effect of a stolen credit card or the potential amount lost by the issuing bank.) Second most frequent was bank account credentials, at $15 to $850; these were offered for sale at 19% of websites in both years. Email accounts were next at $1 to $20, and lists of email addresses went for $1.70 to $15.00 per thousand. At position 10 in 2009 were website administration credentials, costing only $2 to $30. These black market websites demonstrate that the market price of computer assets can be dramatically different from their value to rightful owners.

2025/2026 2025/2026 2025/2026

The value of many assets can change over time, so the degree of harm (and therefore the severity of a threat) can change, too. With unlimited time, money, and capability, we might try to protect against all kinds of harm. But because our resources are limited, we must prioritize our protection, safeguarding only against serious threats and the ones we can control. Choosing the threats we try to mitigate involves a process called risk management, and it includes weighing the seriousness of a threat against our ability to protect.

Risk management involves choosing which threats to control and what resources to devote to protection.

Risk and Common Sense

The number and kinds of threats are practically unlimited because devising an attack requires an active imagination, determination, persistence, and time (as well as access and resources). The nature and number of threats in the computer world reflect life in general:

The causes of harm are limitless and largely unpredictable. Natural disasters like volcanoes and earthquakes happen with little or no warning, as do auto accidents, heart attacks, influenza, and random acts of violence. To protect against accidents or the flu, you might decide to stay indoors, never venturing outside. But by doing so, you trade one set of risks for another; while you are inside, you are vulnerable to building collapse. There are too many possible causes of harm for us to protect ourselves—or our computers—completely against all of them.

In real life we make decisions every day about the best way to provide our security. For example, although we may choose to live in an area that is not prone to earthquakes, we cannot entirely eliminate earthquake risk. Some choices are conscious, such as deciding not to walk down a dark alley in an unsafe neighborhood; other times our subconscious guides us, from experience or expertise, to take some precaution. We evaluate the likelihood and severity of harm, and then consider ways (called countermeasures or controls) to address threats and determine the controls' effectiveness.

Computer security is similar. Because we cannot protect against everything, we prioritize: Only so much time, energy, or money is available for protection, so we address some risks and let others slide. Or we consider alternative courses of action, such as transferring risk by purchasing

2025/2026 2025/2026 2025/2026

insurance or even doing nothing if the side effects of the countermeasure could be worse than the possible harm. The risk that remains uncovered by controls is called ***residual risk.***

A basic model of risk management involves a user's calculating the value of all assets, determining the amount of harm from all possible threats, computing the costs of protection, selecting safeguards (that is, controls or countermeasures) based on the degree of risk and on limited resources, and applying the safeguards to optimize harm averted.

This approach to risk management is a logical and sensible approach to protection, but it has significant drawbacks. In reality, it is difficult to assess the value of each asset; as we have seen, value can change depending on context, timing, and a host of other characteristics. Even harder is determining the impact of all possible threats. The range of possible threats is effectively limitless, and it is difficult (if not impossible in some situations) to know the short- and long-term impacts of an action. For instance, Sidebar 1- 3 describes a study of the impact of security breaches over time on corporate finances, showing that a threat must be evaluated over time, not just at a single instance.

Sidebar 1-3 Short- and Long-term Risks of Security Breaches

2025/2026 2025/2026 2025/2026

It was long assumed that security breaches would be bad for business: that customers, fearful of losing their data, would veer away from insecure businesses and toward more secure ones. But empirical studies suggest that the picture is more complicated. Early studies of the effects of security breaches, such as that of Campbell [CAM03], examined the effects of breaches on stock price. They found that a breach's impact could depend on the nature of the breach itself; the effects were higher when the breach involved unauthorized access to confidential data. Cavusoglu et al. [CAV04] discovered that a breach affects the value not only of the company experiencing the breach but also of security enterprises: On average, the breached firms lost 2.1 percent of market value within two days of the breach's disclosure, but security developers' market value actually increased 1.36 percent.

Myung Ko and Carlos Dorantes [KO06] looked at the longer-term financial effects of publicly announced breaches. Based on the Campbell et al. study, they examined data for four quarters following the announcement of unauthorized access to confidential data. Ko and Dorantes note many types of possible breach-related costs:

رومانی صبحی رزق جرجس مطر

رومانی صبحی رزق جرجس مطر

> "Examples of short-term costs include cost of repairs, cost of replacement of the system, lost business due to the disruption of business operations, and lost productivity of employees.
>
> These are also considered tangible costs. On the other hand, long-term costs include the loss of existing customers due to loss of trust, failing to attract potential future customers due to negative reputation from the breach, loss of business partners due to loss of trust, and potential legal liabilities from the breach. Most of these costs are intangible costs that are difficult to calculate but extremely important in assessing the overall security breach costs to the organization."

Ko and Dorantes compared two groups of companies: one set (the treatment group) with data breaches, and the other (the control group) without a breach but matched for size and industry. Their findings were striking. Contrary to what you might suppose, the breached firms had no decrease in performance for the quarters following the breach, but their return on assets decreased in the third quarter. The comparison of treatment with control companies revealed that the control firms generally outperformed the breached firms. However, the breached firms outperformed the control firms in the fourth quarter.

2025/2026 2025/2026 2025/2026

These results are consonant with the results of other researchers who conclude that there is minimal long-term economic impact from a security breach. There are many reasons why this is so. For example, customers may think that all competing firms have the same vulnerabilities and threats, so changing to another vendor does not reduce the risk. Another possible explanation may be a perception that a breached company has better security since the breach forces the company to strengthen controls and thus reduce the likelihood of similar breaches. Yet another explanation may simply be the customers' short attention span; as time passes, customers forget about the breach and return to business as usual.

All these studies have limitations, including small sample sizes and lack of sufficient data. But they clearly demonstrate the difficulties of quantifying and verifying the impacts of security risks, and point out a difference between short- and long-term effects.

Although we should not apply protection haphazardly, we will necessarily protect against threats we consider most likely or most damaging. For this reason, it is essential to understand how we perceive threats and evaluate their likely occurrence and impact. Sidebar 1-4 summarizes some

of the relevant research in risk perception and decision making. Such research suggests that, for relatively rare instances such as high-impact security problems, we must take into account the ways in which people focus more on the impact than on the actual likelihood of occurrence.

Sidebar 1-4 Perception of the Risk of Extreme Events

When a type of adverse event happens frequently, we can calculate its likelihood and impact by examining both frequency and nature of the collective set of events. For instance, we can calculate the likelihood that it will rain this week and take an educated guess at the number of inches of precipitation we will receive; rain is a fairly frequent occurrence. But security problems are often extreme events: They happen infrequently and under a wide variety of circumstances, so it is difficult to look at them as a group and draw general conclusions.

Paul Slovic's work on risk addresses the particular difficulties with extreme events. He points out that evaluating risk in such cases can be a political endeavor as much as a scientific one. He notes that we tend to let values, process, power, and trust influence our risk analysis [SLO99].

Beginning with Fischoff et al. [FIS78], researchers characterized extreme risk along two perception-based axes: the dread of the risk and the degree to which the risk is unknown. These feelings about risk, called affects by psychologists, enable researchers to discuss relative risks by placing them on a plane defined by the two perceptions as axes. A study by Loewenstein et al. [LOE01] describes how risk perceptions are influenced by association (with events already experienced) and by affect at least as much if not more than by reason. In fact, if the two influences compete, feelings usually trump reason.

This characteristic of risk analysis is reinforced by prospect theory: studies of how people make decisions by using reason and feeling. Kahneman and Tversky [KAH79] showed that people tend to overestimate the likelihood of rare, unexperienced events because their feelings of dread and the unknown usually dominate analytical reasoning about the low likelihood of occurrence. By contrast, if people experience similar outcomes and their likelihood, their feeling of dread diminishes and they can actually underestimate rare events. In other words, if the impact of a rare event is high (high dread), then people focus on the impact, regardless of the likelihood. But if the impact of a rare event is small, then they pay attention to the likelihood.

2025/2026 2025/2026 2025/2026

Let us look more carefully at the nature of a security threat. We have seen that one aspect—its potential harm—is the amount of damage it can cause; this aspect is the ***impact*** component of the risk. We also consider the magnitude of the threat's ***likelihood***.

A likely threat is not just one that someone might want to pull off but rather one that could actually occur. Some people might daydream about getting rich by robbing a bank; most, however, would reject that idea because of its difficulty (if not its immorality or risk). One aspect of likelihood is feasibility: Is it even possible to accomplish the attack? If the answer is no, then the likelihood is zero, and therefore so is the risk. So a good place to start in assessing risk is to look at whether the proposed action is feasible. Three factors determine feasibility, as we describe next.

> ***Spending for security is based on the impact and likelihood of potential harm—both of which are nearly impossible to measure precisely.***

Method–Opportunity–Motive

A malicious attacker must have three things to ensure success: method, opportunity, and motive, depicted in Figure 1-11. Roughly speaking, method is the how; opportunity, the when; and motive, the why of an attack. Deny the attacker any of those three and the attack will not succeed. Let us examine these properties individually.

Method

By method we mean the skills, knowledge, tools, and other things with which to perpetrate the attack. Think of comic figures that want to do something, for example, to steal valuable jewelry, but the characters are so inept that their every move is doomed to fail. These people lack the capability or method to succeed, in part because there are no classes in jewel theft or books on burglary for dummies.

Anyone can find plenty of courses and books about computing, however. Knowledge of specific models of computer systems is widely available in bookstores and on the Internet. Mass-market systems (such as the Microsoft or Apple or Unix operating systems) are readily available for purchase, as are common software products, such as word processors or database management systems, so potential attackers can even get hardware and software on which to experiment and perfect an attack. Some manufacturers release detailed specifications on how the system was

2025/2026 2025/2026 2025/2026

designed or how it operates, as guides for users and integrators who want to implement other complementary products. Various attack tools—scripts, model programs, and tools to test for weaknesses—are available from hackers' sites on the Internet, to the degree that many attacks require only the attacker's ability to download and run a program. The term ***script kiddie*** describes someone who downloads a complete attack code package and needs only to enter a few details to identify the target and let the script perform the attack. Often, only time and inclination limit an attacker.

رومانی صبحی رزق جرجس مطر

2025/2026

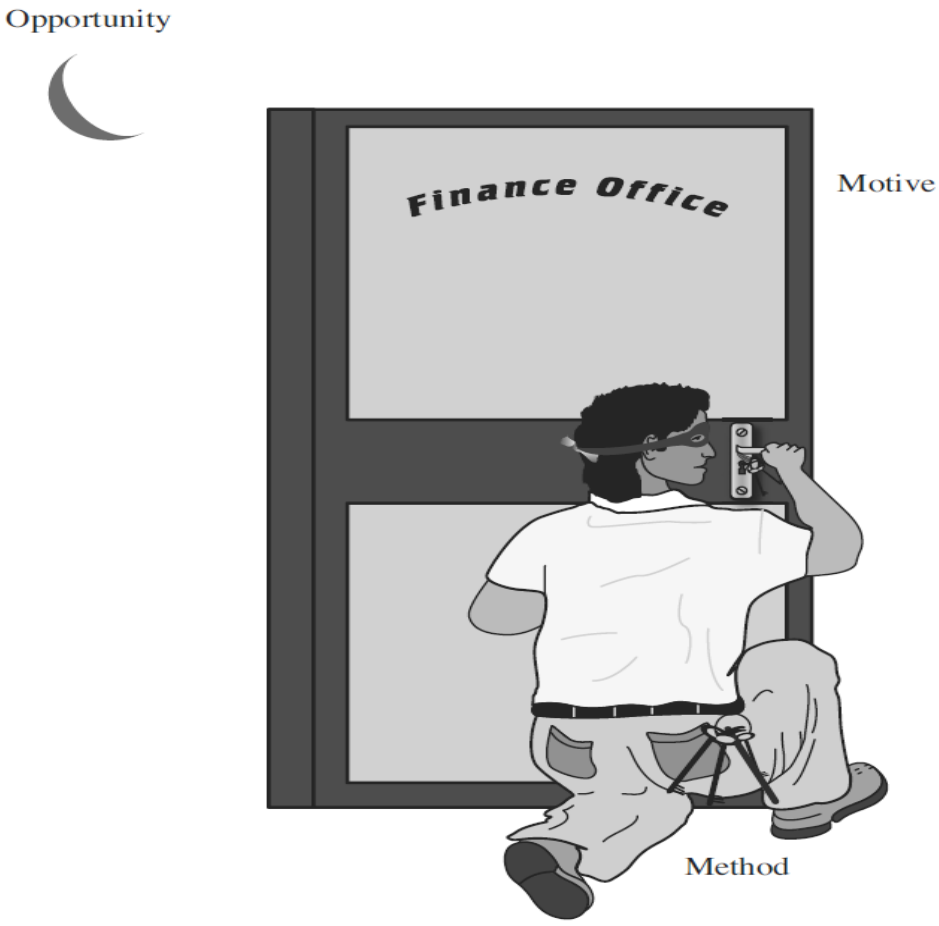


FIGURE 1-11 Method–Opportunity–Motive

Opportunity

Opportunity is the time and access to execute an attack. You hear that a fabulous apartment has just become available, so you rush to the rental agent, only to find someone else rented it five minutes earlier. You missed your opportunity.

Many computer systems present ample opportunity for attack. Systems available to the public are, by definition, accessible; often their owners take special care to make them fully available so that if one hardware component fails, the owner has spares instantly ready to be pressed into service. Other people are oblivious to the need to protect their computers, so unattended laptops and unsecured network connections give ample opportunity for attack. Some systems have private or undocumented entry points for administration or maintenance, but attackers can also find and use those entry points to attack the systems.

Motive

Finally, an attacker must have a motive or reason to want to attack. You probably have ample opportunity and ability to throw a rock through your neighbor's window, but you do not. Why not? Because you have no reason to want to harm your neighbor: You lack motive.

We have already described some of the motives for computer crime: money, fame, self-esteem, politics, terror. It is often difficult to determine motive for an attack. Some places are "attractive targets," meaning they are very appealing to attackers. Popular targets include law enforcement and defense department computers, perhaps because they are presumed to be well protected against attack (so they present a challenge and a successful attack shows the attacker's prowess). Other systems are attacked because they are easy to attack. And some systems are attacked at random simply because they are there.

2025/2026 2025/2026 2025/2026

> ***Method, opportunity, and motive are all necessary for an attack to succeed; deny any of these and the attack will fail.***

By demonstrating feasibility, the factors of method, opportunity, and motive determine whether an attack can succeed. These factors give the advantage to the attacker because they are qualities or strengths the attacker must possess. Another factor, this time giving an advantage to the defender, determines whether an attack will succeed: The attacker needs a vulnerability, an undefended place to attack. If the defender removes vulnerabilities, the attacker cannot attack.

1.4 Vulnerabilities

As we noted earlier in this chapter, a vulnerability is a weakness in the security of the computer system, for example, in procedures, design, or implementation, that might be exploited to cause

رومانی صبحی رزق جرجس مطر

loss or harm. Think of a bank, with an armed guard at the front door, bulletproof glass protecting the tellers, and a heavy metal vault requiring multiple keys for entry. To rob a bank, you would have to think of how to exploit a weakness not covered by these defenses. For example, you might bribe a teller or pose as a maintenance worker.

Computer systems have vulnerabilities, too. In this book we consider many, such as weak authentication, lack of access control, errors in programs, finite or insufficient resources, and inadequate physical protection. Paired with a credible attack, each of these vulnerabilities can allow harm to confidentiality, integrity, or availability. Each attack vector seeks to exploit a particular vulnerability.

Vulnerabilities are weaknesses that can allow harm to occur.

Security analysts speak of a system's attack surface, which is the system's full set of vulnerabilities—actual and potential. Thus, the attack surface includes physical hazards, malicious attacks by outsiders, stealth data theft by insiders, mistakes, and impersonations. Although such attacks range from easy to highly improbable, analysts must consider all possibilities.

Our next step is to find ways to block threats by neutralizing vulnerabilities.

2025/2026 2025/2026 2025/2026

1.5 Controls

A control or countermeasure is a means to counter threats. Harm occurs when a threat is realized against a vulnerability. To protect against harm, then, we can neutralize the threat, close the vulnerability, or both. The possibility for harm to occur is called risk. We can deal with harm in several ways:

- ***prevent*** it, by blocking the attack or closing the vulnerability
- ***deter*** it, by making the attack harder but not impossible
- ***deflect*** it, by making another target more attractive (or this one less so)
- ***mitigate*** it, by making its impact less severe
- ***detect*** it, either as it happens or sometime after the fact
- ***recover*** from its effect

Of course, more than one of these controls can be used simultaneously. So, for example, we might try to prevent intrusions—but if we suspect we cannot prevent all of them, we might also

رومانی صبحی رزق جرجس مطر

install a detection device to warn of an imminent attack. And we should have in place incident-response procedures to help in the recovery in case an intrusion does succeed.

Security professionals balance the cost and effectiveness of controls with the likelihood and severity of harm.

To consider the controls or countermeasures that attempt to prevent exploiting a computing system's vulnerabilities, we begin by thinking about traditional ways to enhance physical security. In the Middle Ages, castles and fortresses were built to protect the people and valuable property inside. The fortress might have had one or more security characteristics, including

- a strong gate or door to repel invaders
- heavy walls to withstand objects thrown or projected against them
- a surrounding moat to control access
- arrow slits to let archers shoot at approaching enemies
- crenellations to allow inhabitants to lean out from the roof and pour hot or vile liquids on attackers

- a drawbridge to limit access to authorized people
- a portcullis to limit access beyond the drawbridge
- gatekeepers to verify that only authorized people and goods could enter

Similarly, today we use a multipronged approach to protect our homes and offices. We may combine strong locks on the doors with a burglar alarm, reinforced windows, and even a nosy neighbor to keep an eye on our valuables. In each case, we select one or more ways to deter an intruder or attacker, and we base our selection not only on the value of what we protect but also on the effort we think an attacker or intruder will expend to get inside.

Computer security has the same characteristics. We have many controls at our disposal. Some are easier than others to use or implement. Some are cheaper than others to use or implement. And some are more difficult than others for intruders to override. Figure 1-12 illustrates how we use a combination of controls to secure our valuable resources. We use one or more controls, according to what we are protecting, how the cost of protection compares with the risk of loss, and how hard we think intruders will work to get what they want.

رومانی صبحی رزق جرجس مطر

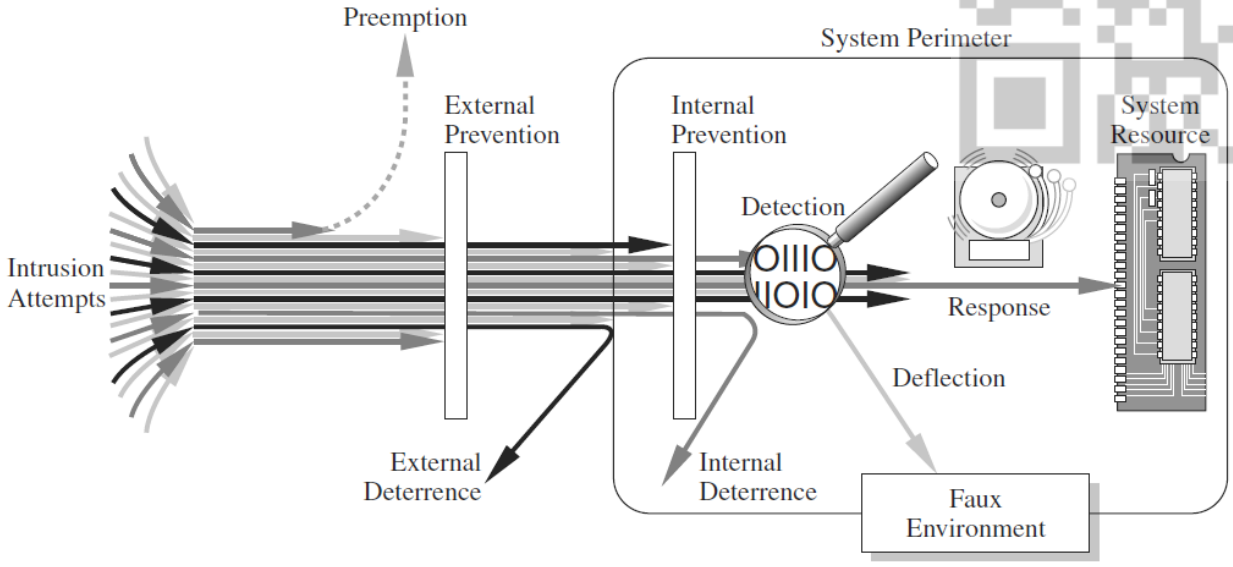


FIGURE 1-12 Effects of Controls

In this section, we present an overview of the controls available to us. In the rest of this book, we examine how to use controls against specific kinds of threats.

We can group controls into three largely independent classes. The following list shows the classes and several examples of each type of control.

2025/2026 2025/2026 2025/2026

- Physical controls stop or block an attack by using something tangible too, such as walls and fences
  - locks
  - (human) guards
  - sprinklers and other fire extinguishers
- Procedural or administrative controls use a command or agreement that
  - requires or advises people how to act; for example,
  - laws, regulations
  - policies, procedures, guidelines
  - copyrights, patents
  - contracts, agreements
- Technical controls counter threats with technology (hardware or software), including
  - passwords
  - program or operating system access controls
  - network protocols

رومانی صبحی رزق جرجس مطر

– firewalls, intrusion detection systems

– encryption

– network traffic flow regulators

(Note that the term “logical controls” is also used, but some people use it to mean administrative controls, whereas others use it to mean technical controls. To avoid confusion, we do not use that term.)

As shown in Figure 1-13, you can think in terms of the property to be protected and the kind of threat when you are choosing appropriate types of countermeasures. None of these classes is necessarily better than or preferable to the others; they work in different ways with different kinds of results. And it can be effective to use overlapping controls or defense in depth: more than one control or more than one class of control to achieve protection.

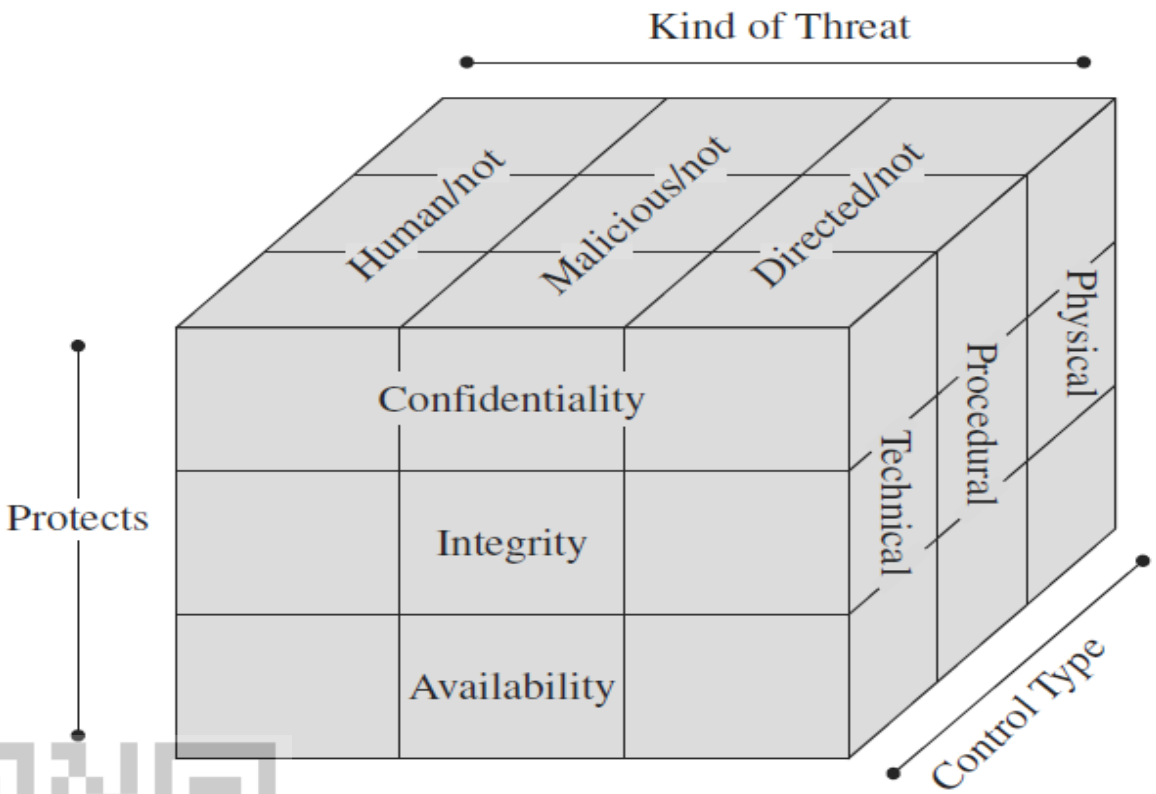


FIGURE 1-13 Types of Countermeasures

رومانی صبحی رزق جرجس مطر

Exercises

1. Distinguish between vulnerability, threat, and control.

2. Theft usually results in some kind of harm. For example, if someone steals your car, you may suffer financial loss, inconvenience (by losing your mode of transportation), and emotional upset (because of invasion of your personal property and space). List three kinds of harm a company might experience from theft of computer equipment.

3. List at least three kinds of harm a company could experience from electronic espionage or unauthorized viewing of confidential company materials.

4. List at least three kinds of damage a company could suffer when the integrity of a program or company data is compromised.

5. List at least three kinds of harm a company could encounter from loss of service, that is, failure of availability. List the product or capability to which access is lost, and explain how this loss hurts the company.

6. Describe a situation in which you have experienced harm as a consequence of a failure of computer security. Was the failure malicious or not? Did the attack target you specifically or was it general and you were the unfortunate victim?

2025/2026 2025/2026 2025/2026

7. Describe two examples of vulnerabilities in automobiles for which auto manufacturers have instituted controls. Tell why you think these controls are effective, somewhat effective, or ineffective.

8. One control against accidental software deletion is to save all old versions of a program. Of course, this control is prohibitively expensive in terms of cost of storage. Suggest a less costly control against accidental software deletion. Is your control effective against all possible causes of software deletion? If not, what threats does it not cover?

9. On your personal computer, who can install programs? Who can change operating system data? Who can replace portions of the operating system? Can any of these actions be performed remotely?

رومانی صبحی رزق جرجس مطر

Chapter 2

Authentication, Access Control, and Cryptography

رومانی صبحی رزق جرجس مطر

Just as doctors have stethoscopes and carpenters have measuring tapes and squares, security professionals have tools they use frequently. Three of these security tools are authentication, access control, and cryptography. In this chapter we introduce these tools, and in later chapters we use these tools repeatedly to address a wide range of security issues.

In some sense, security hasn't changed since sentient beings began accumulating things worth protecting. A system owner establishes a security policy, formally or informally, explicitly or implicitly—perhaps as simple as "no one is allowed to take my food"—and begins taking measures to enforce that policy. The character of the threats changes as the protagonist moves from the jungle to the medieval battlefield to the modern battlefield to the Internet, as does the nature of the available protections, but their strategic essence remains largely constant: An attacker wants something a defender has, so the attacker goes after it. The defender has a number of options—fight, build a barrier or alarm system, run and hide, diminish the target's attractiveness to the attacker—and these options all have analogues in modern computer security. The specifics change, but the broad strokes remain the same.

2025/2026 2025/2026 2025/2026

In this chapter, we lay the foundation for computer security by studying those broad strokes. We look at a number of ubiquitous security strategies, identify the threats against which each of those strategies is effective, and give examples of representative countermeasures. Throughout the rest of this book, as we delve into the specific technical security measures used in operating systems, programming, websites and browsers, and networks, we revisit these same strategies again and again. Years from now, when we're all using technology that hasn't even been imagined yet, this chapter should be just as relevant as it is today.

A security professional analyzes situations by finding threats and vulnerabilities to the confidentiality, integrity, and/or availability of a computing system. Often, controlling these threats and vulnerabilities involves a policy that specifies who (which subjects) can access what (which objects) how (by which means). We introduced that framework in Chapter 1. But now we want to delve more deeply into how such a policy works. To be effective the policy enforcement must determine who accurately. That is, if policy says Adam can access something, security fails

if someone else impersonates Adam. Thus, to enforce security policies properly, we need ways to determine beyond a reasonable doubt that a subject's identity is accurate. The property of accurate identification is called authentication. The first critical tool for security professionals is authentication and its techniques and technologies.

When we introduced security policies we did not explicitly state the converse: A subject is allowed to access an object in a particular mode but, unless authorized, all other subjects are not allowed to access the object. A policy without such limits is practically useless. What good does it do to say one subject can access an object if any other subject can do so without being authorized by policy. Consequently, we need ways to restrict access to only those subjects on the yes list, like admitting theatre patrons to a play (with tickets) or checking in invitees to a party (on a guest list). Someone or something controls access, for example, an usher collects tickets or a host manages the guest list. Allowing exactly those accesses authorized is called access control. Mechanisms to implement access control are another fundamental computer security tool.

Suppose you were trying to limit access to a football match being held on an open park in a populous city. Without a fence, gate, or moat, you could not limit who could see the game. But suppose you had super powers and could cloak the players in invisibility uniforms. You would issue special glasses only to people allowed to see the match; others might look but see nothing. Although this scenario is pure fantasy, such an invisibility technology does exist, called encryption. Simply put, encryption is a tool by which we can transform data so only intended receivers (who have keys, the equivalent of anti-cloaking glasses) can deduce the concealed bits. The third and final fundamental security tool in this chapter is encryption.

In this chapter we describe these tools and then give a few examples to help you understand how the tools work. But most applications of these tools come in later chapters, where we elaborate on their use in the context of a more complete security situation. In most of this chapter we dissect our three fundamental security tools: authentication, access control, and encryption.

2.1 Authentication

Your neighbor recognizes you, sees you frequently, and knows you are someone who should be going into your home. Your neighbor can also notice someone different, especially if that person is doing something suspicious, such as snooping around your doorway, peering up and down the

رومانی صبحی رزق جرجس مطر
2025/2026 2025/2026 2025/2026

walk, or picking up a heavy stone. Coupling these suspicious events with hearing the sound of breaking glass, your neighbor might even call the police.

Computers have replaced many face-to-face interactions with electronic ones. With no vigilant neighbor to recognize that something is awry, people need other mechanisms to separate authorized from unauthorized parties. For this reason, the basis of computer security is controlled access: someone is authorized to take some action on something. We examine access control later in this chapter. But for access control to work, we need to be sure who the "someone" is. In this section we introduce authentication, the process of ascertaining or confirming an identity.

A computer system does not have the cues we do with face-to-face communication that let us recognize our friends. Instead computers depend on data to recognize others. Determining who a person really is consists of two separate steps:

• Identification is the act of asserting who a person is.

• Authentication is the act of proving that asserted identity: that the person is who she says she is.

Identification is asserting who a person is.

Authentication is proving that asserted identity.

2025/2026 2025/2026 2025/2026

We have phrased these steps from the perspective of a person seeking to be recognized, using the term "person" for simplicity. In fact, such recognition occurs between people, computer processes (executing programs), network connections, devices, and similar active entities. In security, all these entities are called ***subjects.***

The two concepts of identification and authentication are easily and often confused. Identities, like names, are often well known, public, and not protected. On the other hand, authentication is necessarily protected. If someone's identity is public, anyone can claim to be that person. What separates the pretenders from the real person is proof by authentication.

Identification versus Authentication

Identities are often well known, predictable, or guessable. If you send email to someone, you implicitly send along your email account ID so the other person can reply to you. In an online discussion you may post comments under a screen name as a way of linking your various postings. Your bank account number is printed on checks you write; your debit card account number is shown on your card, and so on. In each of these cases you reveal a part of your

رومانی صبحی رزق جرجس مطر

identity. Notice that your identity is more than just your name: Your bank account number, debit card number, email address, and other things are ways by which people and processes identify you.

Some account IDs are not hard to guess. Some places assign user IDs as the user's last name followed by first initial. Others use three initials or some other scheme that outsiders can easily predict. Often for online transactions your account ID is your email address, to make it easy for you to remember. Other accounts identify you by telephone, social security, or some other identity number. With too many accounts to remember, you may welcome places that identify you by something you know well because you use it often. But using it often also means other people can know or guess it as well. For these reasons, many people could easily, although falsely, claim to be you by presenting one of your known identifiers.

Identities are typically public or well known. Authentication should be private.

Authentication, on the other hand, should be reliable. If identification asserts your identity, authentication confirms that you are who you purport to be. Although identifiers may be widely known or easily determined, authentication should be private. However, if the authentication process is not strong enough, it will not be secure. Consider, for example, how one political candidate's email was compromised by a not-private-enough authentication process as described in Sidebar 2-1.

Sidebar 2-1 Vice Presidential Candidate Sarah Palin's Email Exposed

During the 2008 U.S. presidential campaign, vice presidential candidate Sarah Palin's personal email account was hacked. Contents of email messages and Palin's contacts list were posted on a public bulletin board. A 20-year-old University of Tennessee student, David Kernell, was subsequently convicted of unauthorized access to obtain information from her computer and sentenced to a year and a day.

How could a college student have accessed the computer of a high-profile public official who at the time was governor of Alaska and a U.S. vice presidential candidate under protection of the U.S. Secret Service? Easy: He simply pretended to be her. But surely nobody (other than, perhaps, comedian Tina Fey) could successfully impersonate her. Here is how easy the attack was. Governor Palin's email account was gov.palin@yahoo.com. The account ID was well known because of news reports of an earlier incident involving Palin's using her personal

account for official state communications; even without the publicity the account name would not have been hard to guess.

But the password? No, the student didn't guess the password. All he had to do was pretend to be Palin and claim she had forgotten her password. Yahoo asked Kernell the security questions Palin had filed with Yahoo on opening the account: birth date (found from Wikipedia), postcode (public knowledge, especially because she had gotten public attention for not using the official governor's mansion), and where she met her husband (part of her unofficial biography circulating during the campaign: she and her husband met in high school). With those three answers, Kernell was able to change her password (to "popcorn," something appealing to most college students). From that point on, not only was Kernell effectively Palin, the real Palin could not access her own email account because did she not know the new password.

Authentication mechanisms use any of three qualities to confirm a user's identity:

- Something the user knows. Passwords, PIN numbers, passphrases, a secret handshake, and mother's maiden name are examples of what a user may know.
- Something the user is. These authenticators, called biometrics, are based on a physical characteristic of the user, such as a fingerprint, the pattern of a person's voice, or a face (picture). These authentication methods are old (we recognize friends in person by their faces or on a telephone by their voices) but are just starting to be used in computer authentications.
- Something the user has. Identity badges, physical keys, a driver's license, or a uniform are common examples of things people have that make them recognizable.

2025/2026 2025/2026 2025/2026

Two or more forms can be combined; for example, a bank card and a PIN combine something the user has (the card) with something the user knows (the PIN).

Authentication is based on something you know, are, or have.

Although passwords were the first form of computer authentication and remain popular, these other forms are becoming easier to use, less expensive, and more common. In the following sections we examine each of these forms of authentication.

رومانی صبحی رزق جرجس مطر

Authentication Based on Phrases and Facts: Something You Know

Password protection seems to offer a relatively secure system for confirming identity related information, but human practice sometimes degrades its quality. Let us explore vulnerabilities in authentication, focusing on the most common authentication parameter, the password. In this section we consider the nature of passwords, criteria for selecting them, and ways of using them for authentication. As you read the following discussion of password vulnerabilities, think about how well these identity attacks would work against security questions and other authentication schemes with which you may be familiar. And remember how much information about us is known—sometimes because we reveal it ourselves—as described in Sidebar 2-2.

Sidebar 2-2 Facebook Pages Answer Security Questions

George Bronk, a 23-year-old resident of Sacramento, California, pleaded guilty on 13 January 2011 to charges including computer intrusion, false impersonation, and possession of child pornography. His crimes involved impersonating women with data obtained from their Facebook accounts.

According to an Associated Press news story [THO11], Bronk scanned Facebook pages for pages showing women's email addresses. He then read their Facebook profiles carefully for clues that could help him answer security questions, such as a favorite color or a father's middle name. With these profile clues, Bronk then turned to the email account providers. Using the same technique as Kernell, Bronk pretended to have forgotten his (her) password and sometimes succeeded at answering the security questions necessary to recover a forgotten password. He sometimes used the same technique to obtain access to Facebook accounts.

After he had the women's passwords, he perused their sent mail folders for embarrassing photographs; he sometimes mailed those to a victim's contacts or posted them on her Facebook page. He carried out his activities from December 2009 to October 2010. When police confiscated his computer and analyzed its contents, they found 3200 Internet contacts and 172 email files containing explicit photographs; police sent mail to all the contacts to ask if they had been victimized, and 46 replied that they had. The victims lived in England, Washington, D.C., and 17 states from California to New Hampshire.

The California attorney general's office advised those using email and social networking sites to pick security questions and answers that aren't posted on public sites, or to add numbers or other

characters to common security answers. Additional safety tips are on the attorney general's website.

Password Use

The use of passwords is fairly straightforward, as you probably already know from experience. A user enters some piece of identification, such as a name or an assigned user ID; this identification can be available to the public or can be easy to guess because it does not provide the real protection. The protection system then requests a password from the user. If the password matches the one on file for the user, the user is authenticated and allowed access. If the password match fails, the system requests the password again, in case the user mistyped.

Even though passwords are widely used, they suffer from some difficulties of use:

- Use. Supplying a password for each access to an object can be inconvenient and time consuming.
- Disclosure. If a user discloses a password to an unauthorized individual, the object becomes immediately accessible. If the user then changes the password to re-protect the object, the user must inform any other legitimate users of the new password because their old password will fail.
- Revocation. To revoke one user's access right to an object, someone must change the password, thereby causing the same problems as disclosure.
- Loss. Depending on how the passwords are implemented, it may be impossible to retrieve a lost or forgotten password. The operators or system administrators can certainly intervene and provide a new password, but often they cannot determine what password a user had chosen previously. If the user loses (or forgets) the password, administrators must assign a new one.

2025/2026 2025/2026 2025/2026

Attacking and Protecting Passwords

How secure are passwords themselves? Passwords are somewhat limited as protection devices because of the relatively small number of bits of information they contain. Worse, people pick passwords that do not even take advantage of the number of bits available:

Choosing a well-known string, such as qwerty, password, or 123456 reduces an attacker's uncertainty or difficulty essentially to zero.

رومانی صبحی رزق جرجس مطر

Knight and Hartley [KNI98] list, in order, 12 steps an attacker might try in order to determine a password. These steps are in increasing degree of difficulty (number of guesses), and so they indicate the amount of work to which the attacker must go in order to derive a password. Here are their password guessing steps:

- no password
- the same as the user ID
- is, or is derived from, the user's name
- on a common word list (for example, password, secret, private) plus common names and patterns (for example, qwerty, aaaaaa)
- contained in a short college dictionary
- contained in a complete English word list
- contained in common non-English-language dictionaries
- contained in a short college dictionary with capitalizations (PaSsWorD) or substitutions (digit 0 for letter O, and so forth)
- contained in a complete English dictionary with capitalizations or substitutions
- contained in common non-English dictionaries with capitalization or substitutions
- obtained by brute force, trying all possible combinations of alphabetic characters
- obtained by brute force, trying all possible combinations from the full character set

Although the last step will always succeed, the steps immediately preceding it are so time consuming that they will deter all but the most dedicated attacker for whom time is not a limiting factor.

Every password can be guessed; password strength is determined by how many guesses are required.

We now expand on some of these approaches.

Dictionary Attacks

Several network sites post dictionaries of phrases, science fiction character names, places, mythological names, Chinese words, Yiddish words, and other specialized lists. These lists help site administrators identify users who have chosen weak passwords, but the same dictionaries can also be used by attackers of sites that do not have such attentive administrators. The COPS

[FAR90], Crack [MUF92], and SATAN [FAR95] utilities allow an administrator to scan a system for weak passwords. But these same utilities, or other homemade ones, allow attackers to do the same. Now Internet sites offer so-called password recovery software as freeware or shareware for under $20. (These are password cracking programs.)

People think they can be clever by picking a simple password and replacing certain characters, such as 0 (zero) for letter O, 1 (one) for letter I or L, 3 (three) for letter E or @ (at) for letter A. But users aren't the only people who could think up these substitutions.

Inferring Passwords Likely for a User

If Sandy is selecting a password, she is probably not choosing a word completely at random. Most likely Sandy's password is something meaningful to her. People typically choose personal passwords, such as the name of a spouse, child, other family member, or pet. For any given person, the number of such possibilities is only a dozen or two. Trying this many passwords by computer takes well under a second! Even a person working by hand could try ten likely candidates in a minute or two.

Thus, what seemed formidable in theory is in fact quite vulnerable in practice, and the likelihood of successful penetration is frightening high. Morris and Thompson [MOR79] confirmed our fears in their report on the results of having gathered passwords from many users, shown in Table 2-1. Figure 2-1 (based on data from that study) shows the characteristics of the 3,289 passwords gathered. The results from that study are distressing, and the situation today is likely to be the same. Of those passwords, 86 percent could be uncovered in about one week's worth of 24-hour-a-day testing, using the very generous estimate of 1 millisecond per password check.

Number	Percentage	Structure
15	< 1%	Single ASCII character
72	2%	Two ASCII character
464	14%	Three ASCII character
477	14%	Four alphabetic letters
706	21%	Five alphabetic letters, all the same case
605	18%	Six lowercase alphabetic letters
492	15%	Words in dictionaries or lists of names
2831	86%	Total of all categories above

.TABLE 2-1 Password Characteristics

2023/2026 2025/2026 2023/2026

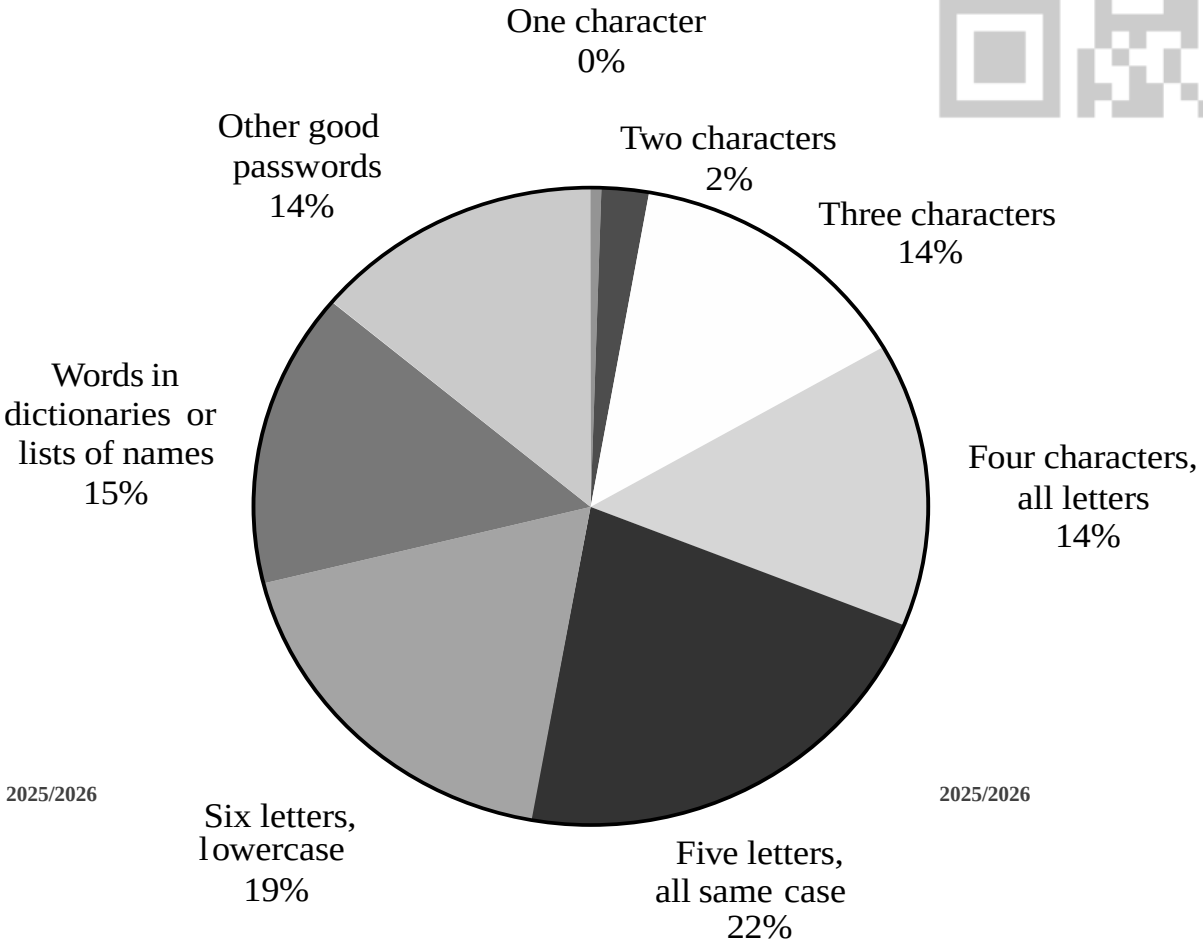


FIGURE 2-1 Distribution of Password Types

Lest you dismiss these results as dated (they were reported in 1979), Klein repeated the experiment in 1990 [KLE90] and Spafford in 1992 [SPA92a]. Each collected approximately 15,000 passwords. Klein reported that 2.7 percent of the passwords were guessed in only 15 minutes of machine time (at the speed of 1990s computers), and 21 percent were guessed within a week! Spafford found that the average password length was 6.8 characters and that 28.9 percent consisted of only lowercase alphabetic characters.

Then, in 2002 the British online bank Egg found its users still choosing weak passwords [BUX02]. A full 50 percent of passwords for their online banking service were family members' names: 23 percent children's names, 19 percent a spouse or partner, and 9 percent their own. Alas, pets came in at only 8 percent, while celebrities and football (soccer) stars tied at 9 percent each. And in 1998, Knight and Hartley [KNI98] reported that approximately 35 percent of passwords are deduced from syllables and initials of the account owner's name. In December

2009 the computer security firm Imperva analyzed 34 million Facebook passwords that had previously been disclosed accidentally, reporting that

- about 30 per cent of users chose passwords of fewer than seven characters.
- nearly 50 per cent of people used names, slang words, dictionary words or trivial passwords—consecutive digits, adjacent keyboard keys and so on.
- most popular passwords included 12345, 123456, 1234567, password, and iloveyou, in the top ten.

Two friends we know told us their passwords as we helped them administer their systems, and their passwords would both have been among the first we would have guessed. But, you say, these are amateurs unaware of the security risk of a weak password. At a recent meeting, a security expert related this experience: He thought he had chosen a solid password, so he invited a class of students to ask him questions and offer guesses as to his password. He was amazed that they asked only a few questions before they had deduced the password. And this was a security expert!

The conclusion we draw from these incidents is that people choose weak and easily guessed passwords more frequently than some might expect. Clearly, people find something in the password process that is difficult or unpleasant: Either people are unable to choose good passwords, perhaps because of the pressure of the situation, or they fear they will forget solid passwords. In either case, passwords are not always good authenticators.

2025/2026 2025/2026 2025/2026

Guessing Probable Passwords

Think of a word. Is the word you thought of long? Is it uncommon? Is it hard to spell or to pronounce? The answer to all three of these questions is probably no.

Penetrators searching for passwords realize these very human characteristics and use them to their advantage. Therefore, penetrators try techniques that are likely to lead to rapid success. If people prefer short passwords to long ones, the penetrator will plan to try all passwords but to try them in order by length. There are only

$$26^1 + 26^2 + 26^3 = 18{,}278$$

(not case sensitive) passwords of length 3 or less. Testing that many passwords would be difficult but possible for a human, but repetitive password testing is an easy computer

رومانی صبحی رزق جرجس مطر

application. At an assumed rate of one password per millisecond, all of these passwords can be checked in 18.278 seconds, hardly a challenge with a computer. Even expanding the tries to 4 or 5 characters raises the count only to 475 seconds (about 8 minutes) or 12,356 seconds (about 3.5 hours), respectively.

This analysis assumes that people choose passwords such as vxlag and msms as often as they pick enter and beer. However, people tend to choose names or words they can remember. Many computing systems have spelling checkers that can be used to check for spelling errors and typographic mistakes in documents. These spelling checkers sometimes carry online dictionaries of the most common English words. One contains a dictionary of 80,000 words. Trying all of these words as passwords takes only 80 seconds at the unrealistically generous estimate of one guess per millisecond.

Nobody knows what the most popular password is, although some conjecture it is “password.” Other common ones are user, abc123, aaaaaa (or aaaaa or aaaaaaa), 123456, and asdfg or qwerty (the arrangement of keys on a keyboard).

2025/2026 2025/2026 2025/2026

Common passwords—such as qwerty, password, 123456—are used astonishingly often.

Defeating Concealment

Easier than guessing a password is just to read one from a table, like the sample table shown in Table 2-2. The operating system authenticates a user by asking for a name and password, which it then has to validate, most likely by comparing to a value stored in a table. But that table then becomes a treasure trove for evil-doers: Obtaining the table gives access to all accounts because it contains not just one but all user IDs and their corresponding passwords.

Identity	Password
Jane	qwerty
Pat	aaaaaa
Phillip	oct31witch
Roz	aaaaaa
Herman	guessme
Claire	aq3wm$oto!4

TABLE 2-2 Sample Password Table

رومانی صبحی رزق جرجس مطر

Operating systems stymie that approach by storing passwords not in their public form but in a concealed form (using encryption, which we describe later in this chapter), as shown in Table 2-3. When a user creates a password, the operating system accepts and immediately conceals it, storing the unreadable version. Later when the user attempts to authenticate by entering a user ID and password, the operating system accepts whatever is typed, applies the same concealing function, and compares the concealed version with what is stored. If the two forms match, the authentication passes.

Identity	Password
Jane	0x471aa2d2
Pat	0x13b9c32f
Phillip	0x01c142be
Roz	0x13b9c32f
Herman	0x5202aae2
Claire	0x488b8c27

TABLE 2-3 Sample Password Table with Concealed Password Values

2025/2026 2025/2026 2025/2026

Operating systems store passwords in hidden (encrypted) form so that compromising the id–password list does not give immediate access to all user accounts.

We used the term “conceal” in the previous paragraph because sometimes the operating system performs some form of scrambling that is not really encryption, or sometimes it is a restricted form of encryption. The only critical point is that the process be one-way:

Converting a password to its concealment form is simple, but going the other way (starting with a concealed version and deriving the corresponding password) is effectively impossible. (For this reason, on some websites if you forget your password, the system can reset your password to a new, random value, but it cannot tell you what your forgotten password was.)

For active authentication, that is, entering identity and authenticator to be able to access a system, most systems lock out a user who fails a small number of successive login attempts. This failure count prevents an attacker from attempting more than a few guesses. (Note, however, that this lockout feature gives an attacker a way to prevent access by a legitimate user: simply enter enough incorrect passwords to cause the system to block the account.) However, if the attacker

obtains an encrypted password table and learns the concealment algorithm, a computer program can easily test hundreds of thousands of guesses in a matter of minutes.

As numerous studies in this chapter confirmed, people often use one of a few predictable passwords. The interceptor can create what is called a rainbow table, a list of the concealed forms of the common passwords, as shown in Table 2-4. Searching for matching entries in an intercepted password table, the intruder can learn that Jan's password is 123456 and Mike's is qwerty. The attacker sorts the table to make lookup fast.

Original Password	Encrypted Password
asdfg	0x023c94fc
p@55w0rd	0x04ff38d9
aaaaaa	0x13b9c32f
password	0x2129f30d
qwerty	0x471aa2d2
12345678	0x4f2c4dd8
123456	0x5903c34d

2025/2026 TABLE 2-4 Sample Rainbow Table for Common Passwords 2025/2026

Rainbow table: precomputed list of popular values, such as passwords

Scrambled passwords have yet another vulnerability. Notice in Table 2-2 that Pat and Roz both chose the same password. Both copies will have the same concealed value, so someone who intercepts the table can learn that users Pat and Roz have the same password. Knowing that, the interceptor can also guess that Pat and Roz both chose common passwords, and start trying the usual ones; when one works, the other will, too.

To counter both these threats, some systems use an extra piece called the salt. A salt is an extra data field different for each user, perhaps the date the account was created or a part of the user's name. The salt value is joined to the password before the combination is transformed by concealment. In this way, Pat+aaaaaa has a different concealment value from Roz+aaaaaa, as shown in Table 2-5. Also, an attacker cannot build a rainbow table because the common passwords now all have a unique component, too.

Identity	ID+password (not stored in table)	Stored Authentication Value
Jane	Jan+qwerty	0x1d46e346
Pat	Pat+aaaaaa	0x2d5d3e44
Phillip	Phi+oct31witch	0xc23c04d8
Roz	Roz+aaaaaa	0xe30f4d27
Herman	Her+guessme	0x8127f48d
Claire	Cla+aq3wm$oto!4	0x5209d942

TABLE 2-5 Sample Password Table with Personalized Concealed Password Values

Salt: user-specific component joined to an encrypted password to distinguish identical passwords

All these techniques to defeat passwords, combined with usability issues, indicate that we need to look for other methods of authentication. In the next section we explore how to implement strong authentication as a control against impersonation attacks. For another example of an authentication problem, see Sidebar 2-3.

Good Passwords

Chosen carefully, passwords can be strong authenticators. The term "password" implies a single word, but you can actually use a nonexistent word or a phrase. So 2Brn2Bti? could be a password (derived from "to be or not to be, that is the question") as could "PayTaxesApril15th." Note that these choices have several important characteristics: The strings are long, they are chosen from a large set of characters, and they do not appear in a dictionary. These properties make the password difficult (but, of course, not impossible) to determine. If we do use passwords, we can improve their security by a few simple practices:

- Use characters other than just a–z. If passwords are chosen from the letters a– z, there are only 26 possibilities for each character. Adding digits expands the number of possibilities to 36. Using both uppercase and lowercase letters plus digits expands the number of

possible characters to 62. Although this change seems small, the effect is large when someone is testing a full space of all possible combinations of characters. It takes about 100 hours to test all 6-letter words chosen from letters of one case only, but it takes about 2 years to test all 6-symbol passwords from upper- and lowercase letters and digits. Although 100 hours is reasonable, 2 years is oppressive enough to make this attack far less attractive.

Sidebar 2-3 Will the Real Earl of Buckingham Please Step Forward?

A man claiming to be the Earl of Buckingham was identified as Charlie Stopford, a man who had disappeared from his family in Florida in 1983 and assumed the identity of Christopher Buckingham, an 8-month-old baby who died in 1963. Questioned in England in 2005 after a check of passport details revealed the connection to the deceased Buckingham baby, Stopford was arrested when he didn't know other correlating family details [PAN06]. (His occupation at the time of his arrest? Computer security consultant.)

The British authorities knew he was not Christopher Buckingham, but what was his real identity? The answer was discovered only because his family in the United States thought it recognized him from photos and a news story: Stopford was a husband and father who had disappeared more than 20 years earlier. Because he had been in the U.S. Navy (in military intelligence, no less) and his adult fingerprints were on file, authorities were able to make a positive identification.

As for the title he appropriated for himself, there has been no Earl of Buckingham since 1687.

In modern society we are accustomed to a full paper trail documenting events from birth through death, but not everybody fits neatly into that model. Consider the case of certain people who for various reasons need to change their identity.

When the government changes someone's identity (for example, when a witness goes into hiding), the new identity includes school records, addresses, employment records, and so forth.

How can we authenticate the identity of war refugees whose home country may no longer exist, let alone civil government and a records office? What should we do to authenticate children born into nomadic tribes that keep no formal birth records? How does an adult confirm an identity after fleeing a hostile territory without waiting at the passport office for two weeks for a document?

- Choose long passwords. The combinatorial explosion of password guessing difficulty begins around length 4 or 5. Choosing longer passwords makes it less likely that a password will be uncovered. Remember that a brute force penetration can stop as soon as the password is

2025/2026 2025/2026 2025/2026

found. Some penetrators will try the easy cases—known words and short passwords—and move on to another target if those attacks fail.

- Avoid actual names or words. Theoretically, there are 26^6 or about 300 million 6-letter "words" (meaning any combination of letters), but there are only about 150,000 words in a good collegiate dictionary, ignoring length. By picking one of the 99.95 percent nonwords, you force the attacker to use a longer brute-force search instead of the abbreviated dictionary search.
- Use a string you can remember. Password choice is a double bind. To remember the password easily, you want one that has special meaning to you. However, you don't want someone else to be able to guess this special meaning. One easy-to-remember password is UcnB2s. That unlikely looking jumble is a simple transformation of "you can never be too secure." The first letters of words from a song, a few letters from different words of a private phrase, or something involving a memorable basketball score are examples of reasonable passwords. But don't be too obvious. Password-cracking tools also test replacements like 0 (zero) for o or O (letter "oh") and 1 (one) for l (letter "ell") or $ for S (letter "ess"). So H0v3U is already in the search file.
- Use variants for multiple passwords. With accounts, websites, and subscriptions, an individual can easily amass 50 or 100 passwords, which is clearly too many to remember. Unless you use a trick. Start with a phrase as in the previous suggestion: Ih1b2s (I have one brother, two sisters). Then append some patterns involving the first few vowels and consonants of the entity for the password: Ih1b2sIvs for vIsa, Ih1b2sAfc for fAcebook, and so forth.
- Change the password regularly. Even if you have no reason to suspect that someone has compromised the password, you should change it from time to time. A penetrator may break a password system by obtaining an old list or working exhaustively on an encrypted list.
- Don't write it down. Note: This time-honored advice is relevant only if physical security is a serious risk. People who have accounts on many machines and servers, and with many applications or sites, may have trouble remembering all the access codes. Setting all codes the same or using insecure but easy-to remember passwords may be more risky than writing passwords on a reasonably well-protected list. (Obviously, you should not tape your PIN to your bank card or post your password on your computer screen.)

2025/2026 2025/2026 2025/2026

• Don’t tell anyone else. The easiest attack is social engineering, in which the attacker contacts the system’s administrator or a user to elicit the password in some way. For example, the attacker may phone a user, claim to be “system administration,” and ask the user to verify the user’s password. Under no circumstances should you ever give out your private password; legitimate administrators can circumvent your password if need be, and others are merely trying to deceive you.

These principles lead to solid password selection, but they lead to a different problem:

People choose simple passwords because they have to create and remember so many passwords. Bank accounts, email access, library services, numerous websites, and other applications all seem to require a password. We cannot blame users for being tempted to use one simple password for all of them when the alternative is trying to remember dozens if not hundreds of strong passwords, as discussed in Sidebar 2-4.

Sidebar 2-4 Usability in the Small versus Usability in the Large

To an application developer seeking a reasonable control, a password seems to be a straightforward mechanism for protecting an asset. But when many applications require passwords, the user’s simple job of remembering one or two passwords is transformed into the nightmare of keeping track of a large number of them. Indeed, a visit to http://www.passwordbook.com suggests that users often have difficulty managing a collection of passwords. The site introduces you to a password and login organizer that is cheaply and easily purchased. In the words of the vendor, it is “The complete password manager for the busy Web master or network administrator … Safe and easy, books don’t crash! Now you can manage all your passwords in one hardbound book.”

Although managing one password or token for an application might seem easy (we call it “usability in the small”), managing many passwords or tokens at once becomes a daunting task (“usability in the large”). The problem of remembering a large variety of items has been documented in the psychology literature since the 1950s, when Miller [MIL56] pointed out that people remember things by breaking them into memorable chunks, whether they are digits, letters, words, or some other identifiable entity. Miller initially documented how young adults had a memory span of seven (plus or minus two) chunks. Subsequent research revealed that the memory span depends on the nature of the chunk: longer chunks led to shorter memory spans:

2025/2026 2025/2026 2025/2026

seven for digits, six for letters, and five for words. Other factors affect a person's memory span, too. Cowan [COW01] suggests that we assume a working memory span of four chunks for young adults, with shorter spans for children and senior citizens. For these reasons, usability should inform not only the choice of appropriate password construction (the small) but also the security architecture itself (the large).

Other Things Known

Passwords, or rather something only the user knows, are one form of strong authentication. Passwords are easy to create and administer, inexpensive to use, and easy to understand. However, users too often choose passwords that are easy for them to remember, but not coincidentally easy for others to guess. Also, users can forget passwords or tell them to others. Passwords come from the authentication factor of something the user knows, and unfortunately people's brains are imperfect.

Consequently, several other approaches to "something the user knows" have been proposed. For example, Sidebar 2-5 describes authentication approaches employing a user's knowledge instead of a password. However, few user knowledge authentication techniques have been well tested and few scale up in any useful way; these approaches are still being researched.

2025/2026 2025/2026 2025/2026

Sidebar 2-5 Using Personal Patterns for Authentication

Lamandé [LAM10] reports that the GrIDSure authentication system (http://www.gridsure.com) has been integrated into Microsoft's Unified Access Gateway (UAG) platform. This system allows a user to authenticate herself with a one-time passcode based on a pattern of squares chosen from a grid. When the user wishes access, she is presented with a grid containing randomly assigned numbers; she then enters as her passcode the numbers that correspond to her chosen pattern. Because the displayed grid numbers change each time the grid is presented, the pattern enables the entered passcode to be a one-time code. GrIDSure is an attempt to scale a "user knowledge" approach from usability in the small to usability in the large. Many researchers (see, for example, [SAS07, BON08, and BID09]) have examined aspects of GrIDSure's security and usability, with mixed results. It remains to be seen how the use of GrIDSure compares with the use of a collection of traditional passwords.

Similarly, the ImageShield product from Confident Technologies (www.confidenttechnologies.com) asks a user to enroll by choosing three categories from a list;

the categories might be cats, cars, and flowers, for example. Then at authentication time, the user is shown a grid of pictures, some from the user's categories and others not. Each picture has a 1-character letter or number. The user's one-time access string is the characters attached to the images from the user's preselected categories. So, if the pictures included a cat with label A, a flower with label 7, and seven other images, the user's access value would be A7. The images, characters and positions change for each access, so the authenticator differs similarly.

Authentication schemes like this are based on simple puzzles that the user can solve easily but that an imposter would be unable to guess successfully. However, with novel authentication schemes, we have to be aware of the phenomenon of usability in the small and the large: Can a user remember squares on a grid and categories of pictures and a favorite vacation spot and the formula 2a+c and many other nonobvious things?

Security Questions

Instead of passwords, some companies use questions to which (presumably) only the right person would know the answer. Such questions include mother's maiden name, street name from childhood, model of first automobile, and name of favorite teacher. The user picks relevant questions and supplies the answers when creating an identity.

2025/2026 2025/2026 2025/2026

The problem with such questions is that the answers to some can be determined with little difficulty, as was the case for Sarah Palin's email account. With the number of public records available online, mother's maiden name and street name are often available, and school friends could guess a small number of possible favorite teachers. Anitra Babic and colleagues [BAB09] documented the weakness of many of the supposedly secret question systems in current use. Joseph Bonneau and Sören Preibusch [BON10] did a detailed survey of website authentication methods and found little uniformity, many weaknesses, and no apparent correlation between the value of a site's data and its authentication requirements.

Passwords are becoming oppressive as many websites now ask users to log in. But when faced with a system that is difficult to handle, users often take the easy route: choosing an easy password and reusing it on many sites. To overcome that weakness, some systems use a form of authentication that cannot be stolen, duplicated, forgotten, lent, or lost: properties of the user, as we discuss in the next section. The technology for passing personal characteristics to a remote

رومانی صبحی رزق جرجس مطر

server requires more than a keyboard and pointing device, but such approaches are becoming more feasible, especially as password table breaches increase.

Authentication Based on Biometrics: Something You Are

Biometrics are biological properties, based on some physical characteristic of the human body. The list of biometric authentication technologies is still growing. Now devices can recognize the following biometrics:

- fingerprint
- hand geometry (shape and size of fingers)
- retina and iris (parts of the eye)
- voice
- handwriting, signature, hand motion
- typing characteristics
- blood vessels in the finger or hand
- face
- facial features, such as nose shape or eye spacing

2025/2026 2025/2026 2025/2026

Authentication with biometrics has advantages over passwords because a biometric cannot be lost, stolen, forgotten, or shared and is always available, always at hand, so to speak. These characteristics are difficult, if not impossible, to forge.

Examples of Biometric Authenticators

Many physical characteristics are possibilities as authenticators. In this section we present examples of two of them, one for the size and shape of the hand, and one for the patterns of veins in the hand.

Figure 2-2 shows a hand geometry reader. The user places a hand on the sensors, which detect lengths and widths of fingers, curvature, and other characteristics.

رومانی صبحی رزق جرجس مطر

رومانی صبحی رزق جرجس مطر

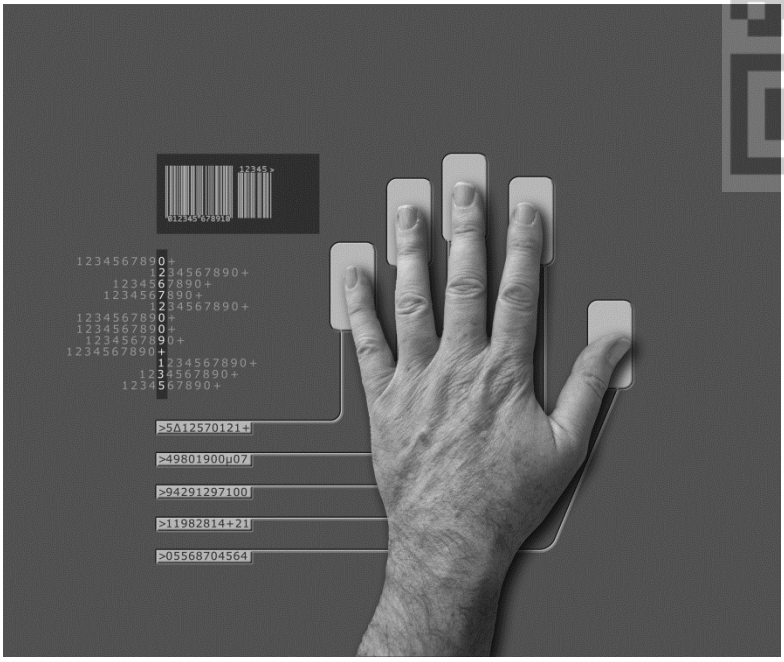


FIGURE 2-2 Hand Geometry Reader (Graeme Dawes/Shutterstock)

An authentication device from Fujitsu reads the pattern of veins in the hand. This device does not require physical contact between the hand and the reader, which is an advantage for hygiene. The manufacturer claims a false acceptance rate of 0.00008 percent and false rejection rate of 0.01 percent, with a response time of less than one second. Figure 2-3 shows this device embedded in a computer mouse, so the user is automatically authenticated.



FIGURE 2-3 Hand Vein Reader (Permission for image provided courtesy of Fujitsu

Problems with Use of Biometrics

Biometrics come with several problems:

رومانی صبحی رزق جرجس مطر

- Biometrics are relatively new, and some people find their use intrusive. For example, people in some cultures are insulted by having to submit to fingerprinting, because they think that only criminals are fingerprinted. Hand geometry and face recognition (which can be done from a camera across the room) are scarcely invasive, but people have real concerns about peering into a laser beam or sticking a finger into a slot. (See [SCH06a] for some examples of people resisting biometrics.)
- Biometric recognition devices are costly, although as the devices become more popular, their cost per device should go down. Still, outfitting every user's workstation with a reader can be expensive for a large company with many employees.
- Biometric readers and comparisons can become a single point of failure. Consider a retail application in which a biometric recognition is linked to a payment scheme: As one user puts it, "If my credit card fails to register, I can always pull out a second card, but if my fingerprint is not recognized, I have only that one finger." (Fingerprint recognition is specific to a single finger; the pattern of one finger is not the same as another.) Manual laborers can actually rub off their fingerprints over time, and a sore or irritation may confound a fingerprint reader. Forgetting a password is a user's fault; failing biometric authentication is not.
- All biometric readers use sampling and establish a threshold for acceptance of a close match. The device has to sample the biometric, measure often hundreds of key points, and compare that set of measurements with a template. Features vary slightly from one reading to the next, for example, if your face is tilted, if you press one side of a finger more than another, or if your voice is affected by a sinus infection. Variation reduces accuracy.
- Although equipment accuracy is improving, false readings still occur. We label a false positive or false accept a reading that is accepted when it should be rejected (that is, the authenticator does not match) and a false negative or false reject one that rejects when it should accept. Often, reducing a false positive rate increases false negatives, and vice versa. Sidebar 2-6 explains why we can never eliminate all false positives and negatives. The consequences for a false negative are usually less than for a false positive, so an acceptable system may have a false positive rate of 0.001 percent but a false negative rate of 1 percent. However, if the population is large and the asset extremely valuable, even these small percentages can lead to catastrophic results.

False positive: incorrectly confirming an identity.

False negative: incorrectly denying an identity.

Biometric matches are not exact; the issue is whether the rate of false positives and false negatives is acceptable.

Biometric authentication means a subject matches a template closely enough. “Close” is a system parameter that can be tuned.

رومانی صبحی رزق جرجس مطر

Authentication Based on Tokens: Something You Have

Something you have means that you have a physical object in your possession. One physical authenticator with which you are probably familiar is a key. When you put your key in your lock, the ridges in the key interact with pins in the lock to let the mechanism turn. In a sense the lock authenticates you for authorized entry because you possess an appropriate key. Of course, you can lose your key or duplicate it and give the duplicate to someone else, so the authentication is not perfect. But it is precise: Only your key works, and your key works only your lock. (For this example, we intentionally ignore master keys.)

2025/2026 2025/2026 2025/2026

Other familiar examples of tokens are badges and identity cards. You may have an “affinity card”: a card with a code that gets you a discount at a store. Many students and employees have identity badges that permit them access to buildings. You must have an identity card or passport to board an airplane or enter a foreign country. In these cases you possess an object that other people recognize to allow you access or privileges.

Another kind of authentication token has data to communicate invisibly. Examples of this kind of token include credit cards with a magnetic stripe, credit cards with an embedded computer chip, or access cards with passive or active wireless technology. You introduce the token into an appropriate reader, and the reader senses values from the card. If your identity and values from your token match, this correspondence adds confidence that you are who you say you are.

We describe different kinds of tokens next.

Active and Passive Tokens

As the names imply, ***passive tokens*** do nothing, and active ones take some action. A photo or key is an example of a passive token in that the contents of the token never change. (And, of

course, with photos permanence can be a problem, as people change hair style or color and their faces change over time.)

An ***active token*** can have some variability or interaction with its surroundings. For example, some public transportation systems use cards with a magnetic strip. When you insert the card into a reader, the machine reads the current balance, subtracts the price of the trip and rewrites a new balance for the next use. In this case, the token is just a repository to hold the current value. Another form of active token initiates a two-way communication with its reader, often by wireless or radio signaling. These tokens lead to the next distinction among tokens, static and dynamic interaction.

Passive tokens do not change. Active tokens communicate with a sensor.

Static and Dynamic Tokens

The value of a ***static token*** remains fixed. Keys, identity cards, passports, credit and other magnetic-stripe cards, and radio transmitter cards (called RFID devices) are examples of static tokens. Static tokens are most useful for onsite authentication: When a guard looks at your picture badge, the fact that you possess such a badge and that your face looks (at least vaguely) like the picture causes the guard to pass your authentication and allow you access.

2025/2026 2025/2026 2025/2026

We are also interested in remote authentication, that is, in your being able to prove your identity to a person or computer somewhere else. With the example of the picture badge, it may not be easy to transmit the image of the badge and the appearance of your face for a remote computer to compare. Worse, distance increases the possibility of forgery: A local guard could tell if you were wearing a mask, but a guard might not detect it from a remote image. Remote authentication is susceptible to the problem of the token having been forged. Tokens are vulnerable to an attack called skimming. Skimming is the use of a device to copy authentication data surreptitiously and relay it to an attacker. Automated teller machines (ATMs) and point-of-sale credit card readers are particularly vulnerable to skimming.

At an ATM the thief attaches a small device over the slot into which you insert your bank card. Because all bank cards conform to a standard format (so you can use your card at any ATM or merchant), the thief can write a simple piece of software to copy and retain the information recorded on the magnetic stripe on your bank card. Some skimmers also have a tiny camera to record your key strokes as you enter your PIN on the keypad. Either instantaneously (using

رومانی صبحی رزق جرجس مطر

wireless communication) or later (collecting the physical device), the thief thus obtains both your account number and its PIN. The thief simply creates a dummy card with your account number recorded and, using the PIN for authentication, visits an ATM and withdraws cash from your account or purchases things with a cloned credit card.

1. Note that this discussion refers to the magnetic-stripe cards popular in the United States. Most other countries use embedded computer chip cards that are substantially less vulnerable to skimming.

Another form of copying occurs with passwords. If you have to enter or speak your password, someone else can look over your shoulder or overhear you, and now that authenticator is easily copied or forged. To overcome copying of physical tokens or passwords, we can use dynamic tokens. A dynamic token is one whose value changes. Although there are several different forms, a dynamic authentication token is essentially a device that generates an unpredictable value that we might call a pass number. Some devices change numbers at a particular interval, for example, once a minute; others change numbers when you press a button, and others compute a new number in response to an input, sometimes called a challenge. In all cases, it does not matter if someone else sees or hears you provide the pass number, because that one value will be valid for only one access (yours), and knowing that one value will not allow the outsider to guess or generate the next pass number.

2025/2026 2025/2026 2025/2026

Dynamic tokens have computing power on the token to change their internal state.

Dynamic token generators are useful for remote authentication, especially of a person to a computer. An example of a dynamic token is the SecurID token from RSA Laboratories, shown in Figure 2-6. To use a SecurID token, you enter the current number displayed on the token when prompted by the authenticating application. Each token generates a distinct, virtually unpredictable series of numbers that change every minute, so the authentication system knows what number to expect from your token at any moment. In this way, two people can have SecurID tokens, but each token authenticates only its assigned owner. Entering the number from another token does not pass your authentication. And because the token generates a new number every minute, entering the number from a previous authentication fails as well.

رومانی صبحی رزق جرجس مطر

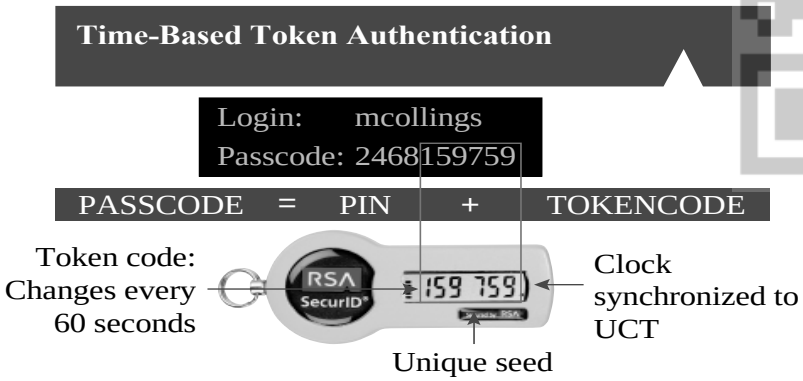


FIGURE 2-6 SecurID Token (Photo courtesy of RSA, the security division of EMS and copyright © RSA Corporation, all rights reserved.)

We have now examined the three bases of authentication: something you know, are, or have. Used in an appropriate setting, each can offer reasonable security. In the next sections we look at some ways of enhancing the basic security from these three forms.

Federated Identity Management

If these different forms of authentication seem confusing and overwhelming, they can be. Consider that some systems will require a password, others a fingerprint scan, others an active token, and others some combination of techniques. As you already know, remembering identities and distinct passwords for many systems is challenging. People who must use several different systems concurrently (email, customer tracking, inventory, and sales, for example) soon grow weary of logging out of one, into another, refreshing a login that has timed out, and creating and updating user profiles. Users rightly call for computers to handle the bookkeeping.

A federated identity management scheme is a union of separate identification and authentication systems. Instead of maintaining separate user profiles, a federated scheme maintains one profile with one authentication method. Separate systems share access to the authenticated identity database. Thus, authentication is performed in one place, and separate processes and systems determine that an already authenticated user is to be activated. Such a process is shown in Figure 2-7.

Federated identity management unifies the identification and authentication process for a group of systems.

2025/2026 2025/2026 2025/2026

Closely related is a single sign-on process, depicted in Figure 2-8. Think of an umbrella procedure to which you log in once per session (for example, once a day). The umbrella procedure maintains your identities and authentication codes for all the different processes you access. When you want to access email, for example, instead of your completing a user ID and password screen, the single sign-on process passes those details to the email handler, and you resume control after the authentication step has succeeded.

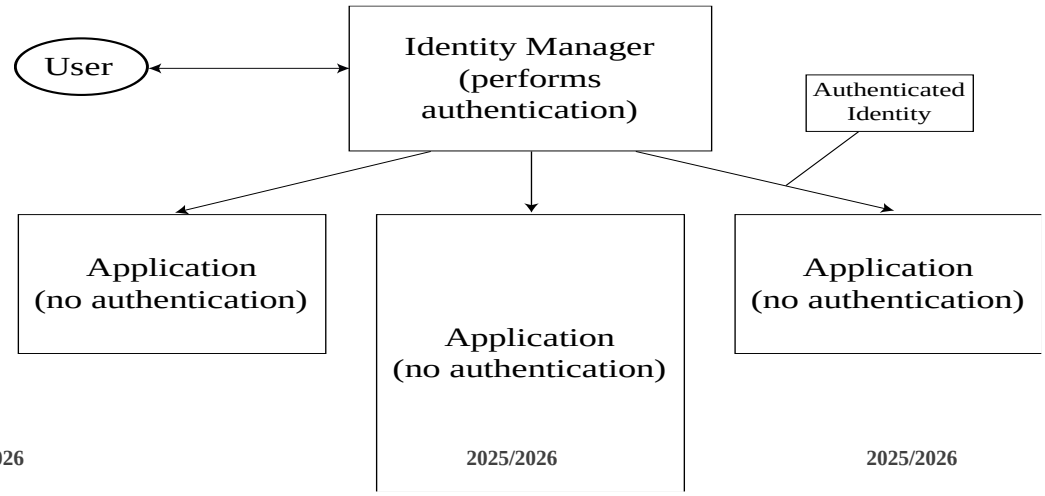


FIGURE 2-7 Federated Identity Manager

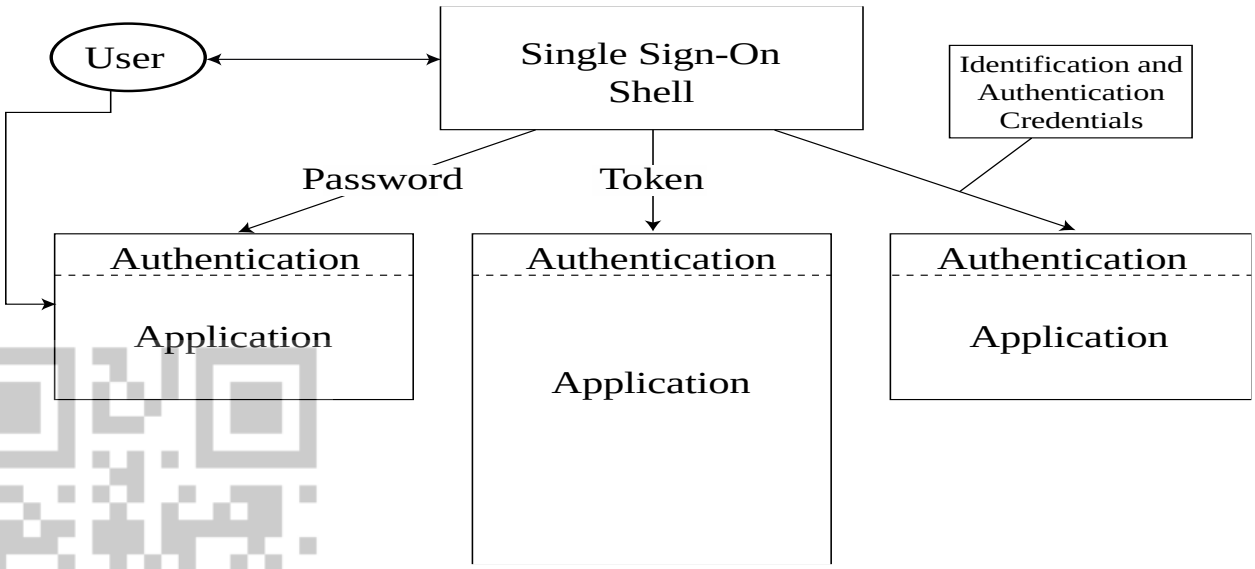


FIGURE 2-8 Single Sign-On

رومانی صبحی رزق جرجس مطر

The difference between these two approaches is that federated identity management involves a single identity management module that replaces identification and authentication in all other systems. Thus all these systems invoke the identity management module. With single sign-on, systems still call for individual identification and authentication, but the umbrella task performs those interactions on behalf of the user.

Single sign-on takes over sign-on and authentication to several independent systems for a user.

Secure Authentication

Passwords, biometrics, and tokens can all participate in secure authentication. Of course, simply using any or all of them is no guarantee that an authentication approach will be secure. To achieve true security, we need to think carefully about the problem we are trying to solve and the tools we have; we also need to think about blocking possible attacks and attackers.

2.2 Access Control

2025/2026 2025/2026 2025/2026

In this section we discuss how to protect general objects, such as files, tables, access to hardware devices or network connections, and other resources. In general, we want a flexible structure, so that certain users can use a resource in one way (for example, read only), others in a different way (for example, allowing modification), and still others not at all. We want techniques that are robust, easy to use, and efficient.

We start with the basic access control paradigm, articulated by Scott Graham and Peter Denning [GRA72]: A subject is permitted to access an object in a particular mode, and only such authorized accesses are allowed.

- Subjects are human users, often represented by surrogate programs running on behalf of the users.
- Objects are things on which an action can be performed: Files, tables, programs, memory objects, hardware devices, strings, data fields, network connections, and processors are examples of objects. So too are users, or rather programs or processes representing users, because the operating system (a program representing the system administrator) can act on a

رومانی صبحی رزق جرجس مطر

user, for example, allowing a user to execute a program, halting a user, or assigning privileges to a user.

- Access modes are any controllable actions of subjects on objects, including, but not limited to, read, write, modify, delete, execute, create, destroy, copy, export, import, and so forth.

Effective separation will keep unauthorized subjects from unauthorized access to objects, but the separation gap must be crossed for authorized subjects and modes. In this section we consider ways to allow all and only authorized accesses.

Access control: limiting who can access what in what ways

Access Policies

Access control is a mechanical process, easily implemented by a table and computer process: A given subject either can or cannot access a particular object in a specified way. Underlying the straightforward decision is a complex and nuanced decision of which accesses should be allowed; these decisions are based on a formal or informal security policy.

Access control decisions are (or should not be) made capriciously. Pat gets access to this file because she works on a project that requires the data; Sol is an administrator and needs to be able to add and delete users for the system. Having a basis simplifies making similar decisions for other users and objects. A policy also simplifies establishing access control rules, because they just reflect the existing policy.

2025/2026 2025/2026 2025/2026

Thus, before trying to implement access control, an organization needs to take the time to develop a higher-level security policy, which will then drive all the access control rules.

Effective Policy Implementation

Protecting objects involves several complementary goals.

- Check every access. We may want to revoke a user's privilege to access an object. If we have previously authorized the user to access the object, we do not necessarily intend that the user should retain indefinite access to the object. In fact, in some situations, we may want to prevent further access immediately after we revoke authorization, for example, if we detect a user being impersonated. For this reason, we should aim to check every access by a user to an object.

رومانی صبحی رزق جرجس مطر

رومانی صبحی رزق جرجس مطر

- Enforce least privilege. The principle of least privilege states that a subject should have access to the smallest number of objects necessary to perform some task. Even if extra information would be useless or harmless if the subject were to have access, the subject should not have that additional access. For example, a program should not have access to the absolute memory address to which a page number reference translates, even though the program could not use that address in any effective way. Not allowing access to unnecessary objects guards against security weaknesses if a part of the protection mechanism should fail.

Least privilege: access to the fewest resources necessary to complete some task

- Verify acceptable usage. Ability to access is a yes-or-no decision. But equally important is checking that the activity to be performed on an object is appropriate. For example, a data structure such as a stack has certain acceptable operations, including push, pop, clear, and so on. We may want not only to control who or what has access to a stack but also to be assured that all accesses performed are legitimate stack accesses.

Access Log

After making an access decision, the system acts to allow that access and leaves the user and the object to complete the transaction. Systems also record which accesses have been permitted, creating what is called an audit log. This log is created and maintained by the system, and it is preserved for later analysis. Several reasons for logging access include the following:

2025/2026 2025/2026 2025/2026

- Records of accesses can help plan for new or upgraded equipment, by showing which items have had heavy use.
- If the system fails, these records can show what accesses were in progress and perhaps help identify the cause of failure.
- If a user misuses objects, the access log shows exactly which objects the user did access.
- In the event of an external compromise, the audit log may help identify how the assailant gained access and which data items were accessed (and therefore revealed or compromised). These data for after-the-fact forensic analysis have been extremely helpful in handling major incidents.

As part of the access control activity, the system builds and protects this audit log. Obviously, granularity matters: A log that records each memory byte accessed is too lengthy to be of much practical value, but a log that says “8:01 user turned on system; 17:21 user turned off system” probably contains too little detail to be helpful.

Implementing Access Control

Access control is often performed by the operating system. Only the operating system can access primitive objects, such as files, to exercise control over them, and the operating system creates and terminates the programs that represent users (subjects). However, current hardware design does not always support the operating system in implementing well-differentiated or fine-grained access control. The operating system does not usually see inside files or data objects, for example, so it cannot perform row- or element-level access control within a database. Also, the operating system cannot easily differentiate among kinds of network traffic. In these cases, the operating system defers to a database manager or a network appliance in implementing some access control aspects. With limited kinds of privileges to allocate, the operating system cannot easily both control a database manager and allow the database manager to control users. Thus, current hardware design limits some operating system designs.

Reference Monitor

James Anderson and his study committee [AND72] gave name and structure to the digital version of a concept that has existed for millennia. To protect their medieval fortresses, rulers had one heavily protected gate as the sole means of ingress. Generals surrounded troop emplacements with forts and sentry guards. Bankers kept cash and other valuables in safes with impregnable doors to which only a select few trusted people had the combinations. Fairy tale villains locked damsels away in towers. All these examples show strong access control because of fail-safe designs.

2025/2026 2025/2026 2025/2026

In Anderson's formulation for computers, access control depends on a combination of hardware and software that is

- always invoked; validates every access attempt
- immune from tampering
- assuredly correct

Reference monitor: access control that is always invoked, tamperproof, and verifiable

Anderson called this construct a reference monitor. It should be obvious why these three properties are essential.

رومانی صبحی رزق جرجس مطر

A reference monitor is a notion, not a tool you can buy to plug into a port. It could be embedded in an application (to control the application's objects), part of the operating system (for system-managed objects) or part of an appliance. Still, you will see these same three properties appear repeatedly in this book. To have an effective reference monitor, we need to consider effective and efficient means to translate policies, the basis for validation, into action. How we represent a policy in binary data has implications for how efficient and even how effective the mediation will be.

In the next sections we present several models of how access rights can be maintained and implemented by the reference monitor.

Access Control Directory

One simple way to protect an object is to use a mechanism that works like a file directory. Imagine we are trying to protect files (the set of objects) from users of a computing system (the set of subjects). Every file has a unique owner who possesses "control" access rights (including the rights to declare who has what access) and to revoke access of any person at any time. Each user has a file directory, which lists all the files to which that user has access.

2025/2026 2025/2026 2025/2026

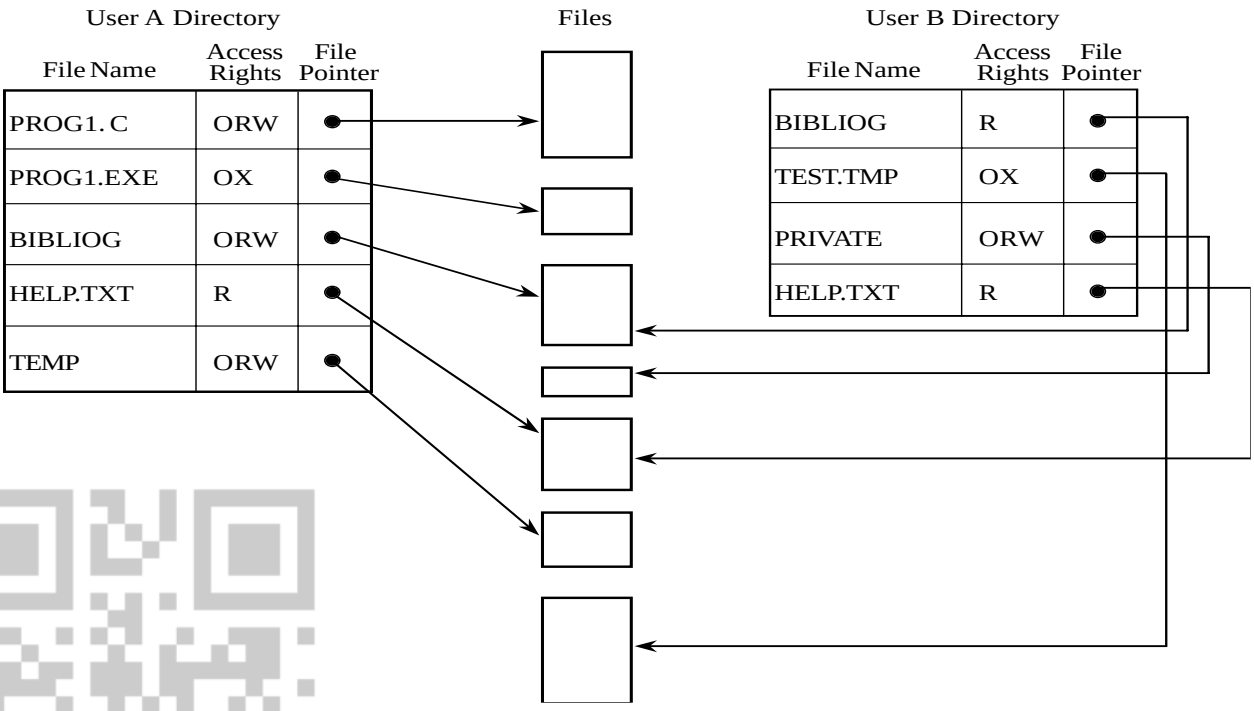


FIGURE 2-9 Directory Access Rights

رومانی صبحی رزق جرجس مطر

Clearly, no user can be allowed to write in the file directory, because that would be a way to forge access to a file. Therefore, the operating system must maintain all file directories, under commands from the owners of files. The obvious rights to files are the common read, write, and execute that are familiar on many shared systems. Furthermore, another right, owner, is possessed by the owner, permitting that user to grant and revoke access rights. Figure 2-9 shows an example of a file directory.

This approach is easy to implement because it uses one list per user, naming all the objects that a user is allowed to access. However, several difficulties can arise. First, the list becomes too large if many shared objects, such as libraries of subprograms or a common table of users, are accessible to all users. The directory of each user must have one entry for each such shared object, even if the user has no intention of accessing the object. Deletion must be reflected in all directories.

A second difficulty is revocation of access. If owner A has passed to user B the right to read file F, an entry for F is made in the directory for B. This granting of access implies a level of trust between A and B. If A later questions that trust, A may want to revoke the access right of B. The operating system can respond easily to the single request to delete the right of B to access F, because that action involves deleting one entry from a specific directory. But if A wants to remove the rights of everyone to access F, the operating system must search each individual directory for the entry F, an activity that can be time consuming on a large system. For example, large systems or networks of smaller systems can easily have 5,000 to 10,000 active accounts. Moreover, B may have passed the access right for F to another user C, a situation known as propagation of access rights, so A may not know that C's access exists and should be revoked. This problem is particularly serious in a network.

2025/2026 2025/2026 2025/2026

A third difficulty involves pseudonyms. Owners A and B may have two different files named F, and they may both want to allow access by S. Clearly, the directory for S cannot contain two entries under the same name for different files. Therefore, S has to be able to uniquely identify the F for A (or B). One approach is to include the original owner's designation as if it were part of the file name, with a notation such as A:F (or B:F).

Suppose, however, that S would like to use a name other than F to make the file's contents more apparent. The system could allow S to name F with any name unique to the directory of S. Then, F from A could be called Q to S. As shown in Figure 2-10, S may have forgotten that Q is F from

رومانی صبحی رزق جرجس مطر

A, and so S requests access again from A for F. But by now A may have more trust in S, so A transfers F with greater rights than before. This action opens up the possibility that one subject, S, may have two distinct sets of access rights to F, one under the name Q and one under the name F. In this way, allowing pseudonyms can lead to multiple permissions that are not necessarily consistent. Thus, the directory approach is probably too simple for most object protection situations.

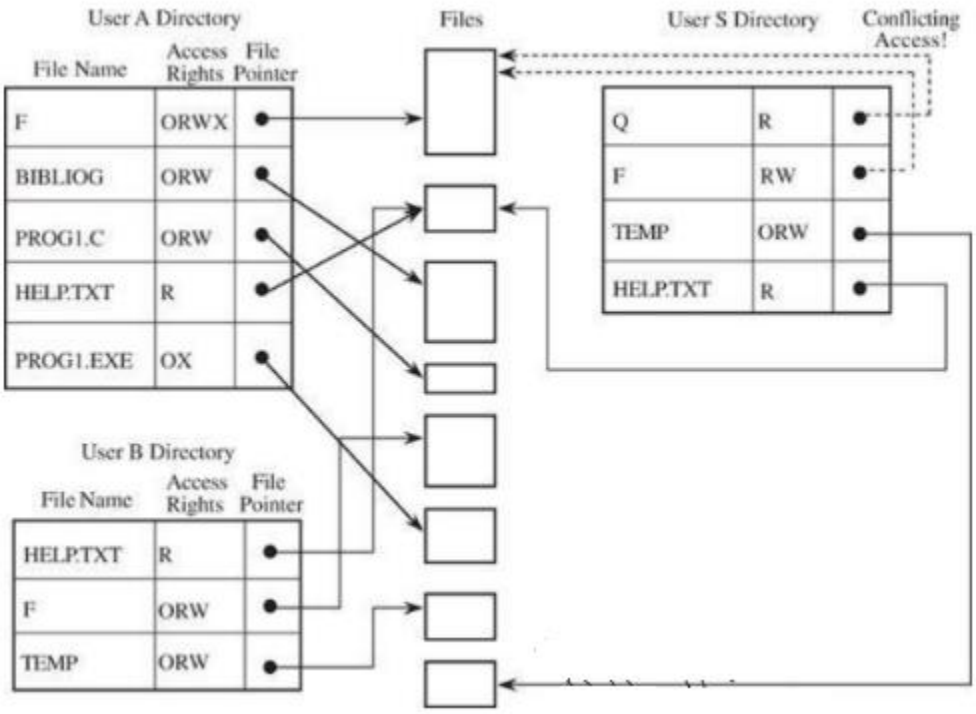


FIGURE 2-10 Ambiguous Access Rights

Access Control Matrix

We can think of the directory as a listing of objects accessible by a single subject, and the access list as a table identifying subjects that can access a single object. The data in these two representations are equivalent, the distinction being the ease of use in given situations.

As an alternative, we can use an access control matrix, shown in Figure 2-11, a table in which each row represents a subject, each column represents an object, and each entry is the set of access rights for that subject to that object.

2025/202

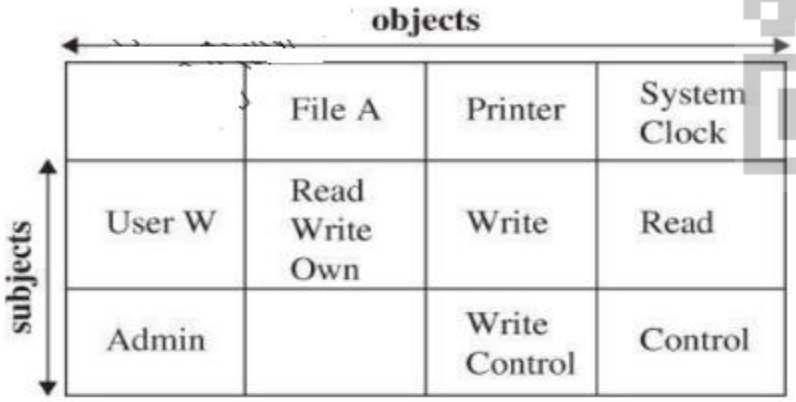


FIGURE 2-11 Access Control Matrix

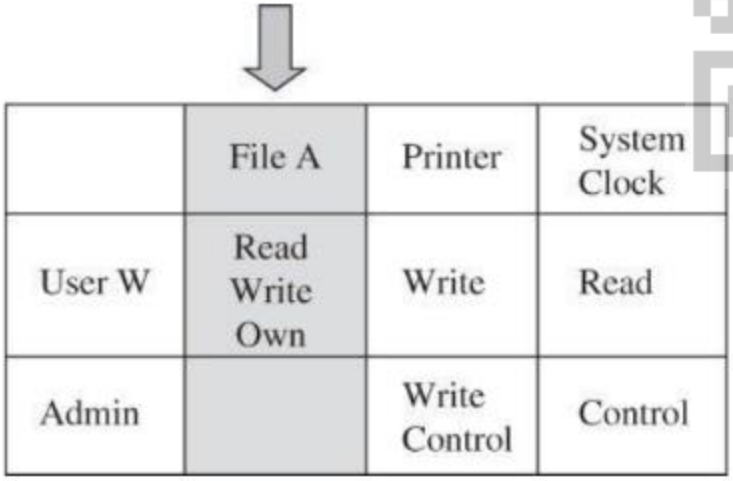
A more detailed example representation of an access control matrix is shown in Table 2- 8. Access rights shown in that table are O, own; R, read; W, write; and X, execute.

	BIBLIOG	TEMP	F	HELP.TXT	C_COMP	LINKER	SYS_CLOCK	PRINTER
USER A	ORW	ORW	ORW	R	X	X	R	W
USER B	R	-	-	R	X	X	R	W
USER S	RW	-	R	R	X	X	R	W
USER T	-	-	-	R	X	X	R	W
SYS_MGR	-	-	-	RW	OX	OX	ORW	O
USER_SVCS	-	-	-	O	X	X	R	W

TABLE 2-8 Access Control Matrix

Access Control List

An alternative representation is the access control list; as shown in Figure 2-12, this representation corresponds to columns of the access control matrix. There is one such list for each object, and the list shows all subjects who should have access to the object and what their access is. This approach differs from the directory list because there is one access control list per object; a directory is created for each subject. Although this difference seems small, there are some significant advantages to this approach.



	File A	Printer	System Clock
User W	Read Write Own	Write	Read
Admin		Write Control	Control

FIGURE 2-12 Access Control List

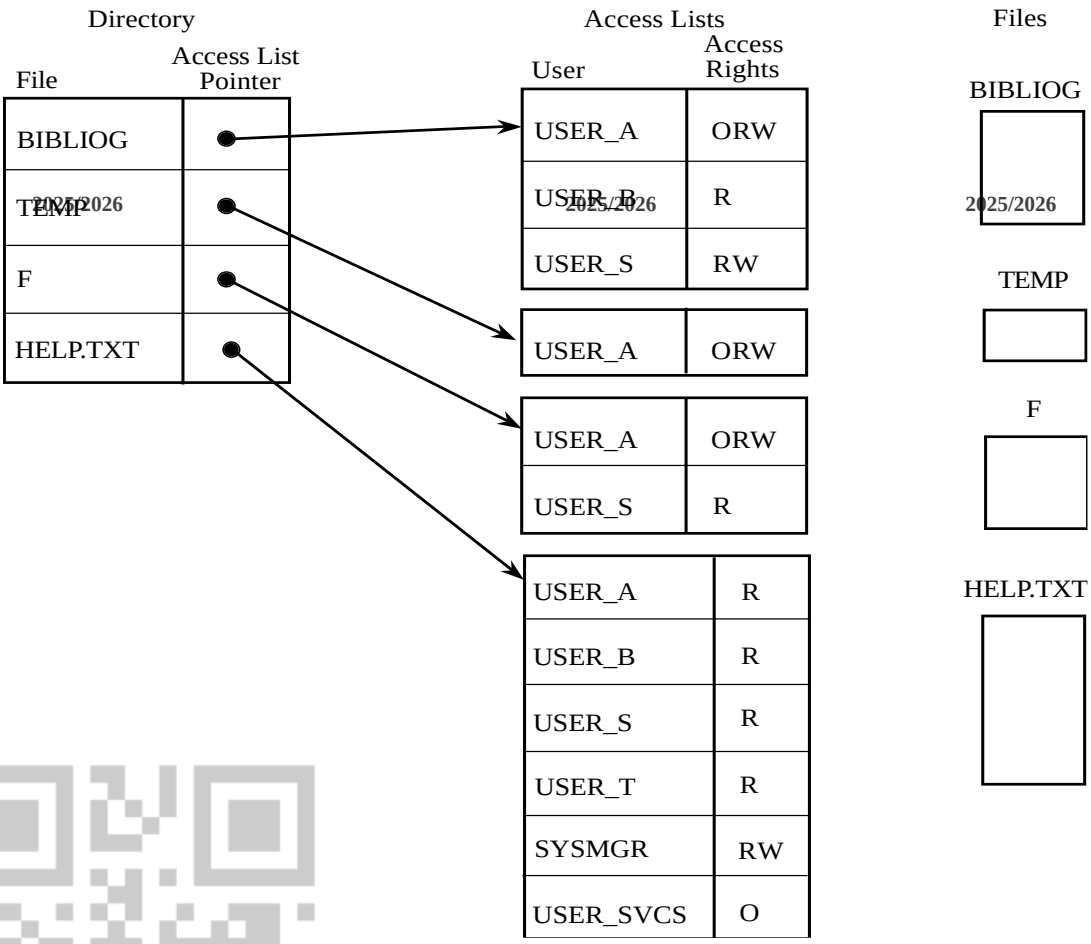


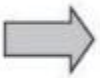
FIGURE 2-13 Access Control List with Two Subjects

The access control list representation can include default rights. Consider subjects A and S, both of whom have access to object F. The operating system will maintain just one access list for F, showing the access rights for A and S, as shown in Figure 2-13. The access control list can include general default entries for any users. In this way, specific users can have explicit rights, and all other users can have a default set of rights. With this organization, all possible users of the system can share a public file or program without the need for an entry for the object in the individual directory of each user.

Privilege List

A privilege list, sometimes called a directory, is a row of the access matrix, showing all those privileges or access rights for a given subject (shown in Figure 2-14). One advantage of a privilege list is ease of revocation: If a user is removed from the system, the privilege list shows all objects to which the user has access so that those rights can be removed from the object.

2025/2026



	File A	Printer	System Clock
User W	Read Write Own	Write	Read
Admin		Write Control	Control

FIGURE 2-14 Privilege Control List

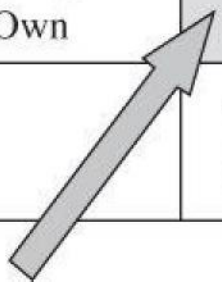
Capability

So far, we have examined protection schemes in which the operating system must keep track of all the protection objects and rights. But other approaches put some of the burden on the user. For example, a user may be required to have a ticket or pass that enables access, much like a ticket or identification card that cannot be duplicated.

More formally, we say that a capability is an unforgeable token that gives the possessor certain rights to an object. The Multics [SAL74], CAL [LAM76], and Hydra [WUL74] systems used capabilities for access control. As shown in Figure 2-15, a capability is just one access control triple of a subject, object, and right. In theory, a subject can create new objects and can specify

the operations allowed on those objects. For example, users can create objects such as files, data segments, or subprocesses and can also specify the acceptable kinds of operations, such as read, write, and execute. But a user can also create completely new objects, such as new data structures, and can define types of accesses previously unknown to the system

	File A	Printer	System Clock
User W	Read Write Own	Write	Read
Admin		Write Control	Control



2025/2026

FIGURE 2-15 Capability

2025/2026 2025/2026

Capability: Single- or multi-use ticket to access an object or service

Procedure-Oriented Access Control

One goal of access control is restricting not just what subjects have access to an object, but also what they can do to that object. Read versus write access can be controlled rather readily by most applications and operating systems, but more complex control is not so easy to achieve.

By ***procedure-oriented*** protection, we imply the existence of a procedure that controls access to objects (for example, by performing its own user authentication to strengthen the basic protection provided by the basic operating system). In essence, the procedure forms a capsule around the object, permitting only certain specified accesses.

Procedures can perform actions specific to a particular object in implementing access control.

Procedures can ensure that accesses to an object be made through a trusted interface. For example, neither users nor general operating system routines might be allowed direct access to the table of valid users. Instead, the only accesses allowed might be through three procedures:

رومانی صبحی رزق جرجس مطر

one to add a user, one to delete a user, and one to check whether a particular name corresponds to a valid user. These procedures, especially add and delete, could use their own checks to make sure that calls to them are legitimate.

Procedure-oriented protection implements the principle of information hiding because the means of implementing an object are known only to the object's control procedure. Of course, this degree of protection carries a penalty of inefficiency. With procedure-oriented protection, there can be no simple, fast access checking, even if the object is frequently used.

Role-Based Access Control

We have not yet distinguished among kinds of users, but we want some users (such as administrators) to have significant privileges, and we want others (such as regular users or guests) to have lower privileges. In companies and educational institutions, this can get complicated when an ordinary user becomes an administrator or a baker moves to the candlestick makers' group. Role-based access control lets us associate privileges with groups, such as all administrators can do this or candlestick makers are forbidden to do that. Administering security is easier if we can control access by job demands, not by person. Access control keeps up with a person who changes responsibilities, and the system administrator does not have to choose the appropriate access control settings for someone. For more details on the nuances of role-based access control, see [FER03].

2025/2026 2025/2026 2025/2026

Access control by role recognizes common needs of all members of a set of subjects.

2.3 Cryptography

Next we introduce the third of our basic security tools, cryptography. In this chapter we present only the rudiments of the topic, just enough so you can see how it can be used and what it can achieve. We leave the internals for Chapter 12 at the end of this book. We do that because most computer security practitioners would be hard-pressed to explain or implement good cryptography from scratch, which makes our point that you do not need to understand the internals of cryptography just to use it successfully. As you read this chapter you may well ask why something is done in a particular way or how something really works. We invite you to jump to Chapter 12 for the details. But this chapter focuses on the tool and its uses, leaving the internal workings for the future.

رومانی صبحی رزق جریس مطر

رومانی صبحی رزق جرجس مطر

Encryption or cryptography—the name means secret writing—is probably the strongest defense in the arsenal of computer security protection. Well-disguised data cannot easily be read, modified, or fabricated. Simply put, encryption is like a machine: you put data in one end, gears spin and lights flash, and you receive modified data out the other end. In fact, some encryption devices used during World War II operated with actual gears and rotors, and these devices were effective at deterring (although not always preventing) the opposite side from reading the protected messages. Now the machinery has been replaced by computer algorithms, but the principle is the same: A transformation makes data difficult for an outsider to interpret.

Cryptography conceals data against unauthorized access.

We begin by examining what encryption does and how it works. We introduce the basic principles of encryption algorithms, introducing two types of encryption with distinct uses. Because weak or flawed encryption creates only the illusion of protection, we also look at how encryption can fail. We briefly describe techniques used to break through the protective cover to void security. Building on these basic types of encryption, we show how to combine them to securely address several general problems of computing and communicating.

Problems Addressed by Encryption

Sometimes we describe encryption in the context of sending secret messages. This framing is just for ease of description: The same concepts apply to protecting a file of data or sensitive information in memory. So when we talk about sending a message, you should also think of protecting any digital object for access only by authorized people.

Consider the steps involved in sending messages from a ***sender***, S, to ***a recipient***, R. If S entrusts the message to T, who then delivers it to R, T then becomes the ***transmission medium***. If an outsider, O, wants to access the message (to read, change, or even destroy it), we call O an ***interceptor or intruder***. Any time after S transmits the message via T, it is vulnerable to exploitation, and O might try to access it in any of the following ways:

- block it, by preventing its reaching R, thereby affecting the availability of the message
- intercept it, by reading or listening to the message, thereby affecting the confidentiality of the message

- modify it, by seizing the message and changing it in some way, affecting the message's integrity
- fabricate an authentic-looking message, arranging for it to be delivered as if it came from S, thereby also affecting the integrity of the message

As you can see, a message's vulnerabilities reflect the four possible security failures we identified in Chapter 1. Fortunately, encryption is a technique that can address all these problems. Encryption is a means of maintaining secure data in an insecure environment. In this book, we study encryption as a security technique, and we see how it is used in protecting programs, databases, networks, and electronic communications.

Terminology

Encryption is the process of encoding a message so that its meaning is not obvious; ***decryption*** is the reverse process, transforming an encrypted message back into its normal, original form. Alternatively, the terms ***encode and decode*** or ***encipher and decipher*** are used instead of encrypt and decrypt. That is, we say we encode, encrypt, or encipher the original message to hide its meaning. Then, we decode, decrypt, or decipher it to reveal the original message. A system for encryption and decryption is called a ***cryptosystem***.

2025/2026 2025/2026 2025/2026

The original form of a message is known as ***plaintext***, and the encrypted form is called ***ciphertext.*** This relationship is shown in Figure 2-18. Think of encryption as a form of opaque paint that obscures or obliterates the plaintext, preventing it from being seen or interpreted accurately. Then, decryption is the process of peeling away the paint to reveal the original plaintext again.

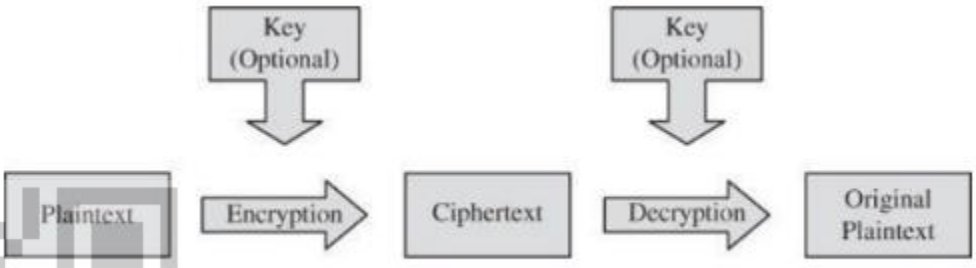


FIGURE 2-18 Plaintext and Ciphertext

Ciphertext: encrypted material; plaintext: material in intelligible form

رومانی صبحی رزق جرجس مطر

We use a formal notation to describe the transformations between plaintext and ciphertext. For example, we write C = E(P) and P = D(C), where C represents the ciphertext, E is the encryption rule, P is the plaintext, and D is the decryption rule. What we seek is a cryptosystem for which P = D(E(P)). In other words, we want to be able to convert the plaintext message to ciphertext to protect it from an intruder, but we also want to be able to get the original message back so that the receiver can read it properly.

رومانی صبحی رزق جرجس مطر

Encryption Keys

A cryptosystem involves a set of rules for how to encrypt the plaintext and decrypt the ciphertext. The encryption and decryption rules, called ***algorithms***, often use a device called a ***key***, denoted by K, so that the resulting ciphertext depends on the original plaintext message, the algorithm, and the key value. We write this dependence as C = E(K, P). Essentially, E is a set of encryption algorithms, and the key K selects one specific algorithm from the set.

This process is similar to using mass-produced locks in houses. As a homeowner, you would pay dearly to contract with someone to invent and make a lock just for your house. In addition, you would not know whether a particular inventor's lock was really solid or how it compared with those of other inventors. A better solution is to have a few well-known, well-respected companies producing standard locks that differ according to the (physical) key. Then, you and your neighbor might have the same brand and style of lock, but your key will open only your lock. In the same way, it is useful to have a few well-examined encryption algorithms for everyone to use, but differing keys would prevent someone from breaking into data you are trying to protect.

2025/2026 2025/2026 2025/2026

Sometimes the encryption and decryption keys are the same, so P = D(K,E(K,P)), meaning that the same key, K, is used both to encrypt a message and later to decrypt it. This form is called ***symmetric or single-key or secret key*** encryption because D and E are mirror-image processes. As a trivial example, the encryption algorithm might be to shift each plaintext letter forward n positions in the alphabet. For n = 1, A is changed to b, B to c, … P to q, … and Z to a, so we say the key value is n, moving n positions forward for encryption and backward for decryption. (You might notice that we have written the plaintext in uppercase letters and the corresponding ciphertext in lowercase; cryptographers sometimes use that convention to help them distinguish the two.)

Symmetric encryption: one key encrypts and decrypts.

At other times, encryption and decryption keys come in pairs. Then, a decryption key, KD, inverts the encryption of key K_E , so that P = D(K_D, E(K_E ,P)). Encryption algorithms of this form are called ***asymmetric or public key*** because converting C back to P involves a series of steps and a key that are different from the steps and key of E. The difference between symmetric and asymmetric encryption is shown in Figure 2-19.

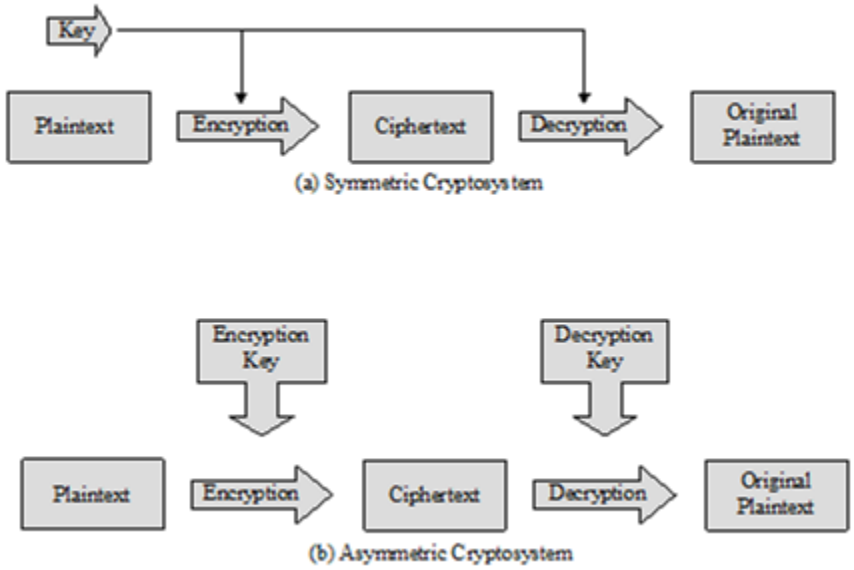


FIGURE 2-19 Symmetric and Asymmetric Encryption

Asymmetric encryption: one key encrypts, a different key decrypts.

A key gives us flexibility in using an encryption scheme. We can create different encryptions of one plaintext message just by changing the key. Moreover, using a key provides additional security. If the encryption algorithm should fall into the interceptor's hands, future messages can still be kept secret because the interceptor will not know the key value. Sidebar 2-14 describes how the British dealt with written keys and codes in World War II. An encryption scheme that does not require the use of a key is called a ***keyless cipher***.

Sidebar 2-14 Silken Codes

Leo Marks [MAR98] describes his life as a code-maker in Britain during World War II. That is, the British hired Marks and others to devise codes that could be used by spies and soldiers in the

2025/2026

field. In the early days, the encryption scheme depended on poems that were written for each spy, and it relied on the spy's ability to memorize and recall the poems correctly.

Marks reduced the risk of error by introducing a coding scheme that was printed on pieces of silk. Silk hidden under clothing could not be felt when the spy was patted down and searched. And, unlike paper, silk burns quickly and completely, so the spy could destroy incriminating evidence, also ensuring that the enemy could not get even fragments of the valuable code. When pressed by superiors as to why the British should use scarce silk (which was also needed for war-time necessities like parachutes) for codes, Marks said that it was a choice "between silk and cyanide."

The history of encryption is fascinating; it is well documented in David Kahn's book [KAH96]. Claude Shannon is considered the father of modern cryptography because he laid out a formal, mathematical foundation for information security and expounded on several principles for secure cryptography at the naissance of digital computing [SHA49]. Encryption has been used for centuries to protect diplomatic and military communications, sometimes without full success. The word ***cryptography*** refers to the practice of using encryption to conceal text. A ***cryptanalyst*** studies encryption and encrypted messages, hoping to find the hidden meanings. A cryptanalyst might also work defensively, probing codes and ciphers to see if they are solid enough to protect data adequately.

Both a ***cryptographer*** and a cryptanalyst attempt to translate coded material back to its original form. Normally, a cryptographer works on behalf of a legitimate sender or receiver, whereas a cryptanalyst works on behalf of an unauthorized interceptor. Finally, ***cryptology*** is the research into and study of encryption and decryption; it includes both cryptography and cryptanalysis.

Cryptanalysis

A cryptanalyst's chore is to break an encryption. That is, the cryptanalyst attempts to deduce the original meaning of a ciphertext message. Better yet, the cryptanalyst hopes to determine which decrypting algorithm, and ideally which key, matches the encrypting algorithm to be able to break other messages encoded in the same way.

For instance, suppose two countries are at war and the first country has intercepted encrypted messages of the second. Cryptanalysts of the first country want to decipher a particular message so as to anticipate the movements and resources of the second. But even better is to discover the

actual decryption method; then the first country can penetrate the encryption of all messages sent by the second country.

An analyst works with a variety of information: encrypted messages, known encryption algorithms, intercepted plaintext, data items known or suspected to be in a ciphertext message, mathematical or statistical tools and techniques, and properties of languages, as well as plenty of ingenuity and luck. Each piece of evidence can provide a clue, and the analyst puts the clues together to try to form a larger picture of a message's meaning in the context of how the encryption is done. Remember that there are no rules. An interceptor can use any means available to tease out the meaning of the message.

Work Factor

An encryption algorithm is called ***breakable*** when, given enough time and data, an analyst can determine the algorithm. However, an algorithm that is theoretically breakable may in fact be impractical to try to break. To see why, consider a 25-character message that is expressed in just uppercase letters. A given cipher scheme may have 26^{25} (approximately 10^{35}) possible decipherments, so the task is to select the right one out of the 26^{25}. If your computer could perform on the order of 10^{10} operations per second, finding this decipherment would require on the order of 10^{25} seconds, or roughly 10^{17} years. In this case, although we know that theoretically we could generate the solution, determining the deciphering algorithm by examining all possibilities can be ignored as infeasible with current technology.

The difficulty of breaking an encryption is called its ***work factor***. Again, an analogy to physical locks may prove helpful. As you know, physical keys have notches or other irregularities, and the notches cause pins to move inside a lock, allowing the lock to open. Some simple locks, such as those sold with suitcases, have only one notch, so these locks can often be opened with just a piece of bent wire; worse yet, some manufacturers produce only a few (and sometimes just one!) distinct internal pin designs; you might be able to open any such lock with a ring of just a few keys. Clearly these locks are cosmetic only.

Common house locks have five or six notches, and each notch can have any of ten depths. To open such a lock requires finding the right combination of notch depths, of which there may be up to a million possibilities, so carrying a ring of that many keys is infeasible. Even though in theory someone could open one of these locks by trying all possible keys, in practice the number

2025/2026 2025/2026 2025/2026

of possibilities is prohibitive. We say that the work factor to open one of these locks without the appropriate key is large enough to deter most attacks. So too with cryptography: An encryption is adequate if the work to decrypt without knowing the encryption key is greater than the value of the encrypted data.

Work factor: amount of effort needed to break an encryption (or mount a successful attack)

Two other important issues must be addressed when considering the breakability of encryption algorithms. First, the cryptanalyst cannot be expected to try only the hard, long way. In the example just presented, the obvious decryption might require 26^{25} machine operations, but a more ingenious approach might require only 10^{15} operations. At the speed of 10^{10} operations per second, 10^{15} operations take slightly more than one day. The ingenious approach is certainly feasible. In fact, newspapers sometimes print cryptogram puzzles that humans solve with pen and paper alone, so there is clearly a shortcut to our computer machine time estimate of years or even one day of effort. The newspaper games give hints about word lengths and repeated characters, so humans are solving an easier problem. As we said, however, cryptanalysts also use every piece of information at their disposal.

Some of the algorithms we study in this book are based on known "hard" problems that take an unreasonably long time to solve. But the cryptanalyst does not necessarily have to solve the underlying problem to break the encryption of a single message. Sloppy use of controls can reveal likely words or phrases, and an analyst can use educated guesses combined with careful analysis to generate all or much of an important message. Or the cryptanalyst might employ a spy to obtain the plaintext entirely outside the system; analysts might then use the pair of plaintext and corresponding ciphertext to infer the algorithm or key used to apply to subsequent messages.

In cryptanalysis there are no rules: Any action is fair play.

Second, estimates of breakability are based on current technology. An enormous advance in computing technology has occurred since 1950. Things that were infeasible in 1940 became possible by the 1950s, and every succeeding decade has brought greater improvements. A conjecture known as "Moore's Law" asserts that the speed of processors doubles every 1.5 years, and this conjecture has been true for over three decades. We dare not pronounce an algorithm

secure just because it cannot be broken with current technology, or worse, that it has not been broken yet.

In this book we write that something is impossible; for example, it is impossible to obtain plaintext from ciphertext without the corresponding key and algorithm. Please understand that in cryptography few things are truly impossible: infeasible or prohibitively difficult, perhaps, but impossible, no.

رومانی صبحی رزق جرجس مطر

Symmetric and Asymmetric Encryption Systems

Recall that the two basic kinds of encryptions are symmetric (also called “secret key”) and asymmetric (also called “public key”). Symmetric algorithms use one key, which works for both encryption and decryption. Usually, the decryption algorithm is closely related to the encryption one, essentially running the encryption in reverse.

The symmetric systems provide a two-way channel to their users: A and B share a secret key, and they can both encrypt information to send to the other as well as decrypt information from the other. As long as the key remains secret, the system also provides ***authenticity***, proof that a message received was not fabricated by someone other than the declared sender. Authenticity is ensured because only the legitimate sender can produce a message that will decrypt properly with the shared key.

2025/2026 2025/2026 2025/2026

Symmetry is a major advantage of this type of encryption, but it also leads to a problem: How do two users A and B obtain their shared secret key? And only A and B can use that key for their encrypted communications. If A wants to share encrypted communication with another user C, A and C need a different shared secret key. Managing keys is the major difficulty in using symmetric encryption. In general, n users who want to communicate in pairs need ***n * (n – 1)/2*** keys. In other words, the number of keys needed increases at a rate proportional to the square of the number of users! So a property of symmetric encryption systems is that they require a means of ***key distribution.***

Asymmetric or public key systems, on the other hand, typically have precisely matched pairs of keys. The keys are produced together or one is derived mathematically from the other. Thus, a process computes both keys as a set.

But for both kinds of encryption, a key must be kept well secured. Once the symmetric or private key is known by an outsider, all messages written previously or in the future can be decrypted

(and hence read or modified) by the outsider. So, for all encryption algorithms, ***key management*** is a major issue. It involves storing, safeguarding, and activating keys.

Asymmetric systems excel at key management. By the nature of the public key approach, you can send a public key in an email message or post it in a public directory. Only the corresponding private key, which presumably is not disclosed, can decrypt what has been encrypted with the public key.

Stream and Block Ciphers

One final characterization of encryption algorithms relates to the nature of the data to be concealed. Suppose you are streaming video, perhaps a movie, from a satellite. The stream may come in bursts, depending on such things as the load on the satellite and the speed at which the sender and receiver can operate. For such application you may use what is called stream encryption, in which each bit, or perhaps each byte, of the data stream is encrypted separately. A model of stream enciphering is shown in Figure 2-20. Notice that the input symbols are transformed one at a time. The advantage of a stream cipher is that it can be applied immediately to whatever data items are ready to transmit. But most encryption algorithms involve complex transformations; to do these transformations on one or a few bits at a time is expensive.

2025/2026 2025/2026 2025/2026

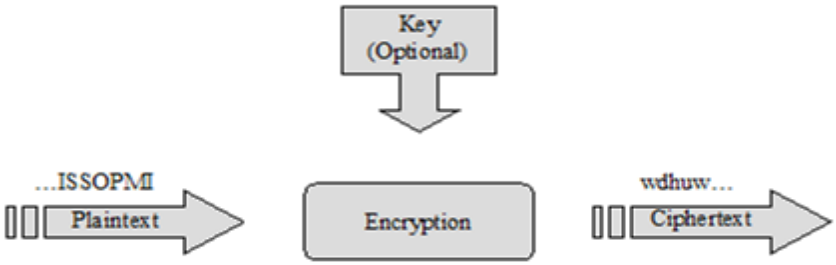


FIGURE 2-20 Stream Enciphering

To address this problem and make it harder for a cryptanalyst to break the code, we can use block ciphers. A block cipher encrypts a group of plaintext symbols as a single block. A block cipher algorithm performs its work on a quantity of plaintext data all at once. Like a machine that cuts out 24 cookies at a time, these algorithms capitalize on economies of scale by operating on large amounts of data at once. Blocks for such algorithms are typically 64, 128, 256 bits or more. The block size need not have any particular relationship to the size of a character. Block ciphers work on blocks of plaintext and produce blocks of ciphertext, as shown in Figure 2-21. In the

figure, the central box represents an encryption machine: The previous plaintext pair is converted to po, the current one being converted is IH, and the machine is soon to convert ES.

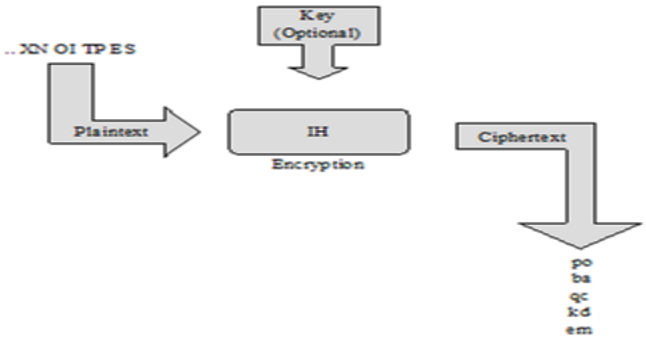


FIGURE 2-21 Block Cipher

Stream ciphers encrypt one bit or one byte at a time; block ciphers encrypt a fixed number of bits as a single chunk.

Table 2-10 lists the advantages and disadvantages of stream and block encryption algorithms.

	Stream	Block
Advantages	• *Speed of transformation.* Because each symbol is encrypted without regard for any other plaintext symbols, each symbol can be encrypted as soon as it is read. Thus, the time to encrypt a symbol depends only on the encryption algorithm itself, not on the time it takes to receive more plaintext. • *Low error propagation.* Because each symbol is separately encoded, an error in the encryption process affects only that character.	• *High diffusion.* Information from the plaintext is diffused into several ciphertext symbols. One ciphertext block may depend on several plaintext letters. • *Immunity to insertion of symbol.* Because blocks of symbols are enciphered, it is impossible to insert a single symbol into one block. The length of the block would then be incorrect, and the decipherment would quickly reveal the insertion.
Disadvantages	• *Low diffusion.* Each symbol is separately enciphered. Therefore, all the information of that symbol is contained in one symbol of ciphertext. • *Susceptibility to malicious insertions and modifications.* Because each symbol is separately enciphered, an active interceptor who has broken the code can splice pieces of previous messages and transmit a spurious new message that may look authentic.	• *Slowness of encryption.* The person or machine doing the block ciphering must wait until an entire block of plaintext symbols has been received before starting the encryption process. • *Padding.* A final short block must be filled with irrelevant data to make a full-sized block. • *Error propagation.* An error will affect the transformation of all other characters in the same block.

TABLE 2-10 Stream and Block Encryption Algorithms

2025/

With this description of the characteristics of different encryption algorithms we can now turn to some widely used encryption algorithms. We present how each works, a bit of the historical context and motivation for each, and some strengths and weaknesses. We identify these algorithms by name because these names appear in the popular literature. We also introduce other symmetric algorithms in Chapter 12. Of course you should recognize that these are just examples of popular algorithms; over time these algorithms may be superseded by others. To a large degree cryptography has become plug-and-play, meaning that in an application developers can substitute one algorithm for another of the same type and similar characteristics. In that way advancements in the field of cryptography do not require that all applications using cryptography be rewritten.

DES: The Data Encryption Standard

The Data Encryption Standard (DES) [NBS77], a system developed for the U.S. government, was intended for use by the general public. Standards organizations have officially accepted it as a cryptographic standard both in the United States and abroad. Moreover, many hardware and software systems have been designed with DES. For many years it was the algorithm of choice for protecting financial, personal, and corporate data; however, researchers increasingly questioned its adequacy as it aged.

Overview of the DES Algorithm

The DES algorithm [NBS77] was developed in the 1970s by IBM for the U.S. National Institute of Standards and Technology (NIST), then called the National Bureau of Standards (NBS). DES is a careful and complex combination of two fundamental building blocks of encryption: substitution and transposition. The algorithm derives its strength from repeated application of these two techniques, one on top of the other, for a total of 16 cycles. The sheer complexity of tracing a single bit through 16 iterations of substitutions and transpositions has so far stopped researchers in the public from identifying more than a handful of general properties of the algorithm.

The algorithm begins by encrypting the plaintext as blocks of 64 bits. The key is 64 bits long, but in fact it can be any 56-bit number. (The extra 8 bits are often used as check digits but do not affect encryption in normal implementations. Thus we say that DES uses a key, the strength of

which is 56 bits.) The user can pick a new key at will any time there is uncertainty about the security of the old key.

DES encrypts 64-bit blocks by using a 56-bit key.

DES uses only standard arithmetic and logical operations on binary data up to 64 bits long, so it is suitable for implementation in software on most current computers. Encrypting with DES involves 16 iterations, each employing replacing blocks of bits (called a substitution step), shuffling the bits (called a permutation step), and mingling in bits from the key (called a key transformation). Although complex, the process is table driven and repetitive, making it suitable for implementation on a single-purpose chip. In fact, several such chips are available on the market for use as basic components in devices that use DES encryption in an application.

Double and Triple DES

As you know, computing power has increased rapidly over the last few decades, and it promises to continue to do so. For this reason, the DES 56-bit key length is not long enough for some people's comfort. Since the 1970s, researchers and practitioners have been interested in a longer-key version of DES. But we have a problem: The DES algorithm design is fixed to a 56-bit key.

2025/2026 2025/2026 2025/2026

Double DES

To address the discomfort, some researchers suggest using a double encryption for greater secrecy. The double encryption works in the following way. Take two keys, k_1 and k_2 , and perform two encryptions, one on top of the other: $E(k_2 , E(k_1,m))$. In theory, this approach should multiply the difficulty of breaking the encryption, just as two locks are harder to pick than one.

Unfortunately, that assumption is false. Ralph Merkle and Martin Hellman [MER81] showed that two encryptions are scarcely better than one: two encryptions with different 56-bit keys are equivalent in work factor to one encryption with a 57-bit key. Thus, the double encryption adds only a small amount of extra work for the attacker who is trying to infer the key(s) under which a piece of ciphertext was encrypted. As we soon describe, some 56-bit DES keys have been derived in just days; two times days is still days, when the hope was to add months if not years of work for the second encryption. Alas, double DES adds essentially no more security.

Triple DES

However, a simple trick does indeed enhance the security of DES. Using three keys adds significant strength. The so-called triple DES procedure is $C = E(k_3 , E(k_2 , E(k_1,m)))$. That is,

you encrypt with one key, then with the second, and finally with a third. This process gives a strength roughly equivalent to a 112-bit key (because the double DES attack defeats the strength of one of the three keys, but it has no effect on the third key).

A minor variation of triple DES, which some people also confusingly call triple DES, is $C = E(k_1, D(k_2, E(k_1, m)))$. That is, you encrypt with one key, decrypt with a second, and encrypt with the first again. This version requires only two keys. (The second decrypt step also makes this process work for single encryptions with one key: The decryption cancels the first encryption, so the net result is one encryption. The encrypt–decrypt–encrypt form is handy because one algorithm can produce results for both conventional single-key DES and the more secure two-key method.) This two-key, three-step version is subject to another tricky attack, so its strength is rated at only about 80 bits. Still, 80 bits is beyond reasonable cracking capability for current hardware.

In summary, ordinary DES has a key space of 56 bits, double DES is scarcely better, but two-key triple DES gives an effective length of 80 bits, and three-key triple DES gives a strength of 112 bits. Remember why we are so fixated on key size: If no other way succeeds, the attacker can always try all possible keys. A longer key means significantly more work for this attack to bear fruit, with the work factor doubling for each additional bit in key length. Now, roughly a half century after DES was created, a 56-bit key is inadequate for any serious confidentiality, but 80- and 112-bit effective key sizes afford reasonable security. We summarize these forms of DES in Table 2-11.

Form	Operation	Properties	Strength
DES	Encrypt with one key	56-bit key	Inadequate for high-security applications by today's computing capabilities
Double DES	Encrypt with first key; then encrypt result with second key	Two 56-bit keys	Only doubles strength of 56-bit key version
Two-key triple DES	Encrypt with first key, then encrypt (or decrypt) result with second key, then encrypt result with first key (E-D-E)	Two 56-bit keys	Gives strength equivalent to about 80-bit key (about 16 million times as strong as 56-bit version)
Three-key triple DES	Encrypt with first key, then encrypt or decrypt result with second key, then encrypt result with third key (E-E-E)	Three 56-bit keys	Gives strength equivalent to about 112-bit key about 72 quintillion ($72*10^{15}$) times as strong as 56-bit version

TABLE 2-11 Forms of DES

رومانی صبحی رزق جرجس مطر

Security of DES

Since it was first announced, DES has been controversial. Many researchers have questioned the security it provides. Because of its association with the U.S. government, specifically the U.S. National Security Agency (NSA) that made certain unexplained changes between what IBM proposed and what the NBS actually published, some people have suspected that the algorithm was somehow weakened, to allow the government to snoop on encrypted data. Much of this controversy has appeared in the open literature, but certain DES features have neither been revealed by the designers nor inferred by outside analysts.

Whitfield Diffie and Martin Hellman [DIF77] argued in 1977 that a 56-bit key is too short. In 1977, it was prohibitive to test all 256 (approximately 10^{15}) keys on then current computers. But they argued that over time, computers would become more powerful and the DES algorithm would remain unchanged; eventually, the speed of computers would exceed the strength of DES. Exactly that happened about 20 years later. In 1997, researchers using a network of over 3,500 machines in parallel were able to infer a DES key in four months' work. And in 1998 for approximately $200,000 U.S. researchers built a special "DES cracker" machine that could find a DES key in approximately four days, a result later improved to a few hours [EFF98].

2025/2026 2025/2026 2025/2026

Does this mean DES is insecure? No, not exactly. No one has yet shown serious flaws in the DES algorithm itself. The 1997 attack required a great deal of cooperation, and the 1998 machine is rather expensive. But even if conventional DES can be attacked, triple DES is still well beyond the power of these attacks. Remember the impact of exponential growth: Let us say, for simplicity, that single-key DES can be broken in one hour. The simple double-key version could then be broken in two hours. But $2^{80} / 2^{56} = 2^{24}$, which is over 16,700,000, meaning it would take 16 million hours, nearly 2,000 years, to defeat a two-key triple DES encryption, and considerably longer for the three-key version.

Nevertheless, the basic structure of DES with its fixed-length 56-bit key and fixed number of iterations makes evident the need for a new, stronger, and more flexible algorithm. In 1995, the NIST began the search for a new, strong encryption algorithm. The response to that search has become the Advanced Encryption Standard, or AES.

AES: Advanced Encryption System

After a public competition and review, NIST selected an algorithm named Rijndael as the new advanced encryption system; Rijndael is now known more widely as AES. AES was adopted for

use by the U.S. government in December 2001 and became Federal Information Processing Standard 197 [NIS01]. AES is likely to be the commercial-grade symmetric algorithm of choice for years, if not decades. Let us look at it more closely.

Overview of Rijndael

Rijndael is a fast algorithm that can easily be implemented on simple processors. Although it has a strong mathematical foundation, it primarily uses substitution, transposition, the shift, exclusive OR, and addition operations. Like DES, AES uses repeat cycles.

There are 10, 12, or 14 cycles for keys of 128, 192, and 256 bits, respectively. In Rijndael, the cycles are called “rounds.” Each round consists of four steps that substitute and scramble bits. Bits from the key are frequently combined with intermediate result bits, so key bits are also well diffused throughout the result. Furthermore, these four steps are extremely fast. The AES algorithm is depicted in Figure 2-22.

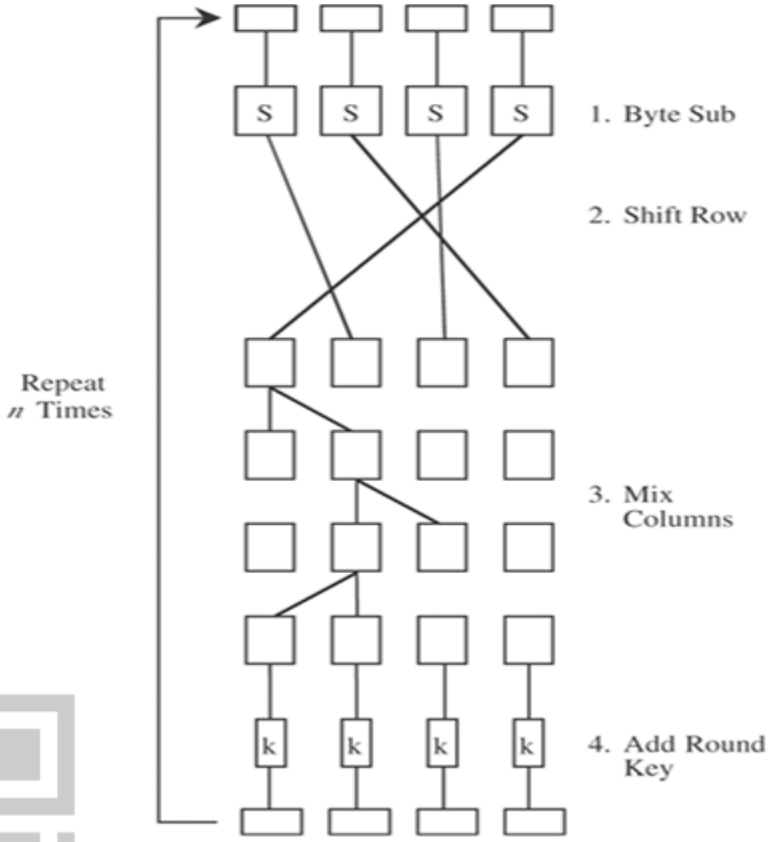


FIGURE 2-22 AES Encryption Algorithm

رومانی صبحی رزق جرجس مطر

Strength of the Algorithm

The characteristics and apparent strength of DES and AES are compared in Table 2-12. Remember, of course, that these strength figures apply only if the implementation and use are robust; a strong algorithm loses strength if used with a weakness that lets outsiders determine key properties of the encrypted data.

	DES	AES
Date designed	1976	1999
Block size	64 bits	128 bits
Key length	56 bits (effective length); up to 112 bits with multiple keys	128, 192, 256 (and possibly more) bits
Operations	16 rounds	10, 12, 14 (depending on key length); can be increased
Encryption primitives	Substitution, permutation	Substitution, shift, bit mixing
Cryptographic primitives	Confusion, diffusion	Confusion, diffusion
Design	Open	Open
Design rationale	Closed	Open
Selection process	Secret	Secret, but open public comments and criticisms invited
Source	IBM, enhanced by NSA	Independent Dutch cryptographers

TABLE 2-12 Comparison of DES and AES

Moreover, the number of cycles can be extended in a natural way. With DES the algorithm was defined for precisely 16 cycles; to extend that number would require substantial redefinition of the algorithm. The internal structure of AES has no a priori limitation on the number of cycles. If a cryptanalyst ever concluded that 10 or 12 or 14 rounds were too low, the only change needed to improve the algorithm would be to change the limit on a repeat loop.

A mark of confidence is that the U.S. government has approved AES for protecting Secret and Top Secret classified documents. This is the first time the United States has ever approved the use of a commercial algorithm derived outside the government (and furthermore, outside the United States) to encrypt classified data.

However, we cannot rest on our laurels. No one can predict now what limitations cryptanalysts might identify in the future. Fortunately, talented cryptologists continue to investigate even

stronger algorithms that will be able to replace AES when it becomes obsolete. At present, AES seems to be a significant improvement over DES, and it can be improved in a natural way if necessary. DES is still in widespread use, but AES is also widely adopted, particularly for new applications.

Public Key Cryptography

So far, we have looked at encryption algorithms from the point of view of making the scrambling easy for the sender (so that encryption is fast and simple) and the decryption easy for the receiver but not for an intruder. But this functional view of transforming plaintext to ciphertext is only part of the picture. We must also figure out how to distribute encryption keys. We have noted how useful keys can be in deterring an intruder, but the key must remain secret for it to be effective. The encryption algorithms we have presented so far are called symmetric or secret-key algorithms. The two most widely used symmetric algorithms, DES and AES, operate similarly: Two users have copies of the same key. One user uses the algorithm to encrypt some plaintext under the key, and the other user uses an inverse of the algorithm with the same key to decrypt the ciphertext. The crux of this issue is that all the power of the encryption depends on the secrecy of the key.

2025/2026 2025/2026 2025/2026

In 1976, Whitfield Diffie and Martin Hellman [DIF76] invented public key cryptography, a new kind of encryption. With a public key encryption system, each user has two keys, one of which does not have to be kept secret. Although counterintuitive, in fact the public nature of the key does not compromise the secrecy of the system. Instead, the basis for public key encryption is to allow the key to be divulged but to keep the decryption technique secret. Public key cryptosystems accomplish this goal by using two keys: one to encrypt and the other to decrypt. Although these keys are produced in mathematically related pairs, an outsider is effectively unable to use one key to derive the other.

In this section, we look at ways to allow the key to be public but still protect the message. We also focus on the RSA algorithm, a popular, commercial-grade public key system. Other algorithms, such as elliptic curve cryptosystems [MIL85, KOB87] and the El Gamal algorithm [ELG85], both of which we cover in Chapter 12, operate similarly (although the underlying mathematics are very different). We concentrate on RSA because many applications use it. We

رومانی صبحی رزق جرجس مطر

also present a mathematical scheme by which two users can jointly construct a secret encryption key without having any prior secrets.

Motivation

Why should making the key public be desirable? With a conventional symmetric key system, each pair of users needs a separate key. But with public key systems, anyone using a single public key can send a secret message to a user, and the message remains adequately protected from being read by an interceptor. Let us investigate why this is so.

Recall that in general, an n-user system requires ***n * (n – 1)/2*** keys, and each user must track and remember a key for each other user with whom he or she wants to communicate. As the number of users grows, the number of keys increases rapidly, as shown in Figure 2-23. Determining and distributing these keys is a problem. A more serious problem is maintaining security for the keys already distributed—we cannot expect users to memorize so many keys. Worse, loss or exposure of one user's keys requires setting up a new key pair with each of that user's correspondents.

2025/2026

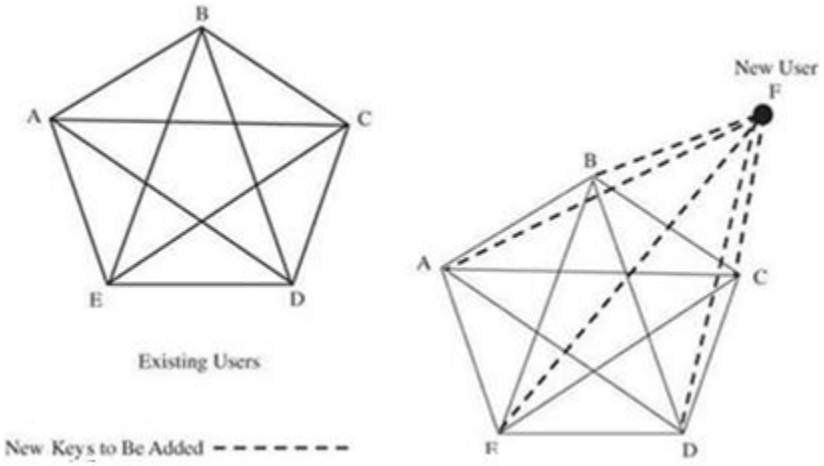


FIGURE 2-23 Explosion in Number of Keys

Characteristics

We can reduce the problem of key proliferation by using a public key approach. In a ***public key*** or ***asymmetric encryption system***, each user has two keys: a ***public key*** and a ***private key***. The user may freely publish the public key because each key does only encryption or decryption, but not both. The keys operate as inverses, meaning that one key undoes the encryption provided by the other key. But deducing one key from the other is effectively impossible.

رومانی صبحی رزق جرجس مطر

To see how, let K_{PRIV} be a user's private key, and let K_{PUB} be the corresponding public key. Then, encrypted plaintext using the public key is decrypted by application of the private key; we write the relationship as

$$P = D(K_{PRIV}, E(K_{PUB}, P))$$

That is, a user can decode with a private key what someone else has encrypted with the corresponding public key. Furthermore, with some public key encryption algorithms, including RSA, we have this relationship:

$$P = D(K_{PUB}, E(K_{PRIV},P))$$

In other words, a user can encrypt a message with a private key, and the message can be revealed only with the corresponding public key.

These two properties tell us that public and private keys can be applied in either order. In particular, the decryption function D can be applied to any argument so that we can decrypt before we encrypt. With conventional encryption, we seldom think of decrypting before encrypting. But the concept makes sense with public keys, where it simply means applying the private transformation first and then the public one.

We have noted that a major problem with symmetric encryption is the sheer number of keys a single user has to store and track. With public keys, only two keys are needed per user: one public and one private. Let us see what difference this makes in the number of keys needed. Suppose we have three users, B, C, and D, who must pass protected messages to user A as well as to each other. Since each distinct pair of users needs a key, each user would need three different keys; for instance, A would need a key for B, a key for C, and a key for D. But using public key encryption, each of B, C, and D can encrypt messages for A by using A's public key. If B has encrypted a message using A's public key, C cannot decrypt it, even if C knew it was encrypted with A's public key. Applying A's public key twice, for example, would not decrypt the message. (We assume, of course, that A's private key remains secret.) Thus, the number of keys needed in the public key system is only two per user.

The Rivest–Shamir–Adelman (RSA) Algorithm

The ***Rivest–Shamir–Adelman (RSA) cryptosystem*** is a public key system. Based on an underlying hard problem and named after its three inventors (Ronald Rivest, Adi Shamir, and

Leonard Adleman), this algorithm was introduced in 1978 [RIV78]. Cryptanalysts have subjected RSA to extensive cryptanalysis, but they have found no serious flaws.

The two keys used in RSA, d and e, are used for decryption and encryption. They are actually interchangeable: Either can be chosen as the public key, but one having been chosen, the other one must be kept private. For simplicity, we call the encryption key e and the decryption key d. We denote plaintext as P and its corresponding ciphertext as C.

C = RSA(P,e). Also, because of the nature of the RSA algorithm, the keys can be applied in either order:

$$P = E(D(P)) = D(E(P))$$

or

$$P = RSA(RSA(P, e), d) = RSA(RSA(P, d), e)$$

(You can think of E and D as two complementary functions, each of which can "undo" the other's effect.)

RSA does have the unfortunate property that the keys are long: 256 bits is considered the minimum usable length, but in most contexts experts prefer keys on the order of 1000 to 2000 bits. Encryption in RSA is done by exponentiation, raising each plaintext block to a power; that power is the key e. In contrast to fast substitution and transposition of symmetric algorithms, exponentiation is extremely time-consuming on a computer. Even worse, the time to encrypt increases exponentially as the exponent (key) grows longer. Thus, RSA is markedly slower than DES and AES.

The encryption algorithm is based on the underlying problem of factoring large numbers in a finite set called a field. So far, nobody has found a shortcut or easy way to factor large numbers in a field. In a highly technical but excellent paper, Dan Boneh [BON99] reviews all the known cryptanalytic attacks on RSA and concludes that none is significant. Because the factorization problem has been open for many decades, most cryptographers consider this problem a solid basis for a secure cryptosystem.

To summarize, the two symmetric algorithms DES and AES provide solid encryption of blocks of 64 to 256 bits of data. The asymmetric algorithm RSA encrypts blocks of various sizes. DES and AES are substantially faster than RSA, by a factor of 10,000 or more, and their rather simple primitive operations have been built into some computer chips, making their encryption even

رومانی صبحی رزق جرجس مطر

more efficient than RSA. Therefore, people tend to use DES and AES as the major cryptographic workhorses, and reserve slower RSA for limited uses at which it excels.

The characteristics of secret key (symmetric) and public key (asymmetric) algorithms are compared in Table 2-13.

	Secret Key (Symmetric)	Public Key (Asymmetric)
Number of keys	1	2
Key size (bits)	56–112 (DES), 128–256 (AES)	Unlimited; typically no less than 256; 1000 to 2000 currently considered desirable for most uses
Protection of key	Must be kept secret	One key must be kept secret; the other can be freely exposed
Best uses	Cryptographic workhorse. Secrecy and integrity of data, from single characters to blocks of data, messages and files	Key exchange, authentication, signing
Key distribution	Must be out-of-band	Public key can be used to distribute other keys
Speed	Fast	Slow, typically by a factor of up to 10,000 times slower than symmetric algorithms

TABLE 2-13 Comparison of Secret Key and Public Key Encryption

Public Key Cryptography to Exchange Secret Keys

Encryption algorithms alone are not the answer to everyone's encryption needs. Although encryption implements protected communications channels, it can also be used for other duties. In fact, combining symmetric and asymmetric encryption often capitalizes on the best features of each.

Suppose you need to send a protected message to someone you do not know and who does not know you. This situation is more common than you may think. For instance, you may want to send your income tax return to the government. You want the information to be protected, but you do not necessarily know the person who is receiving the information. Similarly, you may want to purchase from a shopping website, exchange private (encrypted) email, or arrange for two hosts to establish a protected channel. Each of these situations depends on being able to exchange an encryption key in such a way that nobody else can intercept it. The problem of two previously unknown parties exchanging cryptographic keys is both hard and important. Indeed,

رومانی صبحی رزق جرجس مطر

the problem is almost circular: To establish an encrypted session, you need an encrypted means to exchange keys.

Public key cryptography can help. Since asymmetric keys come in pairs, one half of the pair can be exposed without compromising the other half. In fact, you might think of the public half of the key pair as truly public—posted on a public website, listed in a public directory similar to a telephone listing, or sent openly in an email message. That is the beauty of public key cryptography: As long as the private key is not disclosed, a public key can be open without compromising the security of the encryption.

Simple Key Exchange Protocol

Suppose that a sender, Amy, and a receiver, Bill, both have pairs of asymmetric keys for a common encryption algorithm. We denote any public key encryption function as E(k, X), meaning perform the public key encryption function on X by using key k. Call the keys K_{PRIV-A}, K_{PUB-A}, K_{PRIV-B}, and K_{PUB-B}, for the private and public keys for Amy and Bill, respectively.

The problem we want to solve is for Amy and Bill to be able to establish a secret (symmetric algorithm) encryption key that only they know. The simplest solution is for Amy to choose any symmetric key K, and send E(K_{PRIV-A}, K) to Bill. Bill takes Amy's public key, removes the encryption, and obtains K.

2025/2026 2025/2026 2025/2026

This analysis is flawed, however. How does the sender know the public key really belongs to the intended recipient? Consider, for example, the following scenario. Suppose Amy and Bill do not have a convenient bulletin board. So, Amy just asks Bill for his key. Basically, the key exchange protocol, depicted in Figure 2-24, would work like this:

1. Amy says: Bill, please send me your public key.
2. Bill replies: Here, Amy; this is my public key.
3. Amy responds: Thanks. I have generated a symmetric key for us to use for this interchange. I am sending you the symmetric key encrypted under your public key.

In the subversion shown in Figure 2-25, we insert an attacker, Malvolio, into this communication.

1. Amy says: Bill, please send me your public key.

رومانی صبحی رزق جرجس مطر

1a. Malvolio intercepts the message and fashions a new message to Bill, purporting to come from Amy but with Malvolio's return address, asking for Bill's public key.

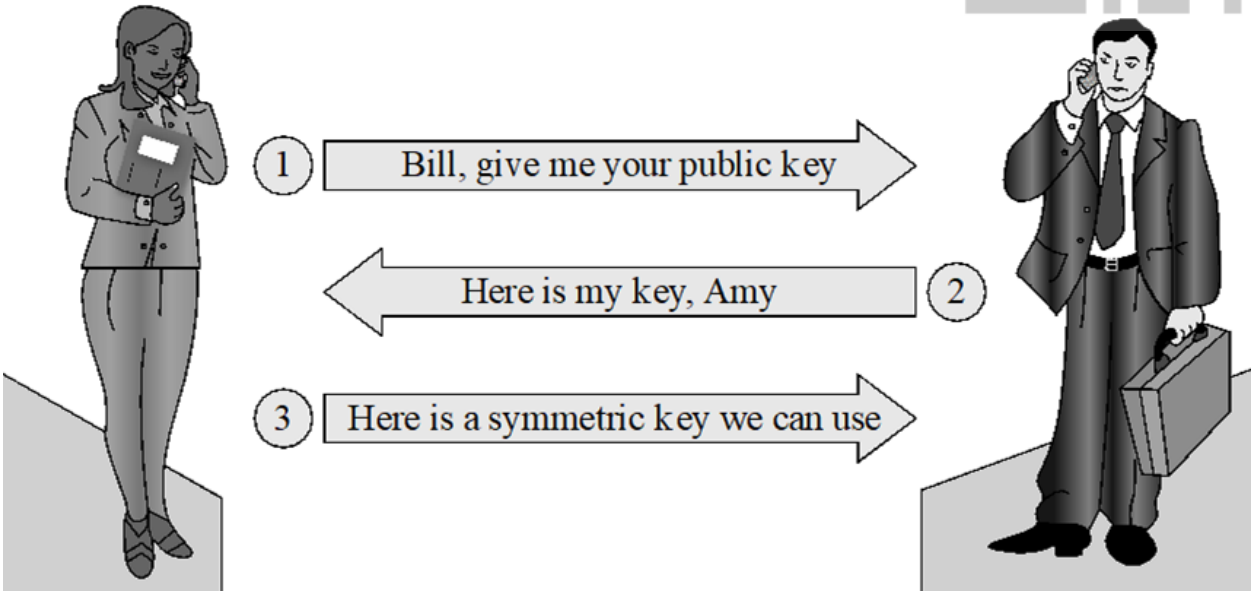


FIGURE 2-24 Key Exchange Protocol

2025/2026 2025/2026 2025/2026

2. Bill replies: Here, Amy; this is my public key. (Because of the return address in step 1a, this reply goes to Malvolio.)

2a. Malvolio holds Bill's public key and sends Malvolio's own public key to Amy, alleging it is from Bill.

3. Amy responds: Thanks. I have generated a symmetric key for us to use for this interchange. I am sending you the symmetric key encrypted under your public key.

3a. Malvolio intercepts this message and obtains and holds the symmetric key Amy has generated.

3b. Malvolio generates a new symmetric key and sends it to Bill, with a message purportedly from Amy: Thanks. I have generated a symmetric key for us to use for this interchange. I am sending you the symmetric key encrypted under your public key.

In summary, Malvolio now holds two symmetric encryption keys, one each shared with Amy and Bill. Not only can Malvolio stealthily obtain all their interchanges, but Amy and Bill cannot communicate securely with each other because neither shares a key with the other.

رومانی صبحی رزق جرجس مطر

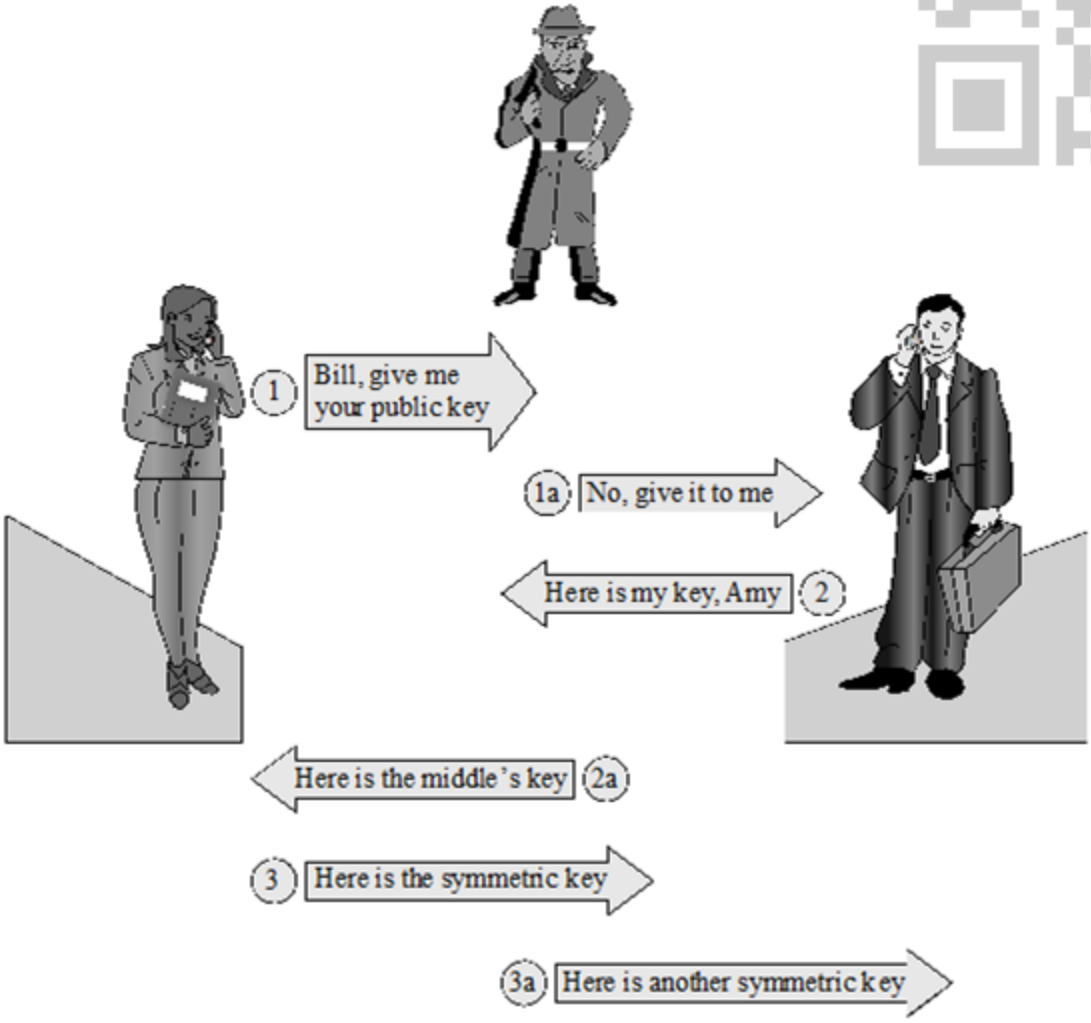


FIGURE 2-25 Key Exchange Protocol with a Man in the Middle

Revised Key Exchange Protocol

Remember that we began this discussion with a man-in-the-middle attack against a simple key exchange protocol. The faulty protocol was

1. A says: B, please send me your public key.
2. B replies: Here, A; this is my public key.
3. A responds: Thanks. I have generated a symmetric key for us to use for this interchange. I am sending you the symmetric key encrypted under your public key.

At step 2 the intruder intercepts B's public key and passes along the intruder's. The intruder can be foiled if A and B exchange half a key at a time. Half a key is useless to the intruder because it

is not enough to encrypt or decrypt anything. Knowing half the key does not materially improve the intruder's ability to break encryptions in the future.

Rivest and Shamir [RIV84] have devised a solid protocol as follows.

1. Amy sends her public key to Bill.
2. Bill sends his public key to Amy.
3. Amy creates a symmetric key, encrypts it using Bill's public key, and sends half of the result to Bill. (Note: half of the result might be the first n/2 bits, all the odd numbered bits, or some other agreed-upon form.)
4. Bill responds to Amy that he received the partial result (which he cannot interpret at this point, so he is confirming only that he received some bits). Bill encrypts any random number with his private key and sends half the bits to Amy.
5. Amy sends the other half of the encrypted result to Bill.
6. Bill puts together the two halves of Amy's result, decrypts it using his private key and thereby obtains the shared symmetric key. Bill sends the other half of his encrypted random number to Amy.
7. Amy puts together the two halves of Bill's random number, decrypts it using her private key, extracts Bill's random number, encrypts it using the now-shared symmetric key, and sends that to Bill.
8. Bill decrypts Amy's transmission with the symmetric key and compares it to the random number he selected in step 6. A match confirms the validity of the exchange.

2025/2026 2025/2026 2025/2026

To see why this protocol works, look at step 3. Malvolio, the intruder, can certainly intercept both public keys in steps 1 and 2 and substitute his own. However, at step 3 Malvolio cannot take half the result, decrypt it using his private key, and reencrypt it under Bill's key. Bits cannot be decrypted one by one and reassembled.

At step 4 Bill picks any random number, which Amy later returns to Bill to show she has successfully received the encrypted value Bill sent. Such a random value is called a nonce, a value meaningless in and of itself, to show activity (liveness) and originality (not a replay). In some protocols the receiver decrypts the nonce, adds 1 to it, reencrypts the result, and returns it. Other times the nonce includes a date, time, or sequence number to show that the value is

رومانی صبحی رزق جرجس مطر

current. This concept is used in computer-to-computer exchanges that lack some of the characteristics of human interaction.

Authenticity

The problem of the person in the middle can be solved in another way: Amy should send to Bill

$$E(K_{PUB\text{-}B}, E(K_{PRIV\text{-}A}, K))$$

This function ensures that only Bill, using $K_{PRIV\text{-}B}$, can remove the encryption applied with $K_{PUB\text{-}B}$, and Bill knows that only Amy could have applied $K_{PRIV\text{-}A}$ that Bill removes with $K_{PUB\text{-}A}$ We can think of this exchange in terms of locks and seals. Anyone can put a letter into a locked mailbox (through the letter slot), but only the holder of the key can remove it. In olden days, important people had seals that they would impress into molten wax on a letter; the seal's imprint showed authenticity, but anyone could break the seal and read the letter. Putting these two pieces together, a sealed letter inside a locked mailbox enforces the authenticity of the sender (the seal) and the confidentiality of the receiver (the locked mailbox).

If Amy wants to send something protected to Bill (such as a credit card number or a set of medical records), then the exchange works something like this. Amy seals the protected information with her private key so that it can be opened only with Amy's public key. This step ensures authenticity: only Amy can have applied the encryption that is reversed with Amy's public key. Amy then locks the information with Bill's public key. This step adds confidentiality because only Bill's private key can decrypt data encrypted with Bill's public key. Bill can use his private key to open the letter box (something only he can do) and use Amy's public key to verify the inner seal (proving that the package came from Amy).

Thus, as we have seen, asymmetric cryptographic functions are a powerful means for exchanging cryptographic keys between people who have no prior relationship. Asymmetric cryptographic functions are slow, but they are used only once, to exchange symmetric keys. Furthermore, if the keys being exchanged are for a symmetric encryption system such as AES or DES, the key length is relatively short, up to 256 bits for AES or 64 for DES. Even if we were to use an expanded form of AES with a key length of 1000 bits, the slow speed of public key cryptography would not be a significant problem because it is performed only once, to establish shared keys.

Asymmetric cryptography is also useful for another important security construct: a digital signature. A human signature on a paper document is a strong attestation: it signifies both

2025/2026 2025/2026 2025/2026

agreement (you agree to the terms in the document you signed) and understanding (you know what you are signing). People accept written signatures as a surrogate for an in-person confirmation. We would like a similarly powerful construct for confirming electronic documents. To build a digital signature we introduce integrity codes, key certificates, and, finally, signatures themselves.

Error Detecting Codes

Communications are notoriously prone to errors in transmission. You may have noticed that occasionally a mobile phone conversation will skip or distort a small segment of the conversation, and television signals sometimes show problems commonly called noise. In these cases, complete and accurate reception is not important as long as the noise is relatively slight or infrequent. You can always ask your phone partner to repeat a sentence, and a winning goal on television is always rebroadcast numerous times.

With important data, however, we need some way to determine that the transmission is complete and intact. Mathematicians and engineers have designed formulas called error detection and correction codes to make transmission errors apparent and to perform minor repairs.

2025/2026 2025/2026 2025/2026

Error detecting codes come under many names, such as hash codes, message digests, checksums, integrity checks, error detection and correction codes, and redundancy tests. Although these terms have fine differences of meaning, the basic purpose of all is to demonstrate that a block of data has been modified. That sentence is worded carefully: A message digest will (sometimes) signal that content has changed, but it is less solid at demonstrating no modification, even though that is what we really want. We want something to show no tampering—malicious or not; we get something that usually shows tampering.

Sam writes a letter, makes and keeps a photocopy, and sends the original to Theresa. Along the way, Fagin intercepts the letter and makes changes; being a skilled forger, Fagin deceives Theresa. Only when Theresa and Sam meet and compare the (modified) original do they detect the change.

The situation is different if Sam and Theresa suspect a forger is nigh. Sam carefully counts the letters in his document, tallying 1 for an a, 2 for a b, and so on up to 26 for a z. He adds those values and writes the sum in tiny digits at the bottom of the letter. When Teresa receives the

رومانی صبحی رزق جرجس مطر

letter she does the same computation and compares her result to the one written at the bottom. Three cases arise:

- the counts to do not agree, in which case Theresa suspects a change
- there is no count, in which case Theresa suspects either that Sam was lazy or forgetful or that a forger overlooked their code
- Teresa's count is the same as written at the bottom

The last case is the most problematic. Theresa probably concludes with relief that there was no change. As you may have already determined, however, she may not be thinking correctly. Fagin might catch on to the code and correctly compute a new sum to match the modifications. Even worse, perhaps Fagin's changes happen to escape detection. Suppose Fagin removes one letter c (value=3) and replaces it with three copies of the letter a (value=1+1+1=3); the sum is the same, or if Fagin only permutes letters, the sum remains the same, because it is not sensitive to order.

These problems all arise because the code is a many-to-one function: two or more inputs produce the same output. Two inputs that produce the same output are called a collision. In fact, all message digests are many-to-one functions, and thus when they report a change, one did occur, but when they report no change, it is only likely—not certain—that none occurred because of the possibility of a collision.

2025/2026 2025/2026 2025/2026

Collisions are usually not a problem for two reasons. First, they occur infrequently. If plaintext is reduced to a 64-bit digest, we expect the likelihood of a collision to be 1 in 2^{64} , or about 1 in 10^{19} , most unlikely, indeed. More importantly, digest functions are unpredictable, so given one input, finding a second input that results in the same output is infeasible. Thus, with good digest functions collisions are infrequent, and we cannot cause or predict them.

We can use error detecting and error correcting codes to guard against modification of data. Detection and correction codes are procedures or functions applied to a block of data; you may be familiar with one type of detecting code: parity. These codes work as their names imply: Error detecting codes detect when an error has occurred, and error correcting codes can actually correct errors without requiring a copy of the original data. The error code is computed and stored safely on the presumed intact, original data; later anyone can recompute the error code and check whether the received result matches the expected value. If the values do not match, a change has certainly occurred; if the values match, it is probable—but not certain—that no change has occurred

رومانی صبحی رزق جرجس مطر

رومانی صبحی رزق جرجس مطر

Parity

The simplest error detection code is a ***parity*** check. An extra bit, which we call a fingerprint, is added to an existing group of data bits, depending on their sum. The two kinds of parity are called even and odd. With even parity the fingerprint is 0 if the sum of the data bits is even, and 1 if the sum is odd; that is, the parity bit is set so that the sum of all data bits plus the parity bit is even. Odd parity is the same except the overall sum is odd. For example, the data stream 01101101 would have an even parity bit of 1 (and an odd parity bit of 0) because 0+1+1+0+1+1+0+1 = 5 + 1 = 6 (or 5 + 0 = 5 for odd parity).

One parity bit can reveal the modification of a single bit. However, parity does not detect two-bit errors—cases in which two bits in a group are changed. One parity bit can detect all single-bit changes, as well as changes of three, five and seven bits. Table 2-14 shows some examples of detected and undetected changes. The changed bits (each line shows changes from the original value of 00000000) are in bold, underlined; the table shows whether parity properly detected that at least one change occurred.

Original Data	Parity Bit	Modified Data	Modification Detected?
0 0 0 0 0 0 0 0	1	0 0 0 0 0 0 0 **1**	Yes
0 0 0 0 0 0 0 0	1	**1** 0 0 0 0 0 0 0	Yes
0 0 0 0 0 0 0 0	1	**1** 0 0 0 0 0 0 **1**	No
0 0 0 0 0 0 0 0	1	0 0 0 0 0 0 **1** **1**	No
0 0 0 0 0 0 0 0	1	0 0 0 0 0 **1** **1** **1**	Yes
0 0 0 0 0 0 0 0	1	0 0 0 0 **1** **1** **1** **1**	No
0 0 0 0 0 0 0 0	1	0 **1** 0 **1** 0 **1** 0 **1**	No
0 0 0 0 0 0 0 0	1	**1** **1** **1** **1** **1** **1** **1** **1**	No

TABLE 2-14 Changes Detected by Parity

Detecting odd numbers of changed bits leads to a change detection rate of about 50 percent, which is not nearly good enough for our purposes. We can improve this rate with more parity bits (computing a second parity bit of bits 1, 3, 5, and 7, for example), but more parity bits increase the size of the fingerprint; each time we increase the fingerprint size we also increase the size of storing these fingerprints.

Parity signals only that a bit has been changed; it does not identify which bit has been changed, much less when, how, or by whom. On hardware storage devices, a code called a ***cyclic redundancy*** check detects errors in recording and playback. Some more complex codes, known as ***error correction codes***, can detect multiple-bit errors (two or more bits changed in a data group) and may be able to pinpoint the changed bits (which are the bits to reset to correct the modification). Fingerprint size, error detection rate, and correction lead us to more powerful codes.

Signatures

The most powerful technique to demonstrate authenticity is a digital signature. Like its counterpart on paper, a digital signature is a way by which a person or organization can affix a bit pattern to a file such that it implies confirmation, pertains to that file only, cannot be forged, and demonstrates authenticity. We want a means by which one party can sign something and, as on paper, have the signature remain valid for days, months, years —indefinitely. Furthermore, the signature must convince all who access the file. Of course, as with most conditions involving digital methods, the caveat is that the assurance is limited by the assumed skill and energy of anyone who would try to defeat the assurance.

A digital signature often uses asymmetric or public key cryptography. As you just saw, a public key protocol is useful for exchange of cryptographic keys between two parties who have no other basis for trust. Unfortunately, the public key cryptographic protocols involve several sequences of messages and replies, which can be time consuming if either party is not immediately available to reply to the latest request. It would be useful to have a technique by which one party could reliably precompute some protocol steps and leave them in a safe place so that the protocol could be carried out even if only one party were active. This situation is similar to the difference between a bank teller and an ATM. You can obtain cash, make a deposit or payment, or check your balance because the bank has pre-established steps for an ATM to handle those simple activities 24 hours a day, even if the bank is not open. But if you need a certified check or foreign currency, you may need to interact directly with a bank agent.

In this section we define digital signatures and compare their properties to those of handwritten signatures on paper. We then describe the infrastructure surrounding digital signatures that lets them be recognizable and valid indefinitely.

رومانی صبحی رزق جرجس مطر

رومانی صبحی رزق جرجس مطر

Components and Characteristics of Signatures

A digital signature is just a binary object associated with a file. But if we want that signature to have the force of a paper-based signature, we need to understand the properties of human signatures. Only then can we express requirements for our digital version.

Properties of Secure Paper-Based Signatures

Consider a typical situation that parallels a common human need: an order to transfer funds from one person to another. In other words, we want to be able to send the electronic equivalent of a computerized check. We understand the properties of this transaction for a conventional paper check:

- A check is a tangible object authorizing a financial transaction.
- The signature on the check confirms authenticity because (presumably) only the legitimate signer can produce that signature.
- In the case of an alleged forgery, a third party can be called in to judge authenticity.
- Once a check is cashed, it is canceled so that it cannot be reused.
- The paper check is not alterable. Or, most forms of alteration are easily detected.

2025/2026 2025/2026 2025/2026

Transacting business by check depends on tangible objects in a prescribed form. But tangible objects do not exist for transactions on computers. Therefore, authorizing payments by computer requires a different model. Let us consider the requirements of such a situation, from the standpoint both of a bank and of a user.

Properties of Digital Signatures

Suppose Sheila sends her bank a message authorizing it to transfer $100 to Rob. Sheila's bank must be able to verify and prove that the message really came from Sheila if she should later disavow sending the message. (This property is called non-repudiation.) The bank also wants to know that the message is entirely Sheila's, that it has not been altered along the way. For her part, Sheila wants to be certain that her bank cannot forge such messages. (This property is called authenticity.) Both parties want to be sure that the message is new, not a reuse of a previous message, and that it has not been altered during transmission. Using electronic signals instead of paper complicates this process.

But we have ways to make the process work. A digital signature is a protocol that produces the same effect as a real signature: It is a mark that only the sender can make but that other people can easily recognize as belonging to the sender. Just like a real signature, a digital signature confirms agreement to a message.

A digital signature must meet two primary conditions:

- It must be unforgeable. If person S signs message M with signature Sig(S,M), no one else can produce the pair [M,Sig(S,M)].
- It must be authentic. If a person R receives the pair [M, Sig(S,M)] purportedly from S, R can check that the signature is really from S. Only S could have created this signature, and the signature is firmly attached to M.

These two requirements, shown in Figure 2-26, are the major hurdles in computer transactions. Two more properties, also drawn from parallels with the paper-based environment, are desirable for transactions completed with the aid of digital signatures:

- It is not alterable. After being transmitted, M cannot be changed by S, R, or an interceptor.
- It is not reusable. A previous message presented again will be instantly detected by R.

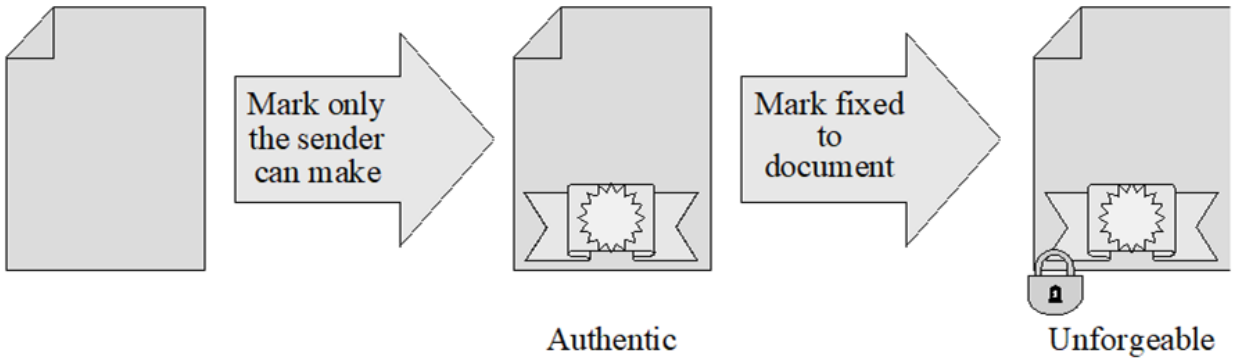


FIGURE 2-26 Digital Signature Requirements

To see how digital signatures work, we first present a mechanism that meets the first two requirements. We then add to that solution to satisfy the other requirements. We develop digital signatures in pieces: first building a piece to address alterations, then describing a way to ensure authenticity, and finally developing a structure to establish identity. Eventually all these parts tie together in a conceptually simple framework.

رومانی صبحی رزق جرجس مطر

We have just described the pieces for a digital signature: public key cryptography and secure message digests. These two pieces together are technically enough to make a digital signature, but they do not address authenticity. For that, we need a structure that binds a user's identity and public key in a trustworthy way. Such a structure is called a certificate. Finally, we present an infrastructure for transmitting and validating certificates.

رومانی صبحی رزق جرجس مطر

Public Keys for Signatures

Public key encryption systems are ideally suited to signing. For simple notation, let us assume that the public key encryption for user U is accessed through $E(M,K_U)$ and that the private key transformation for U is written as $D(M,K_U)$. We can think of E as the privacy transformation (since only U can decrypt it) and D as the authenticity transformation (since only U can produce it). Remember, however, that under some asymmetric algorithms such as RSA, D and E are commutative and either one can be applied to any message. Thus,

$$D(E(M, K_U), K_U) = M = E(D(M, K_U), K_U)$$

If S wishes to send M to R, S uses the authenticity transformation to produce $D(M, K_S)$. S then sends $D(M, K_S)$ to R. R decodes the message with the public key transformation of S, computing $E(D(M, K_S), K_S) = M$. Since only S can create a message that makes sense under $E(–,K_S)$, the message must genuinely have come from S. This test satisfies the authenticity requirement.

2025/2026 2025/2026 2025/2026

R will save $D(M, K_S)$. If S should later allege that the message is a forgery (not really from S), R can simply show M and $D(M, K_S)$. Anyone can verify that since $D(M, K_S)$ is transformed to M with the public key transformation of S—but only S could have produced $D(M, K_S)$—then $D(M, K_S)$ must be from S. This test satisfies the unforgeable requirement.

There are other approaches to signing; some use symmetric encryption, others use asymmetric. The approach shown here illustrates how the protocol can address the requirements for unforgeability and authenticity. To add secrecy, S applies $E(M, K_R)$ as shown in Figure 2-27.

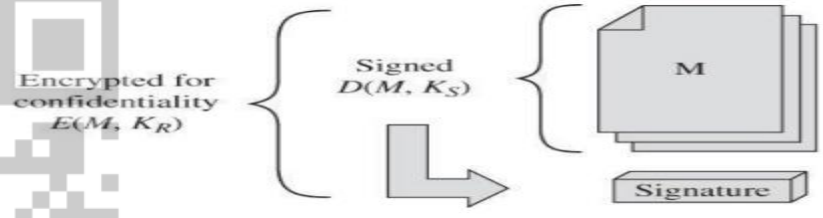


FIGURE 2-27 Use of Two Keys in an Asymmetric Digital Signature

Exercises

1. Describe each of the following four kinds of access control mechanisms in terms of (a) ease of determining authorized access during execution, (b) ease of adding access for a new subject, (c) ease of deleting access by a subject, and (d) ease of creating a new object to which all subjects by default have access.
   - per-subject access control list (that is, one list for each subject tells all the objects to which that subject has access)
   - per-object access control list (that is, one list for each object tells all the subjects who have access to that object)
   - access control matrix
   - capability

2. Suppose a per-subject access control list is used. Deleting an object in such a system is inconvenient because all changes must be made to the control lists of all subjects who did have access to the object. Suggest an alternative, less costly means of handling deletion.

3. File access control relates largely to the secrecy dimension of security. What is the relationship between an access control matrix and the integrity of the objects to which access is being controlled?

4. One feature of a capability-based protection system is the ability of one process to transfer a copy of a capability to another process. Describe a situation in which one process should be able to transfer a capability to another.

5. Suggest an efficient scheme for maintaining a per-user protection scheme. That is, the system maintains one directory per user, and that directory lists all the objects to which the user is allowed access. Your design should address the needs of a system with 1000 users, of whom no more than 20 are active at any time. Each user has an average of 200 permitted objects; there are 50,000 total objects in the system.

6. Calculate the timing of password-guessing attacks:

(a) If passwords are three uppercase alphabetic characters long, how much time would it take to determine a particular password, assuming that testing an individual password requires 5 seconds? How much time if testing requires 0.001 seconds?

(b) Argue for a particular amount of time as the starting point for "secure." That is, suppose an attacker plans to use a brute-force attack to determine a password. For what value of x (the total

2025/2026 2025/2026 2025/2026

amount of time to try as many passwords as necessary) would the attacker find this attack prohibitively long?

(c) If the cutoff between “insecure” and “secure” were x amount of time, how long would a secure password have to be? State and justify your assumptions regarding the character set from which the password is selected and the amount of time required to test a single password.

7. Design a protocol by which two mutually suspicious parties can authenticate each other. Your protocol should be usable the first time these parties try to authenticate each other.

8. List three reasons people might be reluctant to use biometrics for authentication. Can you think of ways to counter those objections?

9. False positive and false negative rates can be adjusted, and they are often complementary: Lowering one raises the other. List two situations in which false negatives are significantly more serious than false positives.

10. In a typical office, biometric authentication might be used to control access to employees and registered visitors only. We know the system will have some false negatives, some employees falsely denied access, so we need a human override, someone who can examine the employee and allow access in spite of the failed authentication. Thus, we need a human guard at the door to handle problems, as well as the authentication device; without biometrics we would have had just the guard. Consequently, we have the same number of personnel with or without biometrics, plus we have the added cost to acquire and maintain the biometrics system. Explain the security advantage in this situation that justifies the extra expense.

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

رومانی صبحی رزق جرجس مطر

Chapter 3

Programs and Programming

Introduction

Programs are simple things but they can wield mighty power. Think about them for a minute: Programs are just strings of 0s and 1s, representing elementary machine commands such as move one data item, compare two data items, or branch to a different command. Those primitive machine commands implement higher-level programming language constructs such as conditionals, repeat loops, case selection, and arithmetic and string operations. And those programming language constructs give us pacemaker functions, satellite control, smart-home technology, traffic management, and digital photography, not to mention streaming video and social networks. The Intel 32- and 64-bit instruction set has about 30 basic primitives (such as move, compare, branch, increment and decrement, logical operations, arithmetic operations, trigger I/O, generate and service interrupts, push, pop, call, and return) and specialized instructions to improve performance on computations such as floating point operations or cryptography. These few machine commands are sufficient to implement the vast range of programs we know today.

2025/2026 2025/2026 2025/2026

Most programs are written in higher-level languages such as Java, C, C++, Perl, or Python; programmers often use libraries of code to build complex programs from pieces written by others. But most people are not programmers; instead, they use already written applications for word processing, web browsing, graphics design, accounting, and the like without knowing anything about the underlying program code. People do not expect to need to understand how power plants operate in order to turn on an electric light. But if the light does not work, the problem could be anywhere from the power plant to the light bulb, and suddenly the user needs to trace potential problems from one end to the other. Although the user does not need to become a physicist or electrical engineer, a general understanding of electricity helps determine how to overcome the problem, or at least how to isolate faults under the user's control (burned out bulb, unplugged lamp).

In this chapter we describe security problems in programs and programming. As with the light, a problem can reside anywhere between the machine hardware and the user interface. Two or more problems may combine in negative ways, some problems can be intermittent or occur only when some other condition is present, and the impact of problems can range from annoying (perhaps not even perceptible) to catastrophic.

Security failures can result from intentional or nonmalicious causes; both can cause harm.

In Chapter 1 we introduce the notion of motive, observing that some security problems result from nonmalicious oversights or blunders, but others are intentional. A malicious attacker can exploit a nonmalicious flaw to cause real harm. Thus, we now study several common program failings to show how simple errors during programming can lead to large-scale problems during execution. Along the way we describe real attacks that have been caused by program flaws. (We use the term flaw because many security professionals use that term or the more evocative term bug. However, as you can see in Sidebar 3-1, the language for describing program problems is not universal.)

2025/2026 2025/2026 2025/2026

Sidebar 3-1 The Terminology of (Lack of) Quality

Thanks to Admiral Grace Murray Hopper, we casually call a software problem a "bug." [KID98] But that term can mean different things depending on context: a mistake in interpreting a requirement, a syntax error in a piece of code, or the (as-yet-unknown) cause of a system crash. The Institute of Electronics and Electrical Engineers (IEEE) suggests using a standard terminology (in IEEE Standard 729) for describing bugs in our software products [IEE83].

When a human makes a mistake, called an error, in performing some software activity, the error may lead to a fault, or an incorrect step, command, process, or data definition in a computer program, design, or documentation. For example, a designer may misunderstand a requirement and create a design that does not match the actual intent of the requirements analyst and the user. This design fault is an encoding of the error, and it can lead to other faults, such as incorrect code and an incorrect description in a user manual. Thus, a single error can generate many faults, and a fault can reside in any development or maintenance product.

A failure is a departure from the system's required behavior. It can be discovered before or after system delivery, during testing, or during operation and maintenance. Since the requirements

documents can contain faults, a failure indicates that the system is not performing as required, even though it may be performing as specified.

Thus, a fault is an inside view of the system, as seen by the eyes of the developers, whereas a failure is an outside view: a problem that the user sees. Every failure has at least one fault as its root cause. But not every fault corresponds to a failure; for example, if faulty code is never executed or a particular state is never entered, the fault will never cause the code to fail.

Although software engineers usually pay careful attention to the distinction between faults and failures, security engineers rarely do. Instead, security engineers use flaw to describe both faults and failures. In this book, we use the security terminology; we try to provide enough context so that you can understand whether we mean fault or failure.

3.1 Unintentional (Nonmalicious) Programming Oversights

Programs and their computer code are the basis of computing. Without a program to guide its activity, a computer is pretty useless. Because the early days of computing offered few programs for general use, early computer users had to be programmers too—they wrote the code and then ran it to accomplish some task. Today's computer users sometimes write their own code, but more often they buy programs off the shelf; they even buy or share code components and then modify them for their own uses. And all users gladly run programs all the time: spreadsheets, music players, word processors, browsers, email handlers, games, simulators, and more. Indeed, code is initiated in myriad ways, from turning on a mobile phone to pressing "start" on a coffee-maker or microwave oven. But as the programs have become more numerous and complex, users are more frequently unable to know what the program is really doing or how.

More importantly, users seldom know whether the program they are using is producing correct results. If a program stops abruptly, text disappears from a document, or music suddenly skips passages, code may not be working properly. (Sometimes these interruptions are intentional, as when a CD player skips because the disk is damaged or a medical device program stops in order to prevent an injury.) But if a spreadsheet produces a result that is off by a small amount or an automated drawing package doesn't align objects exactly, you might not notice—or you notice but blame yourself instead of the program for the discrepancy.

These flaws, seen and unseen, can be cause for concern in several ways. As we all know, programs are written by fallible humans, and program flaws can range from insignificant to

2025/2026 2025/2026 2025/2026

catastrophic. Despite significant testing, the flaws may appear regularly or sporadically, perhaps depending on many unknown and unanticipated conditions.

Program flaws can have two kinds of security implications: They can cause integrity problems leading to harmful output or action, and they offer an opportunity for exploitation by a malicious actor. We discuss each one in turn.

- A program flaw can be a fault affecting the correctness of the program's result —that is, a fault can lead to a failure. Incorrect operation is an integrity failing.

  As we saw in Chapter 1, integrity is one of the three fundamental security properties of the C-I-A triad. Integrity involves not only correctness but also accuracy, precision, and consistency. A faulty program can also inappropriately modify previously correct data, sometimes by overwriting or deleting the original data. Even though the flaw may not have been inserted maliciously, the outcomes of a flawed program can lead to serious harm.
- On the other hand, even a flaw from a benign cause can be exploited by someone malicious. If an attacker learns of a flaw and can use it to manipulate the program's behavior, a simple and nonmalicious flaw can become part of a malicious attack.

2025/2026 2025/2026 2025/2026

Benign flaws can be—often are—exploited for malicious impact.

Thus, in both ways, program correctness becomes a security issue as well as a general quality problem. In this chapter we examine several programming flaws that have security implications. We also show what activities during program design, development, and deployment can improve program security.

Buffer Overflow

We start with a particularly well known flaw, the buffer overflow. Although the basic problem is easy to describe, locating and preventing such difficulties is challenging. Furthermore, the impact of an overflow can be subtle and disproportionate to the underlying oversight. This outsized effect is due in part to the exploits that people have achieved using overflows. Indeed, a buffer overflow is often the initial toehold for mounting a more damaging strike. Most buffer overflows are simple programming oversights, but they can be used for malicious ends. See Sidebar 3-2 for the story of a search for a buffer overflow.

رومانی صبحی رزق جرجس مطر

Buffer overflows often come from innocent programmer oversights or failures to document and check for excessive data.

This example was not the first buffer overflow, and in the intervening time—approaching two decades—far more buffer overflows have been discovered. However, this example shows clearly the mind of an attacker. In this case, David was trying to improve security—he happened to be working for one of this book's authors at the time— but attackers work to defeat security for reasons such as those listed in Chapter 1. We now investigate sources of buffer overflow attacks, their consequences, and some countermeasures.

Anatomy of Buffer Overflows

A string overruns its assigned space or one extra element is shoved into an array; what's the big deal, you ask? To understand why buffer overflows are a major security issue, you need to understand how an operating system stores code and data.

As noted above, buffer overflows have existed almost as long as higher-level programming languages with arrays. Early overflows were simply a minor annoyance to programmers and users, a cause of errors and sometimes even system crashes. More recently, however, attackers have used them as vehicles to cause first a system crash and then a controlled failure with a serious security implication. The large number of security vulnerabilities based on buffer overflows shows that developers must pay more attention now to what had previously been thought to be just a minor annoyance.

Sidebar 3-2 My Phone Number is 5656 4545 7890 1234 2929 2929 2929 …

In 1999, security analyst David Litchfield [LIT99] was intrigued by buffer overflows. He had both an uncanny sense for the kind of program that would contain overflows and the patience to search for them diligently. He happened onto the Microsoft Dialer program, dialer.exe.

Dialer was a program for dialing a telephone. Before cell phones, WiFi, broadband, and DSL, computers were equipped with modems by which they could connect to the land-based telephone network; a user would dial an Internet service provider and establish a connection across a standard voice telephone line. Many people shared one line between voice and computer (data) communication. You could look up a contact's phone number, reach for the telephone, dial the number, and converse; but the computer's modem could dial the same line, so you could feed the number to the modem from an electronic contacts list, let the modem dial your number, and pick

2023/2026 2025/2026 2025/2026

up the receiver when your called party answered. Thus, Microsoft provided Dialer, a simple utility program to dial a number with the modem. (As of 2014, dialer.exe was still part of Windows 10, although the buffer overflow described here was patched shortly after David reported it.)

David reasoned that Dialer had to accept phone numbers of different lengths, given country variations, outgoing access codes, and remote signals (for example, to enter an extension number). But he also suspected there would be an upper limit. So he tried dialer.exe with a 20-digit phone number and everything worked fine. He tried 25 and 50, and the program still worked fine. When he tried a 100-digit phone number, the program crashed. The programmer had probably made an undocumented and untested decision that nobody would ever try to dial a 100-digit phone number … except David.

Having found a breaking point, David then began the interesting part of his work: Crashing a program demonstrates a fault, but exploiting that flaw shows how serious the fault is. By more experimentation, David found that the number to dial was written into the stack, the data structure that stores parameters and return addresses for embedded program calls. The dialer.exe program is treated as a program call by the operating system, so by controlling what dialer.exe overwrote, David could redirect execution to continue anywhere with any instructions he wanted. The full details of his exploitation are given in [LIT99].

2025/2026 2025/2026 2025/2026

Memory Allocation

Memory is a limited but flexible resource; any memory location can hold any piece of code or data. To make managing computer memory efficient, operating systems jam one data element next to another, without regard for data type, size, content, or purpose*. Users and programmers seldom know, much less have any need to know, precisely which memory location a code or data item occupies.

* Some operating systems do separate executable code from nonexecutable data, and some hardware can provide different protection to memory addresses containing code as opposed to data. Unfortunately, however, for reasons including simple design and performance, most operating systems and hardware do not implement such separation. We ignore the few exceptions in this chapter because the security issue of buffer overflow applies even within a more constrained system. Designers and programmers need to be aware of buffer overflows,

رومانی صبحی رزق جرجس مطر

because a program designed for use in one environment is sometimes transported to another less protected one.

Computers use a pointer or register known as a program counter that indicates the next instruction. As long as program flow is sequential, hardware bumps up the value in the program counter to point just after the current instruction as part of performing that instruction. Conditional instructions such as IF(), branch instructions such as loops (WHILE, FOR) and unconditional transfers such as GOTO or CALL divert the flow of execution, causing the hardware to put a new destination address into the program counter. Changing the program counter causes execution to transfer from the bottom of a loop back to its top for another iteration. Hardware simply fetches the byte (or bytes) at the address pointed to by the program counter and executes it as an instruction. Instructions and data are all binary strings; only the context of use says a byte, for example, 0x41 represents the letter A, the number 65, or the instruction to move the contents of register 1 to the stack pointer. If you happen to put the data string “A” in the path of execution, it will be executed as if it were an instruction. In Figure 3-1 we show a typical arrangement of the contents of memory, showing code, local data, the heap (storage for dynamically created data), and the stack (storage for subtask call and return data). As you can see, instructions move from the bottom (low addresses) of memory up; left unchecked, execution would proceed through the local data area and into the heap and stack. Of course, execution typically stays within the area assigned to program code.

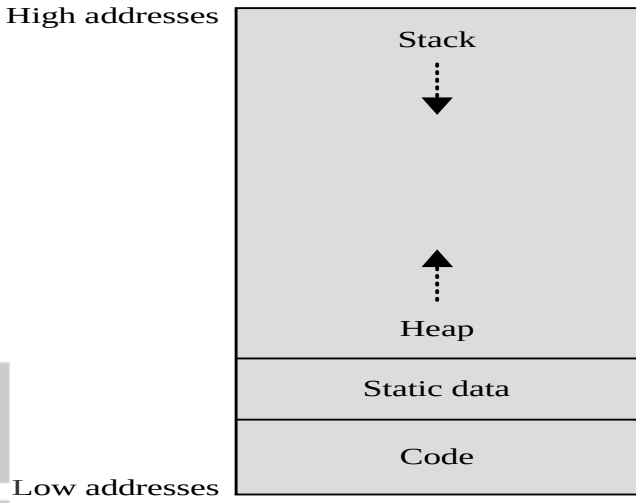


FIGURE 3-1 Typical Memory Organization

Not all binary data items represent valid instructions. Some do not correspond to any defined operation, for example, operation 0x78 on a machine whose instructions are all numbers between 0x01 and 0x6f. Other invalid forms attempt to use nonexistent hardware features, such as a reference to register 9 on a machine with only eight hardware registers.

To help operating systems implement security, some hardware recognizes more than one mode of instruction: so-called privileged instructions that can be executed only when the processor is running in a protected mode. Trying to execute something that does not correspond to a valid instruction or trying to execute a privileged instruction when not in the proper mode will cause a program fault. When hardware generates a program fault, it stops the current thread of execution and transfers control to code that will take recovery action, such as halting the current process and returning control to the supervisor.

Code and Data

Before we can explain the real impact of buffer overflows, we need to clarify one point: Code, data, instructions, the operating system, complex data structures, user programs, strings, downloaded utility routines, hexadecimal data, decimal data, character strings, code libraries, photos, and everything else in memory are just strings of 0s and 1s; think of it all as bytes, each containing a number. The computer pays no attention to how the bytes were produced or where they came from. Each computer instruction determines how data values are interpreted: An Add instruction implies the data item is interpreted as a number, a Move instruction applies to any string of bits of arbitrary form, and a Jump instruction assumes the target is an instruction. But at the machine level, nothing prevents a Jump instruction from transferring into a data field or an Add command operating on an instruction, although the results may be unpleasant. Code and data are bit strings interpreted in a particular way.

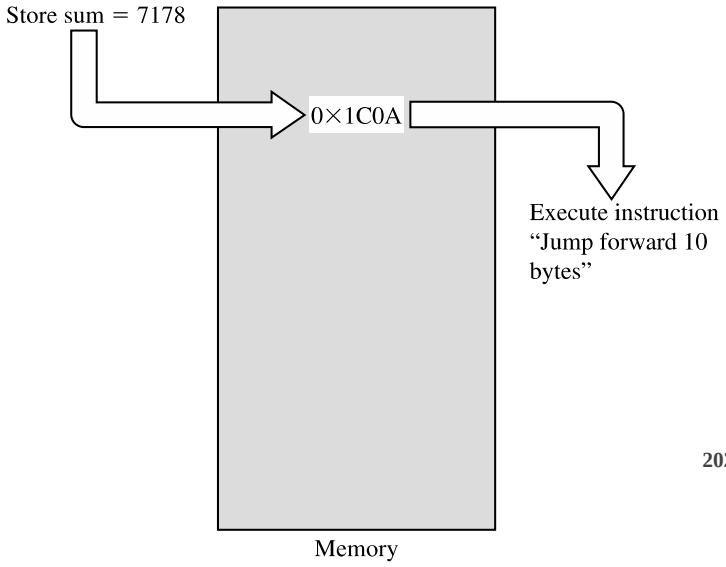
In memory, code is indistinguishable from data. The origin of code (respected source or attacker) is also not visible.

You do not usually try to execute data values or perform arithmetic on instructions. But if 0x1C is the operation code for a Jump instruction, and the form of a Jump instruction is 1C displ, meaning execute the instruction at the address displ bytes ahead of this instruction, the string 0x1C0A is interpreted as jump forward 10 bytes. But, as shown in Figure 3-2, that same bit

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

pattern represents the two-byte decimal integer 7178. So storing the number 7178 in a series of instructions is the same as having programmed a Jump. Most higher-level-language programmers do not care about the representation of instructions in memory, but curious investigators can readily find the correspondence. Manufacturers publish references specifying precisely the behavior of their chips, and utility programs such as compilers, assemblers, and disassemblers help interested programmers develop and interpret machine instructions.



2025/2026
2025/2026

FIGURE 3-2 Bit Patterns Can Represent Data or Instructions

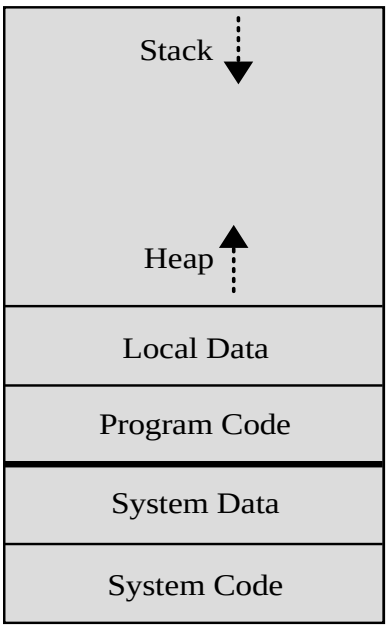
Usually we do not treat code as data, or vice versa; attackers sometimes do, however, especially in memory overflow attacks. The attacker’s trick is to cause data to spill over into executable code and then to select the data values such that they are interpreted as valid instructions to perform the attacker’s goal. For some attackers this is a two-step goal: First cause the overflow and then experiment with the ensuing action to cause a desired, predictable result, just as David did.

Harm from an Overflow

Let us suppose a malicious person understands the damage that can be done by a buffer overflow; that is, we are dealing with more than simply a normal, bumbling programmer. The malicious programmer thinks deviously: What data values could I insert to cause mischief or damage, and what planned instruction codes could I force the system to execute? There are many

possible answers, some of which are more malevolent than others. Here, we present two buffer overflow attacks that are used frequently. (See [ALE96] for more details.)

First, the attacker may replace code in the system space. As shown in Figure 3-3, memory organization is not as simple as shown in Figure 3-1. The operating system's code and data coexist with a user's code and data. The heavy line between system and user space is only to indicate a logical separation between those two areas; in practice, the distinction is not so solid.



2025/2026 2025/2026

FIGURE 3-3 Memory Organization with User and System Areas

Remember that every program is invoked by an operating system that may run with higher privileges than those of a regular program. Thus, if the attacker can gain control by masquerading as the operating system, the attacker can execute commands in a powerful role. Therefore, by replacing a few instructions right after returning from his or her own procedure, the attacker regains control from the operating system, possibly with raised privileges. This technique is called privilege escalation. If the buffer overflows into system code space, the attacker merely inserts overflow data that correspond to the machine code for instructions.

In the other kind of attack, the intruder may wander into an area called the stack and heap. Subprocedure calls are handled with a stack, a data structure in which the most recent item inserted is the next one removed (last arrived, first served). This structure works well because procedure calls can be nested, with each return causing control to transfer back to the immediately preceding routine at its point of execution. Each time a procedure is called, its

parameters, the return address (the address immediately after its call), and other local values are pushed onto a stack. An old stack pointer is also pushed onto the stack, and a stack pointer register is reloaded with the address of these new values. Control is then transferred to the subprocedure.

As the subprocedure executes, it fetches parameters that it finds by using the address pointed to by the stack pointer. Typically, the stack pointer is a register in the processor. Therefore, by causing an overflow into the stack, the attacker can change either the old stack pointer (changing the context for the calling procedure) or the return address (causing control to transfer where the attacker intends when the subprocedure returns). Changing the context or return address allows the attacker to redirect execution to code written by the attacker.

In both these cases, the assailant must experiment a little to determine where the overflow is and how to control it. But the work to be done is relatively small—probably a day or two for a competent analyst. These buffer overflows are carefully explained in a paper by Mudge [MUD95] (real name, Pieter Zatko) of the famed l0pht computer security group. Pincus and Baker [PIN04] reviewed buffer overflows ten years after Mudge and found that, far from being a minor aspect of attack, buffer overflows had been a significant attack vector and had spawned several other new attack types. That pattern continues today.

2025/2026 2025/2026 2025/2026

An alternative style of buffer overflow occurs when parameter values are passed into a routine, especially when the parameters are passed to a web server on the Internet. Parameters are passed in the URL line, with a syntax similar to

Click here to view code image

http://www.somesite.com/subpage/userinput.asp?

parm1=(808)555-1212

In this example, the application script userinput receives one parameter, parm1 with value (808)555-1212 (perhaps a U.S. telephone number). The web browser on the caller's machine will accept values from a user who probably completes fields on a form. The browser encodes those values and transmits them back to the server's web site.

The attacker might question what the server would do with a really long telephone number, say, one with 500 or 1000 digits. This is precisely the question David asked in the example we described in Sidebar 3-2. Passing a very long string to a web server is a slight variation on the classic buffer overflow, but no less effective.

رومانی صبحی رزق جرجس مطر

رومانی صبحی رزق جرجس مطر

Overwriting Memory

Now think about a buffer overflow. If you write an element past the end of an array or you store an 11-byte string in a 10-byte area, that extra data has to go somewhere; often it goes immediately after the last assigned space for the data.

A buffer (or array or string) is a space in which data can be held. A buffer resides in memory. Because memory is finite, a buffer's capacity is finite. For this reason, in many programming languages the programmer must declare the buffer's maximum size so that the compiler can set aside that amount of space.

Let us look at an example to see how buffer overflows can happen. Suppose a C language program contains the declaration

```
char sample[10];
```

The compiler sets aside 10 bytes to store this buffer, one byte for each of the 10 elements of the array, denoted sample[0] through sample[9]. Now we execute the statement

```
sample[10] = 'B';
```

The subscript is out of bounds (that is, it does not fall between 0 and 9), so we have a problem. The nicest outcome (from a security perspective) is for the compiler to detect the problem and mark the error during compilation, which the compiler could do in this case.

2025/2026 2025/2036 2025/2026

However, if the statement were

```
sample[i] = 'B';
```

then the compiler could not identify the problem until i was set during execution either to a proper value (between 0 and 9) or to an out-of-bounds subscript (less than 0 or greater than 9). It would be useful if, during execution, the system produced an error message warning of a subscript exception. Unfortunately, in some languages, buffer sizes do not have to be predefined, so there is no way to detect an out-of-bounds error. More importantly, the code needed to check each subscript against its potential maximum value takes time and space during execution, and resources are applied to catch a problem that occurs relatively infrequently. Even if the compiler were careful in analyzing the buffer declaration and use, this same problem can be caused with pointers, for which there is no reasonable way to define a proper limit. Thus, some compilers do not generate the code to check for exceeding bounds.

رومانی صبحی رزق جرجس مطر

Implications of Overwriting Memory

Let us more closely examine the problem of overwriting memory. Be sure to recognize that the potential overflow causes a serious problem only in some instances. The problem's occurrence depends on what is adjacent to the array sample. For example, suppose each of the ten elements of the array sample is filled with the letter A and the erroneous reference uses the letter B, as follows:

Click here to view code image

```
for (i=0; i<=9; i++)
    sample[i] = 'A';
sample[10] = 'B'
```

All program and data elements are in memory during execution, sharing space with the operating system, other code, and resident routines. So four cases must be considered in deciding where the 'B' goes, as shown in Figure 3-4. If the extra character overflows into the user's data space, it simply overwrites an existing variable value (or it may be written into an as-yet unused location), perhaps affecting the program's result but affecting no other program or data.

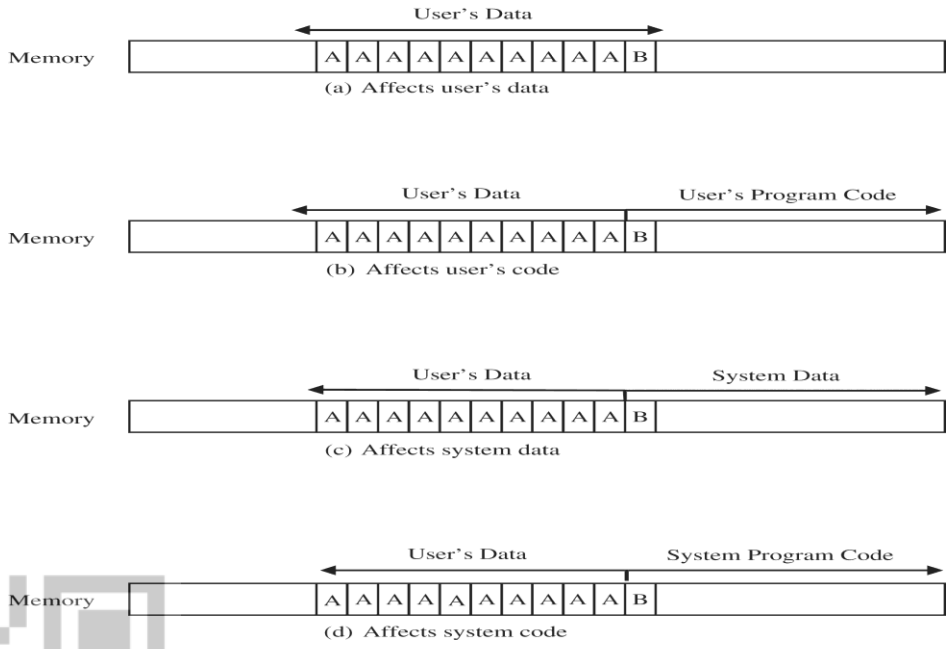


FIGURE 3-4 One-Character Overflow

In the second case, the 'B' goes into the user's program area. If it overlays an already executed instruction (which will not be executed again), the user should perceive no effect. If it overlays an instruction that is not yet executed, the machine will try to execute an instruction with

operation code 0x42, the internal code for the character ‘B’. If there is no instruction with operation code 0x42, the system will halt on an illegal instruction exception. Otherwise, the machine will use subsequent bytes as if they were the rest of the instruction, with success or failure depending on the meaning of the contents. Again, only the user is likely to experience an effect.

The most interesting cases (from a security perspective) occur when the system owns the space immediately after the array that overflows. Spilling over into system data or code areas produces results similar to those for the user’s space: computing with a faulty value or trying to execute an operation.

Program procedures use both local data, data used strictly within one procedure, and shared or common or global data, which are shared between two or more procedures. Memory organization can be complicated, but we simplify the layout as in Figure 3-5. In that picture, local data are stored adjacent to the code of a procedure. Thus, as you can see, a data overflow either falls strictly within a data space or it spills over into an adjacent code area. The data end up on top of one of

2025/2026 2025/2026 2025/2026

- another piece of your data
- an instruction of yours
- data or code belonging to another program
- data or code belonging to the operating system

We consider each of these cases separately.

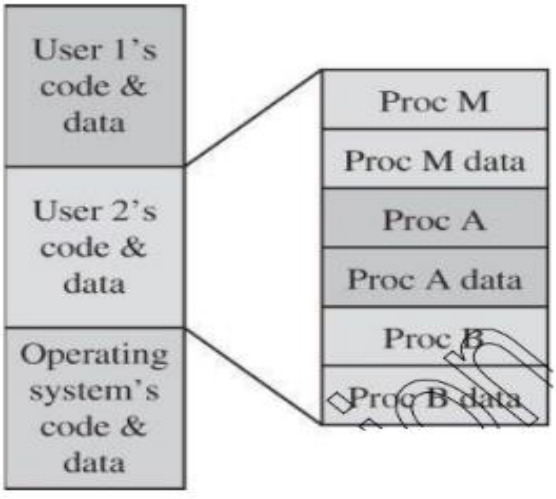


FIGURE 3-5 Memory of Different Procedures for Different Users

رومانی صبحی رزق جرجس مطر

Affecting Your Own Data

Modifying your own data, especially with an unintended value, will obviously affect your computing. Perhaps a loop will repeat too many or too few times, a sum will be compromised, or a date will become garbled. You can imagine these possibilities for yourself. The error may be so egregious that you will easily recognize something is wrong, but a more subtle failure may escape your notice, perhaps forever.

From a security standpoint, few system controls protect you from this kind of error: You own your data space and anything you want to store there is your business. Some, but not all, programming languages generate checking code for things like arrays to ensure that you store elements only within the space allocated. For this reason, the defensive programming technique (discussed later in this chapter) recommends that you always check to ensure that array elements and strings are within their boundaries. As Sidebar 3-3 demonstrates, sometimes such an error lies dormant for a long time.

Sidebar 3-3 Too Many Computers

2025/2026 2025/2026 2025/2026

The ARPANET, precursor to today's Internet, began operation in 1969. Stephen Crocker and Mary Bernstein [CRO89] exhaustively studied the root causes of 17 catastrophic failures of the ARPANET, failures that brought down the entire network or a significant portion of it.

As you might expect, many of these failures occurred during the early 1970s as use of the network caused flaws to surface. The final one of their 17 causes appeared only in 1988, nearly 20 years after the inception of the network. This disruption was caused by an overflow.

The original ARPANET network comprised hosts that connected to specialized communications processors called IMPs. Each interface message processor (IMP) controlled an individual subnetwork, much like today's routers; the IMPs connected to other IMPs through dedicated communications lines. For reliability, each IMP had at least two distinct paths to each other IMP. The IMP connections were added to a table dynamically as communication between two IMPs was required by network traffic.

Affecting an Instruction of Yours

Again, the failure of one of your instructions affects you, and systems give wide latitude to what you can do to yourself. If you store a string that does not represent a valid or permitted

instruction, your program may generate a fault and halt, returning control to the operating system. However, the system will try to execute a string that accidentally does represent a valid instruction, with effects depending on the actual value. Again, depending on the nature of the error, this faulty instruction may have no effect (if it is not in the path of execution or in a section that has already been executed), a null effect (if it happens not to affect code or data, such as an instruction to move the contents of register 1 to itself), or an unnoticed or readily noticed effect.

Destroying your own code or data is unpleasant, but at least you can say the harm was your own fault. Unless, of course, it wasn't your fault. One early flaw in Microsoft's Outlook involved the simple date field: A date is a few bytes long to represent a day, month, year, and time in GMT (Greenwich Mean Time) format. In a former version of Outlook, a message with a date of more than 1000 bytes exceeded the buffer space for message headers and ran into reserved space. Simply downloading such a message from a mail server would cause your system to crash, and each time you tried to restart, Outlook would try to reload the same message and crash again. In this case, you suffered harm from a buffer overflow involving only your memory area.

One program can accidentally modify code or data of another procedure that will not be executed until much later, so the delayed impact can be almost as difficult to diagnose as if the attack came from an unrelated, independent user. The most significant impact of a buffer overflow occurs when the excess data affect the operating system's code or data.

Modification of code and data for one user or another is significant, but it is not a major computer security issue. However, as we show in the next section, buffer overflows perpetrated on the operating system can have serious consequences.

Affecting the Operating System or a Critical Application

The same basic scenarios occur for operating system code or data as for users, although again there are important variations. Exploring these differences also leads us to consider motive, and so we shift from thinking of what are essentially accidents to intentional malicious acts by an attacker.

Because the mix of programs changes continually on a computing system, there is little opportunity to affect any one particular use. We now consider the case in which an attacker who has already overtaken an ordinary user now wants to overtake the operating system. Such an attack can let the attacker plant permanent code that is reactivated each time a machine is

رومانی صبحی رزق جرجس مطر
2025/2026 2025/2026 2025/2026

restarted, for example. Or the attack may expose data, for example, passwords or cryptographic keys that the operating system is entrusted to safeguard. So now let us consider the impact a (compromised) user can have on the operating system.

Users' code and data are placed essentially at random: wherever there is free memory of an appropriate size. Only by tracing through system memory allocation tables can you learn where your program and data appear in memory. However, certain portions of the operating system are placed at particular fixed locations, and other data are located at places that can easily be determined during execution. Fixed or easily determined location distinguishes operating system routines, especially the most critical ones, from a user's code and data.

A second distinction between ordinary users and the operating system is that a user runs without operating system privileges. The operating system invokes a user's program as if it were a subprocedure, and the operating system receives control back when the user's program exits. If the user can alter what the operating system does when it regains control, the user can force the operating system to execute code the user wants to run, but with elevated privileges (those of the operating system). Being able to modify operating system code or data allows the user (that is, an attacker acting as the user) to obtain effective privileged status.

2025/2026 2025/2026 2025/2026

Privilege escalation, executing attack code with higher system permissions, is a bonus for the attacker.

The call and return sequence operates under a well-defined protocol using a data structure called the stack. Aleph One (Elias Levy) describes how to use buffer overflows to overwrite the call stack [ALE96]. In the next section we show how a programmer can use an overflow to compromise a computer's operation.

The Stack and the Heap

The stack is a key data structure necessary for interchange of data between procedures, as we described earlier in this chapter. Executable code resides at one end of memory, which we depict as the low end; above it are constants and data items whose size is known at compile time; above that is the heap for data items whose size can change during execution; and finally, the stack. Actually, as shown earlier in Figure 3-1, the heap and stack are at opposite ends of the memory left over after code and local data.

رومانی صبحی رزق جرجس مطر

When procedure A calls procedure B, A pushes onto the stack its return address (that is, the current value of the program counter), the address at which execution should resume when B exits, as well as calling parameter values. Such a sequence is shown in Figure 3-6.

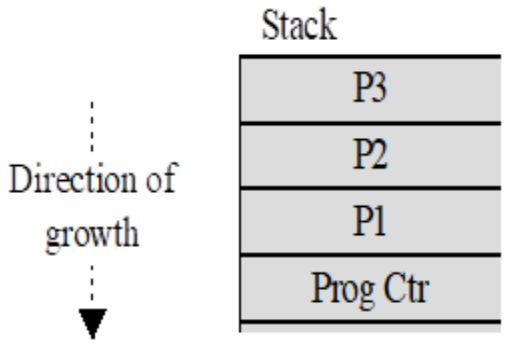


FIGURE 3-6 Parameters and Return Address

To help unwind stack data tangled because of a program that fails during execution, the stack also contains the pointer to the logical bottom of this program's section of the stack, that is, to the point just before where this procedure pushed values onto the stack. This data group of parameters, return address, and stack pointer is called a stack frame, as shown in Figure 3-7.

2025/2026 2025/2026 2025/2026

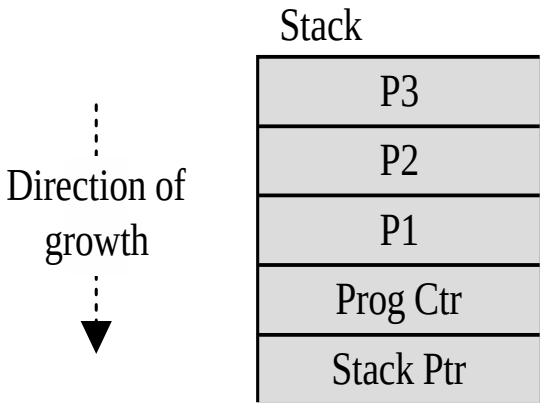


FIGURE 3-7 A Stack Frame

When one procedure calls another, the stack frame is pushed onto the stack to allow the two procedures to exchange data and transfer control; an example of the stack after procedure A calls B is shown in Figure 3-8.

رومانی صبحی رزق جرجس مطر

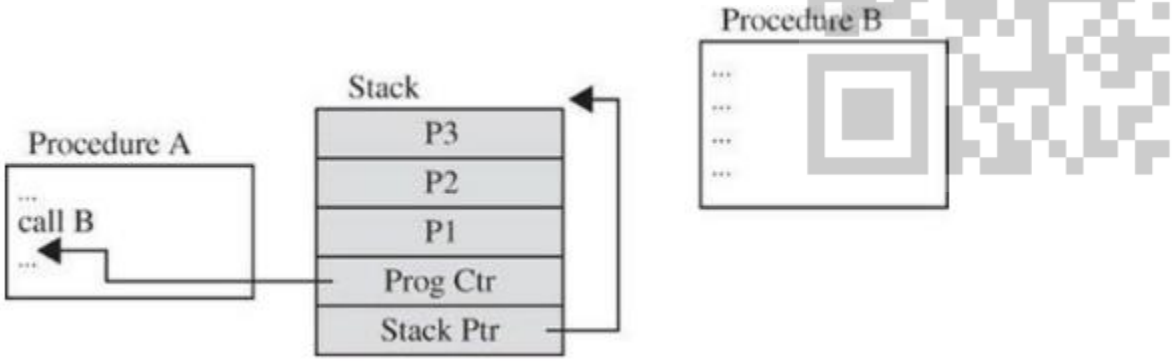


FIGURE 3-8 The Stack after a Procedure Call

Now let us consider a slightly deeper example: Suppose procedure A calls B that in turn calls C. After these two calls the stack will look as shown in Figure 3-7, with the return address to A on the bottom, then parameters from A to B, the return address from C to B, and parameters from B to C, in that order. After procedure C returns to B, the second stack frame is popped off the stack and it looks again like Figure 3-9.

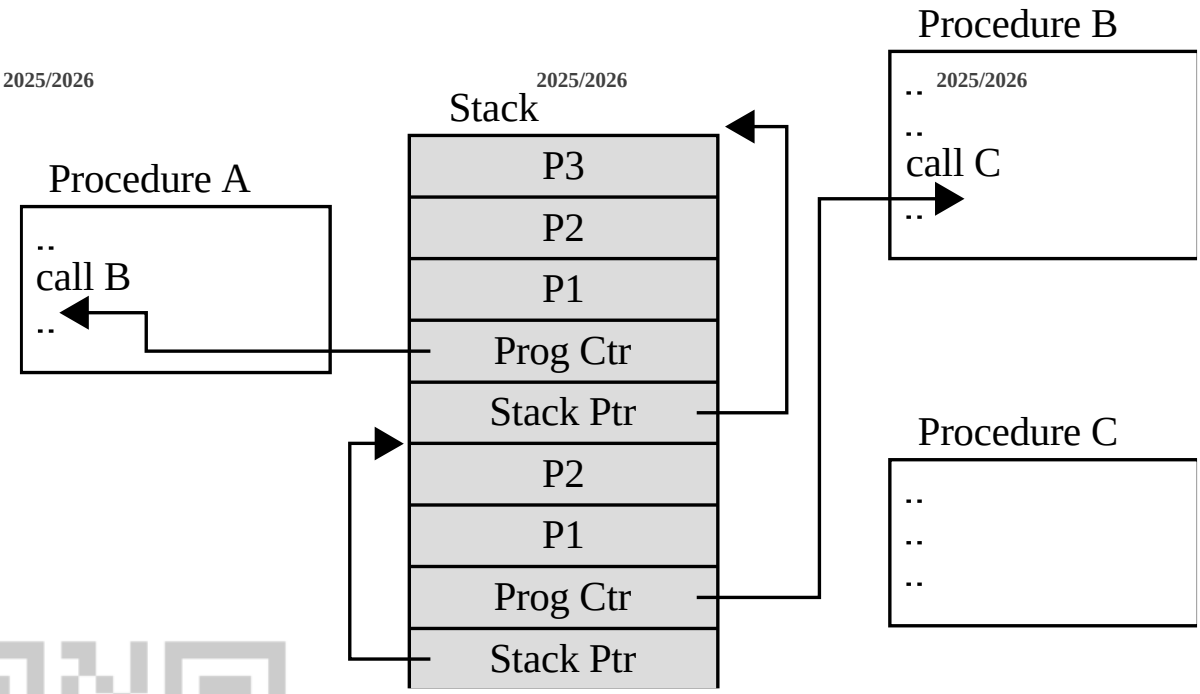


FIGURE 3-9 The Stack after Nested Procedure Calls

The important thing to notice in these figures is the program counter: If the attacker can overwrite the program counter, doing so will redirect program execution after the procedure returns, and that redirection is, in fact, a frequently seen step in exploiting a buffer overflow.

Overflow into system space can redirect execution immediately or on exit from the current called procedure.

Refer again to Figure 3-1 and notice that the stack is at the top of memory, growing downward, and something else, called the heap, is at the bottom growing up. As you have just seen, the stack is mainly used for nested calls to procedures. The ***heap*** provides space for dynamic data, that is, data items whose size is not known when a program is compiled.

If you declare an array of ten elements in the source code of a routine, the compiler allocates enough space for those ten elements, as well as space for constants and individual variables. But suppose you are writing a general-purpose sort routine that works on any data, for example, tables with arbitrarily many rows and columns of any kind of data. You might process an array of 100 integers, a table of 20,000 telephone numbers, or a structure of 2,000 bibliographic references with names, titles, and sources. Even if the table itself is passed as a parameter so that you do not need space to store it within your program, you will need some temporary space, for example, for variables to hold the values of two rows as you compare them and perhaps exchange their positions. Because you cannot know when you write your code how large a row will be, modern programming languages let you defer declaring the size of these variables until the program executes. During execution, code inserted by the compiler into your program determines the sizes and asks the operating system to allocate dynamic memory, which the operating system gets from the heap. The heap grows and shrinks as memory is allocated and freed for dynamic data structures.

2025/2026 2025/2026 2025/2026

As you can see in Figure 3-1, the stack and heap grow toward each other, and you can predict that at some point they might collide. Ordinarily, the operating system monitors their sizes and prevents such a collision, except that the operating system cannot know that you will write 15,000 bytes into a dynamic heap space for which you requested only 15 bytes, or 8 bytes into a 4-byte parameter, or four return parameter values into three parameter spaces.

The attacker wants to overwrite stack memory, sometimes called stack smashing, in a purposeful manner: Arbitrary data in the wrong place causes strange behavior, but particular data in a predictable location causes a planned impact. Here are some ways the attacker can produce effects from an overflow attack:

- Overwrite the program counter stored in the stack so that when this routine exits, control transfers to the address pointed at by the modified program counter address.

رومانی صبحی رزق جرجس مطر

- Overwrite part of the code in low memory, substituting the attacker's instructions for previous program statements.
- Overwrite the program counter and data in the stack so that the program counter now points into the stack, causing the data overwritten into the stack to be executed.

The common feature of these attack methods is that the attacker uses overflow data as code the victim will execute. Because this code runs under the authority of the victim, it carries the victim's privileges, and it can destroy the victim's data by overwriting it or can perform any actions the victim could, for example, sending email as if from the victim. If the overflow occurs during a system call, that is, when the system is running with elevated privileges, the attacker's code also executes with those privileges; thus, an attack that transfers control to the attacker by invoking one of the attacker's routines activates the attacker's code and leaves the attacker in control with privileges. And for many attackers the goal is not simply to destroy data by overwriting memory but also to gain control of the system as a first step in a more complex and empowering attack.

Buffer overflow attacks are interesting because they are the first example of a class of problems called data driven attacks. In a ***data driven attack*** the harm occurs by the data the attacker sends. Think of such an attack this way: A buffer overflows when someone stuffs too much into it. Most people accidentally put one more element in an array or append an additional character into a string. The data inserted relate to the application being computed. However, with a malicious buffer overflow the attacker, like David the nonmalicious researcher, carefully chooses data that will cause specific action, to make the program fail in a planned way. In this way, the selected data drive the impact of the attack.

Data driven attacks are directed by specially chosen data the attacker feeds a program as input.

Malicious exploitation of buffer overflows also exhibit one more important characteristic: They are examples of a multistep approach. Not only does the attacker overrun allocated space, but the attacker also uses the overrun to execute instructions to achieve the next step in the attack. The overflow is not a goal but a stepping stone to a larger purpose.

Buffer overflows can occur with many kinds of data, ranging from arrays to parameters to individual data items, and although some of them are easy to prevent (such as checking an

array's dimension before storing), others are not so easy. Human mistakes will never be eliminated, which means overflow conditions are likely to remain. In the next section we present a selection of controls that can detect and block various kinds of overflow faults.

Overflow Countermeasures

It would seem as if the countermeasure for a buffer overflow is simple: Check before you write. Unfortunately, that is not quite so easy because some buffer overflow situations are not directly under the programmer's control, and an overflow can occur in several ways.

Although buffer overflows are easy to program, no single countermeasure will prevent them. However, because of the prevalence and seriousness of overflows, several kinds of protection have evolved.

The most obvious countermeasure to overwriting memory is to stay within bounds. Maintaining boundaries is a shared responsibility of the programmer, operating system, compiler, and hardware. All should do the following:

- Check lengths before writing.
- Confirm that array subscripts are within limits.

2025/2026 2025/2026 2025/2026

- Double-check boundary condition code to catch possible off-by-one errors.
- Monitor input and accept only as many characters as can be handled.
- Use string utilities that transfer only a bounded amount of data.
- Check procedures that might overrun their space.
- Limit programs' privileges, so if a piece of code is overtaken maliciously, the violator does not acquire elevated system privileges as part of the compromise.

Programming Controls

Later in this chapter we study programming controls in general. You may already have encountered these principles of software engineering in other places. Techniques such as code reviews (in which people other than the programmer inspect code for implementation oversights) and independent testing (in which dedicated testers hypothesize points at which a program could fail) can catch overflow situations before they become problems.

رومانی صبحی رزق جرجس مطر

Language Features

Two features you may have noticed about attacks involving buffer overflows are that the attacker can write directly to particular memory addresses and that the language or compiler allows inappropriate operations on certain data types. Anthony (C.A.R.) Hoare [HOA81] comments on the relationship between language and design:

> Programmers are always surrounded by complexity; we cannot avoid it. Our applications are complex because we are ambitious to use our computers in ever more sophisticated ways. Programming is complex because of the large number of conflicting objectives for each of our programming projects. If our basic tool, the language in which we design and code our programs, is also complicated, the language itself becomes part of the problem rather than part of its solution.

Some programming languages have features that preclude overflows. For example, languages such as Java, .NET, Perl, and Python generate code to check bounds before storing data. The unchecked languages C, C++, and assembler language allow largely unlimited program access. To counter the openness of these languages, compiler writers have developed extensions and libraries that generate code to keep programs in check.

2025/2026 2025/2026 2025/2026

Code Analyzers

Software developers hope for a simple tool to find security errors in programs. Such a tool, called a static code analyzer, analyzes source code to detect unsafe conditions. Although such tools are not, and can never be, perfect, several good ones exist. Kendra Kratkiewicz and Richard Lippmann [KRA05] and the US-CERT Build Security In website at https://buildsecurityin.us-cert.gov/ contain lists of static code analyzers.

Separation

Another direction for protecting against buffer overflows is to enforce containment: separating sensitive areas from the running code and its buffers and data space. To a certain degree, hardware can separate code from data areas and the operating system.

Stumbling Blocks

Because overwriting the stack is such a common and powerful point of attack, protecting it becomes a priority.

رومانی صبحی رزق جرجس مطر

Refer again to Figure 3-8, and notice that each procedure call adds a new stack frame that becomes a distinct slice of the stack. If our goal is to protect the stack, we can do that by wrapping each stack frame in a protective layer. Such a layer is sometimes called a ***canary***, in reference to canary birds that were formerly taken into underground mines; the canary was more sensitive to limited oxygen, so the miners could notice the canary reacting before they were affected, giving the miners time to leave safely.

In this section we show how some manufacturers have developed cushions to guard against benign or malicious damage to the stack.

In a common buffer overflow stack modification, the program counter is reset to point into the stack to the attack code that has overwritten stack data. In Figure 3-10, the two parameters P1 and P2 have been overwritten with code to which the program counter has been redirected. (Two instructions is too short a set for many stack overflow attacks, so a real buffer overflow attack would involve more data in the stack, but the concept is easier to see with a small stack.)

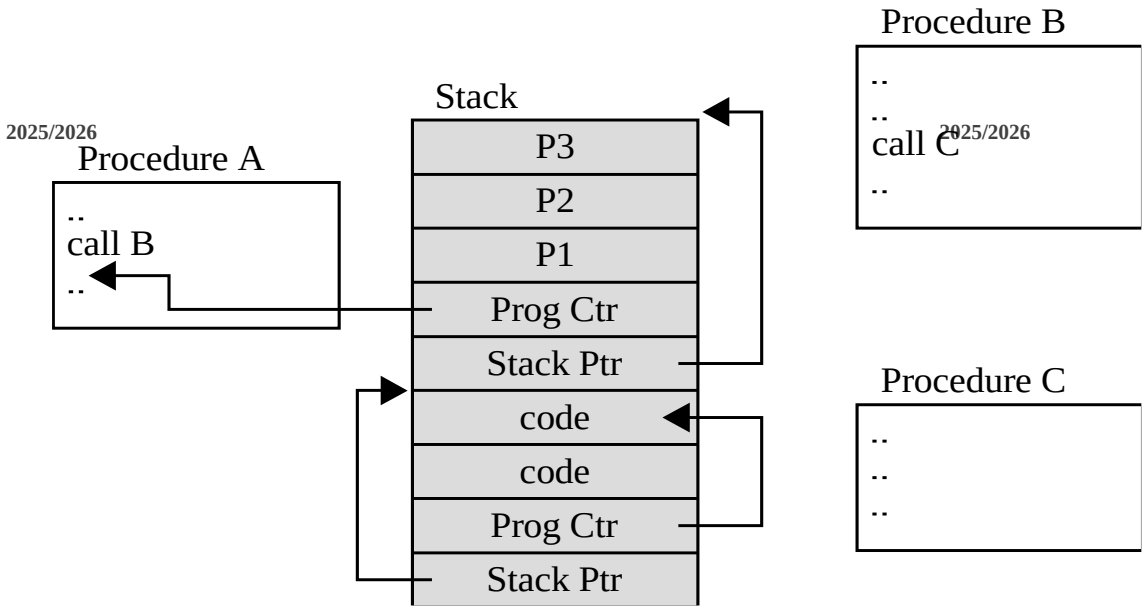


FIGURE 3-10 Compromised Stack

StackGuard is an approach proposed by Crispin Cowan et al. [COW98] The attacker usually cannot tell exactly where the saved program counter is in the stack, only that there is one at an approximate address. Thus, the attacker has to rewrite not just the stack pointer but also some words around it to be sure of changing the true stack pointer, but this uncertainty to the attacker allows StackGuard to detect likely changes to the program counter. Each procedure includes a

prolog code to push values on the stack, set the remainder of the stack frame, and pass control to the called return; then on return, some termination code cleans up the stack, reloads registers, and returns. Just below the program counter, StackGuard inserts a canary value to signal modification; if the attacker rewrites the program counter and the added value, StackGuard augments the termination code to detect the modified added value and signal an error before returning. Thus, each canary value serves as a protective insert to protect the program counter. These protective inserts are shown in Figure 3-11. The idea of surrounding the return address with a tamper-detecting value is sound, as long as only the defender can generate and verify that value.

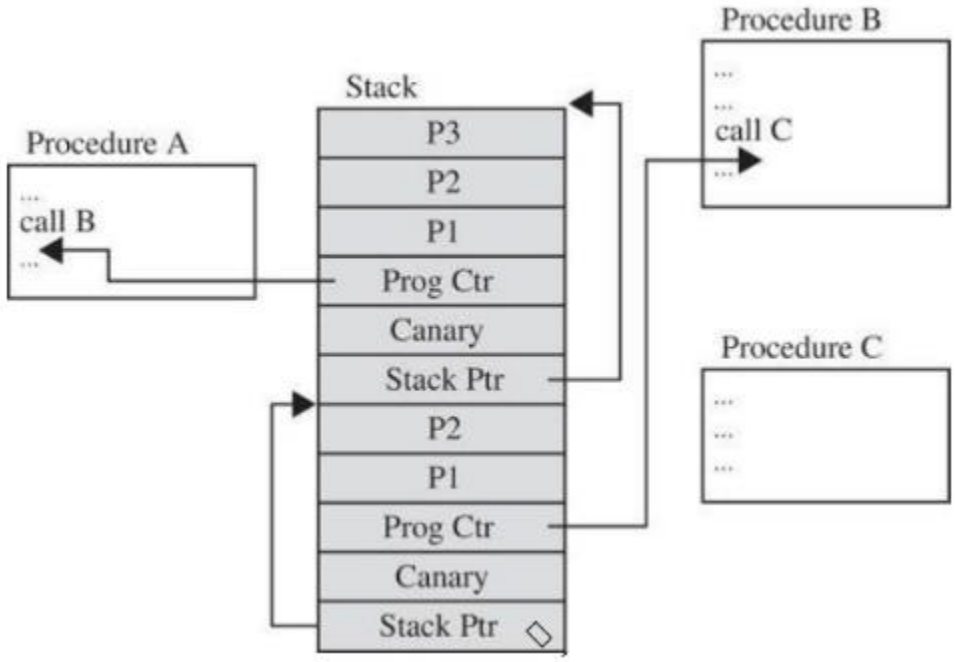


FIGURE 3-11 Canary Values to Signal Modification

Next we consider a programming flaw that is similar to an overflow: a failure to check and control access completely and consistently.

Incomplete Mediation

Mediation means checking: the process of intervening to confirm an actor's authorization before it takes an intended action. In the last chapter we discussed the steps and actors in the authentication process: the access control triple that describes what subject can perform what operation on what object. Verifying that the subject is authorized to perform the operation on an

2025/2026

object is called ***mediation***. Incomplete mediation is a security problem that has been with us for decades: Forgetting to ask “Who goes there?” before allowing the knight across the castle drawbridge is just asking for trouble. In the same way, attackers exploit incomplete mediation to cause security problems.

Definition

Consider the following URL. In addition to a web address, it contains two parameters, so you can think of it as input to a program: Click here to view code image

http://www.somesite.com/subpage/userinput.asp?
parm1=(808)555-1212&parm2=2015Jan17

As a security professional trying to find and fix problems before they occur, you might examine the various parts of the URL to determine what they mean and how they might be exploited. For instance, the parameters parm1 and parm2 look like a telephone number and a date, respectively. Probably the client’s (user’s) web browser enters those two values in their specified format for easy processing on the server’s side.

2025/2026 2025/2026 2025/2026

Users make errors from ignorance, misunderstanding, distraction; user errors should not cause program failures.

Validate All Input

One way to address potential problems is to try to anticipate them. For instance, the programmer in the examples above may have written code to check for correctness on the client’s side (that is, the user’s browser). The client program can search for and screen out errors. Or, to prevent the use of nonsense data, the program can restrict choices to valid ones only. For example, the program supplying the parameters might have solicited them by using a drop-down box or choice list from which only the twelve conventional months could have been selected. Similarly, the year could have been tested to ensure a reasonable value (for example, between 2000 and 2050, according to the application) and date numbers would have to be appropriate for the months in which they occur (no 30th of February, for example). Using such verification, the programmer may have felt well insulated from the possible problems a careless or malicious user could cause.

رومانی صبحی رزق جرجس مطر

Guard Against Users' Fingers

However, the application is still vulnerable. By packing the result into the return URL, the programmer left these data fields in a place where the user can access (and modify) them. In particular, the user can edit the URL line, change any parameter values, and send the revised line. On the server side, the server has no way to tell if the response line came from the client's browser or as a result of the user's editing the URL directly. We say in this case that the data values are not completely mediated: The sensitive data (namely, the parameter values) are in an exposed, uncontrolled condition.

Unchecked data values represent a serious potential vulnerability. To demonstrate this flaw's security implications, we use a real example; only the name of the vendor has been changed to protect the guilty. Things, Inc., was a very large, international vendor of consumer products, called Objects. The company was ready to sell its Objects through a web site, using what appeared to be a standard e-commerce application. The management at Things decided to let some of its in-house developers produce a web site with which its customers could order Objects directly from the web.

2025/2026 2025/2026 2025/2026

So far, so good; everything in the parameter passing looks correct. But this procedure leaves the parameter statement open for malicious tampering. Things should not need to pass the price of the items back to itself as an input parameter. Things presumably knows how much its Objects cost, and they are unlikely to change dramatically since the time the price was quoted a few screens earlier.

There is no reason to leave sensitive data under control of an untrusted user.

A malicious attacker may decide to exploit this peculiarity by supplying instead the following URL, where the price has been reduced from $205 to $25:

Click here to view code image

http://www.things.com/order.asp?custID=101&part=555A&qy=20&price=1&ship=boat&shipcost=5&total=25

Surprise! It worked. The attacker could have ordered Objects from Things in any quantity at any price. And yes, this code was running on the web site for a while before the problem was detected.

رومانی صبحی رزق جرجس مطر

From a security perspective, the most serious concern about this flaw was the length of time that it could have run undetected. Had the whole world suddenly made a rush to Things' web site and bought Objects at a fraction of their actual price, Things probably would have noticed. But Things is large enough that it would never have detected a few customers a day choosing prices that were similar to (but smaller than) the real price, say, 30 percent off. The e-commerce division would have shown a slightly smaller profit than other divisions, but the difference probably would not have been enough to raise anyone's eyebrows; the vulnerability could have gone unnoticed for years. Fortunately, Things hired a consultant to do a routine review of its code, and the consultant quickly found the error.

The vulnerability in this situation is that the customer (computer user) has unmediated access to sensitive data. An application running on the user's browser maintained the order details but allowed the user to change those details at will. In fact, few of these values should have been exposed in the URL sent from the client's browser to the server. The client's application should have specified part number and quantity, but an application on the server's side should have returned the price per unit and total price.

2025/2026 2025/2026 2025/2026

If data can be changed, assume they have been.

This web program design flaw is easy to imagine in other settings. Those of us interested in security must ask ourselves, How many similar problems are in running code today? And how will those vulnerabilities ever be found? And if found, by whom?

Complete Mediation

Because the problem here is incomplete mediation, the solution is complete mediation. Remember from Chapter 2 that one of our standard security tools is access control, sometimes implemented according to the reference monitor concept. The three properties of a reference monitor are (1) small and simple enough to give confidence of correctness, (2) unbypassable, and (3) always invoked. These three properties combine to give us solid, complete mediation.

رومانی صبحی رزق جرجس مطر

Time-of-Check to Time-of-Use

The third programming flaw we describe also involves synchronization. To improve efficiency, modern processors and operating systems usually change the order in which instructions and procedures are executed. In particular, instructions that appear to be adjacent may not actually be executed immediately after each other, either because of intentionally changed order or because of the effects of other processes in concurrent execution.

Definition

Access control is a fundamental part of computer security; we want to make sure that only those subjects who should access an object are allowed that access. Every requested access must be governed by an access policy stating who is allowed access to what; then the request must be mediated by an access-policy-enforcement agent. But an incomplete mediation problem occurs when access is not checked universally. The ***time-of-check to time-of-use (TOCTTOU)*** flaw concerns mediation that is performed with a "bait and switch" in the middle.

2025/2026 2025/2026 2025/2026

Between access check and use, data must be protected against change.

To understand the nature of this flaw, consider a person's buying a sculpture that costs $100. The buyer takes out five $20 bills, carefully counts them in front of the seller, and lays them on the table. Then the seller turns around to write a receipt. While the seller's back is turned, the buyer takes back one $20 bill. When the seller turns around, the buyer hands over the stack of bills, takes the receipt, and leaves with the sculpture. Between the time the security was checked (counting the bills) and the access occurred (exchanging the sculpture for the bills), a condition changed: What was checked is no longer valid when the object (that is, the sculpture) is accessed.

A similar situation can occur with computing systems. Suppose a request to access a file were presented as a data structure, with the name of the file and the mode of access presented in the structure. An example of such a structure is shown in Figure 3-12.

File: my_file	Action: Change byte 4 to A

FIGURE 3-12 File Access Data Structure

The data structure is essentially a work ticket, requiring a stamp of authorization; once authorized, it is put on a queue of things to be done. Normally the access control mediator process receives the data structure, determines whether the access should be allowed, and either rejects the access and stops processing or allows the access and forwards the data structure to the file handler for processing.

To carry out this authorization sequence, the access control mediator would have to look up the file name (and the user identity and any other relevant parameters) in tables. The mediator could compare the names in the table to the file name in the data structure to determine whether access is appropriate. More likely, the mediator would copy the file name into its own local storage area and compare from there. Comparing from the copy leaves the data structure in the user's area, under the user's control.

At this point the incomplete mediation flaw can be exploited. While the mediator is checking access rights for the file my file, the user could change the file name descriptor to your file, the value shown in Figure 3-13. Having read the work ticket once, the mediator would not be expected to reread the ticket before approving it; the mediator would approve the access and send the now-modified descriptor to the file handler.

2025/2026 2025/2026 2025/2026

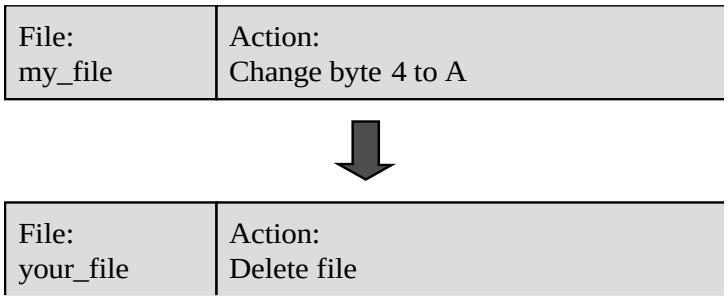


FIGURE 3-13 Unchecked Change to Work Descriptor

The problem is called a time-of-check to time-of-use flaw because it exploits the delay between the two actions: check and use. That is, between the time the access was checked and the time the result of the check was used, a change occurred, invalidating the result of the check.

Security Implication

The security implication here is clear: Checking one action and performing another is an example of ineffective access control, leading to confidentiality failure or integrity failure or

رومانی صبحی رزق جرجس مطر

both. We must be wary whenever a time lag or loss of control occurs, making sure that there is no way to corrupt the check's results during that interval.

Countermeasures

Fortunately, there are ways to prevent exploitation of the time lag, again depending on our security tool, access control. Critical parameters are not exposed during any loss of control. The access-checking software must own the request data until the requested action is complete. Another protection technique is to ensure serial integrity, that is, to allow no interruption (loss of control) during the validation. Or the validation routine can initially copy data from the user's space to the routine's area—out of the user's reach—and perform validation checks on the copy. Finally, the validation routine can seal the request data to detect modification. Really, all these protection methods are expansions on the tamperproof criterion for a reference monitor: Data on which the access control decision is based and the result of the decision must be outside the domain of the program whose access is being controlled.

Undocumented Access Point

2025/2026 2025/2026 2025/2026

Next we describe a common programming situation. During program development and testing, the programmer needs a way to access the internals of a module. Perhaps a result is not being computed correctly so the programmer wants a way to interrogate data values during execution. Maybe flow of control is not proceeding as it should and the programmer needs to feed test values into a routine. It could be that the programmer wants a special debug mode to test conditions. For whatever reason the programmer creates an undocumented entry point or execution mode.

These situations are understandable during program development. Sometimes, however, the programmer forgets to remove these entry points when the program moves from development to product. Or the programmer decides to leave them in to facilitate program maintenance later; the programmer may believe that nobody will find the special entry.

Backdoor

An undocumented access point is called a backdoor or trapdoor. Such an entry can transfer control to any point with any privileges the programmer wanted.

رومانی صبحی رزق جرجس مطر

Few things remain secret on the web for long; someone finds an opening and exploits it. Thus, coding a supposedly secret entry point is an opening for unannounced visitors.

Secret backdoors are eventually found. Security cannot depend on such secrecy.

Another example of backdoors is used once an outsider has compromised a machine. In many cases an intruder who obtains access to a machine wants to return later, either to extend the raid on the one machine or to use the machine as a jumping-off point for strikes against other machines to which the first machine has access. Sometimes the first machine has privileged access to other machines so the intruder can get enhanced rights when exploring capabilities on these new machines. To facilitate return, the attacker can create a new account on the compromised machine, under a user name and password that only the attacker knows.

Protecting Against Unauthorized Entry

Undocumented entry points are a poor programming practice (but they will still be used). They should be found during rigorous code reviews in a software development process. Unfortunately, two factors work against that ideal.

2025/2026 2025/2026

First, being undocumented, these entry points will not be clearly labeled in source code or any of the development documentation. Thus, code reviewers might fail to recognize them during review.

Second, such backdoors are often added after ordinary code development, during testing or even maintenance, so even the scrutiny of skilled reviewers will not find them. Maintenance people who add such code are seldom security engineers, so they are not used to thinking of vulnerabilities and failure modes. For example, as reported by security writer Brian Krebs in his blog Krebs on Security, 24 January 2013, security researcher Stefan Viehböck of SEC Consult Vulnerability Labs in Vienna, Austria found that some products from Barracuda Networks (maker of firewalls and other network devices) accepted remote (network) logins from user name "product" and no password. The engineer who inserted the backdoor probably thought the activity was protected by restricting the address range from which the logins would be accepted: Only logins from the range of addresses assigned to Barracuda would succeed. However, the engineer failed to consider (and a good security engineer would have caught) that the specified range also included hundreds of other companies.

رومانی صبحی رزق جرجس مطر

Thus, preventing or locking these vulnerable doorways is difficult, especially because the people who write them may not appreciate their security implications.

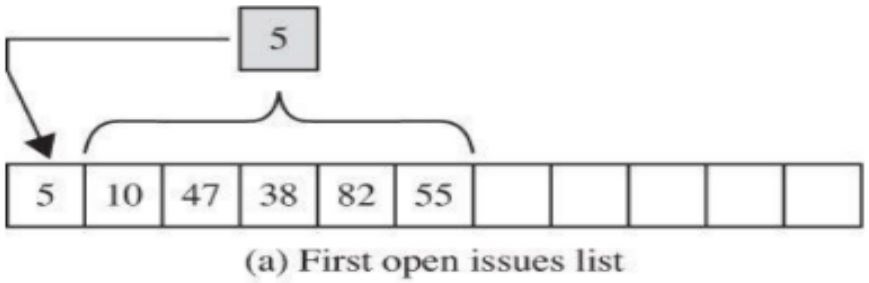
Off-by-One Error

When learning to program, neophytes can easily fail with the ***off-by-one error***: miscalculating the condition to end a loop (repeat while i< = n or i<n? repeat until i=n or i>n?) or overlooking that an array of A[0] through A[n] contains n+1 elements.

Usually the programmer is at fault for failing to think correctly about when a loop should stop. Other times the problem is merging actual data with control data (sometimes called metadata or data about the data). For example, a program may manage a list that increases and decreases. Think of a list of unresolved problems in a customer service department: Today there are five open issues, numbered 10, 47, 38, 82, and 55; during the day, issue 82 is resolved but issues 93 and 64 are added to the list. A programmer may create a simple data structure, an array, to hold these issue numbers and may reasonably specify no more than 100 numbers. But to help with managing the numbers, the programmer may also reserve the first position in the array for the count of open issues.

2025/2026 2025/2026 2025/2026

Thus, in the first case the array really holds six elements, 5 (the count), 10, 47, 38, 82, and 55; and in the second case there are seven, 6, 10, 47, 38, 93, 55, 64, as shown in Figure 3-14. A 100-element array will clearly not hold 100 data items plus one count.



(a) First open issues list

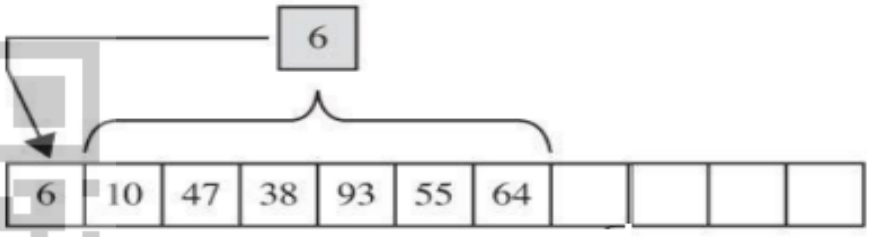


FIGURE 3-14 Both Data and Number of Used Cells in an Array

رومانی صبحی رزق جرجس مطر

In this simple example, the program may run correctly for a long time, as long as no more than 99 issues are open at any time, but adding the 100th issue will cause the program to fail. A similar problem occurs when a procedure edits or reformats input, perhaps changing a one-character sequence into two or more characters (as for example, when the one-character ellipsis symbol "…" available in some fonts is converted by a word processor into three successive periods to account for more limited fonts.) These unanticipated changes in size can cause changed data to no longer fit in the space where it was originally stored. Worse, the error will appear to be sporadic, occurring only when the amount of data exceeds the size of the allocated space.

Unsafe Utility Program

Programming languages, especially C, provide a library of utility routines to assist with common activities, such as moving and copying strings. In C the function strcpy (dest, src) copies a string from src to dest, stopping on a null, with the potential to overrun allocated memory. A safer function is strncpy (dest, src, max), which copies up to the null delimiter or max characters, whichever comes first.

2025/2026 2025/2026 2025/2026

Although there are other sources of overflow problems, from these descriptions you can readily see why so many problems with buffer overflows occur. Next, we describe several classic and significant exploits that have had a buffer overflow as a significant contributing cause. From these examples you can see the amount of harm that a seemingly insignificant program fault can produce.

Race Condition

As the name implies, a race condition means that two processes are competing within the same time interval, and the race affects the integrity or correctness of the computing tasks. For instance, two devices may submit competing requests to the operating system for a given chunk of memory at the same time. In the two-step request process, each device first asks if the size chunk is available, and if the answer is yes, then reserves that chunk for itself. Depending on the timing of the steps, the first device could ask for the chunk, get a "yes" answer, but then not get the chunk because it has already been assigned to the second device. In cases like this, the two requesters "race" to obtain a resource. A race condition occurs most often in an operating system, but it can also occur in multithreaded or cooperating processes.

رومانی صبحی رزق جرجس مطر

Unsynchronized Activity

In a ***race condition or serialization flaw*** two processes execute concurrently, and the outcome of the computation depends on the order in which instructions of the processes execute.

> ***Race condition: situation in which program behavior depends on the order in which two procedures execute***

Imagine an airline reservation system. Each of two agents, A and B, simultaneously tries to book a seat for a passenger on flight 45 on 10 January, for which there is exactly one seat available. If agent A completes the booking before that for B begins, A gets the seat and B is informed that no seats are available. In Figure 3-16 we show a timeline for this situation.

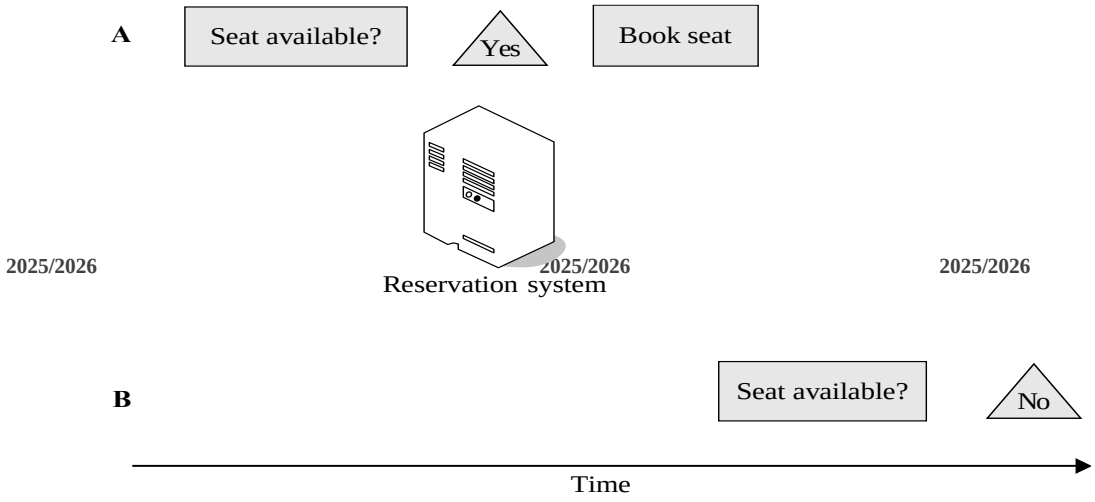


FIGURE 3-16 Seat Request and Reservation Example

However, you can imagine a situation in which A asks if a seat is available, is told yes, and proceeds to complete the purchase of that seat. Meanwhile, between the time A asks and then tries to complete the purchase, agent B asks if a seat is available. The system designers knew that sometimes agents inquire about seats but never complete the booking; their clients often choose different itineraries once they explore their options. For later reference, however, the booking software gives each agent a reference number to make it easy for the server to associate a booking with a particular flight. Because A has not completed the transaction before the system gets a request from B, the system tells B that the seat is available. If the system is not designed properly, both agents can complete their transactions, and two passengers will be confirmed for

that one seat (which will be uncomfortable, to say the least). We show this timeline in Figure 3-17.

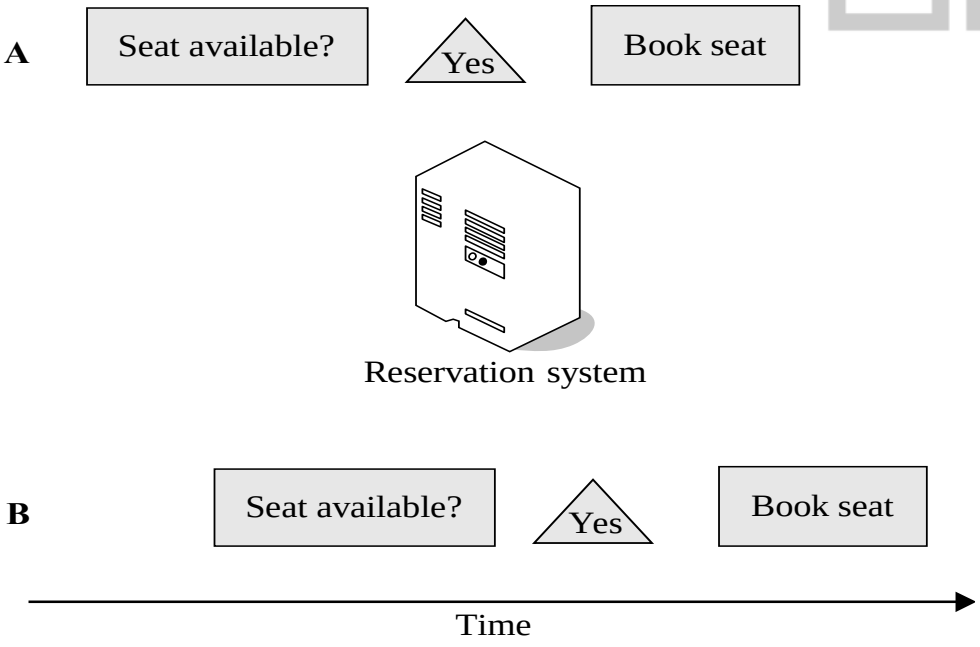


FIGURE 3-17 Overbooking Example

2025/2026 2025/2026 2025/2026

A race condition is difficult to detect because it depends on the order in which two processes execute. But the execution order of the processes can depend on many other things, such as the total load on the system, the amount of available memory space, the priority of each process, or the number and timing of system interrupts to the processes. During testing, and even for a long period of execution, conditions may never cause this particular overload condition to occur. Given these difficulties, programmers can have trouble devising test cases for all the possible conditions under which races can occur. Indeed, the problem may occur with two independent programs that happen to access certain shared resources, something the programmers of each program never envisioned.

The vulnerabilities we have presented here—incomplete mediation, race conditions, time-of-check to time-of-use, and undocumented access points—are flaws that can be exploited to cause a failure of security. Throughout this book we describe other sources of failures because programmers have many process points to exploit and opportunities to create program flaws.

Most of these flaws may have been created because the programmer failed to think clearly and carefully: simple human errors. Occasionally, however, the programmer maliciously planted an intentional flaw. Or, more likely, the assailant found one of these innocent program errors and exploited it for malicious purpose. In the descriptions of program flaws we have pointed out how an attacker could capitalize on the error. In the next section we explain in more detail the harm that malicious code can cause.

رومانی صبحی رزق جرجس مطر

3.2 Malicious Code—Malware

In May 2010, researcher Roger Thompson of the antivirus firm AVG detected malicious code at the web site of the U.S. Bureau of Engraving and Printing, a part of the Treasury Department [MCM10]. The site has two particularly popular sections: a description of the design of the newly redesigned U.S. $100 bill and a set of steps for identifying counterfeit currency.

The altered web site contained a hidden call to a web site in the Ukraine, which then attempted to exploit known vulnerabilities in the web site to lodge malicious code on unsuspecting users' machines. Visitors to the site would download pictures and text, as expected; what visitors couldn't see, and probably did not expect, was that they also downloaded an additional web code script that invoked code at the Ukrainian site.

2025/2026 2025/2026 2025/2026

Malicious code comes in many forms under many names. In this chapter we explore three of the most popular forms: viruses, Trojan horses, and worms. The distinctions among them are small, and we do not need to classify any piece of code precisely. More important is to learn about the nature of attacks from these three: how they can spread, what harm they can cause, and how they can be controlled. We can then apply this knowledge to other types of malicious code, including code forms that do not yet have popular names.

Malware—Viruses, Trojan Horses, and Worms

Malicious code or rogue programs or malware (short for MALicious softWARE) is the general name for programs or program parts planted by an agent with malicious intent to cause unanticipated or undesired effects. The agent is the program's writer or distributor. Malicious intent distinguishes this type of code from unintentional errors, even though both kinds can certainly have similar and serious negative effects. This definition also excludes coincidence, in which minor flaws in two benign programs combine for a negative effect. Most faults found in software inspections, reviews, and testing do not qualify as malicious code; their cause is usually

unintentional. However, unintentional faults can in fact invoke the same responses as intentional malevolence; a benign cause can still lead to a disastrous effect.

Malicious code can be directed at a specific user or class of users, or it can be for anyone.

You may have been affected by malware at one time or another, either because your computer was infected or because you could not access an infected system while its administrators were cleaning up the mess caused by the infection. The malware may have been caused by a worm or a virus or neither; the infection metaphor often seems apt, but the terminology of malicious code is sometimes used imprecisely. Here we distinguish names applied to certain types of malware, but you should focus on methods and impacts, instead of names. That which we call a virus by any other name would smell as vile.

A ***virus*** is a program that can replicate itself and pass on malicious code to other nonmalicious programs by modifying them. The term "virus" was coined because the affected program acts like a biological virus: It infects other healthy subjects by attaching itself to the program and either destroying the program or coexisting with it. Because viruses are insidious, we cannot assume that a clean program yesterday is still clean today. Moreover, a good program can be modified to include a copy of the virus program, so the infected good program itself begins to act as a virus, infecting other programs. The infection usually spreads at a geometric rate, eventually overtaking an entire computing system and spreading to other connected systems.

2025/2026 2025/2026 2025/2026

Virus: code with malicious purpose; intended to spread

A virus can be either transient or resident. A transient virus has a life span that depends on the life of its host; the virus runs when the program to which it is attached executes, and it terminates when the attached program ends. (During its execution, the transient virus may spread its infection to other programs.) A resident virus locates itself in memory; it can then remain active or be activated as a stand-alone program, even after its attached program ends.

The terms worm and virus are often used interchangeably, but they actually refer to different things. A worm is a program that spreads copies of itself through a network. (John Shoch and Jon Hupp [SHO82] are apparently the first to describe a worm, which, interestingly, was created for nonmalicious purposes. Researchers at the Xerox Palo Alto Research Center, Shoch and Hupp wrote the first program as an experiment in distributed computing.) The primary difference

رومانی صبحی رزق جرجس مطر

between a worm and a virus is that a worm operates through networks, and a virus can spread through any medium (but usually uses a copied program or data files). Additionally, the worm spreads copies of itself as a stand-alone program, whereas the virus spreads copies of itself as a program that attaches to or embeds in other programs.

Worm: program that spreads copies of itself through a network

Spreading copies of yourself seems boring and perhaps narcissistic. But worms do have a common, useful purpose. How big is the Internet? What addresses are in use? Worm programs, sometimes called "crawlers" seek out machines on which they can install small pieces of code to gather such data. The code items report back to collection points, telling what connectivity they have found. As we describe in Chapter 6, this kind of reconnaissance can also have a negative security purpose; the worms that travel and collect data do not have to be evil.

As a slightly different example of this type of worm, consider how search engines know about all the pages on the web. A bot (short for robot), is a kind of worm used in vast numbers by search engine hosts like Bing and Google. Armies of these agents run on any computers on which they can install themselves. Their purpose is to scan accessible web content continuously and report back to their controller any new content they have found. In this way, the agents find pages that their controllers then catalog, enabling the search engines to return these results in response to individuals' queries. Thus, when you post a new web page (or modify an old one) with results of your research on why people like peanut butter, a crawler soon notices that page and informs its controller of the contents and whereabouts of your new page.

A ***Trojan horse*** is malicious code that, in addition to its primary effect, has a second, nonobvious, malicious effect. The name is derived from a reference to the Trojan war. Legends tell how the Greeks tricked the Trojans by leaving a great wooden horse outside the Trojans' defensive wall. The Trojans, thinking the horse a gift, took it inside and gave it pride of place. But unknown to the naïve Trojans, the wooden horse was filled with the bravest of Greek soldiers. In the night, the Greek soldiers descended from the horse, opened the gates, and signaled their troops that the way in was now clear to capture Troy. In the same way, Trojan horse malware slips inside a program undetected and produces unwelcome effects later on.

As an example of a computer Trojan horse, consider a login script that solicits a user's identification and password, passes the identification information on to the rest of the system for

2025/2026 2025/2026 2025/2026

login processing, but also retains a copy of the information for later, malicious use. In this example, the user sees only the login occurring as expected, so there is no reason to suspect that any other, unwelcome action took place.

Trojan horse: program with benign apparent effect but second, hidden, malicious effect

To remember the differences among these three types of malware, understand that a Trojan horse is on the surface a useful program with extra, undocumented (malicious) features. It does not necessarily try to propagate. By contrast, a virus is a malicious program that attempts to spread to other computers, as well as perhaps performing unpleasant action on its current host. The virus does not necessarily spread by using a network's properties; it can be spread instead by traveling on a document transferred by a portable device (that memory stick you just inserted in your laptop!) or triggered to spread to other, similar file types when a file is opened. However, a worm requires a network for its attempts to spread itself elsewhere.

Code Type	Characteristics
Virus	Code that causes malicious behavior and propagates copies of itself to other programs
Trojan horse	Code that contains unexpected, undocumented, additional functionality
Worm	Code that propagates copies of itself through a network; impact is usually degraded performance
Rabbit	Code that replicates itself without limit to exhaust resources
Logic bomb	Code that triggers action when a predetermined condition occurs
Time bomb	Code that triggers action when a predetermined time occurs
Dropper	Transfer agent code only to drop other malicious code, such as virus or Trojan horse
Hostile mobile code agent	Code communicated semi-autonomously by programs transmitted through the web
Script attack, JavaScript, Active code attack	Malicious code communicated in JavaScript, ActiveX, or another scripting language, downloaded as part of displaying a web page

رومانی صبحی رزق جرجس مطر

Code Type	Characteristics
RAT (remote access Trojan)	Trojan horse that, once planted, gives access from remote location
Spyware	Program that intercepts and covertly communicates data on the user or the user's activity
Bot	Semi-autonomous agent, under control of a (usually remote) controller or "herder"; not necessarily malicious
Zombie	Code or entire computer under control of a (usually remote) program
Browser hijacker	Code that changes browser settings, disallows access to certain sites, or redirects browser to others
Rootkit	Code installed in "root" or most privileged section of operating system; hard to detect
Trapdoor or backdoor	Code feature that allows unauthorized access to a machine or program; bypasses normal access control and authentication
Tool or toolkit	Program containing a set of tests for vulnerabilities; not dangerous itself, but each successful test identifies a vulnerable host that can be attacked
Scareware	Not code; false warning of malicious code attack

TABLE 3-2 Types of Malicious Code

2025/2026 2025/2026 2025/2026

Beyond this basic terminology, there is much similarity in types of malicious code. Many other types of malicious code are shown in Table 3-2. As you can see, types of malware differ widely in their operation, transmission, and objective. Any of these terms is used popularly to describe malware, and you will encounter imprecise and overlapping definitions. Indeed, people sometimes use virus as a convenient general term for malicious code. Again, let us remind you that nomenclature is not critical; impact and effect are. Battling over whether something is a virus or worm is beside the point; instead, we concentrate on understanding the mechanisms by which malware perpetrates its evil.

Technical Details: Malicious Code

The number of strains of malicious code is unknown. According to a testing service [AVC10], malicious code detectors (such as familiar antivirus tools) that look for malware "signatures" cover over 1 million definitions, although because of mutation, one strain may involve several definitions. Infection vectors include operating systems, document applications (primarily word

processors and spreadsheets), media players, browsers, document-rendering engines (such as Adobe PDF reader) and photo-editing programs. Transmission media include documents, photographs, and music files, on networks, disks, flash media (such as USB memory devices), and even digital photo frames. Infections involving other programmable devices with embedded computers, such as mobile phones, automobiles, digital video recorders, and cash registers, are becoming targets for malicious code.

In this section we explore four aspects of malicious code infections:

- harm—how they affect users and systems
- transmission and propagation—how they are transmitted and replicate, and how they cause further transmission
- activation—how they gain control and install themselves so that they can reactivate
- stealth—how they hide to avoid detection

We begin our study of malware by looking at some aspects of harm caused by malicious code.

Harm from Malicious Code

Viruses and other malicious code can cause essentially unlimited harm. Because malware runs under the authority of the user, it can do anything the user can do. In this section we give some examples of harm malware can cause. Some examples are trivial, more in the vein of a comical prank. But other examples are deadly serious with obvious critical consequences.

2025/2026 2025/2026 2025/2026

We can divide the payload from malicious code into three categories:

- Nondestructive. Examples of behavior are sending a funny message or flashing an image on the screen, often simply to show the author's capability. This category would also include virus hoaxes, messages falsely warning of a piece of malicious code, apparently to cause receivers to panic and forward the message to contacts, thus spreading the panic.
- Destructive. This type of code corrupts files, deletes files, damages software, or executes commands to cause hardware stress or breakage with no apparent motive other than to harm the recipient.
- Commercial or criminal intent. An infection of this type tries to take over the recipient's computer, installing code to allow a remote agent to cause the computer to perform actions on the agent's signal or to forward sensitive data to the agent. Examples of actions include collecting personal data, for example, login credentials to a banking web site, collecting proprietary data, such as corporate plans (as was reported for an infection of computers of

رومانی صبحی رزق جرجس مطر

five petroleum industry companies in February 2011), or serving as a compromised agent for sending spam email or mounting a denial-of-service attack, as described in Chapter 6.

As we point out in Chapter 1, without our knowing the mind of the attacker, motive can be hard to determine. However, this third category has an obvious commercial motive. Organized crime has taken an interest in using malicious code to raise money [WIL01, BRA06, MEN10].

Harm to Users

Most malicious code harm occurs to the infected computer's data. Here are some real-world examples of malice.

- Hiding the cursor.
- Displaying text or an image on the screen.
- Opening a browser window to web sites related to current activity (for example, opening an airline web page when the current site is a foreign city's tourist board).
- Sending email to some or all entries in the user's contacts or alias list. Note that the email would be delivered as having come from the user, leading the recipient to think it authentic. The Melissa virus did this, sending copies of itself as an attachment that unsuspecting recipients would open, which then infected the recipients and allowed the infection to spread to their contacts.
- Opening text documents and changing some instances of "is" to "is not," and vice versa. Thus, "Raul is my friend" becomes "Raul is not my friend." The malware changed only a few instances in random locations, so the change would not be readily apparent. Imagine the effect these changes would have on a term paper, proposal, contract, or news story.
- Deleting all files. The Jerusalem virus did this every Friday that was a 13th day of the month.
- Modifying system program files. Many strains of malware do this to ensure subsequent reactivation and avoid detection.
- Modifying system information, such as the Windows registry (the table of all critical system information).
- Stealing and forwarding sensitive information such as passwords and login details.

In addition to these direct forms of harm, the user can be harmed indirectly. For example, a company's public image can be harmed if the company's web site is hijacked to spread

2025/2026 2025/2026 2025/2026

malicious code. Or if the attack makes some web files or functions unavailable, people may switch to a competitor's site permanently (or until the competitor's site is attacked).

Although the user is most directly harmed by malware, there is secondary harm as the user tries to clean up a system after infection. Next we consider the impact on the user's system.

Harm to the User's System

Malware writers usually intend that their code persist, so they write the code in a way that resists attempts to eradicate it. Few writers are so obvious as to plant a file named "malware" at the top-level directory of a user's disk. Here are some maneuvers by which malware writers conceal their infection; these techniques also complicate detection and eradication.

- Hide the file in a lower-level directory, often a subdirectory created or used b another legitimate program. For example, the Windows operating system maintains subdirectories for some installed programs in a folder named "registered packages." Inside that folder are subfolders with unintelligible names such as {982FB688-E76B-4246-987B-9218318B90A}. Could you tell to what package that directory belongs or what files properly belong there?

2025/2026 2025/2026 2025/2026

- Attach, using the techniques described earlier in this chapter, to a critical system file, especially one that is invoked during system startup (to ensure the malware is reactivated).
- Replace (retaining the name of) a noncritical system file. Some system functionality will be lost, but a cursory look at the system files will not highlight any names that do not belong.
- Hide copies of the executable code in more than one location.
- Hide copies of the executable in different locations on different systems so no single eradication procedure can work.
- Modify the system registry so that the malware is always executed or malware detection is disabled.

Harm to the World

An essential character of most malicious code is its spread to other systems. Except for specifically targeted attacks, malware writers usually want their code to infect many people, and they employ techniques that enable the infection to spread at a geometric rate. The Morris worm of 1988 infected only 3,000 computers, but those computers constituted a significant proportion,

perhaps as much as half, of what was then the Internet. The IloveYou worm (transmitted in an email message with the alluring subject line "I Love You") is estimated to have infected 100,000 servers; the security firm Message Labs estimated that, at the attack's height, 1 email of every 28 transmitted worldwide was an infection from the worm. Code Red is believed to have affected close to 3 million hosts. By some estimates, the Conficker worms (several strains) control a network of 1.5 million compromised and unrepaired hosts under the worms' author's control [MAR09]. Costs of recovery from major infections like these typically exceed $1 million US. Thus, computer users and society in general bear a heavy cost for dealing with malware.

Damage Estimates

How do you determine the cost or damage of any computer security incident? The problem is similar to the question of determining the cost of a complex disaster such as a building collapse, earthquake, oil spill, or personal injury. Unfortunately, translating harm into money is difficult, in computer security and other domains.

The first step is to enumerate the losses. Some will be tangibles, such as damaged equipment. Other losses include lost or damaged data that must be re-created or repaired, and degradation of service in which it takes an employee twice as long to perform a task. Costs also arise in investigating the extent of damage. (Which programs and data are affected and which archived versions are safe to reload?) Then there are intangibles and unmeasurables such as loss of customers or damage to reputation.

2025/2026 2025/2026 2025/2026

> ***Estimating the cost of an incident is hard. That does not mean the cost is zero or insignificant, just hard to determine.***

Transmission and Propagation

A printed copy of code does nothing and threatens no one. Even executable code sitting on a disk does nothing. What triggers code to start? For malware to do its malicious work and spread itself, it must be executed to be activated. Fortunately for malware writers but unfortunately for the rest of us, there are many ways to ensure that programs will be executed on a running computer.

رومانی صبحی رزق جرجس مطر

Setup and Installer Program Transmission

Recall the SETUP program that you run to load and install a new program on your computer. It may call dozens or hundreds of other programs, some on the distribution medium, some already residing on the computer, some in memory. If any one of these programs contains a virus, the virus code could be activated. Let us see how. Suppose the virus code were in a program on the distribution medium, such as a CD, or downloaded in the installation package; when executed, the virus could install itself on a permanent storage medium (typically, a hard disk) and also in any and all executing programs in memory. Human intervention is necessary to start the process; a human being puts the virus on the distribution medium, and perhaps another person initiates the execution of the program to which the virus is attached. (Execution can occur without human intervention, though, such as when execution is triggered by a date or the passage of a certain amount of time.) After that, no human intervention is needed; the virus can spread by itself.

Attached File

A more common means of virus activation is in a file attached to an email message or embedded in a file. In this attack, the virus writer tries to convince the victim (the recipient of the message or file) to open the object. Once the viral object is opened (and thereby executed), the activated virus can do its work. Some modern email handlers, in a drive to "help" the receiver (victim), automatically open attachments as soon as the receiver opens the body of the email message. The virus can be executable code embedded in an executable attachment, but other types of files are equally dangerous. For example, objects such as graphics or photo images can contain code to be executed by an editor, so they can be transmission agents for viruses. In general, forcing users to open files on their own rather than having an application do it automatically is a best practice; programs should not perform potentially security-relevant actions without a user's consent. However, ease-of-use often trumps security, so programs such as browsers, email handlers, and viewers often "helpfully" open files without first asking the user.

Document Viruses

A virus type that used to be quite popular is what we call the document virus, which is implemented within a formatted document, such as a written document, a database, a slide presentation, a picture, or a spreadsheet. These documents are highly structured files that contain

both data (words or numbers) and commands (such as formulas, formatting controls, links). The commands are part of a rich programming language, including macros, variables and procedures, file accesses, and even system calls. The writer of a document virus uses any of the features of the programming language to perform malicious actions.

The ordinary user usually sees only the content of the document (its text or data), so the virus writer simply includes the virus in the commands part of the document, as in the integrated program virus.

Autorun

Autorun is a feature of operating systems that causes the automatic execution of code based on name or placement. An early autorun program was the DOS file autoexec.bat, a script file located at the highest directory level of a startup disk. As the system began execution, it would automatically execute autoexec.bat, so a goal of early malicious code writers was to augment or replace autoexec.bat to get the malicious code executed. Similarly, in Unix, files such as .cshrc and .profile are automatically processed at system startup (depending on version).

One popular technique for transmitting malware is distribution via flash memory, such as a solid state USB memory stick. People love getting something for free, and handing out infected memory devices is a relatively low cost way to spread an infection. Although the spread has to be done by hand (handing out free drives as advertising at a railway station, for example), the personal touch does add to credibility: We would be suspicious of an attachment from an unknown person, but some people relax their guards for something received by hand from another person.

Propagation

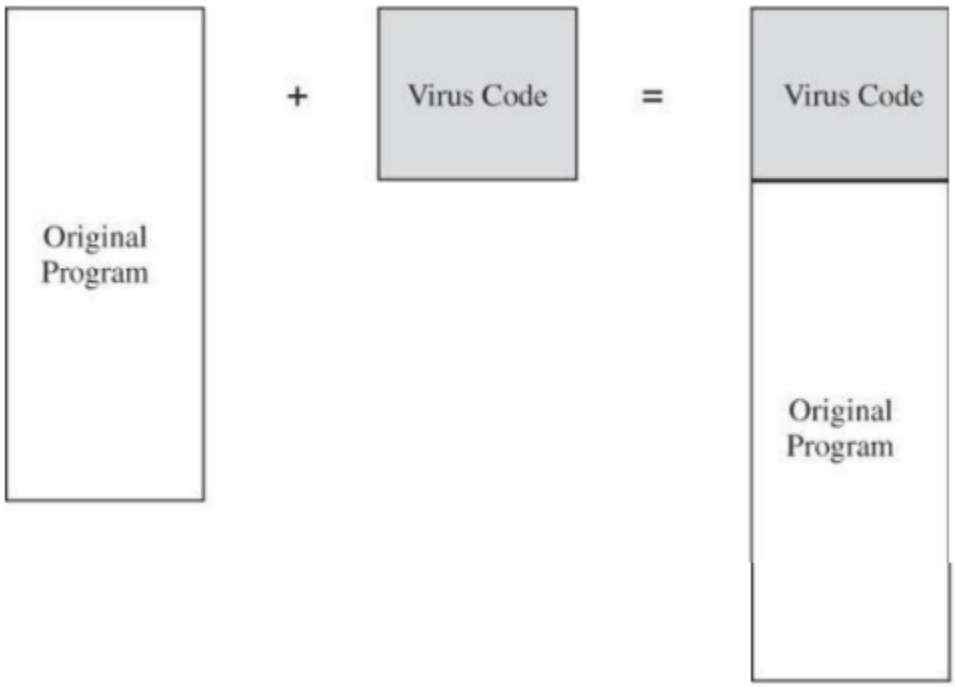
Since a virus can be rather small, its code can be “hidden” inside other larger and more complicated programs. Two hundred lines of a virus could be separated into one hundred packets of two lines of code and a jump each; these one hundred packets could be easily hidden inside a compiler, a database manager, a file manager, or some other large utility.

Appended Viruses

A program virus attaches itself to a program; then, whenever the program is run, the virus is activated. This kind of attachment is usually easy to design and implement. In the simplest case,

2025/2026 2025/2026 2025/2026

a virus inserts a copy of itself into the executable program file before the first executable instruction. Then, all the virus instructions execute first; after the last virus instruction, control flows naturally to what used to be the first program instruction. Such a situation is shown in Figure 3-19.



2025/2026

FIGURE 3-19 Virus Attachment

This kind of attachment is simple and usually effective. The virus writer need not know anything about the program to which the virus will attach, and often the attached program simply serves as a carrier for the virus. The virus performs its task and then transfers to the original program. Typically, the user is unaware of the effect of the virus if the original program still does all that it used to. Most viruses attach in this manner.

Viruses That Surround a Program

An alternative to the attachment is a virus that runs the original program but has control before and after its execution. For example, a virus writer might want to prevent the virus from being detected. If the virus is stored on disk, its presence will be given away by its file name, or its size will affect the amount of space used on the disk. The virus writer might arrange for the virus to attach itself to the program that constructs the listing of files on the disk. If the virus regains

control after the listing program has generated the listing but before the listing is displayed or printed, the virus could eliminate its entry from the listing and falsify space counts so that it appears not to exist. A surrounding virus is shown in Figure 3-20.

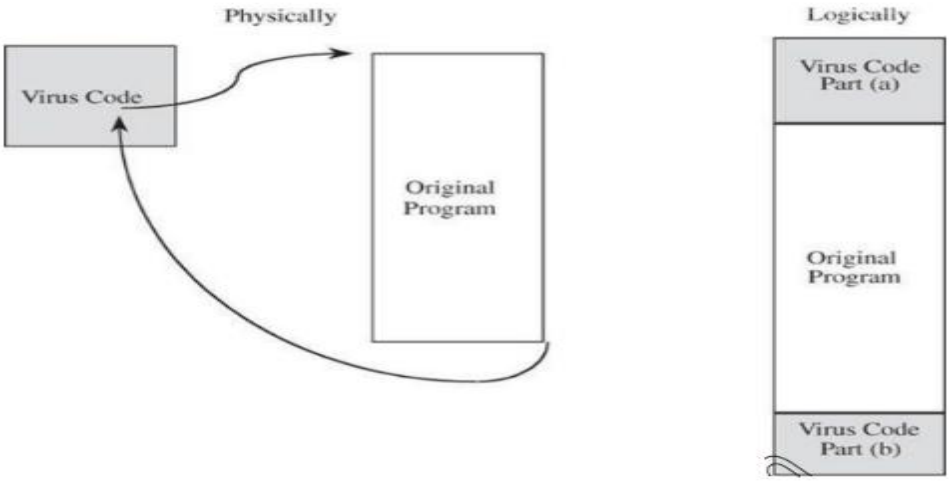


FIGURE 3-20 Surrounding Virus

Integrated Viruses and Replacements

2025/2026 2025/2026 2025/2026

A third situation occurs when the virus replaces some of its target, integrating itself into the original code of the target. Such a situation is shown in Figure 3-21. Clearly, the virus writer has to know the exact structure of the original program to know where to insert which pieces of the virus.

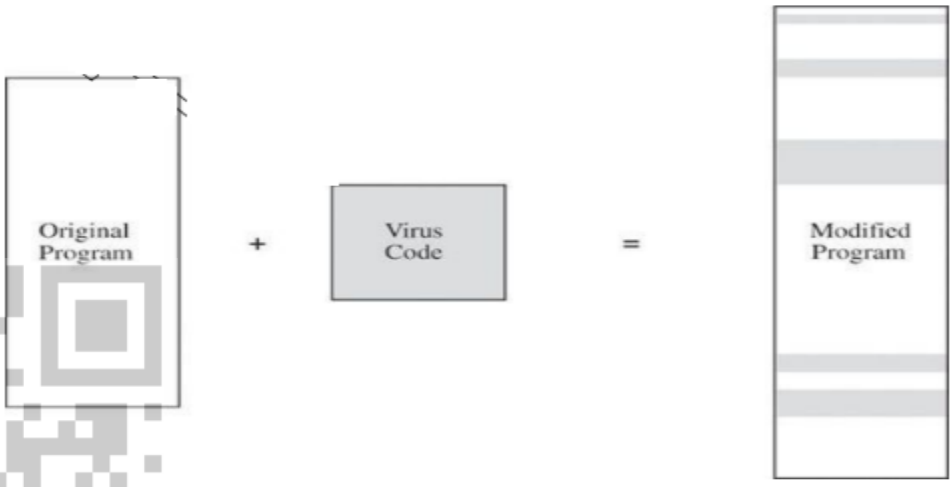


FIGURE 3-21 Virus Insertion

رومانی صبحی رزق جرجس مطر

Finally, the malicious code can replace an entire target, either mimicking the effect of the target or ignoring its expected effect and performing only the virus effect. In this case, the user may perceive the loss of the original program.

Activation

Early malware writers used document macros and scripts as the vector for introducing malware into an environment. Correspondingly, users and designers tightened controls on macros and scripts to guard in general against malicious code, so malware writers had to find other means of transferring their code.

Malware now often exploits one or more existing vulnerabilities in a commonly used program. For example, the Code Red worm of 2001 exploited an older buffer overflow program flaw in Microsoft's Internet Information Server (IIS), and Conficker.A exploited a flaw involving a specially constructed remote procedure call (RPC) request. Although the malware writer usually must find a vulnerability and hope the intended victim has not yet applied a protective or corrective patch, each vulnerability represents a new opening for wreaking havoc against all users of a product.

2025/2026 2025/2026 2025/2026

> ***Is it better to disclose a flaw and alert users that they are vulnerable or conceal it until there is a countermeasure? There is no easy answer***

When an attacker finds a vulnerability to exploit, the next step is using that vulnerability to further the attack. Next we consider how malicious code gains control as part of a compromise.

How Malicious Code Gains Control

To gain control of processing, malicious code such as a virus (V) has to be invoked instead of the target (T). Essentially, the virus either has to seem to be T, saying effectively "I am T," or the virus has to push T out of the way and become a substitute for T, saying effectively "Call me instead of T." A more blatant virus can simply say "invoke me [you fool]."

The virus can assume T's name by replacing (or joining to) T's code in a file structure; this invocation technique is most appropriate for ordinary programs. The virus can overwrite T in storage (simply replacing the copy of T in storage, for example). Alternatively, the virus can change the pointers in the file table so that the virus is located instead of T whenever T is accessed through the file system. These two cases are shown in Figure 3-22.

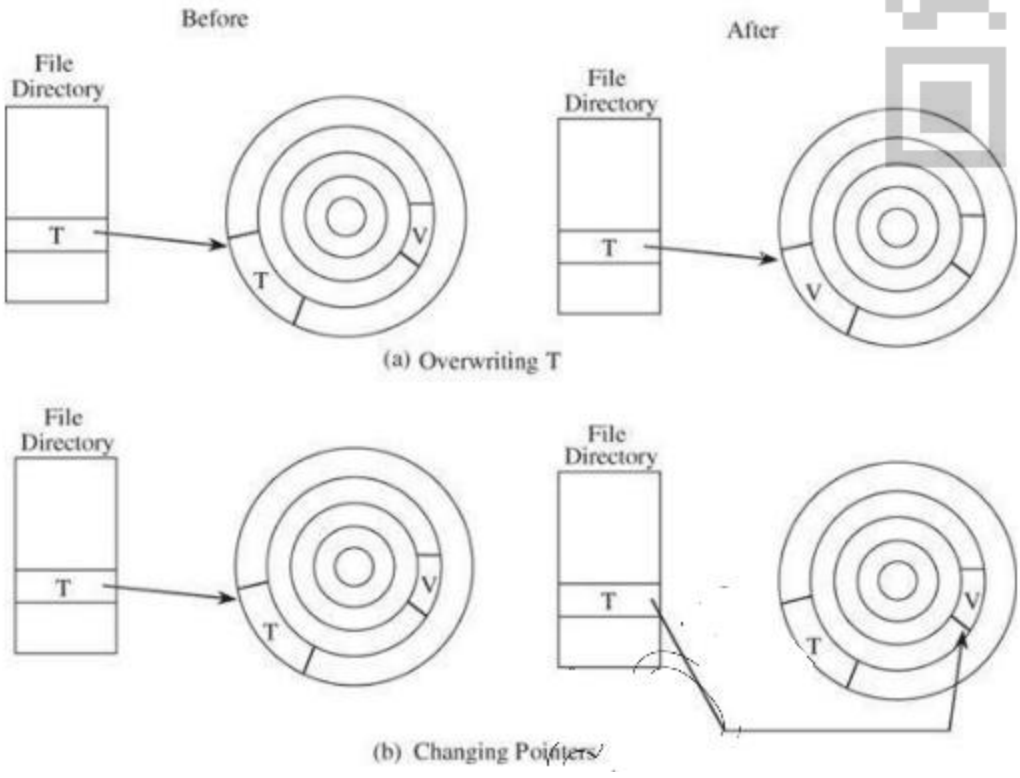


FIGURE 3-22 Virus V Replacing Target T

2025/2026 2025/2026 2025/2026

The virus can supplant T by altering the sequence that would have invoked T to now invoke the virus V; this invocation can replace parts of the resident operating system by modifying pointers to those resident parts, such as the table of handlers for different kinds of interrupts.

Embedding: Homes for Malware

The malware writer may find it appealing to build these qualities into the malware:

- The malicious code is hard to detect.
- The malicious code is not easily destroyed or deactivated.
- The malicious code spreads infection widely.
- The malicious code can reinfect its home program or other programs.
- The malicious code is easy to create.
- The malicious code is machine independent and operating system independent.

Few examples of malware meet all these criteria. The writer chooses from these objectives when deciding what the code will do and where it will reside.

رومانی صبحی رزق جرجس مطر

One-Time Execution (Implanting)

Malicious code often executes a one-time process to transmit or receive and install the infection. Sometimes the user clicks to download a file, other times the user opens an attachment, and other times the malicious code is downloaded silently as a web page is displayed. In any event, this first step to acquire and install the code must be quick and not obvious to the user.

Boot Sector Viruses

A special case of virus attachment, but formerly a fairly popular one, is the so-called boot sector virus. Attackers are interested in creating continuing or repeated harm, instead of just a one-time assault. For continuity the infection needs to stay around and become an integral part of the operating system. In such attackers, the easy way to become permanent is to force the harmful code to be reloaded each time the system is restarted. Actually, a similar technique works for most types of malicious code, so we first describe the process for viruses and then explain how the technique extends to other types.

رومانی صبحی رزق جرجس مطر

Exercises

1. Suppose you are a customs inspector. You are responsible for checking suitcases for secret compartments in which bulky items such as jewelry might be hidden. Describe the procedure you would follow to check for these compartments.
2. Your boss hands you a microprocessor and its technical reference manual. You are asked to check for undocumented features of the processor. Because of the number of possibilities, you cannot test every operation code with every combination of operands. Outline the strategy you would use to identify and characterize unpublicized operations.
3. Your boss hands you a computer program and its technical reference manual. You are asked to check for undocumented features of the program. How is this activity similar to the task of the previous exercises? How does it differ? Which is the more feasible? Why?
4. A program is written to compute the sum of the integers from 1 to 10. The programmer, well trained in reusability and maintainability, writes the program so that it computes the sum of the numbers from k to n. However, a team of security specialists scrutinizes the code. The team certifies that this program properly sets k to 1 and n to 10; therefore, the program is certified as being properly restricted in that it always operates on precisely the range 1 to 10. List different ways that this program can be sabotaged so that during execution it computes a different sum, such as 3 to 20.
5. One way to limit the effect of an untrusted program is confinement: controlling what processes have access to the untrusted program and what access the program has to other processes and data. Explain how confinement would apply to the earlier example of the program that computes the sum of the integers 1 to 10.
6. List three controls that could be applied to detect or prevent off-by-one errors. 7. The distinction between a covert storage channel and a covert timing channel is not clearcut. Every timing channel can be transformed into an equivalent storage channel. Explain how this transformation could be done.
8. List the limitations on the amount of information leaked per second through a covert channel in a multiaccess computing system.
9. An electronic mail system could be used to leak information. First, explain how the leakage could occur. Then, identify controls that could be applied to detect or prevent the leakage.

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

Chapter 4

Operating Systems

In this chapter we explore the role of the operating system in security. Although operating systems are crucial for implementing separation and access control, they are not invulnerable, and therefore compromise of an operating system can lead to security failure. Furthermore, users' objects can be commingled with code and data for applications and support routines, and operating systems are limited in their ability to separate and protect these resources.

We begin this chapter with a brief overview, which for many readers will be a review, of operating system design. We continue by examining aspects of operating system design that enhance security. Finally, we consider rootkits, the most serious compromise of an operating system; with such an exploit the attacker undermines the entire operating system and thus all the security protections it is expected to provide.

2025/2026 2025/2026 2025/2026

4.1 Security in Operating Systems

Many attacks are silent and invisible. What good is an attack if the victim can see and perhaps counter it? As we described in Chapter 3, viruses, Trojan horses, and similar forms of malicious code may masquerade as harmless programs or attach themselves to other legitimate programs. Nevertheless, the malicious code files are stored somewhere, usually on disk or in memory, and their structure can be detected with programs that recognize patterns or behavior. A powerful defense against such malicious code is prevention to block the malware before it can be stored in memory or on disk.

The operating system is the first line of defense against all sorts of unwanted behavior. It protects one user from another, ensures that critical areas of memory or storage are not overwritten by unauthorized processes, performs identification and authentication of people and remote operations, and ensures fair sharing of critical hardware resources. As the powerful traffic cop of a computing system it is also a tempting target for attack because the prize for successfully compromising the operating system is complete control over the machine and all its components.

The operating system is the fundamental controller of all system resources—which makes it a primary target of attack, as well.

When the operating system initializes at system boot time, it initiates tasks in an orderly sequence, such as, first, primitive functions and device drivers, then process controllers, followed by file and memory management routines and finally, the user interface. To establish security, early tasks establish a firm defense to constrain later tasks. Primitive operating system functions, such as interprocess communication and basic input and output, must precede more complex structures such as files, directories, and memory segments, in part because these primitive functions are necessary to implement the latter constructs, and also because basic communication is necessary so that different operating system functions can communicate with each other. Antivirus applications are usually initiated late because they are add-ons to the operating system; still, antivirus code must be in control before the operating system allows access to new objects that might contain viruses. Clearly, prevention software can protect only if it is active before the malicious code.

But what if the malware embeds itself in the operating system, such that it is active before operating system components that might detect or block it? Or what if the malware can circumvent or take over other parts of the operating system? This sequencing leads to an important vulnerability: Gaining control before the protector means that the protector's power is limited. In that case, the attacker has near-complete control of the system: The malicious code is undetectable and unstoppable. Because the malware operates with the privileges of the root of the operating system, it is called a rootkit. Although embedding a rootkit within the operating system is difficult, a successful effort is certainly worth it. We examine rootkits later in this chapter. Before we can study that class of malware, we must first consider the components from which operating systems are composed.

Background: Operating System Structure

An operating system is an executive or supervisor for a piece of computing machinery. Operating systems are not just for conventional computers. Some form of operating system can be found on any of the following objects:

- a dedicated device such as a home thermostat or a heart pacemaker
- an automobile (especially the engine performance sensors and the automated control functions such as antilock brakes); similarly, the avionics components of an airplane or the control system of a streetcar or mass transit system

• a smartphone, tablet, or other web appliance
• a network appliance, such as a firewall or intrusion detection and prevention system
• a controller for a bank of web servers
• a (computer) network traffic management device

In addition to this list, of course, computers—from microcomputers to laptops to huge mainframes—have operating systems. The nature of an operating system varies according to the complexity of the device on which it is installed, the degree of control it exercises, and the amount of interaction it supports, both with humans and other devices. Thus, there is no one simple model of an operating system, and security functions and features vary considerably.

From a security standpoint, we are most interested in an operating system's control of resources: which users are allowed which accesses to which objects, as we explore in the next section.

Security Features of Ordinary Operating Systems

A multiprogramming operating system performs several functions that relate to security. To see how, examine Figure 5-1, which illustrates how an operating system interacts with users, provides services, and allocates resources.

2025/2026 2025/2026 2025/2026

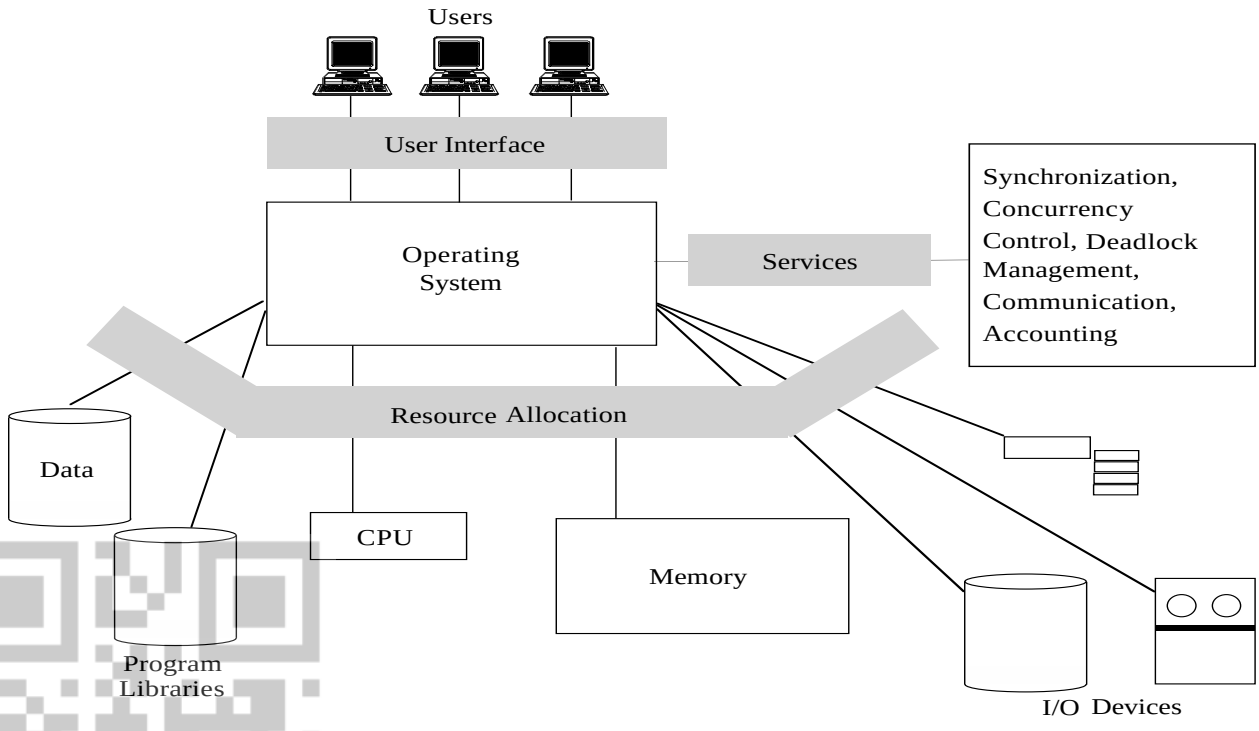


FIGURE 5-1 Operating System Functions

رومانی صبحی رزق جرجس مطر

We can see that the system addresses several particular functions that involve computer security:

- Enforced sharing. Resources should be made available to users as appropriate. Sharing brings about the need to guarantee integrity and consistency. Table lookup, combined with integrity controls such as monitors or transaction processors, is often used to support controlled sharing.
- Interprocess communication and synchronization. Executing processes sometimes need to communicate with other processes or to synchronize their accesses to shared resources. Operating systems provide these services by acting as a bridge between processes, responding to process requests for asynchronous communication with other processes or synchronization. Interprocess communication is mediated by access control tables.
- Protection of critical operating system data. The operating system must maintain data by which it can enforce security. Obviously, if these data are not protected against unauthorized access (read, modify, and delete), the operating system cannot provide enforcement. Various techniques (including encryption, hardware control, and isolation) support protection of operating system security data.

2025/2026 2025/2026 2025/2026

- Guaranteed fair service: All users expect CPU usage and other service to be provided so that no user is indefinitely starved from receiving service. Hardware clocks combine with scheduling disciplines to provide fairness. Hardware facilities and data tables combine to provide control.
- Interface to hardware. All users access hardware functionality. Fair access and controlled sharing are hallmarks of multitask operating systems (those running more than one task concurrently), but a more elementary need is that users require access to devices, communications lines, hardware clocks, and processors. Few users access these hardware resources directly, but all users employ such things through programs and utility functions. Hardware interface used to be more tightly bound into an operating system's design; now, however, operating systems are designed to run on a range of hardware platforms, both to maximize the size of the potential market and to position the operating system for hardware design enhancements.

رومانی صبحی رزق جرجس مطر

• User authentication. The operating system must identify each user who requests access and must ascertain that the user is actually who he or she purports to be. The most common authentication mechanism is password

• Memory protection. Each user's program must run in a portion of memory protected against unauthorized accesses. The protection will certainly prevent outsiders' accesses, and it may also control a user's own access to restricted parts of the program space. Differential security, such as read, write, and execute, may be applied to parts of a user's memory space. Memory protection is usually performed by hardware mechanisms, such as paging or segmentation.

• File and I/O device access control. The operating system must protect user and system files from access by unauthorized users. Similarly, I/O device use must be protected. Data protection is usually achieved by table lookup, as with an

• Allocation and access control to general objects. Users need general objects, such as constructs to permit concurrency and allow synchronization. However, access to these objects must be controlled so that one user does not have a negative effect on other users. Again, table lookup is the common means by which this protection is provided.

2025/2026 2025/2026 2025/2026

You can probably see security implications in many of these primitive operating systems functions. Operating systems show several faces: traffic director, police agent, preschool teacher, umpire, timekeeper, clerk, and housekeeper, to name a few. These fundamental, primitive functions of an operating system are called kernel functions, because they are basic to enforcing security as well as the other higher-level operations an operating system provides. Indeed, the operating system kernel, which we describe shortly, is the basic block that supports all higher-level operating system functions.

Operating systems did not sprout fully formed with the rich feature set we know today. Instead, they evolved from simple support utilities, as we explain next. The history of operating systems is helpful to explain why and how operating systems acquired the security functionality they have today.

A Bit of History

To understand operating systems and their security, it can help to know how modern operating systems evolved. Unlike the evolutions of many other things, operating systems did not progress in a straight line from simplest to most complex but instead had a more jagged progression.

رومانی صبحی رزق جرجس مطر

رومانی صبحی رزق جرجس مطر

Single Users

Once upon a time, there were no operating systems: Users entered their programs directly into the machine in binary by means of switches. In many cases, program entry was done by physical manipulation of a toggle switch; in other cases, the entry was performed with a more complex electronic method, by means of an input device such as a keyboard or a punched card or paper tape reader. Because each user had exclusive use of the computing system, users were required to schedule blocks of time for running the machine. These users were responsible for loading their own libraries of support routines —assemblers, compilers, shared subprograms—and "cleaning up" after use by removing any sensitive code or data.

For the most part there was only one thread of execution. A user loaded a program and any utility support functions, ran that one program, and waited for it to halt at the conclusion of its computation. The only security issue was physical protection of the computer, its programs, and data.

The first operating systems were simple utilities, called executives, designed to assist individual programmers and to smooth the transition from one user to another. The early executives provided linkers and loaders for relocation, easy access to compilers and assemblers, and automatic loading of subprograms from libraries. The executives handled the tedious aspects of programmer support, focusing on a single programmer during execution.

2025/2026 2025/2026 2025/2026

Multiprogramming and Shared Use

Factors such as faster processors, increased uses and demand, larger capacity, and higher cost led to shared computing. The time for a single user to set up a computer, load a program, and unload or shut down at the end was an inefficient waste of expensive machines and labor.

Operating systems took on a much broader role (and a different name) as the notion of multiprogramming was implemented. Realizing that two users could interleave access to the resources of a single computing system, researchers developed concepts such as scheduling, sharing, and concurrent use. ***Multiprogrammed operating systems***, also known as ***monitors***, oversaw each program's execution. Monitors took an active role, whereas executives were passive. That is, an executive stayed in the background, waiting to be called into service by a requesting user. But a monitor actively asserted control of the computing system and gave resources to the user only when the request was consistent with general good use of the system.

Similarly, the executive waited for a request and provided service on demand; the monitor maintained control over all resources, permitting or denying all computing and loaning resources to users as they needed them.

The transition of operating system from executive to monitor was also a shift from supporting to controlling the user.

Multiprogramming brought another important change to computing. When a single person was using a system, the only force to be protected against was that user. Making an error may have made the user feel foolish, but that user could not adversely affect the computation of any other user. However, multiple concurrent users introduced more complexity and risk. User A might rightly be angry if User B's programs or data had a negative effect on A's program's execution. Thus, protecting one user's programs and data from other users' programs became an important issue in multiprogrammed operating systems.

Paradoxically, the next major shift in operating system capabilities involved not growth and complexity but shrinkage and simplicity. The 1980s saw the changeover from multiuser mainframes to personal computers: one computer for one person. With that shift, operating system design went backwards by two decades, forsaking many aspects of controlled sharing and other security features. Those concepts were not lost, however, as the same notions ultimately reappeared, not between two users but between independent activities for the single user.

2025/2026 2025/2026 2025/2026

Controlled sharing also implied security, much of which was lost when the personal computer became dominant.

Multitasking

A user runs a program that generally consists of one ***process***. A process is assigned system resources: files, access to devices and communications, memory, and execution time. The resources of a process are called its domain. The operating system switches control back and forth between processes, allocating, deallocating, and reallocating resources each time a different process is activated. As you can well imagine, significant bookkeeping accompanies each process switch.

رومانی صبحی رزق جرجس مطر

A process consists of one or more ***threads***, separate streams of execution. A thread executes in the same domain as all other threads of the process. That is, threads of one process share a global memory space, files, and so forth. Because resources are shared, the operating system performs far less overhead in switching from one thread to another. Thus, the operating system may change rapidly from one thread to another, giving an effect similar to simultaneous, parallel execution. A thread executes serially (that is, from beginning to end), although execution of one thread may be suspended when a thread of higher priority becomes ready to execute.

Processes have different resources, implying controlled access; threads share resources with less access control.

A server, such as a print server, spawns a new thread for each work package to do. Thus, one print job may be in progress on the printer when the print server receives another print request (perhaps for another user). The server creates a new thread for this second request; the thread prepares the print package to go to the printer and waits for the printer to become ready. In this way, each print server thread is responsible for one print activity, and these separate threads execute the same code to prepare, submit, and monitor one print job.

2025/2026 2025/2026 2025/2026

Finally, a thread may spawn one or more ***tasks***, which is the smallest executable unit of code. Tasks can be interrupted or they can voluntarily relinquish control when they must wait for completion of a parallel task. If there is more than one processor, separate tasks can execute on individual processors, thus giving true parallelism.

Protected Objects

The rise of multiprogramming meant that several aspects of a computing system required protection:

- memory
- sharable I/O devices, such as disks
- serially reusable I/O devices, such as printers and tape drives
- sharable programs and subprocedures
- networks
- sharable data

رومانی صبحی رزق جرجس مطر

As it assumed responsibility for controlled sharing, the operating system had to protect these objects. In the following sections, we look at some of the mechanisms with which operating systems have enforced these objects' protection. Many operating system protection mechanisms have been supported by hardware.

We want to provide sharing for some of those objects. For example, two users with different security levels may want to invoke the same search algorithm or function call. We would like the users to be able to share the algorithms and functions without compromising their individual security needs.

When we think about data, we realize that access can be controlled at various levels: the bit, the byte, the element or word, the field, the record, the file, or the volume. Thus, the granularity of control concerns us. The larger the level of object controlled, the easier it is to implement access control. However, sometimes the operating system must allow access to more than the user needs. For example, with large objects, a user needing access only to part of an object (such as a single record in a file) must be given access to the entire object (the whole file).

Operating System Design to Protect Objects

2025/2026 2029/2026 2025/2026

Operating systems are not monolithic but are instead composed of many individual routines. A well-structured operating system also implements several levels of function and protection, from critical to cosmetic. This ordering is fine conceptually, but in practice, specific functions span these layers. One way to visualize an operating system is in layers, as shown in Figure 5-2. This figure shows functions arranged from most critical (at the bottom) to least critical (at the top). When we say "critical," we mean important to security. So, in this figure, the functions are grouped in three categories: security kernel (to enforce security), operating system kernel (to allocate primitive resources such as time or access to hardware devices), and other operating system functions (to implement the user's interface to hardware). Above the operating system come system utility functions and then the user's applications. In this figure the layering is vertical; other designers think of layering as concentric circles. The critical functions of controlling hardware and enforcing security are said to be in lower or inner layers, and the less critical functions in the upper or outer layers.

رومانی صبحی رزق جرجس مطر

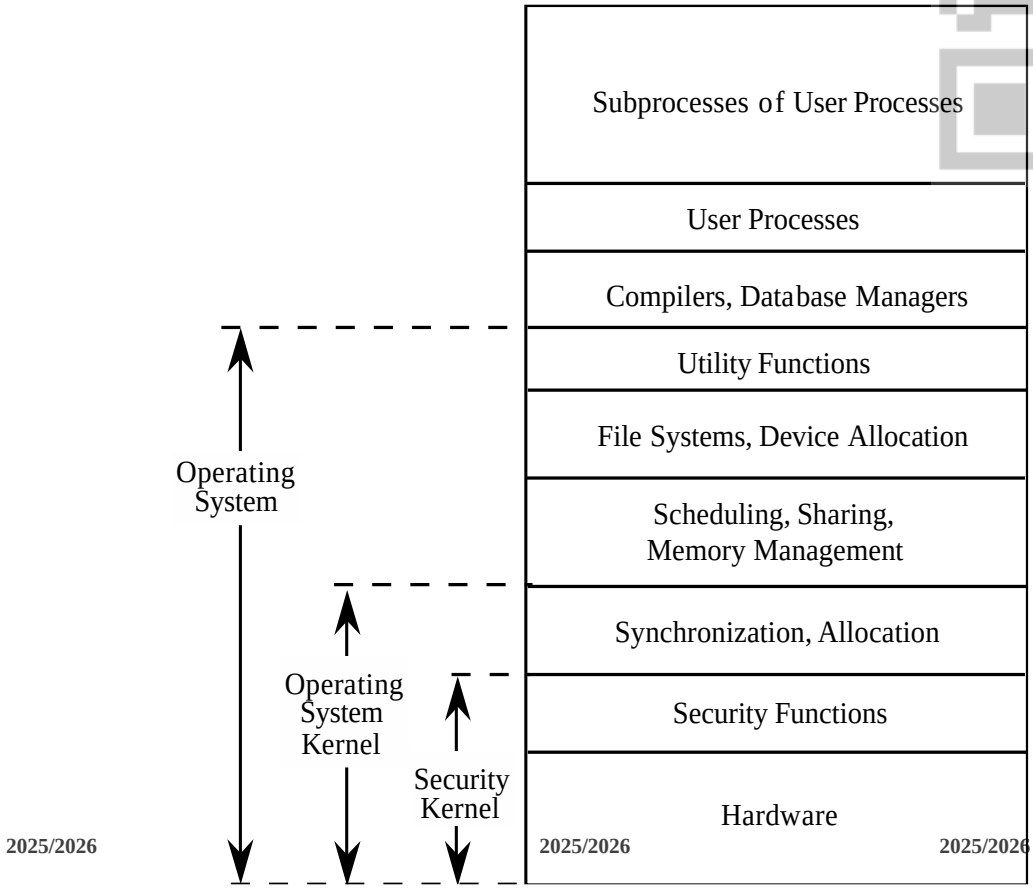
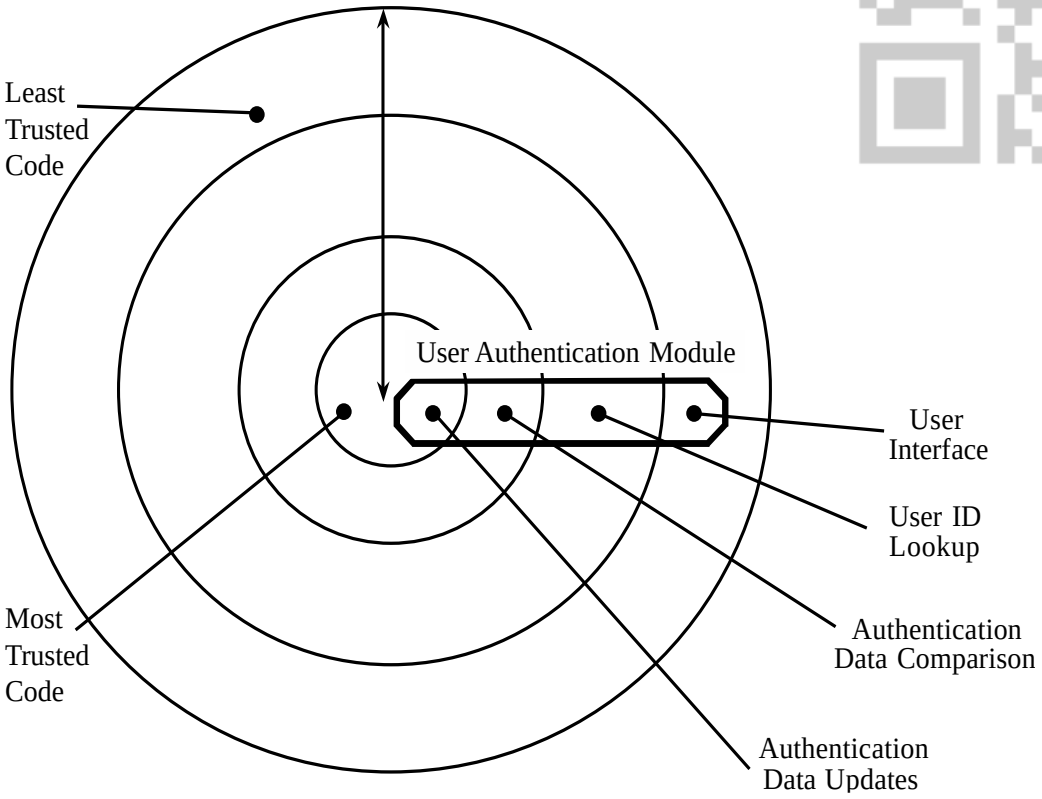


FIGURE 5-2 Layered Operating System

Consider password authentication as an example of a security-relevant operating system activity. In fact, that activity includes several different operations, including (in no particular order) displaying the box in which the user enters a password, receiving password characters but echoing a character such as *, comparing what the user enters to the stored password, checking that a user's identity has been authenticated, or modifying a user's password in the system table. Changing the system password table is certainly more critical to security than displaying a box for password entry, because changing the table could allow an unauthorized user access but displaying the box is merely an interface task. The functions listed would occur at different levels of the operating system. Thus, the user authentication functions are implemented in several places, as shown in Figure 5-3.



2025/2026 FIGURE 5-3 Authentication Functions Spanning Layers in an Operating System 2025/2026

A modern operating system has many different modules, as depicted in Figure 5-4. Not all this code comes from one source. Hardware device drivers may come from the device manufacturer or a third party, and users can install add-ons to implement a different file system or user interface, for example. As you can guess, replacing the file system or user interface requires integration with several levels of the operating system. System tools, such as antivirus code, are said to “hook” or be incorporated into the operating system; those tools are loaded along with the operating system so as to be active by the time user programs execute. Even though they come from different sources, all these modules, drivers, and add-ons may be collectively thought of as the operating system because they perform critical functions and run with enhanced privileges.

From a security standpoint these modules come from different sources, not all trustworthy, and must all integrate successfully. Operating system designers and testers have a nightmarish job to ensure correct functioning with all combinations of hundreds of different add-ons from different sources. All these pieces are maintained separately, so any module can change at any time, but such changes risk incompatibility.

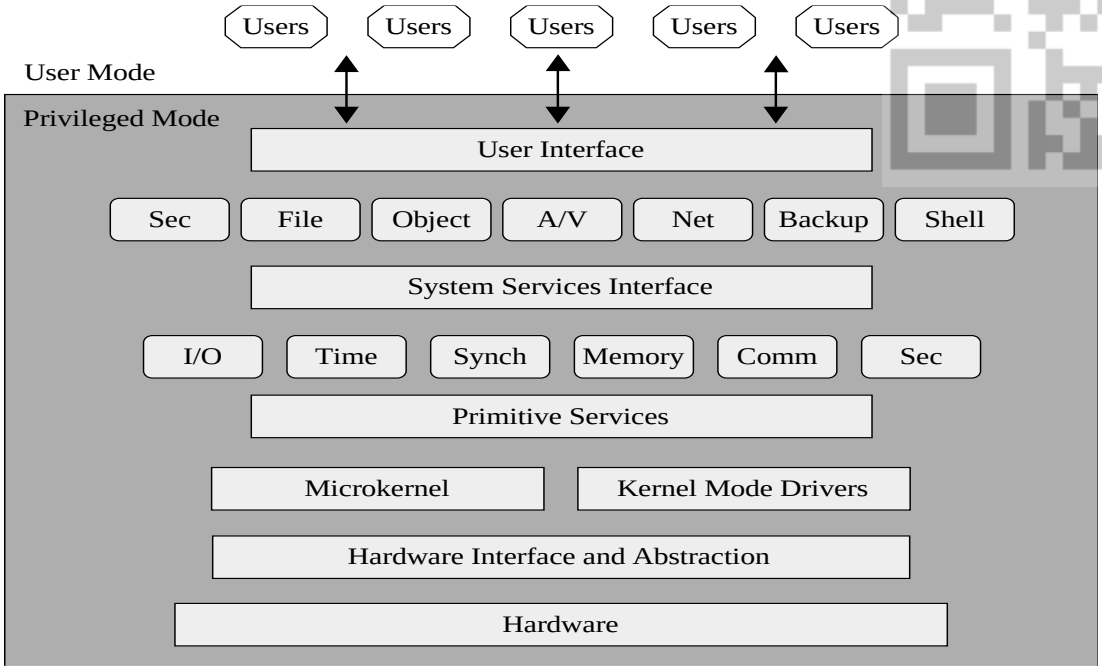


FIGURE 5-4 Operating System Modules

2025/2026 2025/2026 2025/2026

Operating System Design for Self-Protection

An operating system must protect itself against compromise to be able to enforce security. Think of the children's game "king of the hill." One player, the king, stands on top of a mound while the other players scramble up the mound and try to dislodge the king. The king has the natural advantage of being at the top and therefore able to see anyone coming, plus gravity and height work in the king's favor. If someone does force the king off the mound, that person becomes the new king and must defend against attackers. In a computing system, the operating system arrives first and is well positioned by privilege and direct hardware interaction to protect against code that would usurp the operating system's power.

The king of the hill game is simple because there is only one king (at a time). Imagine the chaos if several kings had to repel invaders and also protect against attacks from other kings. One king might even try to dig the mound out from under another king, so attacks on a king could truly come from all directions. Knowing whom to trust and to what degree would become challenges in a multiple-king game. (This political situation can deteriorate into anarchy, which is not good for nations or computing systems.)

The operating system is in a similar situation: It must protect itself not just from errant or malicious user programs but also from harm from incorporated modules, drivers, and add-ons, and with limited knowledge of which ones to trust and for what capabilities.

The operating system must protect itself in order to protect its users and resources.

Next, we consider some of the tools and techniques that operating systems use to enforce protection.

Operating System Tools to Implement Security Functions

In this section we consider how an operating system actually implements the security functions for general objects of unspecified types, such as files, devices, or lists, memory objects, databases, or sharable tables. To make the explanations easier to understand, we sometimes use an example of a specific object, such as a file. Note, however, that a general mechanism can be used to protect any type of object for which access must be limited.

Remember the basic access control paradigm articulated by Scott Graham and Peter Denning [GRA72] and explained in Chapter 2: A subject is permitted to access an object in a particular mode, and only such authorized accesses are allowed. In Chapter 2 we presented several access control techniques: the access control list (ACL), the privilege list, and capabilities. Operating systems implement both the underlying tables supporting access control and the mechanisms that check for acceptable uses.

Another important operating system function related to the access control function is audit: a log of which subject accessed which object when and in what manner. Auditing is a tool for reacting after a security breach, not for preventing one. If critical information is leaked, an audit log may help to determine exactly what information has been compromised and perhaps by whom and when. Such knowledge can help limit the damage of the breach and also help prevent future incidents by illuminating what went wrong this time.

Audit logs show what happened in an incident; analysis of logs can guide prevention of future successful strikes.

2025/2026 2025/2026 2025/2026

An operating system cannot log every action because of the volume of such data. The act of writing to the audit record is also an action, which would generate another record, leading to an infinite chain of records from just the first access. But even if we put aside the problem of auditing the audit, little purpose is served by recording every time a memory location is changed or a file directory is searched. Furthermore, the audit trail is useful only if it is analyzed. Too much data impedes timely and critical analysis.

Virtualization

Another important operating system security technique is virtualization, providing the appearance of one set of resources by using different resources. If you present a plate of cookies to a group of children, the cookies will likely all disappear. If you hide the cookies and put them out a few at a time you limit the children's access. Operating systems can do the same thing.

Virtual Machine

Suppose one set of users, call it the A set, is to be allowed to access only A data, and different users, the B set, can access only B data. We can implement this separation easily and reliably with two unconnected machines. But for performance, economic, or efficiency reasons, that approach may not be desirable. If the A and B sets overlap, strict separation is impossible.

2025/2026 2025/2026 2025/2026

Another approach is ***virtualization***, in which the operating system presents each user with just the resources that class of user should see. To an A user, the machine, called a ***virtual machine***, contains only the A resources. It could seem to the A user as if there is a disk drive, for example, with only the A data. The A user is unable to get to—or even know of the existence of—B resources, because the A user has no way to formulate a command that would expose those resources, just as if they were on a separate machine.

Virtualization: presenting a user the appearance of a system with only the resources the user is entitled to use

Virtualization has advantages other than for security. With virtual machines, an operating system can simulate the effect of one device by using another. So, for example, if an installation decides to replace local disk devices with cloud-based storage, neither the users nor their programs need make any change; the operating system virtualizes the disk drives by covertly modifying each

disk access command so the new commands retrieve and pass along the right data. You execute the command meaning “give me the next byte in this file.” But the operating system has to determine where the file is stored physically on a disk and convert the command to read from sector s block b byte y+1. Unless byte y was the end of a block, in which case the next byte may come from a completely different disk location. Or the command might convert to cloud space c file f byte z. You are oblivious to such transformations because the operating system shields you from such detail.

Next we consider how an operating system can control sharing by separating classes of subjects and objects.

Separation and Sharing

The basis of protection is separation: keeping one user’s objects separate from other users. John Rushby and Brian Randell [RUS83] note that separation in an operating system can occur in several ways:

- physical separation, by which different processes use different physical objects, such as separate printers for output requiring different levels of security
- temporal separation, by which processes having different security requirements are executed at different times
- logical separation, by which users operate under the illusion that no other processes exist, as when an operating system constrains a program’s accesses so that the program cannot access objects outside its permitted domain
- cryptographic separation, by which processes conceal their data and computations in such a way that they are unintelligible to outside processes

Separation occurs by space, time, access control, or cryptograph

Of course, combinations of two or more of these forms of separation are also possible.

The categories of separation are listed roughly in increasing order of complexity to implement, and, for the first three, in decreasing order of the security provided. However, the first two approaches are very stringent and can lead to poor resource utilization. Therefore, we would like to shift the burden of protection to the operating system to allow concurrent execution of processes having different security needs.

But separation is only half the answer. We generally want to separate one user from another user's objects, but we also want to be able to provide sharing for some of those objects. For example, two users with two bodies of sensitive data may want to invoke the same search algorithm or function call. We would like the users to be able to share the algorithms and functions without compromising their individual data. An operating system can support separation and sharing in several ways, offering protection at any of several levels.

- Do not protect. Operating systems with no protection are appropriate when sensitive procedures are being run at separate times.
- Isolate. When an operating system provides isolation, different processes running concurrently are unaware of the presence of each other. Each process has its own address space, files, and other objects. The operating system must confine each process somehow so that the objects of the other processes are completely concealed.
- Share all or share nothing. With this form of protection, the owner of an object declares it to be public or private. A public object is available to all users, whereas a private object is available only to its owner.
- Share but limit access. With protection by access limitation, the operating system checks the allowability of each user's potential access to an object. That is, access control is implemented for a specific user and a specific object. Lists of acceptable actions guide the operating system in determining whether a particular user should have access to a particular object. In some sense, the operating system acts as a guard between users and objects, ensuring that only

authorized accesses occur.

- Limit use of an object. This form of protection limits not just the access to an object but the use made of that object after it has been accessed. For example, a user may be allowed to view a sensitive document but not to print a copy of it. More powerfully, a user may be allowed access to data in a database to derive statistical summaries (such as average salary at a particular grade level), but not to determine specific data values (salaries of individuals).

Again, these modes of sharing are arranged in increasing order of difficulty to implement, but also in increasing order of fineness (which we also describe as granularity) of protection they provide. A given operating system may provide different levels of protection for different

رومانی صبحی رزق جرجس مطر

objects, users, or situations. As we described earlier in this chapter, the granularity of control an operating system implements may not be ideal for the kinds of objects a user needs.

Hardware Protection of Memory

In this section we describe several ways of protecting a memory space. We want a program to be able to share selected parts of memory with other programs and even other users, and especially we want the operating system and a user to coexist in memory without the user's being able to interfere with the operating system. Even in single-user systems, as you have seen, it may be desirable to protect a user from potentially compromisable system utilities and applications. Although the mechanisms for achieving this kind of sharing are somewhat complicated, much of the implementation can be reduced to hardware, thus making sharing efficient and highly resistant to tampering.

Memory protection implements both separation and sharing.

Fence

2025/2026 2025/2026 2025/2026

The simplest form of memory protection was introduced in single-user operating systems, to prevent a faulty user program from destroying part of the resident portion of the operating system. As its name implies, a ***fence*** is a method to confine users to one side of a boundary.

In one implementation, the fence was a predefined memory address, enabling the operating system to reside on one side and the user to stay on the other. An example of this situation is shown in Figure 5-6. Unfortunately, this kind of implementation was very restrictive because a predefined amount of space was always reserved for the operating system, whether the space was needed or not. If less than the predefined space was required, the excess space was wasted. Conversely, if the operating system needed more space, it could not grow beyond the fence boundary.

Another implementation used a hardware register, often called a ***fence register***, containing the address of the end of the operating system. In contrast to a fixed fence, in this scheme the location of the fence could be changed. Each time a user program generated an address for data modification, the address was automatically compared with the fence address.

رومانی صبحی رزق جرجس مطر

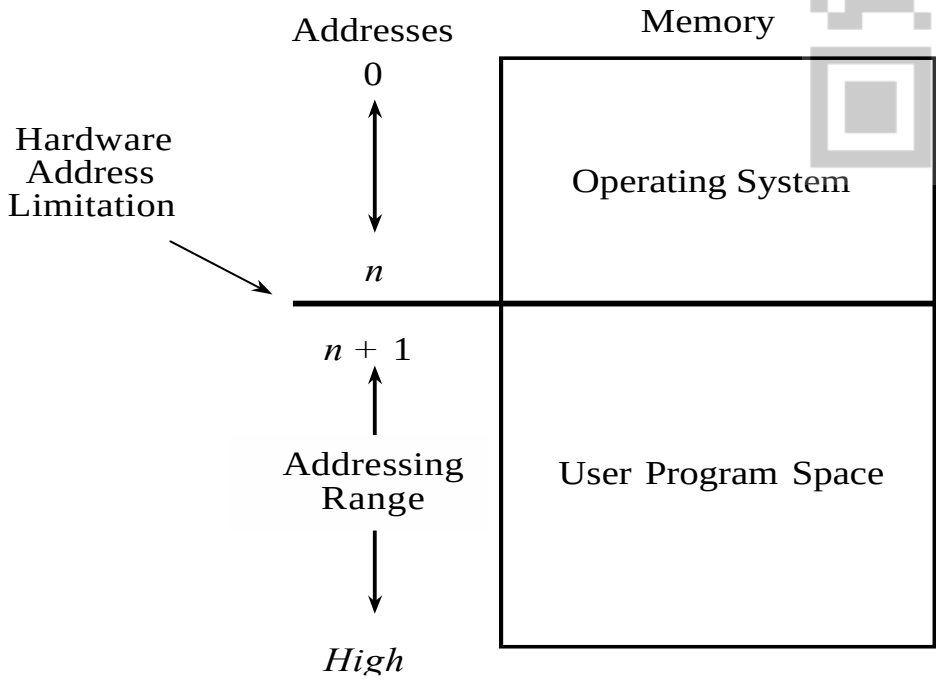


FIGURE 5-6 Fence Protection

If the address was greater than the fence address (that is, in the user area), the instruction was executed; if it was less than the fence address (that is, in the operating system area), an error condition was raised. The use of fence registers is shown in Figure 5-7.

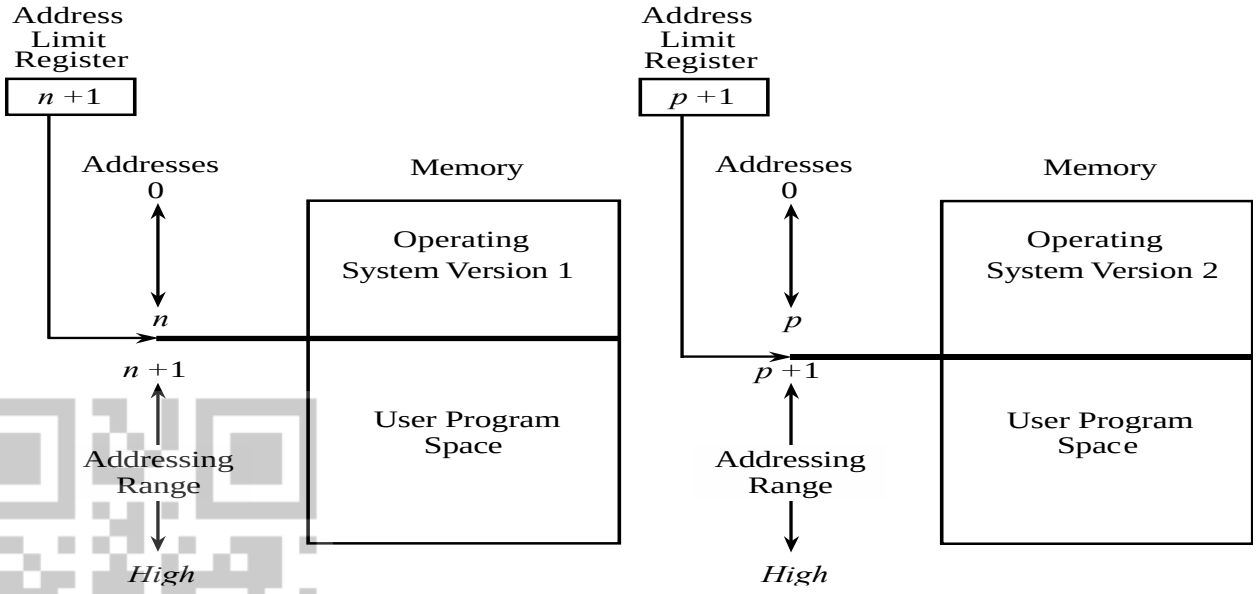


FIGURE 5-7 Fence Registers

A fence register protects in only one direction. In other words, an operating system can be protected from a single user, but the fence cannot protect one user from another user. Similarly, a user cannot identify certain areas of the program as inviolable (such as the code of the program itself or a read-only data area).

Base/Bounds Registers

A major advantage of an operating system with fence registers is the ability to relocate; this characteristic is especially important in a multiuser environment, although it is also useful with multiple concurrent processes loaded dynamically (that is, only when called). With two or more users, none can know in advance where a program will be loaded for execution. The relocation register solves the problem by providing a base or starting address. All addresses inside a program are offsets from that base address. A variable fence register is generally known as a ***base register***.

Fence registers designate a lower bound (a starting address) but not an upper one. An upper bound can be useful in knowing how much space is allotted and in checking for overflows into "forbidden" areas. To overcome this difficulty, a second register is often added, as shown in Figure 5-8. The second register, called a ***bounds register***, is an upper address limit, in the same way that a base or fence register is a lower address limit. Each program address is forced to be above the base address because the contents of the base register are added to the address; each address is also checked to ensure that it is below the bounds address. In this way, a program's addresses are neatly confined to the space between the base and the bounds registers.

This technique protects a program's addresses from modification by another user. When execution changes from one user's program to another's, the operating system must change the contents of the base and bounds registers to reflect the true address space for that user. This change is part of the general preparation, called a context switch, that the operating system must perform when transferring control from one user to another.

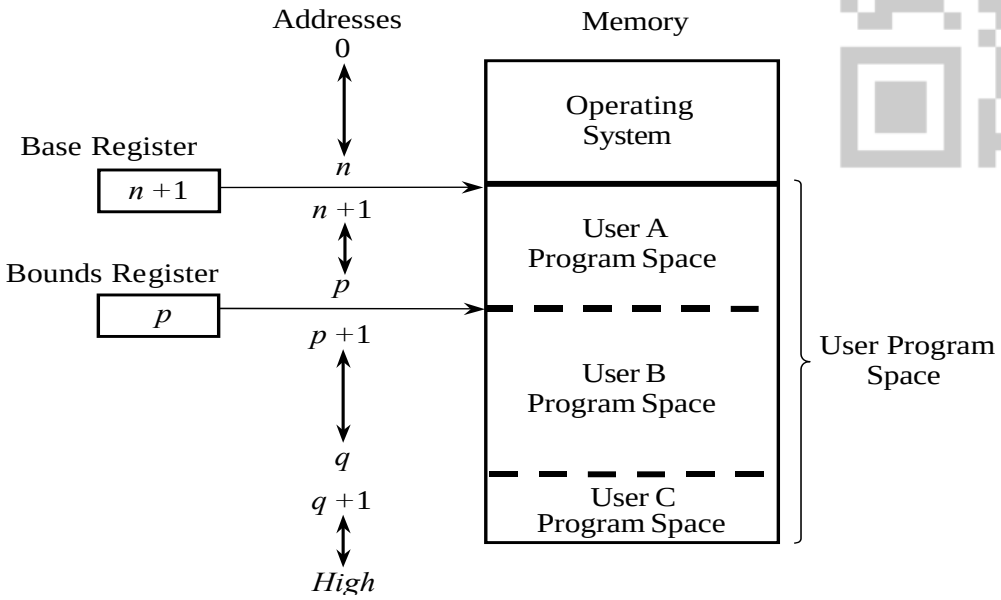


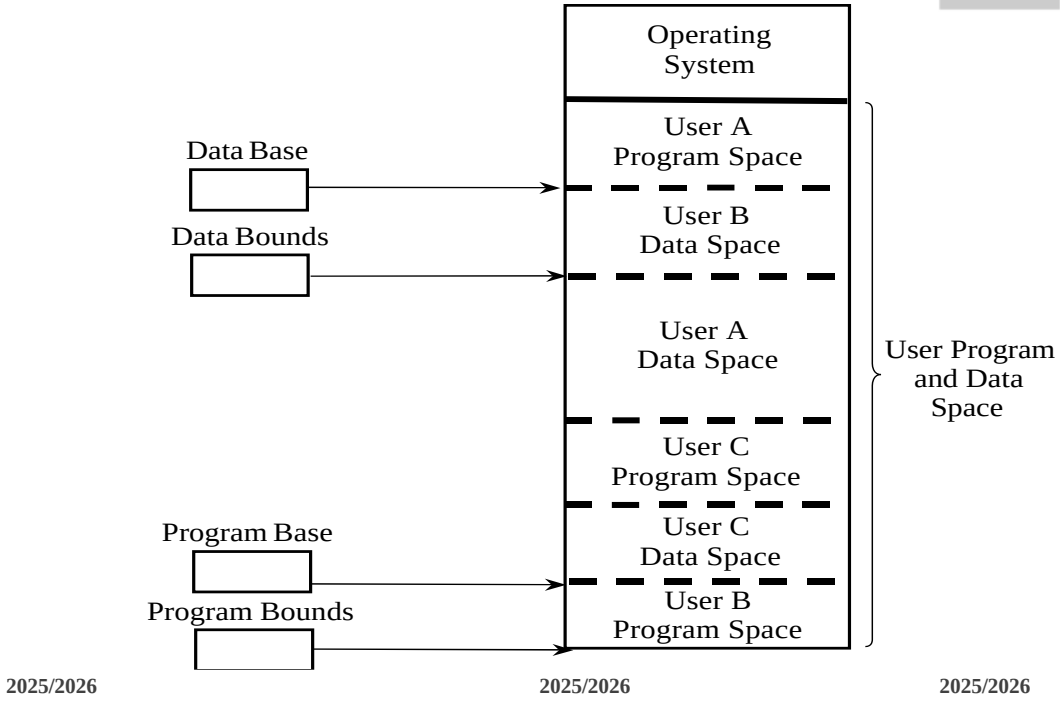
FIGURE 5-8 Base and Bounds Registers

With a pair of base/bounds registers, each user is perfectly protected from outside users, or, more correctly, outside users are protected from errors in any other user's program. Erroneous addresses inside a user's address space can still affect that program because the base/bounds checking guarantees only that each address is inside the user's address space. For example, a user error might occur when a subscript is out of range or an undefined variable generates an address reference within the user's space but, unfortunately, inside the executable instructions of the user's program. In this manner, a user can accidentally store data on top of instructions. Such an error can let a user inadvertently destroy a program, but (fortunately) only that user's own program.

Base/bounds registers surround a program, data area, or domain.

We can solve this overwriting problem by using another pair of base/bounds registers, one for the instructions (code) of the program and a second for the data space. Then, only instruction fetches (instructions to be executed) are relocated and checked with the first register pair, and only data accesses (operands of instructions) are relocated and checked with the second register pair. The use of two pairs of base/bounds registers is shown in Figure 5-9. Although two pairs of registers do not prevent all program errors, they limit the effect of data-manipulating instructions

to the data space. The pairs of registers offer another more important advantage: the ability to split a program into two pieces that can be relocated separately.



2025/2026 2025/2026 2025/2026

FIGURE 5-9 Two Pairs of Base and Bounds Registers

These two features seem to call for the use of three or more pairs of registers: one for code, one for read-only data, and one for modifiable data values. Although in theory this concept can be extended, two pairs of registers are the limit for practical computer design. For each additional pair of registers (beyond two), something in the machine code or state of each instruction must indicate which relocation pair is to be used to address the instruction's operands. That is, with more than two pairs, each instruction specifies one of two or more data spaces. But with only two pairs, the decision can be automatic: data operations (add, bit shift, compare) with the data pair, execution operations (jump) with the code area pair.

Tagged Architecture

Another problem with using base/bounds registers for protection or relocation is their contiguous nature. Each pair of registers confines accesses to a consecutive range of addresses. A compiler

or loader can easily rearrange a program so that all code sections are adjacent and all data sections are adjacent.

However, in some cases you may want to protect some data values but not all. For example, a personnel record may require protecting the field for salary but not office location and phone number. Moreover, a programmer may want to ensure the integrity of certain data values by allowing them to be written when the program is initialized but prohibiting the program from modifying them later. This scheme protects against errors in the programmer's own code. A programmer may also want to invoke a shared subprogram from a common library. We can address some of these issues by using good design, both in the operating system and in the other programs being run. Recall that in Chapter 3 we studied good design characteristics such as information hiding and modularity in program design. These characteristics dictate that one program module must share with another module only the minimum amount of data necessary for both of them to do their work.

Additional, operating-system-specific design features can help, too. Base/bounds registers create an all-or-nothing situation for sharing: Either a program makes all its data available to be accessed and modified or it prohibits access to all. Even if there were a third set of registers for shared data, all shared data would need to be located together. A procedure could not effectively share data items A, B, and C with one module, A, C, and D with a second, and A, B, and D with a third. The only way to accomplish the kind of sharing we want would be to move each appropriate set of data values to some contiguous space. However, this solution would not be acceptable if the data items were large records, arrays, or structures.

An alternative is tagged architecture, in which every word of machine memory has one or more extra bits to identify the access rights to that word. These access bits can be set only by privileged (operating system) instructions. The bits are tested every time an instruction accesses that location.

For example, as shown in Figure 5-10, one memory location may be protected as execute-only (for example, the object code of instructions), whereas another is protected for fetch-only (for example, read) data access, and another accessible for modification (for example, write). In this way, two adjacent locations can have different access rights. Furthermore, with a few extra tag

2025/2026
2025/2026
2025/2026

bits, different classes of data (numeric, character, address, or pointer, and undefined) can be separated, and data fields can be protected for privileged (operating system) access only.

Tag	Memory Word
R	0001
RW	0137
R	0099
X	
X	
X	
X	
X	
X	
R	4091
RW	0002

Code: R = Read-only RW = Read/Write
X = Execute-only

FIGURE 5-10 Tagged Architecture

This protection technique has been used on a few systems, although the number of tag bits has been rather small. The Burroughs B6500-7500 system used three tag bits to separate data words (three types), descriptors (pointers), and control words (stack pointers and addressing control words). The IBM System/38 used a tag to control both integrity and access.

Virtual Memory

We present two more approaches to memory protection, each of which can be implemented on top of a conventional machine structure, suggesting a better chance of acceptance. Although these approaches are ancient by computing standards—they were designed between 1965 and 1975—they have been implemented on many machines since then. Furthermore, they offer important advantages in addressing, with memory protection being a delightful bonus.

2025/2026 2025/2026 2025/2026

Segmentation

The first of these two approaches, segmentation, involves the simple notion of dividing a program into separate pieces. Each piece has a logical unity, exhibiting a relationship among all its code or data values. For example, a segment may be the code of a single procedure, the data of an array, or the collection of all local data values used by a particular module. Segmentation was developed as a feasible means to produce the effect of the equivalent of an unbounded number of base/bounds registers. In other words, segmentation allows a program to be divided into many pieces having different access rights.

Each segment has a unique name. A code or data item within a segment is addressed as the pair <name, offset>, where name is the name of the segment containing the data item and offset is its location within the segment (that is, its distance from the start of the segment).

Logically, the programmer pictures a program as a long collection of segments. Segments can be separately relocated, allowing any segment to be placed in any available memory locations. The relationship between a logical segment and its true memory position is shown in Figure 5-11.

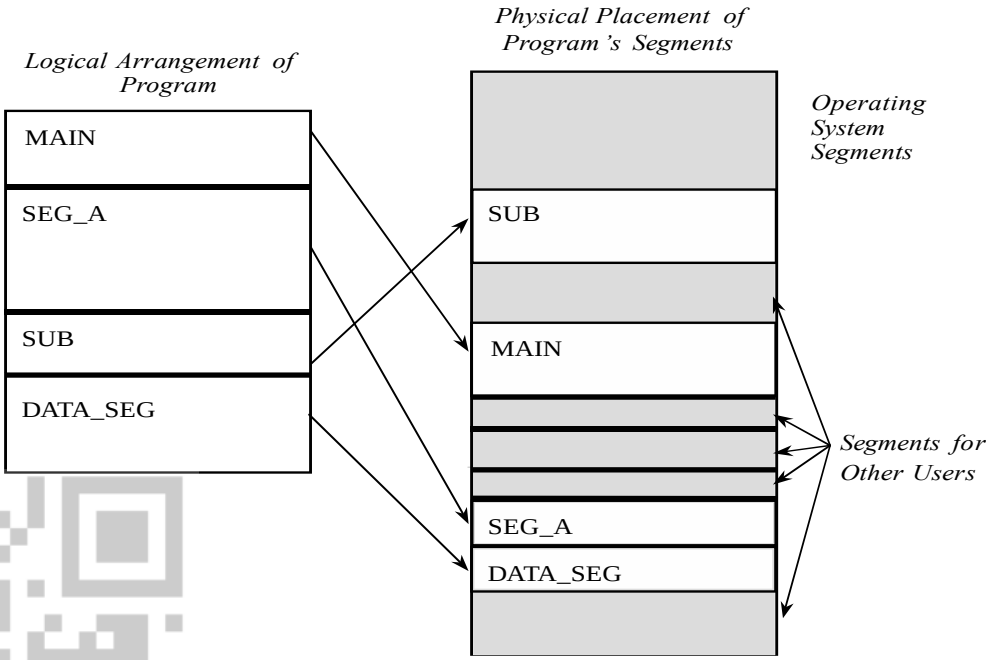


FIGURE 5-11 Segmentation

The operating system must maintain a table of segment names and their true addresses in memory. When a program generates an address of the form <name, offset>, the operating system looks up name in the segment directory and determines its real beginning memory address. To that address the operating system adds offset, giving the true memory address of the code or data item. This translation is shown in Figure 5-12. For efficiency there is usually one operating system segment address table for each process in execution. Two processes that need to share access to a single segment would have the same segment name and address in their segment tables.

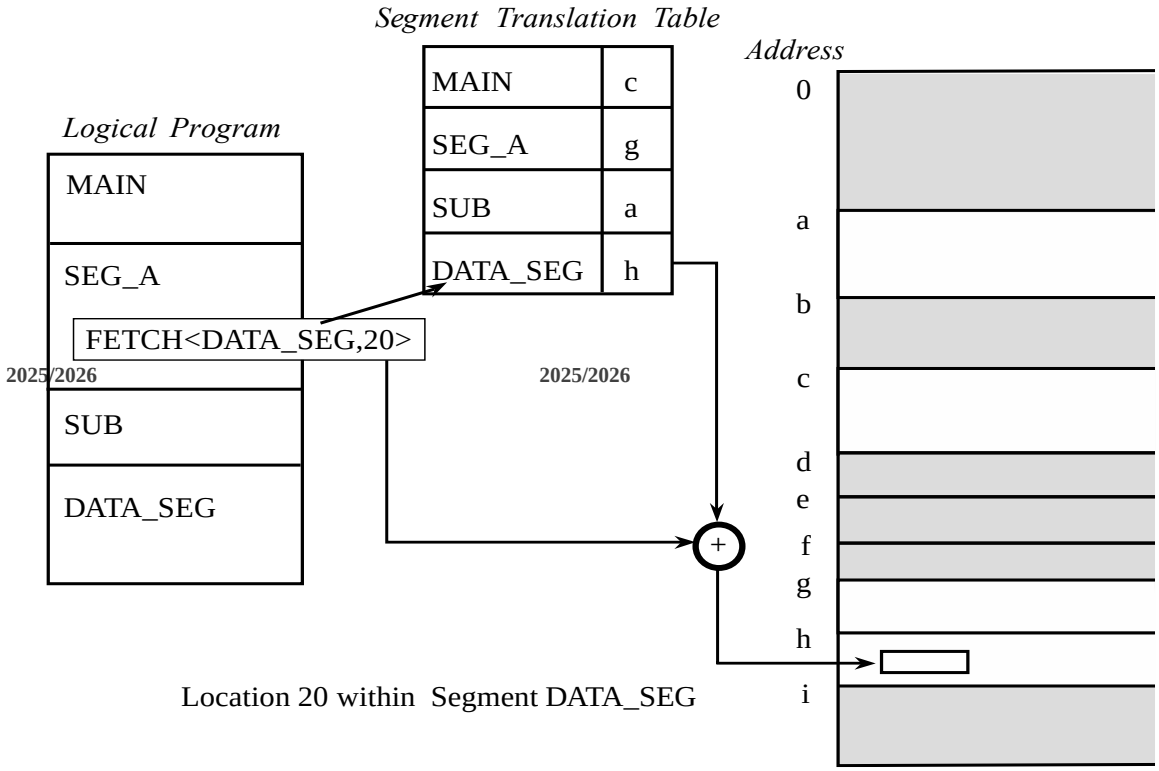
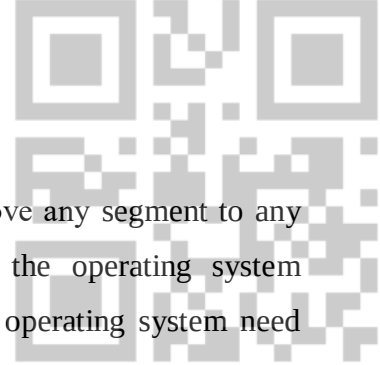


FIGURE 5-12 Segment Address Translation

Thus, a user's program does not know what true memory addresses it uses. It has no way—and no need—to determine the actual address associated with a particular <name, offset>. The <name, offset> pair is adequate to access any data or instruction to which a program should have access.

This hiding of addresses has three advantages for the operating system.



- The operating system can place any segment at any location or move any segment to any location, even after the program begins to execute. Because the operating system translates all address references by a segment address table, the operating system need only update the address in that one table when a segment is moved.

- A segment can be removed from main memory (and stored on an auxiliary device) if it is not being used currently. (These first two advantages explain why this technique is called virtual memory, with the same basis as the virtualization described earlier in this chapter. The appearance of memory to the user is not necessarily what actually exists.)

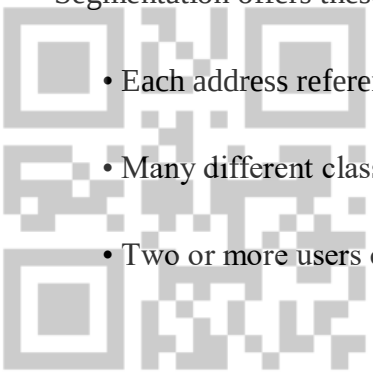
- Every address reference passes through the operating system, so there is an opportunity to check each one for protection.

Because of this last characteristic, a process can access a segment only if that segment appears in that process's segment-translation table. The operating system controls which programs have entries for a particular segment in their segment address tables. This control provides strong protection of segments from access by unpermitted processes. For example, program A might have access to segments BLUE and GREEN of user X but not to other segments of that user or of any other user. In a straightforward way we can allow a user to have different protection classes for different segments of a program. For example, one segment might be read-only data, a second might be execute-only code, and a third might be writeable data. In a situation like this one, segmentation can approximate the goal of separate protection of different pieces of a program, as outlined in the previous section on tagged architecture.

2025/2026 2025/2026 2025/2026

Segmentation allows hardware-supported controlled access to different memory sections in different access modes.

Segmentation offers these security benefits:



- Each address reference is checked—neither too high nor too low—for protection.
- Many different classes of data items can be assigned different levels of protection.
- Two or more users can share access to a segment, with potentially different access rights.

رومانی صبحی رزق جرجس مطر

• A user cannot generate an address or access to an unpermitted segment.

One protection difficulty inherent in segmentation concerns segment size. Each segment has a particular size. However, a program can generate a reference to a valid segment name, but with an offset beyond the end of the segment. For example, reference <A,9999> looks perfectly valid, but in reality segment A may be only 200 bytes long. If left unplugged, this security hole could allow a program to access any memory address beyond the end of a segment just by using large values of offset in an address.

Paging

An alternative to segmentation is paging. The program is divided into equal-sized pieces called pages, and memory is divided into equal-sized units called page frames. (For implementation reasons, the page size is usually chosen to be a power of 2 between 512 and 4096 bytes.) As with segmentation, each address in a paging scheme is a two-part object, this time consisting of <page, offset>.

2025/2026 2025/2026 2025/2026

Each address is again translated by a process similar to that of segmentation: The operating system maintains a table of user page numbers and their true addresses in memory. The page portion of every <page, offset> reference is converted to a page frame address by a table lookup; the offset portion is added to the page frame address to produce the real memory address of the object referred to as <page, offset>. This process is illustrated in Figure 5-13.

Unlike segmentation, all pages in the paging approach are of the same fixed size, so fragmentation is not a problem. Each page can fit in any available page in memory, thus obviating the problem of addressing beyond the end of a page. The binary form of a < page, offset> address is designed so that the offset values fill a range of bits in the address. Therefore, an offset beyond the end of a particular page results in a carry into the page portion of the address, which changes the address.

Paging allows the security advantages of segmentation with more efficient memory management.

رومانی صبحی رزق جرجس مطر

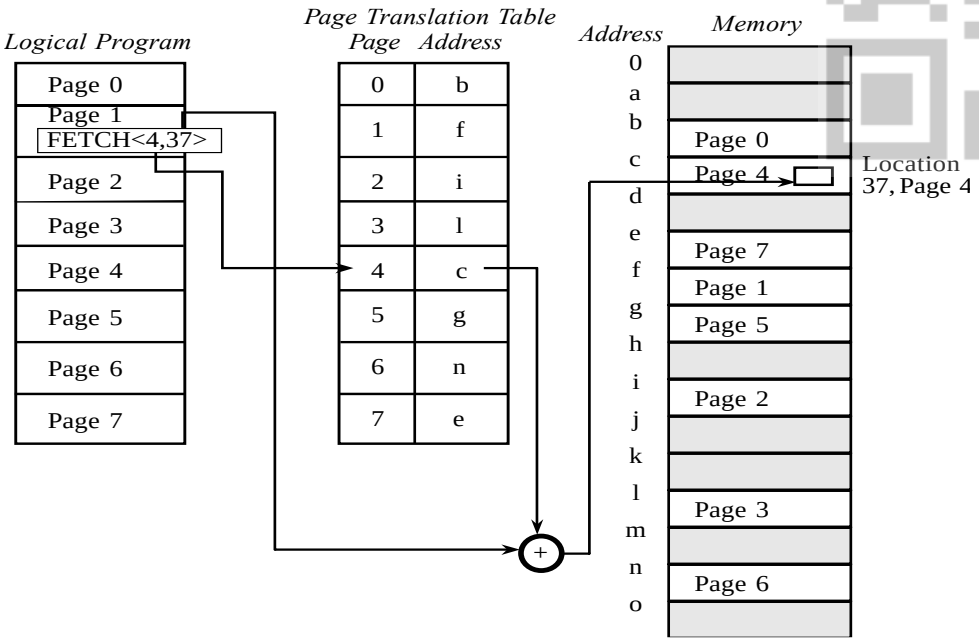


FIGURE 5-13 Page Address Translation

To see how this idea works, consider a page size of 1024 bytes ($1024 = 2^{10}$) , where 10 bits are allocated for the offset portion of each address. A program cannot generate an offset value larger than 1023 in 10 bits. Moving to the next location after <x, 1023> causes a carry into the page portion, thereby moving translation to the next page. During the translation, the paging process checks to verify that a <page, offset> reference does not exceed the maximum number of pages the process has defined.

With a segmentation approach, a programmer must be conscious of segments. However, a programmer is oblivious to page boundaries when using a paging-based operating system. Moreover, with paging there is no logical unity to a page; a page is simply the next 2^n bytes of the program. Thus, a change to a program, such as the addition of one instruction, pushes all subsequent instructions to lower addresses and moves a few bytes from the end of each page to the start of the next. This shift is not something about which the programmer need be concerned, because the entire mechanism of paging and address translation is hidden from the programmer.

However, when we consider protection, this shift is a serious problem. Because segments are logical units, we can associate different segments with individual protection rights, such as read-only or execute-only. The shifting can be handled efficiently during address translation. But with

paging, there is no necessary unity to the items on a page, so there is no way to establish that all values on a page should be protected at the same level, such as read-only or execute-only.

Combined Paging with Segmentation

We have seen how paging offers implementation efficiency, while segmentation offers logical protection characteristics. Since each approach has drawbacks as well as desirable features, the two approaches have been combined.

The IBM 390 family of mainframe systems used a form of paged segmentation. Similarly, the Multics operating system (implemented on a GE-645 machine) applied paging on top of segmentation. In both cases, the programmer could divide a program into logical segments. Each segment was then broken into fixed-size pages. In Multics, the segment name portion of an address was an 18-bit number with a 16-bit offset. The addresses were then broken into 1024-byte pages. The translation process is shown in Figure 5-14. This approach retained the logical unity of a segment and permitted differentiated protection for the segments, but it added an additional layer of translation for each address. Additional hardware improved the efficiency of the implementation.

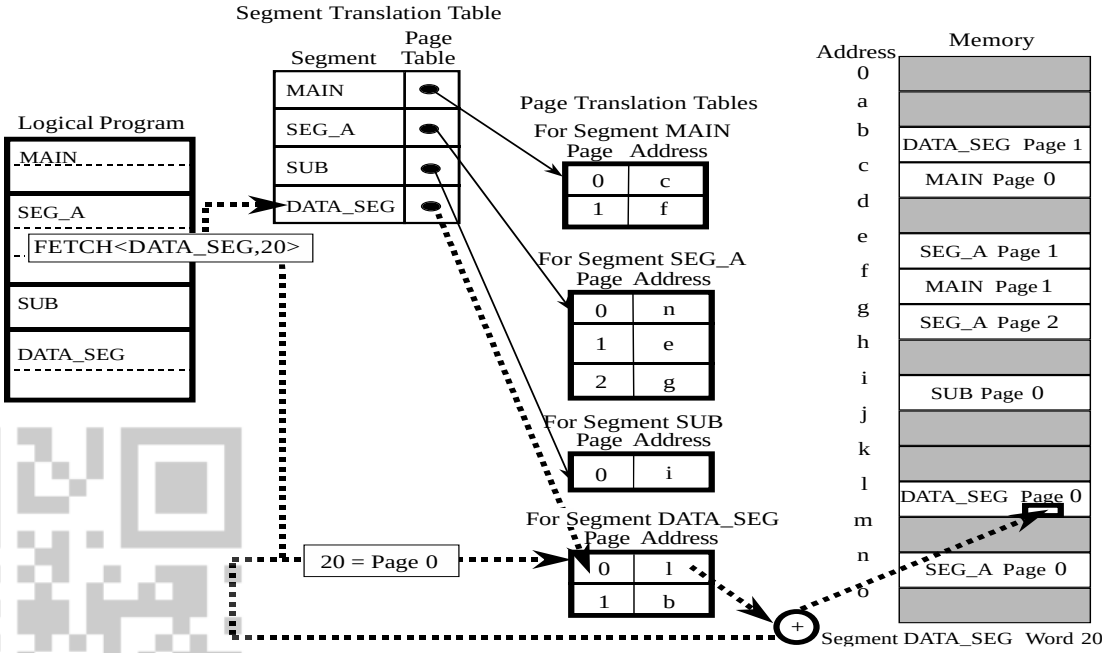


FIGURE 5-14 Address Translation with Paged Segmentation

These hardware mechanisms provide good memory protection, even though their original purpose was something else indeed: efficient memory allocation and data relocation, with security a fortuitous side effect. In operating systems, security has been a central requirement and design element since the beginning, as we explore in the next section.

4.2 Security in the Design of Operating Systems

As we just discussed, operating systems are complex pieces of software. The components come from many sources, some pieces are legacy code to support old functions; other pieces date back literally decades, with long-forgotten design characteristics. And some pieces were written just yesterday. Old and new pieces must interact and interface successfully, and new designers must ensure that their code works correctly with all existing previous versions, not to mention the numerous applications that exist.

Exploit authors capitalize on this complexity by experimenting to locate interface mismatches: a function no longer called, an empty position in the table of interrupts handled, a forgotten device driver. The operating system opens many points to which code can later attach as pieces are loaded during the boot process; if one of these pieces is not present, the malicious code can attach instead.

Obviously, not all complex software is vulnerable to attack. The point we are making is that the more complex the software, the more possibilities for unwanted software introduction. A house with no windows leaves no chance for someone to break in through a window, but each additional window in a house design increases the potential for this harm and requires the homeowner to apply more security. Now extend this metaphor to modern operating systems that typically include millions of lines of code: What is the likelihood that every line is perfect for its use and fits perfectly with every other line?

The principles of secure program design we introduced in Chapter 3 apply equally well to operating systems. Simple, modular, loosely coupled designs present fewer opportunities to the attacker.

Simplicity of Design

Operating systems by themselves (regardless of their security constraints) are difficult to design. They handle many duties, are subject to interruptions and context switches, and must minimize overhead so as not to slow user computations and interactions. Adding the responsibility for security enforcement to the operating system increases the difficulty of design.

Layered Design

As described previously, a nontrivial operating system consists of at least four levels: hardware, kernel, operating system, and user. Each of these layers can include sublayers. For example, in [SCH83], the kernel has five distinct layers. The user level may also have quasi-system programs, such as database managers or graphical user interface shells, that constitute separate layers of security themselves.

Layered Trust

2025/2026 2025/2026 2025/2026

As we discussed earlier in this chapter, the layered structure of a secure operating system can be thought of as a series of concentric circles, with the most sensitive operations in the innermost layers. An equivalent view is as a building, with the most sensitive tasks assigned to lower floors. Then, the trustworthiness and access rights of a process can be judged by the process's proximity to the center: The more trusted processes are closer to the center or bottom.

Implicit in the use of layering as a countermeasure is separation. Earlier in this chapter we described ways to implement separation: physical, temporal, logical, and cryptographic. Of these four, logical (software-based) separation is most applicable to layered design, which means a fundamental (inner or lower) part of the operating system must control the accesses of all outer or higher layers to enforce separation.

Peter Neumann [NEU86] describes the layered structure used for the Provably Secure Operating System (PSOS). Some lower-level layers present some or all of their functionality to higher levels, but each layer properly encapsulates those things below itself.

Kernelized Design

A kernel is the part of an operating system that performs the lowest-level functions. In standard operating system design, the kernel implements operations such as synchronization, interprocess communication, message passing, and interrupt handling. The kernel is also called a nucleus or core. The notion of designing an operating system around a kernel is described by Butler Lampson and Howard Sturgis [LAM76] and by Gerald Popek and Charles Kline [POP78].

A ***security kernel*** is responsible for enforcing the security mechanisms of the entire operating system. The security kernel provides the security interfaces among the hardware, operating system, and other parts of the computing system. Typically, the operating system is designed so that the security kernel is contained within the operating system kernel. Security kernels are discussed in detail by Stan Ames [AME83].

Security kernel: locus of all security enforcemen

There are several good design reasons why security functions may be isolated in a security kernel.

2025/2026 2025/2026 2025/2026

- Coverage. Every access to a protected object must pass through the security kernel. In a system designed in this way, the operating system can use the security kernel to ensure that every access is checked.
- Separation. Isolating security mechanisms both from the rest of the operating system and from the user space makes it easier to protect those mechanisms from penetration by the operating system or the users.
- Unity. All security functions are performed by a single set of code, so it is easier to trace the cause of any problems that arise with these functions.
- Modifiability. Changes to the security mechanisms are easier to make and easier to test. And because of unity, the effects of changes are localized so interfaces are easier to understand and control.

رومانی صبحی رزق جرجس مطر

• Compactness. Because it performs only security functions, the security kernel is likely to be relatively small.

• Verifiability. Being relatively small, the security kernel can be analyzed rigorously. For example, formal methods can be used to ensure that all security situations (such as states and state changes) have been covered by the design.

Reference Monitor

The most important part of a security kernel is the reference monitor, the portion that controls accesses to objects [AND72, LAM71]. We introduced reference monitors in Chapter 2. The reference monitor separates subjects and objects, enforcing that a subject can access only those objects expressly allowed by security policy. A reference monitor is not necessarily a single piece of code; rather, it is the collection of access controls for devices, files, memory, interprocess communication, and other kinds of objects. As shown in Figure 5-15, a reference monitor acts like a brick wall around the operating system or trusted software to mediate accesses by subjects (S) to objects (O).

2025/2026 2025/2026

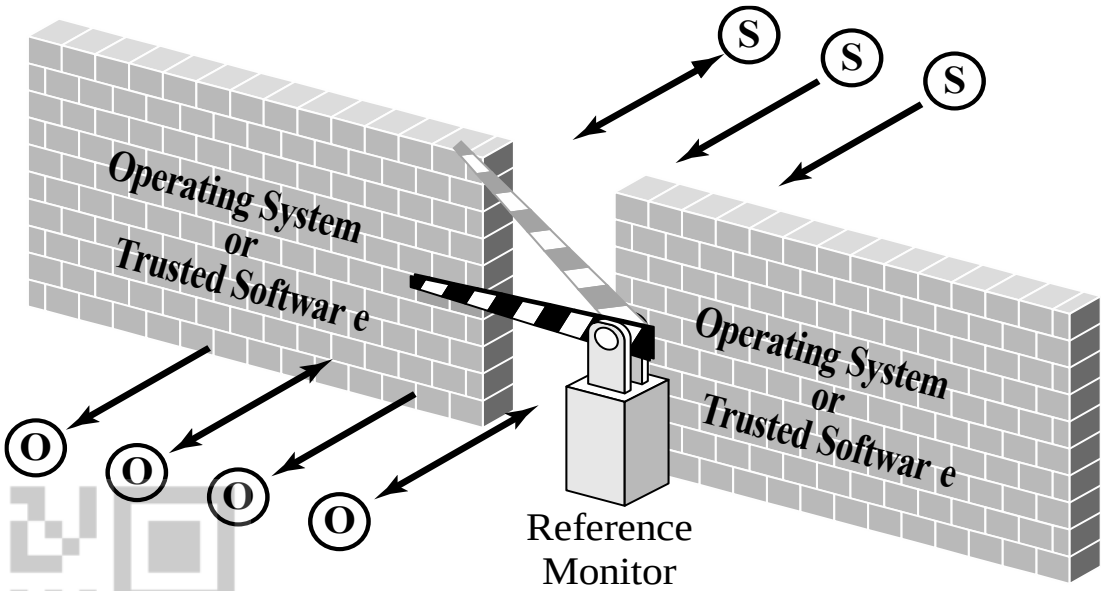


FIGURE 5-15 Reference Monitor

As stated in Chapter 2, a reference monitor must be

- tamperproof, that is, impossible to weaken or disable
- unbypassable, that is, always invoked when access to any object is required
- analyzable, that is, small enough to be subjected to analysis and testing, the completeness of which can be ensured

The reference monitor is not the only security mechanism of a trusted operating system. Other parts of the security suite include auditing and identification and authentication processing, as well as setting enforcement parameters, such as who are allowable subjects and what objects they are allowed to access. These other security parts interact with the reference monitor, receiving data from the reference monitor or providing it with the data it needs to operate.

The reference monitor concept has been used for many trusted operating systems and also for smaller pieces of trusted software. The validity of this concept is well supported both in research and in practice. Paul Karger [KAR90, KAR91] and Morrie Gasser [GAS88] describe the design and construction of the kernelized DEC VAX operating system that adhered strictly to use of a reference monitor to control access.

Correctness and Completeness

That security considerations pervade the design and structure of operating systems requires correctness and completeness. Correctness implies that because an operating system controls the interaction between subjects and objects, security must be considered in every aspect of its design. That is, the operating system design must include definitions of which objects will be protected in what ways, what subjects will have access and at what levels, and so on. There must be a clear mapping from the security requirements to the design so that all developers can see how the two relate.

Moreover, after designers have structured a section of the operating system, they must check to see that the design actually implements the degree of security that it is supposed to enforce. This checking can be done in many ways, including formal reviews or simulations. Again, a mapping

is necessary, this time from the requirements to design to tests, so that developers can affirm that each aspect of operating system security has been tested and shown to work correctly. Because security appears in every part of an operating system, security design and implementation cannot be left fuzzy or vague until the rest of the system is working and being tested.

Completeness requires that security functionality be included in all places necessary. Although this requirement seems self-evident, not all developers are necessarily thinking of security as they design and write code, so security completeness is challenging. It is extremely hard to retrofit security features to an operating system designed with inadequate security. Leaving an operating system's security to the last minute is much like trying to install plumbing or electrical wiring in a house whose foundation is set, floors laid, and walls already up and painted; not only must you destroy most of what you have built, but you may also find that the general structure can no longer accommodate all that is needed (and so some has to be left out or compromised). And last-minute additions are often done hastily under time pressure, which does not encourage completeness.

Security enforcement must be correct and complete.

Secure Design Principles

Good design principles are always good for security, as we have noted above. But several important design principles are particular to security and essential for building a solid, trusted operating system. These principles, articulated well by Jerome Saltzer and Michael Schroeder [SAL74, SAL75], were raised in Chapter 3; we repeat them here because of their importance in the design of secure operating systems.

- least privilege
- economy of mechanism
- open design
- complete mediation
- permission based
- separation of privilege

• least common mechanism
• ease of use

Although these design principles were suggested several decades ago, they are as accurate now as they were when originally written. The principles have been used repeatedly and successfully in the design and implementation of numerous trusted systems. More importantly, when security problems have been found in operating systems in the past, they almost always derive from failure to abide by one or more of these principles. These design principles led to the development of "trusted" computer systems or "trusted" operating systems.

Trusted Systems

Trusted systems can also help counter the malicious software problem. A trusted system is one that has been shown to warrant some degree of trust that it will perform certain activities faithfully, that is, in accordance with users' expectations. Contrary to popular usage, "trusted" in this context does not mean hope, in the sense of "gee, I hope this system protects me from malicious code." Hope is trust with little justification; trusted systems have convincing evidence to justify users' trust. See Sidebar 5-4 for further discussion of the meaning of the word.

Trusted system: one with evidence to substantiate the claim it implements some function or policy

Sidebar 5-4 What Does "Trust" Mean for a System?

Before we begin to examine a trusted operating system in detail, let us look more carefully at the terminology involved in understanding and describing trust. What would it take for us to consider something to be secure?

The word secure reflects a dichotomy: Something is either secure or not secure. If secure, it should withstand all attacks, today, tomorrow, and a century from now. And if we claim that it is secure, you either accept our assertion (and buy and use it) or reject it (and either do not use it or use it but do not expect much from it).

How does security differ from quality? If we claim that something is good, you are less interested in our claims and more interested in an objective appraisal of whether the thing meets your performance and functionality needs. From this perspective, security is only one facet of goodness or quality; you may choose to balance security with other characteristics (such as speed or user friendliness) to select a system that is best, given the choices you may have. In particular, the system you build or select may be pretty good, even though it may not be as secure as you would like it to be.

Security professionals prefer to speak of trusted instead of secure operating systems. A trusted system connotes one that meets the intended security requirements, is of high enough quality, and justifies the user's confidence in that quality. That is, trust is perceived by the system's receiver or user, not by its developer, designer, or manufacturer. As a user, you may not be able to evaluate that trust directly. You may trust the design, a professional evaluation, or the opinion of a valued colleague. But in the end, it is your responsibility to sanction the degree of trust you require.

2025/2026 2025/2026 2025/2026

We say that software is trusted software if we know that the code has been rigorously developed and analyzed, giving us reason to trust that the code does what it is expected to do and nothing more. Typically, trusted code can be a foundation on which other, untrusted, code runs. That is, the untrusted system's quality depends, in part, on the trusted code; the trusted code establishes the baseline for security of the overall system. In particular, an operating system can be trusted software when there is a rational or objective basis for trusting that it correctly controls the accesses of components or systems run from it.

To trust any program, we base our trust on rigorous analysis and testing, looking for certain key characteristics:

- Functional correctness. The program does what it is supposed to, and it works correctly.
- Enforcement of integrity. Even if presented erroneous commands or commands from unauthorized users, the program maintains the correctness of the data with which it has contact.

رومانی صبحی رزق جرجس مطر

- Limited privilege. The program is allowed to access secure data, but the access is minimized and neither the access rights nor the data are passed along to other untrusted programs or back to an untrusted caller.

- Appropriate confidence level. The program has been examined and rated at a degree of trust appropriate for the kind of data and environment in which it is to be used.

Trusted software is often used as a safe way for general users to access sensitive data. Trusted programs are used to perform limited (safe) operations for users without allowing the users to have direct access to sensitive data.

History of Trusted Systems

Trusted systems have had a long and fitful history in computer security. The need for secure systems became apparent early in the days of multiuser, shared computing, in the 1960s. Willis Ware [WAR70] chaired a committee expressing the need for stronger security enforcement in systems. During the 1970s, research and actual systems demonstrated the capability of and need for such systems, culminating in the report from James Anderson's committee [AND72] that called for development of a process for obtaining more trustworthy systems.

2025/2026 2025/2026 2025/2026

Starting with drafts in the late 1970s, the U.S. Department of Defense wrote the Trusted Computer System Evaluation Criteria (called the TCSEC or Orange Book, because of the color of its cover), a document that specified functionality, design principles, and an evaluation methodology for trusted computer systems. Over time, the same approach was extended to network components and database management systems. For reasons we explain shortly, this scheme did not reach its intended degree of acceptance. Nevertheless, the TCSEC laid the groundwork for a progression of advancements on that foundation. Also important is that this progression started in the United States, but rapidly expanded to involve Canada, Germany, England, the Netherlands, and France (as well as work in other countries), engendering a truly international approach to trusted computer systems, depicted in the timeline of Figure 5-16.

Orange Book (TCSEC): First attempt to codify principles and requirements for secure computing systems

The 1980s and 1990s saw several candidates for evaluating the degree of a system's trustedness, and these approaches converged between 1995 and 2003 in an international process for evaluation, called the Common Criteria for Information Technology Security Evaluation. Today, thanks to that standard, the market has many products whose trustworthiness has been independently confirmed.

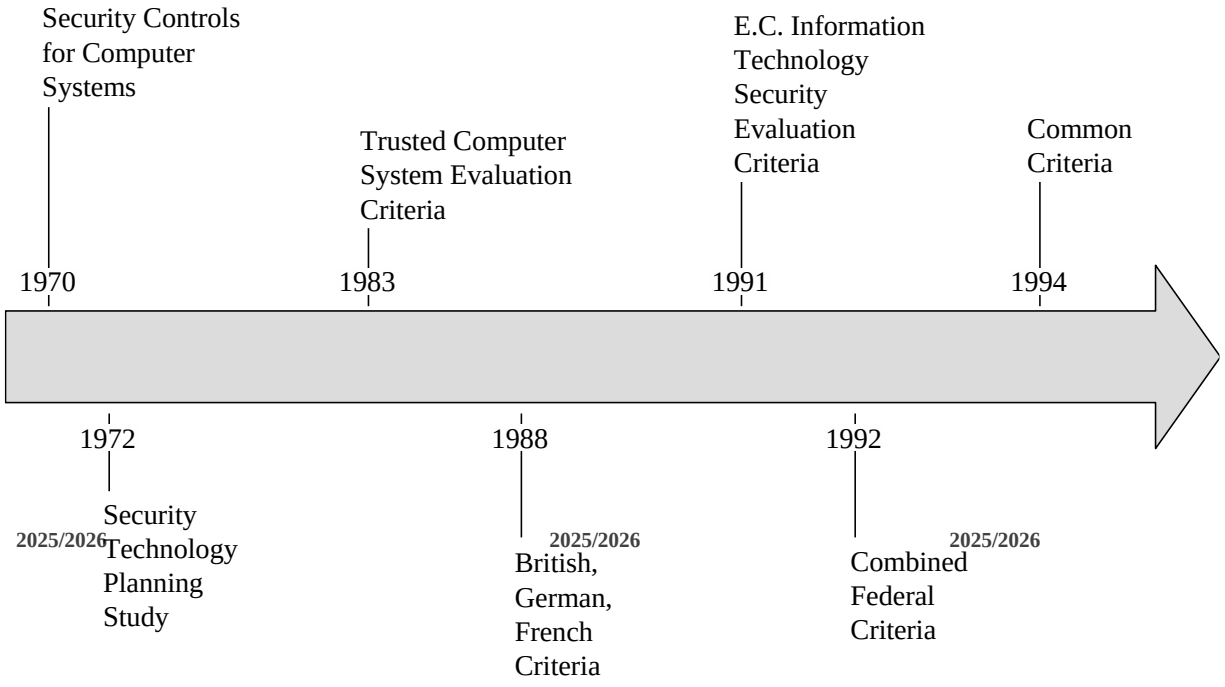


FIGURE 5-16 Trusted Systems Design and Evaluation Criteria

In the next section we examine the functions important to a trusted system. Then, in the following section, we briefly describe the current trusted system evaluation and certification process.

Trusted System Functions

Trusted systems contain certain functions to ensure security. In this section we look over several aspects of a trusted system, starting with the trusted computing base.

Trusted Computing Base (TCB)

The trusted computing base, or TCB, is the name we give to everything in the trusted operating system that is necessary to enforce the security policy. Alternatively, we say that the TCB consists of the parts of the trusted operating system on which we depend for correct enforcement of policy.

Trusted computing base (TCB): everything necessary for a system to enforce its security policy

We can think of the TCB as a coherent whole in the following way. Suppose you divide a trusted operating system into the parts that are in the TCB and those that are not, and you allow the most skillful malicious programmers to write all the non-TCB parts. Since the TCB handles all the security (including protecting itself), nothing the malicious non-TCB parts do can impair the correct security policy enforcement of the TCB. This definition gives you a sense that the TCB forms the fortress-like shell that protects whatever in the system needs protection. But the analogy also clarifies the meaning of trusted in trusted operating system: Our trust in the security of the whole system depends on the TCB.

2025/2026 2025/2026 2025/2026

Obviously, the TCB must be both correct and complete. Thus, to understand how to design a good TCB, we focus on the division between the TCB and non-TCB elements of the operating system and concentrate our effort on ensuring the correctness of the TCB.

TCB Functions

Just what constitutes the TCB? We can answer this question by listing system elements on which security enforcement could depend:

- hardware, including processors, memory, registers, a clock, and I/O devices
- some notion of processes, so that we can separate and protect security-critical processes
- primitive files, such as the security access control database and identification and authentication data
- protected memory, so that the reference monitor can be protected against tampering

رومانی صبحی رزق جرجس مطر

• some interprocess communication, so that different parts of the TCB can pass data to and activate other parts; for example, the reference monitor can invoke and pass data securely to the audit routine

It may seem as if this list encompasses most of the operating system, but in fact the TCB is only a small subset. For example, although the TCB requires access to files of enforcement data, it does not need an entire file structure of hierarchical directories, virtual devices, indexed files, and multidevice files. Thus, the TCB might contain a primitive file manager to handle only the small, simple security data files needed for the TCB. The more complex file manager to provide externally visible files could be outside the TCB. Figure 5-17 shows a typical division into TCB and non-TCB sections

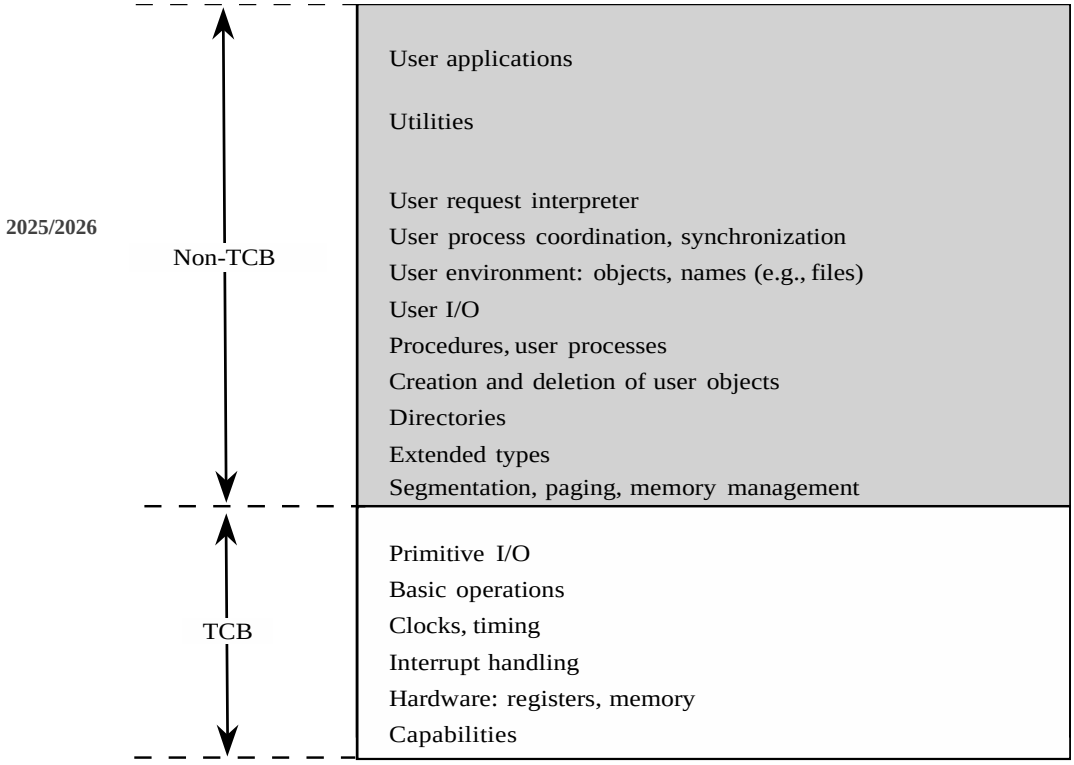


FIGURE 5-17 System Separated into TCB and Non-TCB Sections

The TCB, which must maintain the secrecy and integrity of each domain, monitors four basic interactions.

• Process activation. In a multiprogramming environment, activation and deactivation of processes occur frequently. Changing from one process to another requires a complete change of registers, relocation maps, file access lists, process status information, and other pointers, much of which is security sensitive information.

• Execution domain switching. Processes running in one domain often invoke processes in other domains to obtain more or less sensitive data or services.

• Memory protection. Because each domain includes code and data stored in memory, the TCB must monitor memory references to ensure secrecy and integrity for each domain.

• I/O operation. In some systems, software is involved with each character transferred in an I/O operation. This software connects a user program in the outermost domain to an I/O device in the innermost (hardware) domain. Thus, I/O operations can cross all domains.

TCB Design

2025/2026 2025/2026 2025/2026

The division of the operating system into TCB and non-TCB aspects is convenient for designers and developers because it means that all security-relevant code is located in one (logical) part. But the distinction is more than just logical. To ensure that the security enforcement cannot be affected by non-TCB code, TCB code must run in some protected state that distinguishes it and protects it from interference or compromise by any code outside the TCB. Thus, the structuring into TCB and non-TCB must be done consciously.

The TCB is separated to achieve self-protection and independence.

TCB Implementation

Security-related activities are likely to be performed in different places. Security is potentially related to every memory access, every I/O operation, every file or program access, every activation or termination of a user, every creation of a new execution thread, and every interprocess communication. In modular operating systems, these separate activities can be handled in independent modules. Each of these separate modules, then, has both security-related and other functions.

رومانی صبحی رزق جرجس مطر

Collecting all security functions into the TCB may destroy the modularity of an existing operating system. A unified TCB may also be too large to be analyzed easily. Nevertheless, a designer may decide to separate the security functions of an existing operating system, creating a security kernel. This form of kernel is depicted in Figure 5-18.

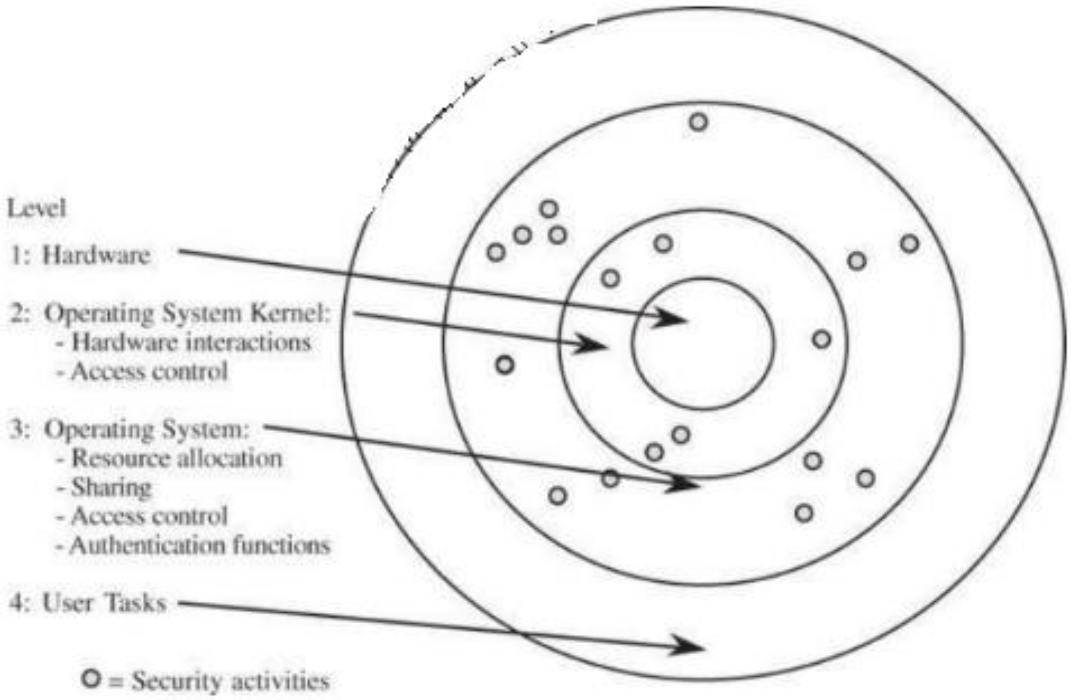


FIGURE 5-18 Security Kernel

A more sensible approach is to design the security kernel first and then design the operating system around it. This technique was used by Honeywell in the design of a prototype for its secure operating system, Scomp. That system contained only twenty modules to perform the primitive security functions, and these modules consisted of fewer than 1,000 lines of higher-level-language source code. Once the actual security kernel of Scomp was built, its functions grew to contain approximately 10,000 lines of code.

In a security-based design, the security kernel forms an interface layer, just atop system hardware. The security kernel monitors all operating system hardware accesses and performs all protection functions. The security kernel, which relies on support from hardware, allows the operating system itself to handle most functions not related to security. In this way, the security kernel can be small and efficient. As a byproduct of this partitioning, computing systems have at

least three execution domains: security kernel, operating system, and user. This situation was depicted in Figure 5-1 at the start of this chapter.

Secure Startup

Startup is a known weak point in system design. Before the operating system is fully functional, its protection capabilities are limited. As more pieces become operational, they exercise more complete control over the resources. During startup, the nature of the threats is also lowered because users are not yet active and network connections have not yet been established.

Designers of trusted systems recognized the vulnerability at system startup, especially if it was a restart from a previous failure. Thus, trusted system design documents such as the Orange Book [DOD85] require developers to demonstrate that when the system starts, all security functions are working properly and no effects remain from any previous system session.

Secure startup ensures no malicious code can block or interfere with security enforcement.

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

رومانی صبحی رزق جرجس مطر

Exercises

1. Give an example of the use of physical separation for security in a computing environment.
2. Give an example of the use of temporal separation for security in a computing environment.
3. Give an example of an object whose sensitivity may change during execution.
4. Respond to the allegation "An operating system requires no protection for its executable code (in memory) because that code is a duplicate of code maintained on disk."
5. Explain how a fence register is used for relocating a user's program.
6. Can any number of concurrent processes be protected from one another by just one pair of base/bounds registers?
7. The discussion of base/bounds registers implies that program code is execute only and that data areas are read-write-only. Is this ever not the case? Explain your answer.
8. A design using tag bits presupposes that adjacent memory locations hold dissimilar things: a line of code, a piece of data, a line of code, two pieces of data, and so forth. Most programs do not look like that. How can tag bits be appropriate in a situation in which programs have the more conventional arrangement of code and data?
9. What are some other modes of access that users might want to apply to code or data, in addition to the common read, write, and execute permission?
10. If two users share access to a segment, they must do so by the same name. Must their protection rights to it be the same? Why or why not?
11. A problem with either segmented or paged address translation is timing. Suppose a user wants to read some data from an input device into memory. For efficiency during data transfer, often the actual memory address at which the data are to be placed is provided to an I/O device. The real address is passed so that time consuming address translation does not have to be performed during a very fast data transfer. What security problems does this approach bring?

2025/2026 2025/2026 2025/2026

Chapter 5

Databases

رومانی صبحی رزق جرجس مطر

Protecting data is at the heart of many secure systems, and many users (people, programs, or systems) rely on a database management system (DBMS) to manage the protection of structured data. For this reason, we devote this chapter to the security of databases and database management systems, as an example of how application security can be designed and implemented for a specific task.

Databases are essential to many business and government organizations, holding data that reflect the organization's core activities. Often, when business processes are reengineered to make them more effective and more in tune with new or revised goals, one of the first systems to receive careful scrutiny is the set of databases supporting the business processes. Thus, databases are more than software-related repositories. Their organization and contents are considered valuable corporate assets that must be carefully protected.

2025/2026 2025/2026 2025/2026

However, the protection provided by database management systems has had mixed results. Over time, we have improved our understanding of database security problems, and several good controls have been developed. But there are still more security concerns for which no controls are available.

We begin this chapter with a brief summary of database terminology. We then consider the security requirements for database management systems. Two major security problems integrity and secrecy—are explained in a database context. We continue the chapter by studying two important (but related) database security problems, the inference problem and the multilevel problem. Both problems are complex, and there are no immediate solutions. However, by understanding the problems, we become more sensitive to ways of reducing potential threats to the data. Finally, we conclude the chapter by looking at big data collection and data mining, a technology for deriving patterns from one or more databases. Data mining involves many of the security issues we raise in this chapter.

5.1 Introduction to Databases

We begin by describing a database and defining terminology related to its use. We draw on examples from what is called the relational database because it is one of the most widely used types. However, the concepts described here apply to any type of database.

We first define the basic concepts and then use them to discuss security concerns.

Concept of a Database

A ***database*** is a collection of data and a set of rules that organize the data by specifying certain relationships among the data. Through these rules, the user describes a logical format for the data. The data items are stored in a file, but the precise physical format of the file is of no concern to the user. A ***database administrator*** is a person who defines the rules that organize the data and also controls who should have access to what parts of the data. The user interacts with the database through a program called a database manager or a ***database management system (DBMS)***, informally known as a ***front end***.

2025/2026 2025/2026 2025/2026

Components of Databases

The database file consists of ***records***, each of which contains one related group of data. As shown in the example in Table 7-1, a record in a name and address file consists of one name and address. Each record contains ***fields or elements***, the elementary data items themselves. The fields in the name and address record are NAME, ADDRESS, CITY, STATE, and ZIP (where ZIP is the U.S. postal code). This database can be viewed as a two-dimensional table, where a record is a row and each field of a record is an element of the table.

A database is a collection of tables, each containing records having one or more fields or elements.

Not every database is easily represented as a single, compact table. The database in Figure 7-1 logically consists of three files with possibly different uses. These three files could be represented as one large table, but that depiction may not improve the utility of or access to the data.

رومانی صبحی رزق جرجس مطر

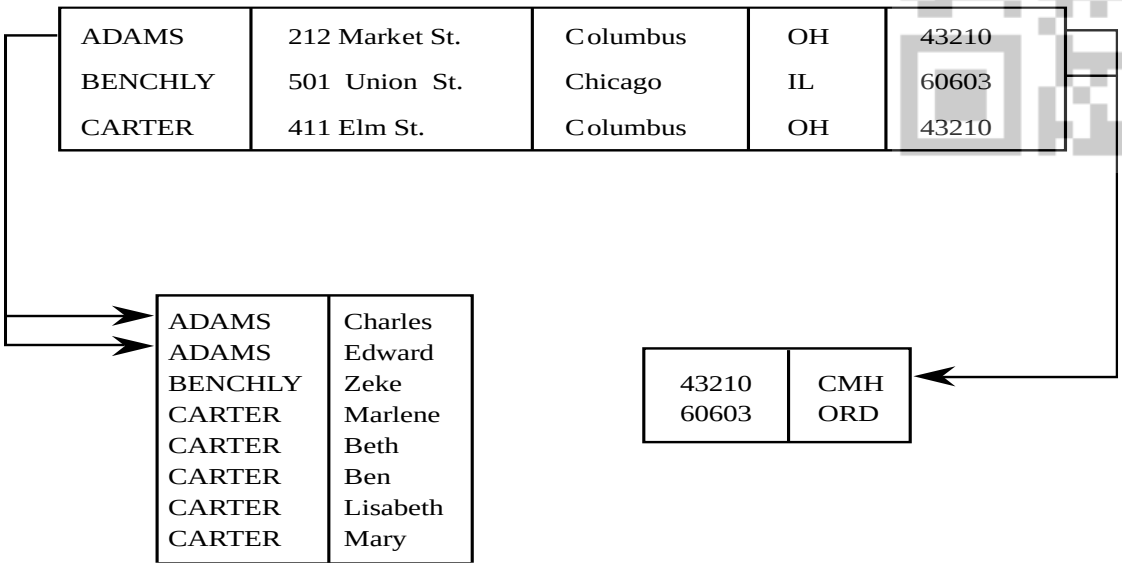


FIGURE 7-1 Database of Several Related Tables

The logical structure of a database is called a schema. A particular user may have access to only part of the database, called a subschema. The overall schema of the database in Figure 7-1 is detailed in Table 7-2. The three separate blocks of the figure are examples of subschemas, although other subschemas of this database can be defined. We can use schemas and subschemas to present to users only those elements they wish or need to see. For example, if Table 7-1 represents the employees at a company, the subschema on the lower left can list employee names without revealing personal information such as home address.

ADAMS	212 Market St.	Columbus	OH	43210
BENCHLY	501 Union St.	Chicago	IL	60603
CARTER	411 Elm St.	Columbus	OH	43210

TABLE 7-1 Example of a Database

The rules of a database identify the columns with names. The name of each column is called an attribute of the database. A relation is a set of columns. For example, using the database in Table 7-2, we see that NAME–ZIP is a relation formed by taking the NAME and ZIP columns, as shown in Table 7-3. The relation specifies clusters of related data values, in much the same way

that the relation “mother of” specifies a relationship among pairs of humans. In this example, each cluster contains a pair of elements, a NAME and a ZIP. Other relations can have more columns, so each cluster may be a triple, a 4-tuple, or an n-tuple (for some value n) of elements.

Name	First	Address	City	State	Zip	Airport
ADAMS	Charles	212 Market St.	Columbus	OH	43210	CMH
ADAMS	Edward	212 Market St.	Columbus	OH	43210	CMH
BENCHLY	Zeke	501 Union St.	Chicago	IL	60603	ORD
CARTER	Marlene	411 Elm St.	Columbus	OH	43210	CMH
CARTER	Beth	411 Elm St.	Columbus	OH	43210	CMH
CARTER	Ben	411 Elm St.	Columbus	OH	43210	CMH
CARTER	Lisabeth	411 Elm St.	Columbus	OH	43210	CMH
CARTER	Mary	411 Elm St.	Columbus	OH	43210	CMH

TABLE 7-2 Schema of Database from Figure 7-1

Relations in a database show some connection among data in tables.

2025/2026

Name	Zip
ADAMS	43210
BENCHLY	60603
CARTER	43210

TABLE 7-3 Relation in a Database

Queries

Users interact with database managers through commands to the DBMS that retrieve, modify, add, or delete fields and records of the database. A command is called a query. Database management systems have precise rules of syntax for queries. Most query languages use an English-like notation, and many are based on SQL, a structured query language originally

developed by IBM. We have written the example queries in this chapter to resemble English sentences so that they are easy to understand. For example, the query

SELECT NAME = ‘ADAMS’

retrieves all records having the value ADAMS in the NAME field.

The result of executing a query is a subschema. One way to form a subschema of a database is by selecting records meeting certain conditions. For example, we might select records in which ZIP=43210, producing the result shown in Table 7-4.

TABLE 7-4 Results of a Select Query

Other, more complex, selection criteria are possible, with logical operators such as and (∧) and or (∨), and comparisons such as less than (<). An example of a select query is

SELECT (ZIP=‘43210’) ∧ (NAME=‘ADAMS’)

2025/2026 2025/2026 2025/2026

After having selected records, we may project these records onto one or more attributes. The select operation identifies certain rows from the database, and a project operation extracts the values from certain fields (columns) of those records. The result of a select-project operation is the set of values of specified attributes for the selected records. For example, we might select records meeting the condition ZIP=43210 and project the results onto the attributes NAME and FIRST, as in Table 7-5. The result is the list of first and last names of people whose addresses have zip code 43210.

ADAMS	Charles
ADAMS	Edward
CARTER	Marlene
CARTER	Beth
CARTER	Ben
CARTER	Lisabeth
CARTER	Mary

TABLE 7-5 Results of a Select–Project Query

رومانی صبحی رزق جرجس مطر

Notice that we do not have to project onto the same attribute(s) on which the selection is done. For example, we can build a query using ZIP and NAME but project the result onto FIRST:

Click here to view code image

SHOW FIRST WHERE (ZIP='43210') ∧ (NAME='ADAMS')

The result would be a list of the first names of people whose last names are ADAMS and ZIP is 43210.

We can also merge two subschema on a common element by using a join query. The result of this operation is a subschema whose records have the same value for the common element. For example, Figure 7-2 shows that the subschema NAME–ZIP and the subschema ZIP–AIRPORT can be joined on the common field ZIP to produce the subschema NAME–AIRPORT.

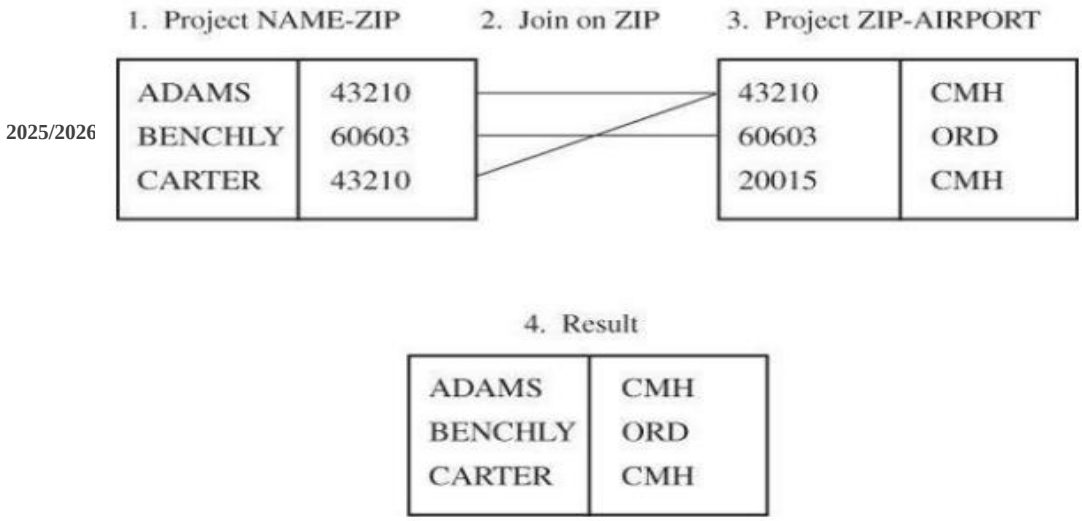


FIGURE 7-2 Result of a Select–Project–Join Query

Users extract data through use of queries.

Advantages of Using Databases

The logical idea behind a database is this: A database is a single collection of data, stored and maintained at one central location, to which many people have access as needed. However, the actual implementation may involve some other physical storage arrangement or access. The

2025/2026

essence of a good database is that the users are unaware of the physical arrangements; the unified logical arrangement is all they see. As a result, a database offers many advantages over a simple file system:

- shared access, so that many users can use one common, centralized set of data
- controlled access, so that only authorized users are allowed to view or to modify data values
- minimal redundancy, so that individual users do not have to collect and maintain their own sets of data
- data consistency, so that a change to a data value affects all users of the data value
- data integrity, so that data values are protected against accidental or malicious undesirable changes

You should notice many familiar security concepts in this list. Although security is only one of several motivations for using a database, some users appreciate having a degree of secure access.

2025/2026 2025/2026 2025/2026

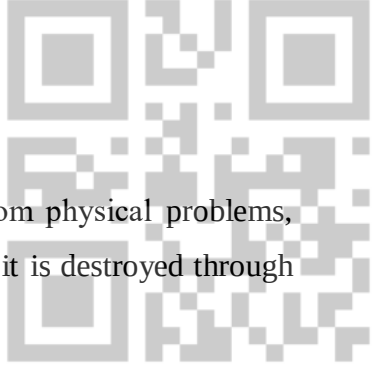
Databases support controlled, shared access to a single repository of data.

A DBMS is designed to provide these advantages efficiently. However, as often happens, the objectives can conflict with each other. In particular, as we shall see, security interests can conflict with performance. This clash is not surprising, because measures taken to enforce security often increase the computing system's size or complexity. What is surprising, though, is that security interests may also reduce the system's ability to provide data to users by limiting certain queries that would otherwise seem innocuous.

5.2 Security Requirements of Databases

The basic security requirements of database systems are not unlike those of other computing systems we have studied. The basic problems—access control, exclusion of spurious data, authentication of users, and reliability—have appeared in many contexts so far in this book. Following is a list of requirements for database security.

رومانی صبحی رزق جرجس مطر



- Physical database integrity. The data of a database are immune from physical problems, such as power failures, and someone can reconstruct the database if it is destroyed through a catastrophe.

- Logical database integrity. The structure of the database is preserved. With logical integrity of a database, a modification to the value of one field does not affect other fields, for example.

- Element integrity. The data contained in each element are accurate.

- Auditability. It is possible to track who or what has accessed (or modified) the elements in the database.

- Access control. A user is allowed to access only authorized data, and different users can be restricted to different modes of access (such as read or write).

- User authentication. Every user is positively identified, both for the audit trail and for permission to access certain data.

2025/2026 2025/2026 2025/2026

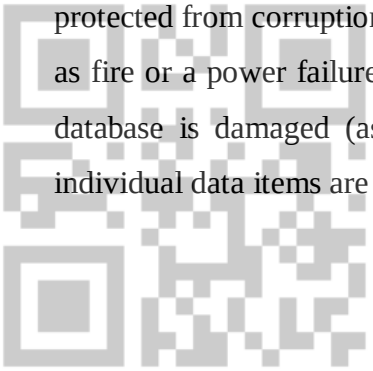
- Availability. Users can access the database in general and all the data for which they are authorized.

We briefly examine each of these requirements.

Integrity of the Database

If a database is to serve as a central repository of data, users must be able to trust the accuracy of the data values. This condition implies that the database administrator must be assured that updates are performed only by authorized individuals. It also implies that the data must be protected from corruption, either by an outside illegal program action or by an outside force such as fire or a power failure. Two situations can affect the integrity of a database: when the whole database is damaged (as happens, for example, if its storage medium is damaged) or when individual data items are unreadable.

رومانی صبحی رزق جرجس مطر



Integrity of the database as a whole is the responsibility of the DBMS, the operating system, and the (human) computing system manager. From the perspective of the operating system and the computing system manager, databases and DBMSs are files and programs, respectively. Therefore, one way of protecting the database as a whole is to regularly back up all files on the system. These periodic backups can be adequate controls against catastrophic failure.

Sometimes an administrator needs to be able to reconstruct the database at the point of a failure. For instance, when the power fails suddenly, a bank's clients may be in the middle of making transactions or students may be registering online for their classes. In these cases, owners want to be able to restore the systems to a stable point without forcing users to redo their recently completed transactions.

To handle these situations, the DBMS must maintain a log of transactions. For example, suppose the banking system is designed so that a message is generated in a log (electronic or paper or both) each time a transaction is processed. In the event of a system failure, the system can obtain accurate account balances by reverting to a backup copy of the database and reprocessing all later transactions from the log.

2025/2026 2025/2026 2025/2026

Element Integrity

The integrity of database elements is their correctness or accuracy. Ultimately, authorized users are responsible for entering correct data in databases. However, users and programs make mistakes collecting data, computing results, and entering values. Therefore, DBMSs sometimes take special action to help catch errors as they are made and to correct errors after they are inserted.

Databases achieve integrity of the database, its structure, and its individual elements.

This corrective action can be taken in three ways: by field checks, through access control, and with change log.

First, the DBMS can apply field checks, activities that test for appropriate values in a position. A field might be required to be numeric, an uppercase letter, or one of a set of acceptable characters. The check ensures that a value falls within specified bounds or is not greater than the

رومانی صبحی رزق جرجس مطر

sum of the values in two other fields. These checks prevent simple errors as the data are entered.

A second integrity action is afforded by access control. To see why, consider life without databases. Data files may contain data from several sources, and redundant data may be stored in several different places. For example, a student's mailing address may be stored in many different campus files: in the registrar's office for formal correspondence, in the food service office for dining hall privileges, at the bookstore for purchases, and in the financial aid office for accounting. Indeed, the student may not even be aware that each separate office has the address on file. If the student moves from one residence to another, each of the separate files requires correction.

Without a database, you can imagine the risks to the data's integrity. First, at a given time, some data files could show the old address (they have not yet been updated) and some simultaneously have the new address (they have already been updated). Second, there is always the possibility that someone misentered a data field, again leading to files with incorrect information. Third, the student may not even be aware of some files, so he or she does not know to notify the file owner about updating the address information. These problems are solved by databases. They enable collection and control of this data at one central source, ensuring the student and users of having the correct address.

2025/2026 2025/2026 2025/2026

However, the centralization is easier said than done. Who owns this shared central file?

Who is authorized to update which elements? What if two people apply conflicting modifications? What if modifications are applied out of sequence? How are duplicate records detected? What action is taken when duplicates are found? These are policy questions that must be resolved by the database administrator.

The third means of providing database integrity is maintaining a change log for the database. A change log lists every change made to the database; it contains both original and modified values. Using this log, a database administrator can undo any changes that were made in error. For example, a library fine might erroneously be posted against Charles W. Robertson, instead of Charles M. Robertson, flagging Charles W. Robertson as ineligible to participate in varsity athletics. Upon discovering this error, the database administrator obtains Charles W.'s original eligibility value from the log and corrects the database.

رومانی صبحی رزق جرجس مطر

Auditability

For some applications administrators may want to generate an audit record of all access (read or write) to a database. Such a record can help to maintain the database's integrity, or at least to discover after the fact who had affected what values and when. A second advantage, as we see later, is that users can access protected data incrementally; that is, no single access reveals protected data, but a set of sequential accesses viewed together reveals the data, much like discovering the clues in a detective novel. In this case, an audit trail can identify which clues a user has already been given, as a guide to whether to tell the user more.

As we note in Chapter 2, granularity can become an impediment in auditing. Audited events in operating systems are actions like open file or call procedure; they are seldom as specific as write record 3 or execute instruction I. To be useful for maintaining integrity, database audit trails should include accesses at the record, field, and even element levels. This detail is prohibitive for most database applications.

Furthermore, the database management system may access a record but not report the data to a user, as when the user performs a select operation. For example, a residence hall advisor might want a count of all students who have failed elementary French, and the database management system reports 462. To get that number the system had to inspect all student records and note those with failing grades, and it performed this lookup on behalf of the advisor who is appropriately listed in the log as receiving the data. Thus, in a sense, the advisor accessed all those student grades, although from the number 462 the advisor cannot determine the grade of any individual student. (Accessing a record or an element without transferring to the user the data received is called the pass-through problem.) Thus, a log of all records accessed directly may both overstate and understate what a user actually learns. The problem is even more nuanced than what we describe here, and we consider some intricacies of disclosure later in this chapter.

2025/2026 2025/2026 2025/2026

Access Control

Databases are often separated logically by user access privileges. For example, all users can be granted access to general data, but only the personnel department can obtain salary data and only the marketing department can obtain sales data. Databases are useful because they centralize the

رومانی صبحی رزق جرجس مطر

storage and maintenance of data. Limited access is both a responsibility and a benefit of this centralization.

The database administrator specifies who should be allowed access to which data, at the view, relation, field, record, or even element level. The DBMS must enforce this policy, granting access to all specified data or no access where prohibited. Furthermore, the number of modes of access can be many. A user or program may have the right to read, change, delete, or append to a value, add or delete entire fields or records, or reorganize the entire database.

Superficially, access control for a database seems like access control for an operating system or any other component of a computing system. However, the database problem is more complicated, as we see throughout this chapter. Operating system objects, such as files, are unrelated items, whereas records, fields, and elements are related. Although a user probably cannot determine the contents of one file by reading others, a user might be able to determine one data element just by reading others. The problem of obtaining data values from others is called ***inference***, and we consider it in depth later in this chapter.

2025/2026 2025/2026 2025/2026

It is important to notice that you can access data by inference without needing direct access to the secure object itself. Restricting inference may mean prohibiting certain paths to prevent possible inferences. However, restricting access to control inference also limits queries from users who do not intend unauthorized access to values. Moreover, attempts to check requested accesses for possible unacceptable inferences may actually degrade the DBMS's performance.

Finally, size or granularity is different between operating system objects and database objects. An operating system can readily control access to files, as we explain in Chapter 5. However an access control list of several hundred files is much easier to implement than an access control list for a database with several hundred files of perhaps a hundred fields each. Size affects the efficiency of processing. Operating systems do not usually "see into" a file to control access to items within a file.

Database management systems implement their own access control at a level finer than what an operating system handles.

رومانی صبحی رزق جرجس مطر

User Authentication

The DBMS can require rigorous user authentication. For example, a DBMS might insist that a user pass both specific password and time-of-day checks. This authentication supplements the authentication performed by the operating system. Typically, the DBMS runs as an application program on top of the operating system. This system design means that there is no trusted path from the DBMS to the operating system, so the DBMS must be suspicious of any data it receives, including a user identity from the operating system. Thus, the DBMS is forced to do its own authentication.

Availability

A DBMS has aspects of both a program and a system. It is a program that uses other hardware and software resources, yet to many users it is the only application run. Users often take the DBMS for granted, employing it as an essential tool with which to perform particular tasks. But when the system is not available—busy serving other users or down to be repaired or upgraded—the users are very aware of a DBMS's unavailability. For example, two users may request the same record, and the DBMS must arbitrate; one user is bound to be denied access for a while. Or the DBMS may withhold unprotected data to avoid revealing protected data, leaving the requesting user unhappy. We examine these problems in more detail later in this chapter. Problems like these result in high availability requirements for a DBMS.

2025/2026 2025/2026 2025/2026

Integrity/Confidentiality/Availability

The three aspects of computer security—integrity, confidentiality, and availability—clearly relate to database management systems. As we have described, integrity applies to the individual elements of a database as well as to the database as a whole. Integrity is also a property of the structure of the database (elements in one table correspond one to one with those of another) and of the relationships of the database (records having the same unique identifier, called a key, are related). Thus, integrity is a major concern in the design of database management systems. We look more closely at integrity issues in the next section.

رومانی صبحی رزق جرجس مطر

Confidentiality is likewise a key issue with databases because databases are often used to implement controlled sharing of sensitive data. Access to data can be direct (you request a record and the database provides it) or indirect (you request some records and from those results infer or intuit other data). Controlling direct access employs the access control techniques we describe in Chapters 2 and 5. Indirect access, however, is more difficult to control, and we explore it in more depth later in this chapter.

Finally, availability is important because of the shared access motivation underlying database development. However, availability conflicts with confidentiality. The last sections of the chapter address availability in an environment in which confidentiality is also important.

5.3 Reliability and Integrity

Databases amalgamate data from many sources, and users expect a DBMS to provide access to the data in a reliable way. When software engineers say that software has reliability, they mean that the software runs for very long periods of time without failing. Users certainly expect a DBMS to be reliable, since the data usually are key to business or organizational needs. Moreover, users entrust their data to a DBMS and rightly expect it to protect the data from loss or damage. Concerns for reliability and integrity are general security issues, but they are more apparent with databases.

A DBMS guards against loss or damage in several ways, which we study in this section. However, the controls we consider are not absolute: No control can prevent an authorized user from inadvertently entering an acceptable but incorrect value.

Database concerns about reliability and integrity can be viewed from three dimensions:

- Database integrity: concern that the database as a whole is protected against damage, as from the failure of a disk drive or the corruption of the master database index. These concerns are addressed by operating system integrity controls and recovery procedures.
- Element integrity: concern that the value of a specific data element is written or changed only by authorized users. Proper access controls protect a database from corruption by unauthorized users.

• Element accuracy: concern that only correct values are written into the elements of a database. Checks on the values of elements can help prevent insertion of improper values. Also, constraint conditions can detect incorrect values.

Protection Features from the Operating System

In Chapter 4 we discuss the protection an operating system provides for its users. A responsible system administrator backs up the files of a database periodically along with other user files. During normal execution, the operating system's standard access control facilities protect the files against outside access. Finally, the operating system performs certain integrity checks for all data as a part of normal read and write operations for I/O devices. These controls provide basic security for databases, but the database manager must enhance them.

Two-Phase Update

A serious problem for a database manager is the failure of the computing system in the middle of data modification. If the data item to be modified was a long field or a record consisting of several attributes, only some of the new data might have been written to permanent storage. Therefore, the database file would contain incorrect data that had not been updated. Even if errors of this type were spotted easily (which they are not), a more subtle problem occurs when several fields are updated and no single field appears to be in obvious error. The solution to this problem, proposed first by Lampson and Sturgis [LAM76] and adopted by most DBMSs, uses a two-phase update.

2025/2026 2025/2026 2025/2026

Update Technique

During the first phase, called the intent phase, the DBMS gathers the resources it needs to perform the update. It may gather data, create dummy records, open files, lock out other users, and calculate final answers; in short, it does everything to prepare for the update, but it makes no changes to the database. The first phase is repeatable an unlimited number of times because it takes no permanent action. If the system fails during execution of the first phase, no harm is done because all these steps can be restarted and repeated after the system resumes processing.

رومانی صبحی رزق جرجس مطر

The last event of the first phase, called ***committing***, involves the writing of a ***commit flag*** to the database. The commit flag means that the DBMS has passed the point of no return: After committing, the DBMS begins making permanent changes.

The second phase makes the permanent changes. During the second phase, no actions from before the commit can be repeated, but the update activities of phase two can also be repeated as often as needed. If the system fails during the second phase, the database may contain incomplete data, but the system can repair these data by performing all activities of the second phase. After the second phase has been completed, the database is again complete.

Two-Phase Update Example

Consider a database that contains the inventory of a company's office supplies. The company's central stockroom stores paper, pens, paper clips, and the like, and the different departments requisition items as they need them. The company buys in bulk to obtain the best prices. Each department has a budget for office supplies, so there is a charging mechanism by which the cost of supplies is recovered from the department. Also, the central stockroom monitors quantities of supplies on hand so as to order new supplies when the stock becomes low.

Suppose the process begins with a requisition from the accounting department for 50 boxes of paper clips. Assume that there are 107 boxes in stock and a new order is placed if the quantity in stock ever falls below 100. Here are the steps followed after the stockroom receives the requisition.

1. The stockroom checks the database to determine that 50 boxes of paper clips are on hand. If not, the requisition is rejected and the transaction is finished.

2. If enough paper clips are in stock, the stockroom deducts 50 from the inventory figure in the database (107 – 50 = 57).

3. The stockroom charges accounting's supplies budget (also in the database) for 50 boxes of paper clips.

4. The stockroom checks its remaining quantity on hand (57) to determine whether the remaining quantity is below the reorder point. Because it is, a notice to order more paper clips is generated, and the item is flagged as “on order” in the database.

5. A delivery order is prepared, enabling 50 boxes of paper clips to be sent to accounting.

All five of these steps must be completed in the order listed for the database to be accurate and for the transaction to be processed correctly.

Suppose a failure occurs while these steps are being processed. If the failure occurs before step 1 is complete, no harm ensues, because the entire transaction can be restarted. However, during steps 2, 3, and 4, changes are made to elements in the database. If a failure occurs then, the values in the database are inconsistent. Worse, the transaction cannot be reprocessed because a requisition would be deducted twice or a department would be charged twice or two delivery orders would be prepared.

When a two-phase commit is used, ***shadow values*** are maintained for key data points. A shadow data value is computed and stored locally during the intent phase, and it is copied to the actual database during the commit phase. The operations on the database would be performed as follows for a two-phase commit.

2025/2026 2025/2026 2025/2026

Intent:

1. Check the value of COMMIT-FLAG in the database. If it is set, this phase cannot be performed. Halt or loop, checking COMMIT-FLAG until it is not set.

2. Compare number of boxes of paper clips on hand to number requisitioned; if more are requisitioned than are on hand, halt.

3. Compute TCLIPS = ONHAND – REQUISITION.

4. Obtain BUDGET, the current supplies budget remaining for accounting department. Compute TBUDGET = BUDGET – COST, where COST is the cost of 50 boxes of clips.

رومانی صبحی رزق جرجس مطر

5. Check whether TCLIPS is below reorder point; if so, set TREORDER = TRUE; else set TREORDER = FALSE.

Commit:

1. Set COMMIT-FLAG in database.

2. Copy TCLIPS to CLIPS in database.

3. Copy TBUDGET to BUDGET in database.

4. Copy TREORDER to REORDER in database.

5. Prepare notice to deliver paper clips to accounting department. Indicate transaction completed in log.

6. Unset COMMIT-FLAG.

رومانی صبحی رزق جرجس مطر

With this example, each step of the intent phase depends only on unmodified values from the database and the previous results of the intent phase. Each variable beginning with T is a shadow variable used only in this transaction. The steps of the intent phase can be repeated an unlimited number of times without affecting the integrity of the database.

2025/2026 2025/2026 2025/2026

Once the DBMS begins the commit phase, it writes a COMMIT flag. When this flag is set, the DBMS will not perform any steps of the intent phase. Intent steps cannot be performed after committing because database values are modified in the commit phase. Notice, however, that the steps of the commit phase can be repeated an unlimited number of times, again with no negative effect on the correctness of the values in the database.

The one remaining flaw in this logic occurs if the system fails after writing the “transaction complete” message in the log but before clearing the commit flag in the database. It is a simple matter to work backward through the transaction log to find completed transactions for which the commit flag is still set and to clear those flags.

Redundancy/Internal Consistency

Many DBMSs maintain additional information to detect internal inconsistencies in data. The additional information ranges from a few check bits to duplicate or shadow fields, depending on the importance of the data.

Error Detection and Correction Codes

One form of redundancy is error detection and correction codes, such as parity bits, Hamming codes [HAM50], and cyclic redundancy checks. (We introduce such codes in Chapter 2.) These codes can be applied to single fields, records, or the entire database. Each time a data item is placed in the database, the appropriate check codes are computed and stored; each time a data item is retrieved, a similar check code is computed and compared to the stored value. If the values are unequal, they signify to the DBMS that an error has occurred in the database. Some of these codes point out the place of the error; others show precisely what the correct value should be. The more information provided, the more space required to store the codes.

2025/2026 2025/2026 2025/2026

Shadow Fields

Entire attributes or entire records can be duplicated in a database. If the data are irreproducible, this second copy can provide an immediate replacement if an error is detected. Obviously, redundant fields require substantial storage space.

Recovery

In addition to these error correction processes, a DBMS can maintain a log of user accesses, particularly changes. In the event of a failure, the database is reloaded from a backup copy and all later changes are then applied from the audit log.

Concurrency/Consistency

Database systems are often multiuser systems. Accesses by two users sharing the same database must be constrained so that neither interferes with the other. Simple locking is done by the

رومانی صبحی رزق جرجس مطر

DBMS. If two users attempt to read the same data item, there is no conflict because both obtain the same value.

If both users try to modify the same data items, we often assume that there is no conflict because each knows what to write; the value to be written does not depend on the previous value of the data item. However, this supposition is not quite accurate.

To see how concurrent modification can get us into trouble, suppose that the database consists of seat reservations for a particular airline flight. Agent A, booking a seat for passenger Mock, submits a query to find what seats are still available. The agent knows that Mock prefers a right aisle seat, and the agent finds that seats 5D, 11D, and 14D are open. At the same time, Agent B is trying to book seats for a family of three traveling together. In response to a query, the database indicates that 8A–B–C and 11D–E–F are the two remaining groups of three adjacent unassigned seats. Agent A submits the update command

SELECT (SEAT-NO = ‘11D’)

2025/2026 2025/2026 2025/2026

ASSIGN ‘MOCK,E’ TO PASSENGER-NAME

while agent B submits the update sequence

SELECT (SEAT-NO = ‘11D’)

ASSIGN ‘EDWARDS,S’ TO PASSENGER-NAME

as well as commands for seats 11E and 11F. Then two passengers have been booked into the same seat (which would be uncomfortable, to say the least).

Both agents have acted properly: Each sought a list of empty seats, chose one seat from the list, and updated the database to show to whom the seat was assigned. The difficulty in this situation is the time delay between reading a value from the database and writing a modification of that value. During the delay time, another user has accessed the same data.

To resolve this problem, a DBMS treats the entire query–update cycle as a single atomic operation. The command from the agent must now resemble “read the current value of seat

رومانی صبحی رزق جرجس مطر

PASSENGER-NAME for seat 11D; if it is ‘UNASSIGNED’, modify it to ‘MOCK,E’ (or ‘EDWARDS,S’).” The read–modify cycle must be completed as an uninterrupted item without allowing any other users access to the PASSENGER-NAME field for seat 11D.

The second agent’s request to book would not be considered until after the first agent’s had been completed; at that time, the value of PASSENGER-NAME would no longer be ‘UNASSIGNED.’

A final problem in concurrent access is read–write. Suppose one user is updating a value when a second user wishes to read it. If the read is done while the write is in progress, the reader may receive data that are only partly updated. Consequently, the DBMS locks any read requests until a write has been completed.

Database management systems serve multiple users at once by implementing concurrency and sequencing.

5.4 Database Disclosure

2025/2026 **2025/2026** **2025/2026**

As we describe in Chapter Privacy, more data are being collected about more people than ever before. In the past a single company, organization, or government office knew only about its clients or patrons; there was little sharing between organizations. And the number or kinds of places that collected data were small. Yes, we expect offices to keep records on us, but not to include every street, cash register, and web site we visit. Computers, of course, have made feasible not only the collection but also the sharing of these massive amounts of data.

Databases contain thoughts, preferences, opinions, activities (or their descriptions), fantasies, friends, and connections. From these databases people can draw inferences that may be accurate or false: Jamie is your friend. Jamie likes frogs. Ergo, you like frogs. Obviously, this is faulty logic, although it might also be true. In the next section we explore how people and computers analyze such databases for data connections that lead to unacceptable data disclosure.

رومانی صبحی رزق جرجس مطر

Sensitive Data

Some databases contain what is called sensitive data. As a working definition, let us say that sensitive data are data that should not be made public. Determining which data items and fields are sensitive depends both on the individual database and the underlying meaning of the data. Obviously, some databases, such as a public library catalog, contain no sensitive data; other databases, such as defense-related ones, are wholly sensitive. These two cases—nothing sensitive and everything sensitive—are the easiest to handle, because they can be covered by access controls to the database as a whole. Someone either is or is not an authorized user. These controls can be provided by the operating system.

The more difficult problem, which is also the more interesting one, is the case in which some but not all of the elements in the database are sensitive. There may be varying degrees of sensitivity. For example, a university database might contain student data consisting of name, financial aid, dorm, drug use, sex, parking fines, and race. An example of this database is shown in Table 7-6. Name and dorm are probably the least sensitive; financial aid, parking fines, and drug use the most; sex and race somewhere in between. That is, many people may have legitimate access to name, some to sex and race, and relatively few to financial aid, parking fines, or drug use. Indeed, knowledge of the existence of some fields, such as drug use, may itself be sensitive. Thus, security concerns not only the data elements but their context and meaning.

2025/2026 2025/2026 2025/2026

Name	Sex	Race	Aid	Fines	Drugs	Dorm
Adams	M	C	5000	45.	1	Holmes
Bailey	M	B	0	0.	0	Grey

TABLE 7-6 Example Database

Furthermore, we must account for different degrees of sensitivity. For instance, although all the fields are highly sensitive, the financial aid, parking fines, and drug-use fields may not have the same kinds of access restrictions. Our security requirements may demand that a few people be authorized to see each field, but no one be authorized to see all three. The challenge of the access control problem is to limit users' access so that they can obtain only the data to which they have

رومانی صبحی رزق جرجس مطر

legitimate access. Alternatively, the access control problem forces us to ensure that sensitive data are not released to unauthorized people.

Several factors can make data sensitive.

- Inherently sensitive. The value itself may be so revealing that it is sensitive. Examples are the locations of defensive missiles or the median income of barbers in a town with only one barber.
- From a sensitive source. The source of the data may indicate a need for confidentiality. An example is information from an informer whose identity would be compromised if the information were disclosed.
- Declared sensitive. The database administrator or the owner of the data may have declared the data to be sensitive. Examples are classified military data or the name of the anonymous donor of a piece of art.
- Part of a sensitive attribute or record. In a database, an entire attribute or record may be classified as sensitive. Examples are the salary attribute of a personnel database or a record describing a secret space mission.
- Sensitive in relation to previously disclosed information. Some data become sensitive in the presence of other data. For example, the longitude coordinate of a secret gold mine reveals little, but the longitude coordinate in conjunction with the latitude coordinate pinpoints the mine.

All of these factors must be considered when the sensitivity of the data is being determined.

Databases protect sensitive data by controlling direct or indirect access to the data.

Types of Disclosures

We all know that some data are sensitive. However, sometimes even characteristics of the data are sensitive. In this section, we see that even descriptive information about data (such as their existence or whether they have an element that is nonzero) is a form of disclosure.

رومانی صبحی رزق جرجس مطر

Exact Data

The most serious disclosure is the exact value of a sensitive data item itself. The user may know that sensitive data are being requested, or the user may request general data without knowing that some of it is sensitive. A faulty database manager may even deliver sensitive data by accident, without the user's having requested it. In all these cases, the result is the same: The security of the sensitive data has been breached.

Bounds

Another exposure is disclosing bounds on a sensitive value, that is, indicating that a sensitive value, y, is between two values, L and H. Sometimes, by using a narrowing technique not unlike the binary search, the user may first determine that $L \leq y \leq H$ and then see whether $L \leq y \leq H/2$, and so forth, thereby permitting the user to determine y to any desired precision. In another case, merely revealing that a value such as the athletic scholarship budget or the number of CIA agents exceeds a certain amount may be a serious breach of security.

Sometimes, however, bounds are a useful way to present sensitive data. It is common to release upper and lower bounds for data without identifying the specific records. For example, a company may announce that its salaries for programmers range from $50,000 to $82,000. If you are a programmer earning $79,700, you would suppose you are fairly well off, so you have the information you want; however, the announcement does not disclose who are the highest- and lowest-paid programmers.

2025/2026 2025/2026 2025/2026

Negative Result

Sometimes we can word a query to determine a negative result. That is, we can learn that z is not the value of y. For example, knowing that 0 is not the total number of felony convictions for a person reveals that the person was convicted of a felony. The distinction between 1 and 2 or 46 and 47 felonies is not as sensitive as the distinction between 0 and 1. Therefore, disclosing that a value is not 0 can be a significant disclosure. Similarly, if a student does not appear on the honors list, you can infer that the person's grade point average is below 3.50. This information is not too revealing, however, because the range of grade point averages from 0.0 to 3.49 is rather wide.

رومانی صبحی رزق جرجس مطر

Existence

In some cases, the ***existence*** of data is itself a sensitive piece of data, regardless of the actual value. For example, an employer may not want employees to know that their telephone use is being monitored. In this case, discovering a NUMBER OF PERSONAL TELEPHONE CALLS field in a personnel file would reveal sensitive data.

Probable Value

Finally, it may be possible to determine the probability that a certain element has a certain value. To see how, suppose you want to find out whether the president of the United States is registered in the Tory party. Knowing that the president is in the database, you submit two queries to the database:

> ***A database manager can control access by direct queries; disclosure can occur in more subtle ways that are harder to control.***

2025/2026 2025/2026 2025/2026

Count(Residence="1600 Pennsylvania Avenue") = 4

Count(Residence="1600 Pennsylvania Avenue" AND Tory=TRUE) = 1

From these queries you conclude there is a 25 percent likelihood that the president is a registered Tory.

Direct Inference

Inference is a way to infer or derive sensitive data from nonsensitive data. The inference problem is a subtle vulnerability in database security.

The database in Table 7-7 illustrates the inference problem; this database has the same form as the one introduced in Table 7-6, but we have added more data to make some points related to multiple data items. Recall that AID is the amount of financial aid a student is receiving. FINES is the amount of parking fines still owed. DRUGS is the result of a drug-use survey: 0 means never used and 3 means frequent user. Obviously this information should be kept confidential. We assume that AID, FINES, and DRUGS are sensitive fields, although only when the values

are related to a specific individual. In this section, we look at ways to determine sensitive data values from the database.

Name	Sex	Race	Aid	Fines	Drugs	Dorm
Adams	M	C	5000	45.	1	Holmes
Bailey	M	B	0	0.	0	Grey
Chin	F	A	3000	20.	0	West
Dewitt	M	B	1000	35.	3	Grey
Earhart	F	C	2000	95.	1	Holmes
Fein	F	C	1000	15.	0	West
Groff	M	C	4000	0.	3	West
Hill	F	B	5000	10.	2	Holmes
Koch	F	C	0	0.	1	West
Liu	F	A	0	10.	2	Grey
Majors	M	C	2000	0.	2	Grey

2025/2026 2025/2026 2025/2026

TABLE 7-7 Database to Illustrate Inferences

Direct Attack

In a direct attack, a user tries to determine values of sensitive fields by seeking them directly with queries that yield few records. The most successful technique is to form a query so specific that it matches exactly one data item.

In Table 7-7, a sensitive query might be

List NAME where

SEX=M ∧ DRUGS=1

This query discloses that for record ADAMS, DRUGS=1. However, it is an obvious attack because it selects people for whom DRUGS=1, and the DBMS might reject the query because it selects records for a specific value of the sensitive attribute DRUGS.

A less obvious query is

List NAME where

(SEX=M ∧ DRUGS=1) ∨
(SEX=M ∧ SEX=F) ∨
(DORM=AYRES)

On the surface, this query looks as if it should conceal drug usage by selecting other non-drug-related records as well. However, this query still retrieves only one record, revealing a name that corresponds to the sensitive DRUG value. The DBMS needs to know that SEX has only two possible values, so that the second clause will select no records. Even if that were possible, the DBMS would also need to know that no records exist with DORM=AYRES, even though AYRES might in fact be an acceptable value for DORM.

Inference by Arithmetic

Another procedure, used by the U.S. Census Bureau and other organizations that gather sensitive data, is to release only statistics. The organizations suppress individual names, addresses, or other characteristics by which a single individual can be recognized. Only neutral statistics, such as count, sum, and mean, are released. 2025/2026 2025/2026

The indirect attack seeks to infer a final result based on one or more intermediate statistical results. But this approach requires work outside the database itself. In particular, a statistical attack seeks to use some apparently anonymous statistical measure to infer individual data. In the following sections, we present several examples of indirect attacks on databases that report statistics.

Sum

An attack by sum tries to infer a value from a reported sum. For example, with the sample database in Table 7-7, it might seem safe to report student aid total by sex and dorm. Such a report is shown in Table 7-8. This seemingly innocent report reveals that no female living in Grey is receiving financial aid. Thus, we can infer that any female living in Grey (such as Liu) is certainly not receiving financial aid. This approach often allows us to determine a negative result.

	Holmes	Grey	West	Total
M	5000	3000	4000	12000
F	7000	0	4000	11000
Total	12000	3000	8000	23000

TABLE 7-8 Table Showing Negative Result

Count

The ***count*** can be combined with the sum to produce some even more revealing results. Often these two statistics are released for a database to allow users to determine average values. (Conversely, if count and mean are released, sum can be deduced.)

Table 7-9 shows the count of records for students by dorm and sex. This table is innocuous by itself. Combined with the sum table, however, this table demonstrates that the two males in Holmes and West are receiving financial aid in the amount of $5000 and $4000, respectively. We can obtain the names by selecting the subschema of NAME, DORM, which is not sensitive because it delivers only low-security data on the entire database.

2025/2026 2025/2026 2025/2026

Sex	Holmes	Grey	West	Total
M	1	3	1	5
F	2	1	3	6
Total	3	4	4	11

TABLE 7-9 Inference from Count and Sum Results

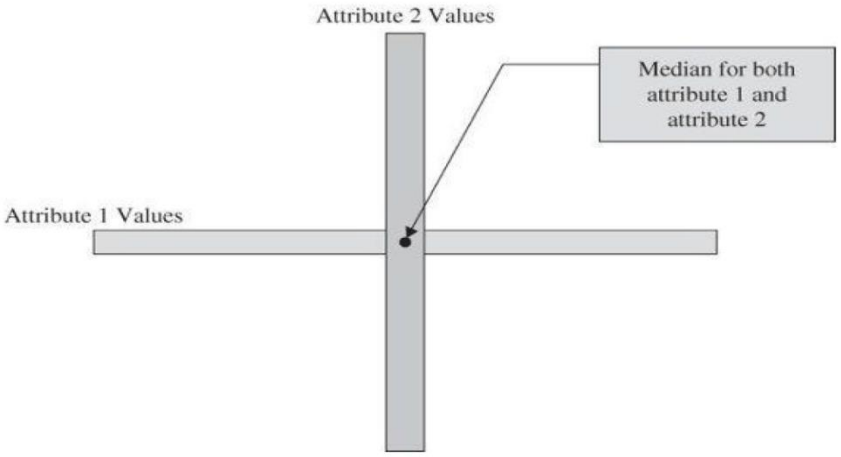
Mean

The arithmetic mean (average) allows exact disclosure if the attacker can manipulate the subject population. As a trivial example, consider salary. Given the number of employees, the mean

salary for a company and the mean salary of all employees except the president, it is easy to compute the president's salary.

Median

By a slightly more complicated process, we can determine an individual value from the median, the midpoint of an ordered list of values. The attack requires finding selections having one point of intersection that happens to be exactly in the middle, as shown in Figure 7-3.



2025/2026

FIGURE 7-3 Intersecting Medians

For example, in our sample database, there are five males and three persons whose drug use value is 2. Arranged in order of aid, these lists are shown in Table 7-10. Notice that Majors is the only name common to both lists, and conveniently that name is in the middle of each list. Someone working at the Health Clinic might be able to find out that Majors is a white male whose drug-use score is 2. That information identifies Majors as the intersection of these two lists and pinpoints Majors' financial aid as $2000. In this example, the queries

q = median(AID where SEX = M)

p = median(AID where DRUGS = 2)

reveal the exact financial aid amount for Majors.

رومانی صبحی رزق جرجس مطر

Name	Sex	Drugs	Aid
Bailey	M	0	0
Dewitt	M	3	1000
Majors	M	2	2000
Groff	M	3	4000
Adams	M	1	5000
Liu	F	2	0
Majors	M	2	2000
Hill	F	2	5000

TABLE 7-10 Drug Use and Aid Results

Tracker Attacks

2025/2026 2025/2026 2025/2026

As already explained, database management systems may conceal data when a small number of entries make up a large proportion of the data revealed. A ***tracker attack*** can fool the database manager into locating the desired data by using additional queries that produce small results. The tracker adds additional records to be retrieved for two different queries; the two sets of records cancel each other out, leaving only the statistic or data desired. The approach is to use intelligent padding of two queries. In other words, instead of trying to identify a unique value, we request n–1 other values (where there are n values in the database). Given n and n–1, we can easily compute the desired single element.

For instance, suppose we want to know how many female Caucasians live in Holmes Hall. A query posed might be

$$\text{count}((\text{SEX}=\text{F}) \wedge (\text{RACE}=\text{C}) \wedge (\text{DORM}=\text{Holmes}))$$

The database management system might consult the database, find that the answer is 1, and block the answer to that query because one record dominates the result of the query.

Aggregation

Related to the inference problem is ***aggregation***, which means building sensitive results from less sensitive inputs. We saw earlier that knowing either the latitude or longitude of a gold mine does you no good. But if you know both latitude and longitude, you can pinpoint the mine. For a more realistic example, consider how police use aggregation frequently in solving crimes: They determine who had a motive for committing the crime, when the crime was committed, who had alibis covering that time, who had the skills, and so forth. Typically, you think of police investigation as starting with the entire population and narrowing the analysis to a single person. But if the police officers work in parallel, one may have a list of possible suspects, another may have a list with possible motive, and another may have a list of capable persons. When the intersection of these lists is a single person, the police have their prime suspect.

> ***Aggregation is nearly impossible for a database management system to control because combining the data can occur outside the system, even by multiple colluding users.***

analysis 2025/2026 2025/2026

Analysis on Data

As we just described, the attacker has time and computing power to analyze data. Correlating seemingly unrelated bits of information can, as we showed, help build a larger picture. Even supposedly anonymized data can be revealing.

Hidden Data Attributes

A picture is just a picture and a document is just a document, right? Not quite, in the digital age. Objects such as pictures, music files, and documents are actually complex data structures having properties or attributes that add meaning to the data. These properties, called metadata, are not displayed with the picture or document, but they are not concealed; in fact, numerous applications support selecting, searching, sorting, and editing based on metadata.

File Tags

One use of attributes is tags for pictures. You might organize your photo collection with tags telling who or what landmarks are in each photo. Thus, you could search for all photos including

Zane Wellman or from your trip to Stockholm. However, this tagging can sometimes reveal more than intended. Suppose your photo with Zane was taken on a rather embarrassing night out and shows him rather unflatteringly. If the photo were posted without a narrative description, only people who knew Zane would see the image and know it was he. But when Zane applies for a job and the company does a web search to find out anything about him, the photo pops up because his name is in the metadata. And the picture can lead to questions; Zane may not even know you posted the photo, so he may be stunned to learn that a potential employer has seen him in that situation.

Geotagging

On 11 August 2010 the New York Times published a story about geotagging, the practice of many cameras and smartphones of tagging each photo they take with the GPS coordinates where the photo was taken. According to the story, Adam Savage, host of the program Myth Busters, took a photo of his car in front of his house and posted it to his Twitter account. However, because the photo contained location coordinates, Savage inadvertently disclosed the location of his house. With relatively little work, anyone can extract this metadata for offline analysis.

2025/2026 2025/2026 2025/2026

Suppression and concealing are two controls applied to data items. With suppression, sensitive data values are not forthcoming; the query is rejected without response. With concealing, the answer is close to but not exactly the actual value.

> ***Data suppression blocks release of sensitive data; data concealing releases part or an approximation of sensitive data.***

These two controls reflect the contrast between security and precision. With suppression, any results given are correct, yet many responses must be withheld to maintain security. With concealing, more results can be given, but their precision is lower. The choice between suppression and concealing depends on the context of the database.

Security Versus Precision

Our examples have illustrated how difficult it is to determine what data are sensitive and how to protect them. The situation is complicated by a desire to share nonsensitive data. For reasons of

رومانی صبحی رزق جرجس مطر

confidentiality we want to disclose only those data that are not sensitive. Such an outlook encourages a conservative philosophy in determining what data to disclose: less is better than more.

On the other hand, consider the users of the data. The conservative philosophy suggests rejecting any query that mentions a sensitive field. We may thereby reject many reasonable and nondisclosing queries. For example, a researcher may want a list of grades for all students using drugs, or a statistician may request lists of salaries for all men and for all women. These queries probably do not compromise the identity of any individual. We want to disclose as much data as possible so that users of the database have access to the data they need. This goal, called ***precision***, aims to protect all sensitive data while revealing as much nonsensitive data as possible.

We can depict the relationship between security and precision with concentric circles. As Figure 7-4 shows, the sensitive data in the central circle should be carefully concealed. The outside band represents data we willingly disclose in response to queries. But we know that the user may put together pieces of disclosed data and infer other, more deeply hidden, data. The figure shows us that beneath the outer layer may be yet more nonsensitive data that the user cannot infer.

2025/2026 2025/2026 2025/2026

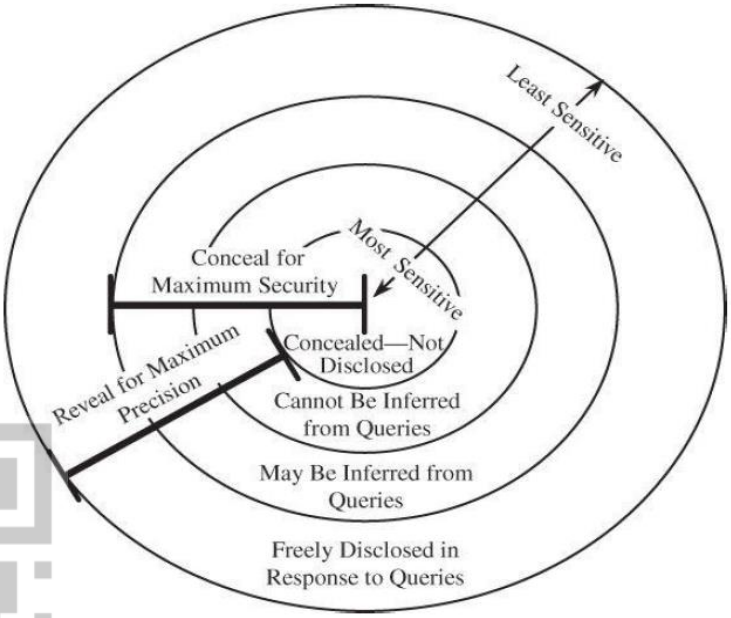


FIGURE 7-4 Security versus Precision

رومانی صبحی رزق جرجس مطر

The ideal combination of security and precision allows us to maintain perfect confidentiality with maximum precision; in other words, we disclose all and only the nonsensitive data. But achieving this goal is not as easy as it might seem, as we show in the next section. In the next section, we consider ways in which sensitive data can be obtained from queries that appear harmless.

Statistical Suppression

Because the attacks to obtain data used features of statistics, it may not be surprising that statistics also give some clues to countering those attacks.

Limited Response Suppression

Limited response suppression eliminates certain low-frequency elements from being displayed. It is not sufficient to delete them, however, if their values can also be inferred. To see why, consider Table 7-11, which shows counts of students by dorm and sex.

2025/2026 2025/2026

Sex	Holmes	Grey	West	Total
M	1	3	1	5
F	2	1	3	6
Total	3	4	4	11

TABLE 7-11 Count of Students by Dorm and Sex

The data in this table suggest that the cells with counts of 1 should be suppressed; their counts are too revealing. But it does no good to suppress the Male–Holmes cell when the value 1 can be determined by subtracting Female–Holmes (2) from the total (3) to determine 1, as shown in Table 7-12.

Sex	Holmes	Grey	West	Total
M	–	3	–	5
F	2	–	3	6
Total	3	4	4	11

TABLE 7-12 Using Subtraction to Derive Suppressed Cells

رومانی صبحی رزق جرجس مطر

When one cell is suppressed in a table with totals for rows and columns, it is necessary to suppress at least one additional cell on the row and one on the column to confuse a snooper. Using this logic, all cells (except totals) would have to be suppressed in this small sample table. When totals are not presented, single cells in a row or column can be suppressed.

Combined Results

Another control combines rows or columns to protect sensitive values. For example, Table 7-13 shows several sensitive results that identify single individuals. (Even though these counts may not seem sensitive, someone could use them to infer sensitive data such as NAME; therefore, we consider them to be sensitive.)

2025/2026

	Drug Use			
Sex	0	1	2	3
M	1	1	1	2
F	2	2	2	0

2025/2026

TABLE 7-13 Combining Values to Derive Sensitive Results

These counts, combined with other results such as sum, permit us to infer individual drug-use values for the three males, as well as to infer that no female was rated 3 for drug use. To suppress such sensitive information, one can combine the attribute values for 0 and 1, and also for 2 and 3, producing the less sensitive results shown in Table 7-14. In this instance, it is impossible to identify any single value.

	Drug Use	
Sex	0 or 1	2 or 3
M	2	3
F	4	2

TABLE 7-14 Combining Values to Suppress Sensitive Data

رومانی صبحی رزق جرجس مطر

Random Sample

With random sample control, a result is not derived from the whole database; instead the result is computed on a random sample of the database. The sample chosen is large enough to be valid. Because the sample is not the whole database, a query against this sample will not necessarily match the result for the whole database. Thus, a result of 5 percent for a particular query means that 5 percent of the records chosen for the sample for this query had the desired property. You would expect that approximately 5 percent of the entire database would have the property in question, but the actual percentage may be quite different.

So that averaging attacks from repeated, equivalent queries are prevented, the same sample set should be chosen for equivalent queries. In this way, all equivalent queries will produce the same result, although that result will be only an approximation for the entire database.

Concealment

Aggregation need not directly threaten privacy. An aggregate (such as sum, median, or count) often depends on so many data items that the sensitivity of any single contributing item is hidden. Government statistics show this well: Census data, labor statistics, and school results show trends and patterns for groups (such as a neighborhood or school district), but do not violate the privacy of any single person.

2025/2026 2025/2026 2025/2026

Blocking Small Sample Sizes

Organizations that publish personal statistical data, such as the U.S. Census Bureau, do not reveal results when a small number of people make up a large proportion of a category. The rule of “n items over k percent” means that data should be withheld if n items represent over k percent of the result reported. In the previous case, the one person selected represents 100 percent of the data reported, so there would be no ambiguity about which person matches the query.

رومانی صبحی رزق جرجس مطر

Random Data Perturbation

It is sometimes useful to perturb the values of the database by a small error. For each x_i that is the true value of data item i in the database, we can generate a small random error term εi and add it to xi for statistical results. The ε values are both positive and negative, so some reported values will be slightly higher than their true values and other reported values will be lower. Statistical measures such as sum and mean will be close but not necessarily exact. Data perturbation is easier to use than random sample selection because it is easier to store all the ε values in order to produce the same result for equivalent queries.

Data perturbation works for aggregation, as well. With perturbation you add a small positive or negative error term to each data value. Agrawal and Srikant [AGR00] show that given the distribution of data after perturbation and given the distribution of added errors, a researcher can determine the distribution (not the values) of the underlying data. The underlying distribution is often what researchers want. This result demonstrates that data perturbation can help protect privacy without sacrificing the accuracy of results.

2025/2026 2025/2026 2025/2026

Swapping

Correlation involves joining databases on common fields. Thus, the ID number in the AOL queries of Sidebar 7-4 lets researchers combine queries to derive one user's name and hometown. That act of joining or linking permits researchers to draw conclusions by inference.

To counter this kind of linking, some database administrators randomly perturb the data. The sex for Bailey and Chin might be interchanged, as might the race of Dewitt and Earhart. The count of individuals with these values would still be correct. Total aid for all males would be off by a bit. In this tiny example, these changes could affect the results significantly. However, if we had a larger database and performed just a few interchanges, most statistics would be close, probably close enough for most analytic purposes. Researchers could be warned that to protect confidentiality some exact data might be compromised. A researcher might conclude that Adams was receiving 5000 in aid, but could not be sure that conclusion was accurate because of the data swapping.

رومانی صبحی رزق جرجس مطر

Security

Security of data is the second major challenge of big data architectures. Reconsider the C–I–A triad from Chapter 1. Confidentiality is closely related to privacy, but there are other confidentiality concerns. Big data often involves big money: Data collectors pay to harvest data that they then sell. A data harvester reaps continuing profits by collecting data once and selling it many times to different buyers. Search companies such as Google collect data from users' search terms (such as "hotel San Francisco") which they might then sell to Hilton, Marriott, and Sheraton, as well as airlines, restaurants, tour companies, and so on. If Google sells data to Hilton and Hilton then resells it to Marriott and the others, Google loses out on the subsequent revenue stream. Thus, Google wants to control the confidentiality of its proprietary data. Integrity of data—that it is correct and intact— matters, as does ensuring availability of the data. Thus, all three elements of the triad matter for big data.

Here we list some of the security issues of big data.

2025/2026 2025/2026 2025/2026

Secure Data Storage

In the Hadoop model, data items are replicated and stored in any convenient location. If one data store becomes so full that performance suffers, controllers automatically split the data and move some elsewhere. The application developer generally does not know, much less care, where the data are physically stored.

Data-storage providers seek the lowest cost, and hosting data in the center of New York City, London, or Tokyo is likely to be far more costly than Alcoa, Tennessee, or Pateley Bridge, England. With adequate power and network infrastructure, any place is as good as any other.

Almost. Suppose data are stored in a politically sensitive region, even a war zone: A mortar attack or fallout from a missile strike is not desirable. Or suppose the data are housed in a country whose ruler decides to nationalize all foreign-held assets. Or consider a locale where hungry citizens storm an installation to steal anything they can later resell as scrap in order to buy food. In other chapters we have described access control in terms of magnetic cards and

fingerprint readers for polite persons, but access control on a global basis must address physical and political issues, as well.

Transaction Logs and Auditing

Activity logs are important for monitoring who did what. Review of audit logs can help an administrator regulate access permissions, and logs also help determine the extent of damage if an error or security breach occurs. Determining what to track is challenging, however: Too little tracking can limit the usefulness of the access logs, but too much data can overwhelm humans and technology, making it difficult to find the proverbial needle in a haystack of accesses. Data granularity affects volume of tracking data. It may be useless to know that user A accessed database D on Monday, when what would really help is to know that A modified records 2 and 17, or even that A changed the address field in 2, and the salary field of 17. The tracking data available depend on the granularity of access recorded for the data.

Tracking access is expensive, especially if accesses are numerous; detailed access auditing is uncommon for big data.

2025/2026 2025/2026 2025/2026

In big data applications the unit of access is often the file, so a log would record only that an application accessed file F, not the specific data within F.

Real-time Security Monitoring

As described in Chapter 6, intrusion detection, and protection systems allow administrators to monitor activity, perhaps detect anomalous behavior or attacks, and apply countermeasures while an incident is underway. Big data architectures involve nimble movement of data and computation, but the connecting network may be a large shared network, often the Internet. Real-time security monitoring is not intended for complex, shared, fluid network architectures.

Integrity

Finally, integrity deserves its own consideration separate from confidentiality and availability, because correctness, accuracy, and reliability are so important for data users. In this section we identify some integrity problems relevant for big data.

Data Accuracy

You may have experienced some piece of personal data being incorrectly entered into a database. We presented the example earlier in this chapter of 510 Thames Street instead of 519. The frustrating part is that, try as you will, you often can never trace the ultimate place where the number is incorrect. So 510s keep popping up for years. We used the earlier example as an aspect of element integrity.

Big datasets have problems with integrity, as we just described. However, another characteristic of big data use complicates the situation. Big data often uses many data streams collected from many sources, for example, photo recognition of auto license plates, human transcription from written public records, voice recognition from recordings, and input from handwritten forms, all of which are prone to error.

Rich, structured databases often have one or more identifying keys, such as telephone number, national insurance number, account number, date of birth, or some other solid data item on which to join two datasets. Big data collections tend not to have such strong keys, so they are joined on weaker attributes, such as name (which can be presented in several forms and misspelled in even more ways). The accuracy of results from such joins is lower.

> ***Need for data accuracy depends on the intended use; users need to consider their accuracy requirements when acting on results from big data operations.***

For many uses, high accuracy of big data is not important: Whether 90, 100, or 110 people of 500 in a neighborhood have pets is less important than that ownership of pets is in the range of 15 percent to 25 percent. However, users need to appreciate this limited degree of accuracy.

Source Provenance

Big data usually involves collecting and analyzing data from several sources. As we just pointed out, datasets will have differing degrees of quality depending in part on where the data come from. Big data applications must control for such variability, although big data models do not always give applications a way to learn the exact source of data, or even the nature of the source. Thus, application writers cannot readily account for data provenance in results they generate.

End-Point Filtering and Validation

Finally, after an application has processed data from a big data collection, the application might want to filter and validate the results. Current big data frameworks do not support that kind of data revision and manipulation.

Security Additions for Big Data Applications

As described in other chapters, adding security to an existing product or system is seldom a successful strategy. Nevertheless, sometimes that is the only available approach; not only is the specification and design complete, but one or more versions of the product are in use. Such is the case with Hadoop and other proprietary big data application frameworks: Product design and implementation were complete before security was considered seriously. Security engineers have now recommended changes and additions to Hadoop to support well-known security tools and techniques.

2025/2026 2025/2026 2025/2026

To address these errors, the team proposed a secure mode that involves the following extensions to Hadoop:

- authentication for end-user web devices
- mutual (user–process–Hadoop service) authentication with Kerberos
- access control to files in the Hadoop file system
- delegation tokens for continuous authentication between internal clients and services
- job tokens for distributing access authorization to multiple distributed platforms that collectively implement a data search across disparate data stores
- SSL encryption for network traffic

A driving force in this design was performance: The developers decided that security functions could not reduce performance by more than 3 percent.

Performance tends to outweigh accuracy or security for data-mining applications.

رومانی صبحی رزق جرجس مطر

Exercises

1. (a) In an environment in which several users share access to a single database, can one user ever block another's access for an unlimited period of time? (This situation is called indefinite postponement.)

   (b) Describe a scenario in which two users could cause the indefinite postponement of each other.

   (c) Describe a scenario in which a single user could cause the indefinite postponement of all users.

2. Using the two-step commit presented in the beginning of this chapter, describe how to avoid assigning one seat to two people, as in the airline example. That is, list precisely which steps the database manager should follow in assigning passengers to seats.

3. Suppose a database manager were to allow nesting of one transaction inside another. That is, after having updated part of one record, the DBMS would allow you to select another record, update it, and then perform further updates on the first record. What effect would nesting have on the integrity of a database? Suggest a mechanism by which nesting could be allowed.

2025/2026 2025/2026 2025/2026

4. Can a database contain two identical records without a negative effect on the integrity of the database? Why or why not?

5. Some operating systems perform buffered I/O. In this scheme, an output request is accepted from a user and the user is informed of the normal I/O completion. However, the actual physical write operation is performed later, at a time convenient to the operating system. Discuss the effect of buffered I/O on integrity in a DBMS.

رومانی صبحی رزق جرجس مطر

Chapter 6

Legal Issues and Ethics

رومانی صبحی رزق جرجس مطر

In this chapter we study human controls applicable to computer security: the legal system and ethics. The legal system has adapted quite well to computer technology by reusing some old forms of legal protection (copyrights and patents) and creating laws where no adequate ones existed (malicious access). Still, the courts are not a perfect form of protection for computer resources, for two reasons. First, the courts tend to be reactive instead of proactive. That is, we have to wait for a transgression to occur and then adjudicate it, rather than try to prevent it in the first place. Second, fixing a problem through the courts can be time consuming (sometimes taking years) and expensive; the latter characteristic prevents all but the wealthy from addressing most security issues.

2025/2026 2025/2026 2025/2026

On the other hand, ethics has not had to change, because ethics is more situational and personal than the law. For example, the privacy of personal information is becoming an important part of computer security. And although technically this issue is just an aspect of confidentiality, practically it has a long history in both law and ethics. This chapter rounds out our study of protection for computing systems by considering the context in which security is assessed and applied.

Not always are conflicts resolved pleasantly. Some people will think that they have been treated unfairly, and some people do indeed act unfairly. In some countries, a citizen reacts to a wrongful act by going to court. The courts are seen as the ultimate arbiters and enforcers of fairness. But, as most lawyers will tell you, the courts' definition of fair may not coincide with yours. Even if you could be sure the courts would side with you, a legal battle can be emotionally draining. Our purpose in this section is not only to understand how the legal system helps protect computer security but also to know how and when to use the legal system wisely.

Law and computer security are related in several ways. First, international, national, state, and city laws can affect privacy and secrecy. These statutes often apply to the rights of individuals to keep personal matters private. Second, laws regulate the use, development, and ownership of

data and programs. Patents, copyrights, and trade secrets are legal devices to protect the rights of developers and owners of programs and data. Similarly, one aspect of computer security is controlling access to programs and data; that access control is supported by these mechanisms of the law. Third, laws affect actions that can be taken to protect the secrecy, integrity, and availability of computer information and service. These basic concerns in computer security are both strengthened and constrained by applicable laws. Thus, legal means interact with other controls to establish computer security.

However, the law does not always provide an adequate control. When computer systems are concerned, the law is slowly evolving because the issues are similar to but not the same as those for property rights. Computers are new, compared to houses, land, horses, or money. As a consequence, the place of computer systems in law is not yet firmly established. As statutes are written and cases decided, the roles of computers and the people, data, and processes involved are becoming more defined in the law. However, laws do not yet address all improper acts committed with computers. Finally, some judges, lawyers, and police officers do not understand computing, so they cannot determine how computing relates to other, more established, parts of the law.

2025/2026 2025/2026 2025/2026

The laws dealing with computer security affect programmers, designers, users, and maintainers of computing systems and computerized data banks. These laws protect, but they also regulate the behavior of people who use computers. Furthermore, computer professionals are among the best-qualified advocates for changing old laws and creating new ones regarding computers. Before recommending change, however, professionals must understand the current state of computers and the law. Therefore, we have three motivations for studying the legal section of this chapter:

- to know what protection the law provides for computers and data
- to appreciate laws that protect the rights of others with respect to computers, programs, and data
- to understand existing laws as a basis for recommending new laws to protect computers, data, and people

The next few sections address the following aspects of protection of the security of computers.

- Protecting computing systems against criminals. Computer criminals violate the principles of confidentiality, integrity, and availability for computer systems. Preventing the

رومانی صبحی رزق جرجس مطر

violation is better than prosecuting it after the fact. However, if other controls fail, legal action may be necessary. In this section we study several representative laws to determine what acts are punishable under the law.

- Protecting code and data. Copyrights, patents, and trade secrets are all forms of legal protection that can be applied to programs and, sometimes, data. However, we must understand the fundamental differences between the kind of protection these three provide and the methods of obtaining that protection.
- Protecting programmers' and employers'rights. The law protects both programmers and people who employ programmers. Generally, programmers have only limited legal rights to access programs they have written while employed. This section contains a survey of the rights of employees and employers regarding programs written for pay.
- Protecting users of programs. When you buy a program, you expect it to work properly. If it doesn't, you want the legal system to protect your rights as a consumer. This section surveys the legal recourse you have to address faulty programs.

رومانی صبحی رزق جرجس مطر

Computer law is complex and emerging rather rapidly as it tries to keep up with the rapid technological advances in and enabled by computing. We present the fundamentals in this book not in full detail as you would expect by someone with a law degree, but as a situational analysis to heighten the awareness of those who are not lawyers but who must deal with the law's implications. To apply the material of this section to any specific case, you should consult a lawyer who understands and specializes in computer law. And, as most lawyers will advise, ensuring legal protection by doing things correctly from the beginning is far easier—and cheaper—than hiring a lawyer to sort out a web of conflict after things have gone wrong.

2025/2026 2025/2026 2025/2026

6.1 Protecting Programs and Data

Suppose Martha wrote a computer program to play a video game. She invited some friends over to play the game and gave them copies so that they could play at home. Steve took a copy and rewrote parts of Martha's program to improve the quality of the screen display. After Steve shared the changes with her, Martha incorporated them into her program. Now Martha's friends have convinced her that the program is good enough to sell, so she wants to advertise and offer

the game for sale by mail. She wants to know what legal protection she can apply to protect her software.

Copyrights, patents, and trade secrets are legal devices that can protect computers, programs, and data. However, in some cases, precise steps must be taken to protect the work before anyone else is allowed access to it. In this section, we explain how each of these forms of protection was originally designed to be used and how each is currently used in computing. We focus primarily on U.S. law, to provide examples of intent and consequence. Readers from other countries or doing business in other countries should consult lawyers in those countries to determine the specific differences and similarities.

Copyrights

In the United States, the basis of copyright protection is presented in the U.S. Constitution. The body of legislation supporting constitutional provisions contains laws that elaborate on or expand the constitutional protections. Relevant statutes include the U.S. copyright law of 1978, which was updated in 1998 as the Digital Millennium Copyright Act (DMCA) specifically to deal with computers and other electronic media such as digital video and music. The 1998 changes brought U.S. copyright law into general conformance with the World Intellectual Property Organization treaty of 1996, an international copyright standard to which 95 countries adhere.

Copyrights are designed to protect the expression of ideas. Thus, a copyright applies to a creative work, such as a story, photograph, song, or pencil sketch. The right to copy an expression of an idea is protected by a copyright. Ideas themselves, the law alleges, are free; anyone with a bright mind can think up anything anyone else can, at least in theory. The intention of a copyright is to allow regular and free exchange of ideas.

Copyright protects expression of a creative work and promotes exchange of ideas.

The author of a book translates ideas into words on paper. The paper embodies the expression of those ideas and is the author's livelihood. That is, an author hopes to earn a living by presenting ideas in such an appealing manner that others will pay to read them.

Copyright gives the author the exclusive right to make copies of the expression and sell them to the public. That is, only the author (or booksellers or others working as the author's agents) can sell new copies of the author's book. (We consider resales later in this chapter.)

Definition of Intellectual Property

The U.S. copyright law (§102) states that a copyright can be registered for "original works of authorship fixed in any tangible medium of expression, … from which they can be perceived, reproduced, or otherwise communicated, either directly or with the aid of a machine or device." Again, the copyright does not cover the idea being expressed. "In no case does copyright protection for an original work of authorship extend to any idea." The copyright must apply to an original work, and it must be in some tangible medium of expression.

Copyright protects artifacts—expressions of ideas—not the ideas themselves.

The copyrighted expression must also be in some tangible medium. A story or art work must be written, printed, painted, recorded (on a physical medium such as a plastic record), stored on a magnetic medium (such as a disk or tape), or fixed in some other way. Furthermore, the purpose of the copyright is to promote distribution of the work; therefore, the work must be distributed, even if a fee is charged for a copy.

Originality of Work

The work being copyrighted must be original to the author. As noted previously, some expressions in the public domain are not subject to copyright. A work can be copyrighted even if it contains some public domain material, as long as there is some originality, too. The author does not even have to identify what is public and what is original.

Fair Use of Material

The copyright law indicates that the copyrighted object is subject to fair use. A purchaser has the right to use the product in the manner for which it was intended and in a way that does not interfere with the author's rights. Specifically, the law allows "fair use of a copyrighted work, including such use by reproduction in copies … for purposes such as criticism, comment, news

reporting, teaching (including multiple copies for classroom use), scholarship or research." The purpose and effect of the use on the potential market for or the value of the work affect the decision of what constitutes fair use. For example, fair use allows making a backup copy of copyrighted software you acquired legally: Your backup copy protects your use against system failures but it doesn't affect the author because you have no need for nor do you want use of two copies at once. The copyright law usually upholds the author's right to a fair return for the work, while encouraging others to use the underlying ideas. Unfair use of a copyrighted item is called piracy.

Fair use allows copies for scholarship and research.

Requirements for Registering a Copyright

The copyright is easy to obtain, and mistakes in securing a copyright can be corrected. The first step of registration is notice. Any potential user must be made aware that the work is copyrighted. Each copy must be marked with the copyright symbol ©, the word Copyright, the year, and the author's name. (At one time, these items were followed by All rights reserved to preserve the copyright in certain South American countries. Adding the phrase now is unnecessary but harmless.)

The order of the elements can be changed, and either © or Copyright can be omitted (but not both). Each copy distributed must be so marked, although the law will forgive failure to mark copies if a reasonable attempt is made to recall and mark any ones distributed without a mark.

The copyright must also be officially filed. In the United States a form is completed and submitted to the Copyright Office, along with a nominal fee and a copy of the work. Actually, the Copyright Office requires only the first 25 and the last 25 pages of the work, to help it justify a claim in the event of a court case. The filing must be done within three months after the first distribution of the work. The law allows filing up to five years late, but no infringements before the time of filing can be prosecuted.

Copyright Infringement

The holder of the copyright must go to court to prove that someone has infringed on the copyright. The infringement must be substantial, and it must be copying, not independent work.

رومانی صبحی رزق جرجس مطر

In theory, two people might write identically the same song independently, neither knowing the other. These two people would both be entitled to copyright protection for their work. Neither would have infringed on the other, and both would have the right to distribute their work for a fee. Again, copyright is most easily understood for written works of fiction because it is extremely unlikely that two people would express an idea with the same or highly similar wording.

The independence of nonfiction works is not nearly so clear. Consider, for example, an arithmetic book. Long division can be explained in only so many ways, so two independent books could use similar wording for that explanation. The number of possible alternative examples is limited, so that two authors might independently choose to write the same simple example. However, it is far less likely that two textbook authors would have the same pattern of presentation and all the same examples from beginning to end.

Copyrights for Computer Software

The original copyright law envisioned protection for things such as books, songs, and photographs. People can rather easily detect when these items are copied. The separation between public domain and creativity is fairly clear. And the distinction between an idea (feeling, emotion) and its expression is pretty obvious. Works of nonfiction understandably have less leeway for independent expression. Because of programming language constraints and speed and size efficiency, computer programs have less leeway still.

2025/2026 2025/2026 2025/2026

Can a computer program be copyrighted? Yes. The 1976 copyright law was amended in 1980 to include an explicit definition of computer software. However, copyright protection may not be an especially desirable form of protection for computer works. To see why, consider the algorithm used in a given program. The algorithm is the idea, and the statements of the programming language are the expression of the idea. Therefore, protection is allowed for the program statements themselves, but not for the algorithmic concept: copying the code intact is prohibited, but reimplementing the algorithm is permitted. Remember that one purpose of copyright is to promote the dissemination of ideas The algorithm, which is the idea embodied in the computer program, is to be shared.

رومانی صبحی رزق جرجس مطر

A second problem with copyright protection for computer works is the requirement that the work be published. A program may be published by distribution of copies of its object code, for example, on a disk. However, if the source code is not distributed, it has not been published. An alleged infringer cannot have violated a copyright on source code if the source code was never published.

Copyrights for Digital Objects

The ***Digital Millennium Copyright Act (DMCA)*** of 1998 clarified some issues of digital objects (such as music files, graphics images, data in a database, and also computer programs), but it left others unclear.

Among the provisions of the DMCA are these:

- Digital objects can be subject to copyright.
- It is a crime to circumvent or disable antipiracy functionality built into an object.
- It is a crime to manufacture, sell, or distribute devices that disable antipiracy functionality or that copy digital objects.
- However, these devices can be used (and manufactured, sold, or distributed) for research and educational purposes.
- It is acceptable to make a backup copy of a digital object as a protection against hardware or software failure or to store copies in an archive.
- Libraries can make up to three copies of a digital object for lending to other libraries.

2025/2026 2025/2026 2025/2026

So, a user can make reasonable copies of an object in the normal course of its use and as a protection against system failures. If a system is regularly backed up and so a digital object (such as a software program) is copied onto many backups, that is not a violation of copyright.

Patents

Patents are unlike copyrights in that they protect inventions, tangible objects, or ways to make them, not works of the mind. The distinction between patents and copyrights is that patents were intended to apply to the results of science, technology, and engineering, whereas copyrights were meant to cover works in the arts, literature, and written scholarship. A patent can protect a "new and useful process, machine, manufacture, or composition of matter." The U.S. law excludes "newly discovered laws of nature … [and] mental processes." Thus "2+2=4" is not a proper

subject for a patent because it is a law of nature. Similarly, that expression is in the public domain and would thus be unsuitable for a copyright. Finally, you can argue that mathematics is purely mental, just ideas. Nobody has ever seen or touched a two, two horses, yes, but not just a two. A patent is designed to protect the device or process for carrying out an idea, not the idea itself.

Patents protect inventions, tangible objects, not their design or idea.

Requirement of Novelty

If two composers happen to compose the same song independently at different times, copyright law would allow both of them to have copyright. If two inventors devise the same invention, the patent goes to the person who invented it first, regardless of who first filed the patent. A patent can be valid only for something that is truly novel or unique, so there can be only one patent for a given invention.

An object patented must also be nonobvious. If an invention would be obvious to a person ordinarily skilled in the field, it cannot be patented. The law states that a patent cannot be obtained "if the differences between the subject matter sought to be patented and the prior art are such that the subject matter as a whole would have been obvious at the time the invention was made to a person having ordinary skill in the art to which said subject matter pertains." For example, a piece of cardboard to be used as a bookmark would not be a likely candidate for a patent because the idea of a piece of cardboard would be obvious to almost any reader.

2025/2026 2025/2026 2025/2026

Procedure for Registering a Patent

An applicant registers a copyright by filing a brief form, marking a copyright notice on the creative work, and distributing the work. The whole process takes less than an hour.

To obtain a patent, an inventor must convince the U.S. Patent and Trademark Office that the invention deserves a patent. For a fee, a patent attorney will research the patents already issued for similar inventions. This search accomplishes two things. First, it determines that the invention to be patented has not already been patented (and, presumably, has not been previously invented). Second, the search can help identify similar things that have been patented. These similarities can be useful when describing the unique features of the invention that make it worthy of patent protection. The Patent Office compares an application to those of all other

similar patented inventions and decides whether the application covers something truly novel and nonobvious. If the office decides the invention is novel, a patent is granted.

Typically, an inventor writes a patent application listing many claims of originality, from very general to very specific. The Patent Office may disallow some of the more general claims while upholding some of the more specific ones. The patent is valid for all the upheld claims. The patent applicant reveals what is novel about the invention in sufficient detail to allow the Patent Office and the courts to judge novelty; that degree of detail may also tell the world how the invention works, thereby opening the possibility of infringement.

The patent owner uses the patented invention by producing products or by licensing others to produce them. Patented objects are sometimes marked with a patent number to warn others that the technology is patented. The patent holder hopes this warning will prevent others from infringing.

Patent Infringement

2025/2026 2025/2026 2025/2026

A patent holder must oppose all infringement. With a copyright, the holder can choose which cases to prosecute, ignoring small infringements and waiting for serious infractions where the infringement is great enough to ensure success in court or to justify the cost of the court case. However, failing to sue a patent infringement—even a small one or one the patent holder does not know about—can mean losing the patent rights entirely. But, unlike copyright infringement, a patent holder does not have to prove that the infringer copied the invention; a patent infringement occurs even if someone independently invents the same thing, without knowledge of the patented invention.

Patent holders must act against all infringers.

Applicability of Patents to Computer Objects

The Patent Office has not encouraged patents of computer software. For a long time, computer programs were seen as the representation of an algorithm, and an algorithm was a fact of nature, which is not subject to patent. An early software patent case, Gottschalk v. Benson, involved a request to patent a process for converting decimal numbers into binary. The Supreme Court

رومانی صبحی رزق جرجس مطر

rejected the claim, saying it seemed to attempt to patent an abstract idea, in short, an algorithm. But the underlying algorithm is precisely what most software developers would like to protect.

Software can be patented, and the courts increasingly recognize the patentability of a novel technique, that is, an algorithm.

Trade Secrets

A trade secret is unlike a patent or copyright in that it must be kept a secret. The information has value only as a secret, and an infringer is one who divulges the secret. Once divulged, the information usually cannot be made secret again.

A trade secret is a secret valuable to a business owner.

Characteristics of Trade Secrets

A ***trade secret*** is information that gives one company a competitive edge over others. For example, the formula for a soft drink is a trade secret, as is a mailing list of customers or information about a product due to be announced in a few months.

2025/2026 2025/2026 2025/2026

The distinguishing characteristic of a trade secret is that it must always be kept secret. Employees and outsiders who have access to the secret must be required not to divulge the secret. The owner must take precautions to protect the secret, such as storing it in a safe, encrypting it in a computer file, or requiring employees to sign a statement that they will not disclose the secret.

If someone obtains a trade secret improperly and profits from it, the owner can recover profits, damages, lost revenues, and legal costs. The court will do whatever it can to return the holder to the same competitive position it had while the information was secret and may award damages to compensate for lost sales. However, trade secret protection evaporates in the case of independent discovery. If someone else happens to discover the secret independently, there is no infringement and trade secret rights are gone.

Reverse Engineering

Another way trade secret protection can vanish is by reverse engineering. Suppose a secret is the way to pack tissues in a cardboard box to make one pop up as another is pulled out. Anyone can

رومانی صبحی رزق جرجس مطر

cut open the box and study the process. Therefore, the trade secret is easily discovered. In ***reverse engineering***, one studies a finished object to determine how it is manufactured or how it works.

Through reverse engineering someone might discover how a telephone is built; the design of the telephone is obvious from the components and how they are connected. Therefore, a patent is the appropriate way to protect an invention such as a telephone. However, something like a soft drink is not just the combination of its ingredients. Making a soft drink may involve time, temperature, presence of oxygen or other gases, and similar factors that could not be learned from a straight chemical decomposition of the product. The recipe of a soft drink is a closely guarded trade secret. Trade secret protection works best when the secret is not apparent in the product.

Applicability to Computer Objects

Trade secret protection applies very well to computer software. The underlying algorithm of a computer program is novel, but its novelty depends on nobody else's knowing it. Trade secret protection allows distribution of the result of a secret (the executable program) while still keeping the program design hidden. Trade secret protection does not cover copying a product (specifically a computer program), so it cannot protect against a pirate who sells copies of someone else's program without permission. However, trade secret protection makes it illegal to steal a secret algorithm and use it in another product.

2025/2026 2025/2026 2025/2026

The difficulty with computer programs is that reverse engineering works. Decompiler and disassembler programs can produce a source version of an executable program. Of course, this source does not contain the descriptive variable names or the comments to explain the code, but it is an accurate version that someone else can study, reuse, or extend.

Difficulty of Enforcement

Trade secret protection is of no help when someone infers a program's design by studying its output or, worse yet, decoding the object code. Both of these are legitimate (that is, legal) activities, and both cause trade secret protection to disappear.

رومانی صبحی رزق جرجس مطر

The confidentiality of a trade secret must be ensured with adequate safeguards. If source code is distributed loosely or if the owner fails to impress on people (such as employees) the importance of keeping the secret, any prosecution of infringement will be weakened. Employment contracts typically include a clause stating that the employee will not divulge any trade secrets received from the company, even after leaving a job. Additional protection, such as marking copies of sensitive documents or controlling access to computer files of secret information, may be necessary to impress people with the importance of secrecy.

In Table 11-1 we compare these three kinds of protection.

	Copyright	Patent	Trade Secret
Protects	Expression of idea, not idea itself	Invention—the way something works	A secret, competitive advantage
Protected object made public	Yes; intention is to promote publication	Design filed at Patent Office	No
Requirement to distribute	Yes	No	No
Ease of filing	Very easy, do-it-yourself	Very complicated; specialist lawyer suggested	No filing
Duration	Varies by country; approximately 75–100 years is typical	19 years	Indefinite
Legal protection	Sue if unauthorized copy sold	Sue if invention copied	Sue if secret improperly obtained

TABLE 11-1 Comparing Copyrights, Patents, and Trade Secrets

Rights of Employees and Employers

Employers hire employees to generate ideas and make products. The protection offered by copyrights, patents, and trade secrets appeals to employers because it applies to ideas and products. However, the issue of who owns the ideas and products is complex. Ownership is a computer security concern because it relates to the rights of an employer to protect the secrecy and integrity of works produced by the employees. In this section we study the respective rights of employers and employees to their computer products.

رومانی صبحی رزق جرجس مطر

Ownership of Products

Suppose Edye works for a computer software company. As part of her job, she develops a program to manage windows for a computer screen display. The program belongs to her company because it paid Edye to write the program: she wrote it as a part of a work assignment. Thus, Edye cannot market this program herself. She could not sell it even if she worked for a non-software-related company but developed the software as part of her job. Most employees understand this aspect of their responsibilities to their employer.

An employment contract clarifies for both parties an employee's rights to computer products.

Ownership of a Patent

The person who owns a work under patent or copyright law is the inventor; in the examples described earlier, the owner is the programmer or the employer. Under patent law, it is important to know who files the patent application. If an employee lets an employer patent an invention, the employer is deemed to own the patent and therefore the rights to the invention.

The employer also has the right to patent if the employee's job functions included inventing the product. For instance, in a large company a scientist may be hired to do research and development, and the results of this inventive work become the property of the employer. Even if an employee patents something, the employer can argue for a right to use the invention if the employer contributed some resources (such as computer time or access to a library or database) in developing the invention.

2025/2026 2025/2026 2025/2026

Ownership of a Copyright

Owning a copyright is similar to owning a patent. The author (programmer) is the presumed owner of the work, and the owner has all rights to an object. However, a special situation known as work for hire applies to many copyrights for developing software or other products.

Work for Hire

In a ***work for hire*** situation, the employer, not the employee, is considered the author of a work. Work for hire is not easy to identify and depends in part on the laws of the state in which the employment occurs. The relationship between an employee and employer is considered a work

for hire if some or all of the following conditions are true. (The more of these conditions that are true, the more a situation resembles work for hire.)

- The employer has a supervisory relationship, overseeing the manner in which the creative work is done.
- The employer has the right to fire the employee.
- The employer arranges for the work to be done before the work was created (as opposed to the sale of an existing work).
- A written contract between the employer and employee states that the employer has hired the employee to do certain work.

In the situation in which Edye develops a program on her job, her employer will certainly claim a work for hire relationship. Then, the employer owns all copyright rights and should be identified in place of the author on the copyright notice.

In a work for hire, the employer is the creator and owner of an employee's work product.

Licenses

An alternative to a work for hire arrangement is licensed software. In this situation, the programmer develops and retains full ownership of the software. In return for a fee, the programmer grants to a company a license to use the program. The license can be granted for a definite or unlimited period of time, for one copy or for an unlimited number, to use at one location or many, to use on one machine or all, at specified or unlimited times. This arrangement is highly advantageous to the programmer, just as a work for hire arrangement is highly advantageous to the employer. The choice between work for hire and license is largely what the two parties will agree to.

2025/2026 2025/2026 2025/2026

Trade Secret Protection

A trade secret is different from either a patent or a copyright in that there is no registered inventor or author; there is no registration office for trade secrets. In the event a trade secret is revealed, the owner can prosecute the revealer for damages suffered. But first, ownership must be established because only the owner can be harmed.

A company owns the trade secrets of its business-confidential data. As soon as a secret is developed, the company becomes the owner. For example, as soon as sales figures are

رومانی صبحی رزق جرجس مطر

accumulated, a company has trade secret right to them, even if the figures are not yet compiled, totaled, summarized, printed, or distributed. As with copyrights, an employer may argue about having contributed to the development of trade secrets. If your trade secret is an improved sorting algorithm and part of your job involves investigating and testing sorting algorithms, your employer will probably claim at least partial ownership of the algorithm you try to market.

Employment Contracts

An employment contract often spells out rights of ownership. But sometimes the software developer and possible employer have no contract. Having a contract is desirable both for employees and employers so that both will understand their rights and responsibilities.

Typically, an employment contract specifies that the employee be hired to work as a programmer exclusively for the benefit of the company. The company states that this is a work for hire situation. The company claims all rights to any programs developed, including all copyright rights and the right to market. The contract may further state that the employee is receiving access to certain trade secrets as a part of employment, and the employee agrees not to reveal those secrets to anyone.

2025/2026 2025/2026 2025/2026

6.2 Computer Crime

The law related to contracts and employment is difficult, but at least employees, objects, contracts, and owners are fairly standard entities for which legal precedents have been developed over centuries. The definitions in copyright and patent law are strained when applied to digital objects because old forms must be made to fit new objects; for these situations, however, cases being decided now are establishing legal precedents. But crimes involving computers are an area of the law that is even less clear than the other areas. In this section we study computer crime and consider why new laws are needed to address some of its problems.

Why a Separate Category for Computer Crime Is Needed

Crimes can be organized into certain recognized categories, including murder, robbery, and littering. We do not separate crime into categories for different weapons, such as gun crime or knife crime, but we separate crime victims into categories, depending on whether they are people or other objects. Nevertheless, driving into your neighbor’s picture window can be as bad as

driving into his evergreen tree or pet sheep. Let us look at an example to see why computer crime categories are not sufficient and why we need special laws relating to computers as subjects and objects of crime.

Rules of Property

Donn Parker and Susan Nycom [PAR84] describe the theft of a trade secret proprietary software package. The theft occurred across state boundaries by means of a telephone line; this interstate aspect is important because it means that the crime is subject to federal law as well as state law. The California Supreme Court ruled that this software acquisition was not theft because

> Implicit in the definition of "article" in Section 499c(a) is that it must be something tangible … Based on the record here, the defendant did not carry any tangible thing … from the computer to his terminal unless the impulses which defendant allegedly caused to be transmitted over the telephone wire could be said to be tangible. It is the opinion of the Court that such impulses are not tangible and hence do not constitute an "article."

The legal system has explicit rules about what constitutes property. Generally, property is tangible, unlike magnetic impulses. For example, unauthorized use of a neighbor's lawn mower constitutes theft, even if the lawn mower was returned in essentially the same condition as it was when taken. To a computer professional, taking a copy of a software package without permission is clear-cut theft. Fortunately, laws evolve to fit the times, and this interpretation from the 1980s has been refined so that bits are now recognized items of property, even if you cannot hold a bit in your hand.

2025/2026 2025/2026 2025/2026

Rules of Evidence

Computer printouts have been used as evidence in many successful prosecutions. Frequently used are computer records generated in the ordinary course of operation, such as system audit logs.

Under the rules of evidence, courts prefer the best version of a piece of evidence (called the best evidence rule). An original document is preferable to a copy, but the original may be unavailable. As long as the original is unavailable for some reason other than the fault of the party presenting the evidence, a copy is acceptable. A copy is strengthened if someone testifies, for example, to having heard the parties agreeing to the terms of the contract the day before, or someone recognizes and attests to the signatures on the copy. Business records are perfectly acceptable as

رومانی صبحی رزق جرجس مطر

evidence. A computer printout showing activity around a time period of interest is solid evidence, especially if a system administrator or other person in charge can testify that the system generates this log data continually, and this printout is an accurate depiction of the contents of the log file.

All pieces of evidence contribute to a judge's or jury's weighing the evidence. Credible evidence carries more weight in reaching a conclusion.

رومانی صبحی رزق جرجس مطر

Threats to Integrity and Confidentiality

The integrity and secrecy of data are also issues in many court cases. Parker and Nycom [PAR84] describe a case in which a trespasser gained remote access to a computing system. The computing system contained confidential records about people, and the integrity of the data was important. The prosecution of this case had to be phrased in terms of theft of computer time and valued as such, even though that was insignificant compared with loss of privacy and integrity. Why? Because the law as written recognized theft of computer time as a loss, but not loss of privacy or destruction of data.

Value of Data

In another computer crime, a person was found guilty of having stolen a substantial amount of data from a computer data bank. However, the court determined that the "value" of that data was the cost of the paper on which it was printed, which was only a few dollars. Because of that valuation, this crime was classified as a misdemeanor and considered to be a minor crime.

Fortunately, the courts have since determined that information and other intangibles can have significant value. Digital data, like many other things of value, is worth what a willing buyer would pay for it.

Why Computer Crime Is Hard to Define

From these examples, it is clear that the legal community has slowly accommodated advances in computers. Some people in the legal process do not understand computers and computing, so crimes involving computers are not always treated properly. Creating and changing laws are slow processes, intended to involve substantial thought about the effects of proposed changes.

This deliberate process is very much out of pace with a technology that is progressing as fast as computing.

Adding to the problem of a rapidly changing technology is that a computer can perform many roles in a crime. A particular computer can be the subject, object, or medium of a crime. A computer can be attacked (attempted unauthorized access), used to attack (impersonating a legitimate node on a network), and used as a means to commit crime (Trojan horse or fake login). By some laws, hitting a person on the head with a computer is a computer crime. Computer crime statutes must address all of these evils.

Why Computer Crime Is Hard to Prosecute

Even when everyone acknowledges that a computer crime has been committed, computer crime is hard to prosecute for the following reasons.

- Lack of understanding. Courts, lawyers, police agents, or jurors do not necessarily understand computers. Many judges began practicing law before the invention of computers, and most began before the widespread use of the personal computer. Fortunately, computer literacy in the courts is improving as judges, lawyers, and police officers use computers in their daily activities.
- Lack of physical evidence. Police and courts have for years depended on tangible evidence, such as fingerprints. As readers of Sherlock Holmes know, seemingly minuscule clues can lead to solutions to the most complicated crimes (or so Doyle would have you believe). But with many computer crimes there simply are no fingerprints and no physical clues of any sort.
- Lack of political impact. Solving and obtaining a conviction for a murder or robbery is popular with the public, and so it gets high priority with prosecutors and police chiefs. Solving and obtaining a conviction for an obscure high-tech crime, especially one not involving obvious and significant loss, may get less attention. However, as computing becomes more pervasive, the visibility and impact of computer crime will increase.
- Complexity of case. Basic crimes that everyone understands, such as murder, kidnapping, or auto theft, can be easy to prosecute. A complex money laundering or tax fraud case may be more difficult to present to a jury because jurors have a hard time following a circuitous

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

accounting trail. But the hardest crime to present may be a high-tech crime, described, for example, as root

access by a buffer overflow in which memory was overwritten by other instructions, which allowed the attacker to copy and execute code at will and then delete the code, eliminating all traces of entry (after disabling the audit logging, of course).

- Age of defendant. Many computer crimes are committed by juveniles. Society understands immaturity and disregards even very serious crimes by juveniles because the juveniles did not understand the impact of their actions. A more serious, related problem is that many adults see juvenile computer crimes as childhood pranks, the modern equivalent of tipping over an outhouse.

Even when there is clear evidence of a crime, the victim may not want to prosecute because of possible negative publicity. Banks, insurance companies, investment firms, the government, and healthcare groups think their trust by the public will be diminished if a computer vulnerability is exposed. Also, they may fear repetition of the same crime by others: so-called copycat crimes. For all of these reasons, computer crimes are often not prosecuted.

2025/2026 2025/2026 2025/2026

Computer crime is often complex, so explaining it to a jury is difficult and uncertain. Faced with free choice, prosecutors may prefer a simpler murder or robbery.

Ethical Issues in Computer Security

This final section helps clarify thinking about the ethical issues involved in computer security. We offer no answers. Rather, after listing and explaining some ethical principles, we present several problem studies to which the principles can be applied. Each story is followed by a list of possible ethical issues involved, although the list is not necessarily all-inclusive or conclusive. The primary purpose of this section is to explore some of the ethical issues associated with computer security and to show how ethics functions as a control.

Differences Between the Law and Ethics

As we noted earlier, law is not always the appropriate way to deal with issues of human behavior. It is difficult to define a law to preclude only the events we want it to. For example, a law that restricts animals from public places must be refined to permit guide dogs for the blind. Lawmakers, who are not computer professionals, are hard pressed to think of all the exceptions

when they draft a law concerning computer affairs. Even when a law is well conceived and well written, its enforcement may be difficult. The courts are overburdened, and prosecuting relatively minor infractions may be excessively time consuming relative to the benefit.

Thus, it is impossible or impractical to develop laws to describe and enforce all forms of behavior acceptable to society. Instead, society relies on ***ethics or morals*** to prescribe generally accepted standards of proper behavior. (In this section the terms ethics and morals are used interchangeably.) An ***ethic*** is an objectively defined standard of right and wrong. Ethical standards are often idealistic principles because they focus on one objective. In a given situation, however, several moral objectives may be involved, so people have to determine an action that is appropriate considering all the objectives. Even though religious groups and professional organizations promote certain standards of ethical behavior, ultimately each person is responsible for deciding what to do in a specific situation. Therefore, through our choices, each of us defines a personal set of ethical practices. A set of ethical principles is called an ***ethical system***.

Law	Ethics
Described by formal, written documents	Described by unwritten principles
Interpreted by courts	Interpreted by each individual
Established by legislatures representing all people	Presented by philosophers, religions, professional groups
Applied to everyone	Chosen personally
Priority determined by courts if two laws conflict	Priority determined by an individual if two principles conflict
"Right" arbitrated finally by court	Not arbitrated externally
Enforced by police and courts	Enforced by intangibles such as principles and beliefs

TABLE 11-3 Comparison of Law and Ethics

رومانی صبحی رزق جرجس مطر

Ethics and Religion

Ethics is a set of principles or norms for justifying what is right or wrong in a given situation. To understand what ethics is we may start by trying to understand what it is not. Ethical principles are different from religious beliefs. Religion is based on personal notions about the creation of the world and the existence of controlling forces or beings. Many moral principles are embodied in the major religions, and the basis of a personal morality is a matter of belief and conviction, much the same as for religions. However, two people with different religious backgrounds may develop the same ethical philosophy, while two exponents of the same religion might reach opposite ethical conclusions in a particular situation. Finally, we can analyze a situation from an ethical perspective and reach ethical conclusions without appealing to any particular religion or religious framework. Thus, it is important to distinguish ethics from religion.

Ethical Principles Are Not Universal

Ethical values vary by society, and from person to person within a society. For example, the concept of privacy is important in Western cultures. But in Eastern cultures, privacy is not desirable because people associate privacy with having something to hide. Not only is a Westerner's desire for privacy not understood but in fact it has a negative connotation. Therefore, the attitudes of people may be affected by culture or background.

Also, an individual's standards of behavior may be influenced by past events in life. A person who grew up in a large family may place greater emphasis on personal control and ownership of possessions than would an only child who seldom had to share. Major events or close contact with others can also shape one's ethical position. Despite these differences, the underlying principles of how to make moral judgment are the same.

Although these aspects of ethics are quite reasonable and understandable, they lead people to distrust ethics because it is not founded on basic principles all can accept. Also, people from a scientific or technical background expect precision and universality.

Ethics Does Not Provide Answers

Ethical pluralism is recognizing or admitting that more than one position may be ethically justifiable—even equally so—in a given situation. Pluralism is another way of noting that two people may legitimately disagree on issues of ethics. We expect and accept disagreement in such areas as politics and religion.

رومانی صبحی رزق جرجس مطر

Exercises

1. List the issues involved in the software vulnerability reporting argument. What are the technical issues? The psychological and sociological ones? The managerial ones? The economic ones? The ethical ones? Select a vulnerability reporting process that you think is appropriate and explain why it meets more requirements than any other process.
2. List the issues involved in the software reliability (correct functioning of a product purchased) argument. What are the technical issues? The psychological/sociological ones? The managerial ones? The economic ones? The ethical ones? Select a policy on compensation for incorrect software you think is appropriate and explain why it meets more requirements than any other process.
3. Would you hire Goli (the computer security consultant and hacker from incident VIII) to protect your computer system? How would you respond if she came to you describing a vulnerability in your system and offering to help you fix it? Explain your answer.
4. Prepare an argument for or against the proposition that the following is ethical behavior. You and some friends decide to share music from CDs. You copy some to your computer and then burn identical copies for your friends. Does the argument change if the exchange is done with unknown people, through an anonymous file-sharing service on the order of Napster?
5. Prepare an argument for or against the proposition that the following is ethical behavior. While visiting a friend in another city you turn on your laptop and your wireless adapter senses a strong signal of an unsecured access point named siren-island. You connect to it and use Internet access throughout the weekend. Does the argument change if the time period is not just a weekend but unlimited (you are not just visiting but you live there) and the access point name obviously relates to the person who lives in the next apartment?

2025/2026 2025/2026 2025/2026

رومانی صبحی رزق جرجس مطر

With best wishes

Dr. Moustafa El-Ashry